

Contents

Basic definitions	17
World Space, ViewPort, and Screen Space	17
World Space	17
Viewport.....	17
Screen Space	17
Pan	17
Difference between them.....	18
Simple example.....	18
Illustrative example of World Space and Viewport.....	19
Diagram Coordination System	20
center-based world coordinate system.....	21
JavaScript and C# Usage	23
Description	23
Purpose of Using Both JavaScript and C#	23
JavaScript Usage	24
Role of JavaScript	24
JavaScript Responsibilities	24
JavaScript and Immediate Interaction	26
JavaScript Should Not Be the Main Authority for Permanent Data.....	26
Why JavaScript Is Suitable Here	27
Examples of JavaScript Usage in This Project	27
C# Usage	28
Role of C#.....	28
C# Responsibilities	28
C# as the Trusted Application Layer	28
Why C# Is Suitable Here	29
Examples of C# Usage in This Project.....	29
Communication Between JavaScript and C#	29
Recommended Responsibility Split.....	30
Important Design Rule.....	31
Recommended Rule for the Diagram Project.....	31
Short Developer Summary	32

Function Interaction Contract for JavaScript and C# Files	34
Required definition for each function	34
Trigger	34
Input	34
Validation / Preconditions	34
Processing Responsibility	34
Output	34
State Change / Persistence Effect	38
Next Possible Call(s)	38
Error Output / Failure Path	39
Recommended standard template	39
Recommended architectural rule	40
Recommended flow for project	40
Examples	41
Example 1 — JavaScript drag function	41
Example 2 — JavaScript save request	42
Example 3 — C# save endpoint	42
Example 4 — C# service function	43
DiagramCanvas	44
Purpose	44
Coordinate system	44
Center point	44
Axis directions	44
World space and viewport	45
Units	46
Item placement	46
Layering and Z behavior	46
Origin behavior	47
Item movement	47
Selection and hit testing	48
Snapping and overlap	48
Precision and rounding	48
Default canvas behavior	49
Rectangle Resize Control Point	49
Recommended developer rules	51

Recommended short description for developers	52
Recommendations	52
Suggested variables for DiagramCanvas	52
Rectangle Definition	59
Canonical stored variables	60
Default values	61
Fill definition	61
Line definition	62
Padding definition	62
Derived visible geometry variables.....	63
Derived Hover Padding boundaries	63
Derived Protection Padding boundaries	63
Boundary definition.....	64
Hover region rule	64
Resize Control Points	64
Junction point attachment.....	66
Rectangle as a container	66
Layering	66
Non-overlap using Protection Padding	67
Constraints.....	67
Recommendation	68
Square Junction Point definition.....	68
Meaning of Junction Point	68
Visual definition	69
No Padding	69
Junction Sensor Area	69
Geometry definition	71
Placement / attachment definition	72
For line attachment.....	73
For rectangle attachment.....	74
For circle attachment	74
Multiple junction points at the same place	75
Activation rule.....	75
Constraint rules	76
Recommendation	78

Circle Definition	79
Visual Definition.....	79
Padding.....	80
Geometry definition	81
Layering	82
Appearance variables	83
Boundary equation	83
Inside / outside equations	84
Hover region rule.....	84
Bounding box	84
Junction point attachment on a circle	85
Connection points.....	86
Non-overlap with another circle.....	86
Non-overlap with a rectangle	87
Recommended formal definition	88
Constraint rules	88
Clean variable list	89
Circle Resize Control Point.....	90
Recommendation	94
Rectangle Container	94
Parent Rectangle Container Definition	95
Derived parent boundaries	96
Recommended internal usable boundary	97
Parent container hover rule	98
Child Rectangle Definition	98
Nested Rectangle Container Definition	99
Important rule for nested containers.....	101
Child Circle Definition	101
General containment rule	102
General clamped movement equations.....	103
Strict no-touch rule	103
Multiple nesting levels	104
Recommendations	104
Final summary.....	105
Circle Center on Rectangle Container Edge Definition	107

Meaning and purpose	107
Visual definition	107
Geometry definition	107
Placement / attachment definition	108
Movement along rectangle container edge.....	109
Protection Padding and containment interpretation	109
Constraint rules	110
Recommendations	110
Final summary.....	110
Mathematical Calculations	110
Purpose and scope	110
Coordinate conversion calculations.....	111
Hover, selection, and visual activation calculations	111
Move-update calculations and MaxMoveValidationRate	113
Parent-container expansion calculations.....	114
Render-rate detection calculation	114
Protection Padding, Overlap, Containment, and Persistence Calculations.....	115
Purpose	115
Main architectural rule	116
World-space rule	117
Trigger conditions.....	117
Insert trigger.....	117
Move trigger.....	117
Optional later resize trigger	117
Canonical geometry and derived Protection Padding geometry.....	117
Rectangle Protection Padding model.....	118
Canonical rectangle variables	118
Derived rectangle Protection Padding values.....	118
Circle Protection Padding model.....	118
Overlap validation rules	119
Rectangle against rectangle	119
Circle against circle	120
Rectangle against circle	120
Validation when geometry is not inside a container	121
Free-canvas validation scope	121

Free-canvas validation flow	121
Validation when geometry is inside a container	122
Immediate-parent rule	122
Validation count inside a container	122
Child container case.....	122
Container containment equations	123
Child rectangle inside container.....	123
Child circle inside container	123
Insertion without a parent container	123
Required steps.....	123
Insertion within a container	124
Required steps.....	124
Parent resizing rule	124
Movement on free canvas	124
Overlap return rule during mouse movement	125
Movement inside a container	125
Recommended rule.....	125
Overlap return rule during mouse movement inside a container	126
Large moving rectangle container against smaller container	126
Previous valid position rule	127
Client-side and server-side execution rule	128
Better to run on client side.....	128
Better to run on server side	128
Combined rule	129
Required save behavior after calculation finishes.....	129
What shall be saved.....	129
What shall not be saved	130
Recalculation before save	130
Required event-to-function routing.....	131
Insert on free canvas.....	131
Insert inside container	131
Move on free canvas.....	131
Move inside container	132
Save committed diagram.....	132
Function contract rule.....	132

Recommended function list	133
JavaScript functions	133
C# functions.....	134
Final implementation rule	134
Different Keyboard and Mouse actions.....	135
Purpose	135
Main interaction rule	135
Architectural ownership rule.....	135
Interaction target categories	136
Canvas-level targets.....	136
Shape-level targets	136
Editing targets	136
Core mouse actions.....	137
Left click.....	137
Double left click.....	137
Right click	137
Mouse hover	138
Ctrl + mouse wheel up/down	138
Left click then drag shape.....	138
Left click then resize.....	139
Core keyboard actions.....	139
Arrow keys	139
Ctrl combinations	139
Shift and Alt modifiers.....	140
Delete key	140
Escape key	140
Interaction result depends on target.....	141
Hover over center.....	141
Hover over interior.....	141
Hover over edge or perimeter	141
Hover over resize control point	141
Hover over junction point.....	142
Hover over empty canvas.....	142
Recommended missing mouse actions to include	142
Mouse down.....	142

Mouse move	142
Mouse up	143
Click on canvas background	143
Drag over canvas	143
Drop on canvas	143
Pan action	144
Selection box / marquee drag	144
Middle click / mouse wheel click	144
Home / End	144
Page Up / Page Down	144
Copy / cut / paste shortcuts	144
Undo / redo shortcuts	145
Recommended action-state categories	145
Detection actions	145
Selection actions	146
Transformation actions	146
Context actions	146
Commit / cancel actions	146
Recommended section rule for developers	147
Short formal summary	147
Development phases	148
Important Rules	148
Milestone 1 — Shapes Panel Developer Plan	149
Overview	149
Shapes Panel terminology	150
Mandatory existing JavaScript before starting Milestone 1	151
Mathematical Calculation Rules for Milestone 1	152
Panel layout calculation	152
Search calculation	152
Drag-start handoff calculation	152
Milestone 1 phases	152
Phase 1.1 — Define Shapes Panel structure and visible behavior	152
Phase 1.2 — Add category selection and dynamic item loading	156
Phase 1.3 — Add search, splitter resize, and drag-start from library to canvas	159
Final result after Milestone 1	163

Milestone 1 persistence note	163
Save test criteria for Milestone 1	164
Coding files structure for Milestone 1	164
Best folder layout for Milestone 1	165
Milestone 2 — DiagramCanvas Developer Plan	166
Overview	166
Prerequisites	167
Mandatory existing JavaScript before starting Milestone 2	167
DiagramCanvas variables for Milestone 2	168
Recommended database table structure	170
Recommended files for Milestone 2	172
Important rules	192
Mathematical Calculation Rules for Milestone 2	193
Viewport boundaries	193
Coordinate conversion	193
Pan and zoom	193
Phase 2.1 — Define DiagramCanvas variables and bind them to visible behavior	193
Phase 2.2 — Drag a simple picture and validate world space, viewport, and screen space	196
Phase 2.3 — Save the active diagram to JSON and reopen it	199
Exact variables to save in this phase	199
Save regression rule for this phase	199
Save test criteria for this phase	199
Phase 2.4 — Save the active diagram to the database and reopen it	201
Exact variables to save in this phase	201
Save regression rule for this phase	201
Save test criteria for this phase	201
Phase 2.5 — Load multiple diagrams from JSON	203
Phase 2.6 — Load multiple diagrams from the database	204
Final result after Milestone 2	206
Recommended implementation order	207
Milestone 3 — Rectangle and Circle Definition Developer Plan	207
Overview	207
Prerequisites	208
Mandatory existing JavaScript before starting Milestone 3	208
Milestone 3 variables for Devices, Containers, Rectangles, and Circles	209

Recommended database table structure	212
Recommended files for Milestone 3.....	216
Important rules	231
Mathematical Calculation Rules for Milestone 3.....	232
Rectangle derived geometry.....	232
Circle derived geometry	232
Persistence rule	232
Phase 3.1 — Define Rectangle canonical variables and bind them to visible behavior	232
Phase 3.2 — Define Circle canonical variables and bind them to visible behavior	235
Phase 3.3 — Build the active shape-definition model and validate derived geometry	237
Phase 3.4 — Save the active shape definitions to JSON and reopen them	239
Exact variables to save in this phase.....	239
Save regression rule for this phase	239
Save test criteria for this phase	239
Phase 3.5 — Save the active shape definitions to the database and reopen them	241
Exact variables to save in this phase.....	241
Save regression rule for this phase	241
Save test criteria for this phase	241
Final result after Milestone 3.....	243
Recommended implementation order	244
Milestone 4 — Drag and Drop Rectangle or Circle from Left Panel into DiagramCanvas	244
Overview.....	244
Prerequisites	244
Mandatory existing JavaScript before starting Milestone 4.....	244
Milestone 4 variables.....	245
Recommended database table structure	245
Recommended files for Milestone 4.....	248
Important rules	263
Mathematical Calculation Rules for Milestone 4.....	264
Drop coordinate calculation	264
Default geometry calculation	264
Creation validation calculation	264
Phase 4.1 - Start dragging Rectangle or Circle from the left panel and show drag preview over the canvas	264
Phase 4.2 - Drop the Rectangle or Circle into DiagramCanvas and create a real item in world space	265

Phase 4.3 - Activate the new item, show post-drop state, and prepare for later item editing milestones.....	266
Phase 4.4 - Save the diagram to a JSON file and reopen it.....	266
Exact variables to save in this phase.....	266
Save regression rule for this phase	267
Save test criteria for this phase	267
Phase 4.5 - Save the diagram to the database and reopen it	268
Exact variables to save in this phase.....	268
Save regression rule for this phase	268
Save test criteria for this phase	268
Final result after Milestone 4.....	269
Recommended implementation order	269
Recommended implementation order	269
Milestone 5 — Rectangle Interaction Behavior on DiagramCanvas	270
Overview.....	270
Recommended files for Milestone 5.....	275
Recommended database table structure	275
Milestone 5 variables.....	276
Prerequisites	276
Mandatory existing JavaScript before starting Milestone 5.....	276
Milestone 5 terminology.....	276
Important milestone rules	278
Mathematical Calculation Rules for Milestone 5.....	279
Rectangle hover and selection	279
Move calculation	279
Resize calculation	280
Milestone 5 phases	280
Phase 5.1 — Hover detection and rectangle selection behavior	280
Phase 5.2 — Drag / move the selected rectangle on the canvas under movement constraints	283
Phase 5.3 — Resize control point hover and 8-direction rectangle resize behavior	287
Phase 5.4 — Finalize rectangle interaction state and prepare for future milestones	291
Phase 5.5 — Save rectangle-related committed diagram changes to a JSON file	292
Exact variables to save in this phase.....	292
Save regression rule for this phase	292
Save test criteria for this phase	293
Phase 5.6 — Save rectangle-related committed diagram changes to the database.....	295

Exact variables to save in this phase	295
Save regression rule for this phase	296
Save test criteria for this phase	296
Phase 5.7 — Recalculate committed rectangle geometry and persist the diagram to both JSON and database	299
Exact variables to save in this phase	299
Save regression rule for this phase	300
Save test criteria for this phase	300
Final result after Milestone 5	306
Recommended implementation order	307
Recommended coding files structure additions for Milestone 5	307
More ideas you can define in Milestone 5	308
Milestone 6 — Circle Interaction Behavior on DiagramCanvas	311
Overview	311
Prerequisites	311
Milestone 6 committed variables	311
Milestone 6 runtime-only variables	312
Recommended files for Milestone 6	312
Important Milestone 6 rules	313
Mathematical Calculation Rules for Milestone 6	313
Circle hover and selection	314
Circle move calculation	314
Circle resize calculation	314
Milestone 6 phases	314
Phase 6.1 — Circle hover detection and selection behavior	314
Phase 6.2 — Move the selected circle on DiagramCanvas	314
Phase 6.3 — Resize the circle from four resize control points	314
Phase 6.4 — Finalize circle interaction state and event handoff	315
Phase 6.5 — Save circle-related committed changes to JSON	315
Phase 6.6 — Save circle-related committed changes to the database	315
Phase 6.7 — Recalculate committed circle geometry and persist to both JSON and database	315
Final result after Milestone 6	315
Recommended implementation order	316
Milestone 7 — Connecting Two Geometric Items Using Junction Points and Lines	316
Overview	316
Goal	316

Scope	316
Prerequisites	317
Milestone 7 terminology.....	318
Important milestone rules	319
Connect-To workflow	319
Connection-type lookup source	320
Global Variables — Connections tab.....	320
Milestone 7 committed variables.....	321
Recommended database table structure	322
Table — ConnectionTypeLookup	322
Table — DiagramConnections	322
Table — DiagramConnectionDetails	322
Table — ConnectionStyleDefaults	323
Recommended files for Milestone 7	323
Function contract requirement	325
Mathematical Calculation Rules for Milestone 7	325
JavaScript calculation ownership.....	325
Canonical geometry inputs	326
Rectangle boundary attachment calculation	326
Rectangle host location parameter	327
Circle boundary attachment calculation	327
Shortest-path endpoint calculation by geometry pair.....	327
Connection route point calculation.....	328
Intermediate route-point persistence rule.....	329
Connection label position calculations	329
Move recalculation rule	330
Junction-point recalculation on resize	330
Database-driven connection recalculation	331
Milestone 7 phases	331
Phase 7.1 — Right-click connection workflow and Connect-To mode lock.....	331
Goal.....	331
Scope	332
Function contract deliverables	332
Validation / Preconditions	332
Error / failure expectations.....	332

Success criteria.....	332
Phase 7.2 — Select the second geometry and retrieve possible connection types	332
Goal.....	332
Scope	332
Function contract deliverables	332
Validation / Preconditions	332
Error / failure expectations.....	332
Success criteria.....	332
Phase 7.3 — Display connection options menu and apply default connection style	333
Goal.....	333
Scope	333
Function contract deliverables	333
Validation / Preconditions	333
Error / failure expectations.....	333
Success criteria.....	333
Phase 7.4 — Create a single-line shortest connection using junction points.....	334
Goal.....	334
Scope	334
Function contract deliverables	334
Validation / Preconditions	334
Error / failure expectations.....	334
Success criteria.....	334
Phase 7.5 — Create multi-link connection routes using derived intermediate route points	335
Goal.....	335
Scope	335
Function contract deliverables	335
Validation / Preconditions	335
Error / failure expectations.....	335
Success criteria.....	335
Phase 7.6 — Display source, destination, and middle-line connection text.....	336
Goal.....	336
Scope	336
Function contract deliverables	336
Validation / Preconditions	336
Error / failure expectations.....	336

Success criteria.....	336
Phase 7.7 — Recalculate connections when connected geometries move or resize	337
Goal.....	337
Scope	337
Function contract deliverables	337
Validation / Preconditions	337
Error / failure expectations.....	337
Success criteria.....	337
Phase 7.8 — Save and reload connection and junction-point information from JSON and database	338
Goal.....	338
Scope	338
Function contract deliverables	338
Validation / Preconditions	338
Error / failure expectations.....	338
Success criteria.....	338
Connection database-driven test criteria.....	338
Milestone 7 save test criteria	339
Final result after Milestone 7.....	339
Recommended implementation order	340
Recommended coding files structure additions for Milestone 7	340
More ideas you can define in Milestone 7	342
Milestone 8 — Circle Center on Rectangle Container Edge and Edge Movement Behavior.....	342
Overview.....	342
Recommended files for Milestone 8.....	343
Recommended database table structure	347
Milestone 8 variables.....	348
Prerequisites	349
Mandatory existing JavaScript before starting Milestone 8	350
Milestone 8 terminology.....	351
Important milestone rules	351
Core Circle On Container equations used in this milestone	352
Mathematical Calculation Rules for Milestone 8	352
Container edge segment parameter calculation	352
Candidate center projection/clamping.....	353
Edge movement calculation	353

Protection Padding and sibling validation calculation	353
Milestone 8 phases	353
Phase 8.1 — Define Edge-Attached Circle geometry model and committed variables	353
Phase 8.2 — Attach the circle center to a valid rectangle container edge	354
Phase 8.3 — Move the circle center across any valid edge of the container	354
Phase 8.4 — Handle edge endpoints and mandatory corner transition behavior	355
Phase 8.5 — Validate Protection Padding and sibling overlap while moving along edge	355
Phase 8.6 — Resize Circle On Container within global radius constraints	356
Phase 8.7 — Recalculate Circle On Container centers when the parent container moves or resizes	357
Phase 8.8 — Recalculate committed Circle On Container geometry and persist to JSON and database	369
Exact variables to save in this phase	357
Save regression rule for this phase	357
Save test criteria for this phase	357
Final result after Milestone 8	358
Recommended implementation order	358
Recommended coding files structure additions for Milestone 8	358
Best folder layout additions for Milestone 8	359
More ideas you can define in Milestone 8	360
Milestone 9 — Protection Padding Calculation and Containment Behavior Inside Rectangle Containers	360
Overview	360
Recommended files for Milestone 9	365
Recommended database table structure	365
Milestone 9 variables	366
Prerequisites	366
Mandatory existing JavaScript before starting Milestone 9	366
Milestone 9 terminology	367
Important milestone rules	369
Core Protection Padding equations used in this milestone	370
Mathematical Calculation Rules for Milestone 9	372
General containment calculation	372
Child rectangle center clamp	372
Child circle center clamp	372
Parent expansion calculation	372

Milestone 9 phases	372
Phase 9.1 — Single child rectangle inside parent rectangle container: free movement under Protection Padding	373
Phase 9.2 — Single child rectangle inside parent rectangle container: child resize under Protection Padding	376
Phase 9.3 — Parent rectangle resize and parent-child reference-center preservation for child rectangle	379
Phase 9.4 — Single child circle inside parent rectangle container: free movement under Protection Padding	382
Phase 9.5 — Single child circle inside parent rectangle container: child resize under Protection Padding	386
Phase 9.6 — Parent rectangle resize and parent-child reference-center preservation for child circle	389
Phase 9.7 — Recalculate committed Protection Padding geometry and persist the containment state to JSON and database	392
Phase 9.8 — Validate Circle On Container Protection Padding and radius constraints on a single rectangle container	407
Exact variables to save in this phase	392
Save regression rule for this phase	392
Save test criteria for this phase	392
Final result after Milestone 9	395
Recommended implementation order	396
Recommended coding files structure additions for Milestone 9	396
Best folder layout additions for Milestone 9	396
More ideas you can define in Milestone 9	397
Milestone 10 — Protection Padding Calculation and Containment Behavior for Multiple Children Inside One Parent Rectangle Container	398
Overview	398
The table below lists the main database tables used or extended by Milestone 10.	403
The table below includes the main Milestone 10 working variables.	403
Milestone 10 variables	404
Prerequisites	405
Mandatory existing JavaScript before starting Milestone 10	405
Milestone 10 terminology	405
Important milestone rules	407
Core Protection Padding equations used in this milestone	408
Mathematical Calculation Rules for Milestone 10	411
Validation set calculation	411

Sibling non-overlap calculation	411
Parent containment calculation.....	411
Milestone 10 phases	411
Phase 10.1 — Multiple child rectangles inside one parent rectangle container: free movement under Protection Padding.....	411
Phase 10.2 — Multiple child rectangles inside one parent rectangle container: child resize under Protection Padding.....	415
Phase 10.3 — Parent rectangle resize and parent-child reference-center preservation for multiple child rectangles	418
Phase 10.4 — Multiple child circles inside one parent rectangle container: free movement under Protection Padding.....	421
Phase 10.5 — Multiple child circles inside one parent rectangle container: child resize under Protection Padding.....	425
Phase 10.6 — Parent rectangle resize and parent-child reference-center preservation for multiple child circles	428
Phase 10.7 — Recalculate committed multi-child Protection Padding geometry and persist the containment state to JSON and database.....	431
Phase 10.8 — Validate multiple Circle On Container items on one container boundary system	447
Exact variables to save in this phase.....	431
Save regression rule for this phase	431
Save test criteria for this phase	431
Final result after Milestone 10.....	434
Recommended implementation order	435
Recommended coding files structure additions for Milestone 10	435
Best folder layout additions for Milestone 10.....	436
More ideas you can define in Milestone 10	436
Milestone 11 — Moving a Rectangle Container with Multiple Levels of Nested Containers and Multiple Geometric Children	437
Overview.....	437
Recommended files for Milestone 11	441
Recommended database table structure	441
Milestone 11 variables.....	442
Prerequisites.....	442
Mandatory existing JavaScript before starting Milestone 11	442
Milestone 11 terminology.....	443
Important milestone rules	444
Core movement equations used in this milestone	445

Large moved-container overlap equation for Milestone 11	447
Mathematical Calculation Rules for Milestone 11	447
Subtree movement delta	447
Validation scope calculation	448
Rejection calculation	448
Milestone 11 phases.....	448
Phase 11.1 — Define the movable container subtree and its descendant collections.....	448
Phase 11.2 — Move a nested rectangle container inside its immediate parent container.....	451
Phase 11.3 — Move an intermediate rectangle container with mixed descendants.....	455
Phase 11.4 — Move a top-level rectangle container with multiple nested containers and multiple geometric descendants	458
Phase 11.5 — Recalculate derived geometry and Protection Padding for the full moved subtree .	460
Phase 11.6 — Move one sibling container among multiple sibling containers inside the same immediate parent	463
Phase 11.7 — Recalculate committed hierarchical move state and persist the subtree move to JSON and database.....	465
Phase 11.8 — Move container subtrees with Circle On Container items attached to descendant container edges.....	482
Exact variables to save in this phase.....	465
Save regression rule for this phase	465
Save test criteria for this phase	465
Final result after Milestone 11	469
Recommended implementation order	469
Recommended coding files structure additions for Milestone 11.....	469
Best folder layout additions for Milestone 11	470
More ideas you can define in Milestone 11	470
Milestone 12 — Attaching an SVG Image to a Rectangle or a Circle Using Host Geometry Rules	471
Overview.....	471
Recommended files for Milestone 12.....	475
Recommended database table structure	476
Milestone 12 variables.....	477
Prerequisites	477
Mandatory existing JavaScript before starting Milestone 12	477
Milestone 12 terminology.....	477
Important milestone rules	480
Core SVG attachment equations used in this milestone.....	482

Mathematical Calculation Rules for Milestone 12	484
SVG host placement calculation	485
Rectangle host fit calculation	485
Circle host fit calculation	485
Persistence and validation calculation	485
Milestone 12 phases	485
Phase 12.1 — Upload, validate, and save the SVG asset as text	485
Phase 12.2 — Attach the SVG asset to a rectangle host	488
Phase 12.3 — Resize the SVG on a rectangle and make the rectangle host reflect that size	492
Phase 12.4 — Attach the SVG asset to a circle host	495
Phase 12.5 — Resize the SVG on a circle and make the circle host reflect that size	498
Phase 12.6 — Apply padding, container restrictions, and overlap rules to SVG through the host geometry	501
Phase 12.7 — Recalculate committed SVG asset, attachment, and host-linked geometry and persist to JSON and database	505
Exact variables to save in this phase	505
Save regression rule for this phase	505
Save test criteria for this phase	505
Final result after Milestone 12	510
Recommended implementation order	511
Recommended coding files structure additions for Milestone 12	511
Best folder layout additions for Milestone 12	512
More ideas you can define in Milestone 12	512

Basic definitions

World Space, ViewPort, and Screen Space

World Space

World Space is the main logical coordinate system of the diagram.

It is the space where items truly exist in the model.

Examples:

- Rectangle center = (20, 30)

- Circle center = (100, -50)
- Line start = (0, 0)

These values stay the same even if the user zooms in, zooms out, or pans the view.

So:

World Space = the real mathematical space of the diagram.

Viewport

Viewport is the currently visible window into world space.

The world may be infinite, but the user can only see part of it at one time.

That visible part is the **Viewport**.

For example:

- the diagram may contain items from $X = -100000$ to $X = 100000$
- but the user may currently only see the area from $X = -200$ to $X = 200$

That visible area is the viewport.

So:

Viewport = the part of world space currently being shown to the user.

Important:

- panning moves the viewport
- zooming changes how much world space the viewport shows
- the items themselves do not move in world space just because the viewport changes

Screen Space

Screen Space is the actual display coordinate system of the monitor or application window.

This is usually measured in pixels.

Examples:

- mouse click at pixel (500, 300)
- item drawn at pixel (1200, 700)

Screen space depends on:

- window size
- monitor size
- zoom level
- viewport position

So:

Screen Space = the final on-screen pixel space used for drawing and input.

Pan

Pan is the action of moving the viewport across world space without changing the true world-space coordinates of diagram items.

Difference between them

World Space

- stores the true item positions
- belongs to the diagram model
- does not change when the user pans or zooms

Viewport

- selects which part of world space is visible
- belongs to the viewing system
- changes when the user pans or zooms

Screen Space

- is the actual pixel location on the display
- belongs to the rendering and input system
- changes with window size, zoom, and viewport position

Simple example

Suppose a rectangle center is:

(20, 30)

This is its **World Space** position.

Now suppose the user is looking at a certain area of the diagram.
That visible area is the **Viewport**.

Inside that viewport, the renderer may draw the rectangle at:

(600, 250)

pixels on the monitor.

That is its **Screen Space** position.

So:

- (20, 30) = world space
- "current visible area containing that point" = viewport
- (600, 250) pixels = screen space

Very short summary

World Space = where the item really is

Viewport = what part of the world is currently visible

Screen Space = where it appears on the screen

Best way for developers to think about it

Use this flow:

World Space -> Viewport -> Screen Space

and for mouse clicks:

Screen Space -> Viewport -> World Space

That is the clean model for this diagram library.

Illustrative example of World Space and Viewport

For clarity, assume the diagram uses one global world space and one active viewport.

The **world space** is the full logical coordinate space of the diagram. In this example, it is represented by the thick outer boundary. All diagram items exist in this world space whether they are currently visible or not.

The **viewport** is the currently visible window into the world space. In this example, it is represented by the very thick inner boundary. Only the items located inside the viewport are currently visible to the user.

Inside the viewport, there are three visible items:

- Rectangle **R1** with center at **(-40, 20)**
- Rectangle **R2** with center at **(50, -10)**
- Circle **C1** with center at **(10, 30)**

Outside the viewport, but still inside world space, there are four additional circles:

- top-left circle at **(-150, 90)**
- top-right circle at **(170, 80)**
- bottom-right circle at **(150, -80)**

These three circles still belong to the diagram model because they exist in world space, but they are not currently visible because they lie outside the viewport.

This example shows the core rule clearly:

World space stores where items really exist.

Viewport defines which part of that world is currently visible.

If the viewport moves, the item coordinates in world space do not change. Only the visible portion of the world changes. This matches the document's model that world space contains the true item positions, while the viewport is only the visible window into that world.

For a shorter caption under the figure, use this:

Figure description: The thick outer boundary represents world space. The very thick inner boundary represents the viewport. Two rectangles and one circle are inside the viewport and are therefore visible. Four circles are outside the viewport but still inside world space, which means they exist in the diagram model but are not currently visible.

Diagram Coordination System

is the rule set that defines how positions are represented and understood inside the diagram.

It defines:

- where the origin is located, such as (0,0) or (0,0,0)
- how the axes behave, such as:
 - o +X to the right
 - o +Y upward or downward
- how item positions are measured
- how all diagram items share the same reference space
- how the diagram model interprets movement, distance, size, and placement

A cleaner formal definition is:

Diagram Coordinate System is the mathematical and logical reference framework used to define the position, direction, and spatial relationship of all items in the diagram.

For this project, it can be defined like this:

The diagram coordinate system is a center-based world coordinate system with origin at (0,0,0), where X increases to the right, Y increases upward, and Z represents stacking order.

So in simple words:

Diagram Coordinate System = the positioning system of the whole diagram.

It is broader than:

- **Diagram Origin Definition**, which only says where the origin is
- **Axis Orientation**, which only says which way the axes go

The coordinate system includes both of those, plus the general spatial rules of the diagram.

The following are different diagram coordination systems.

- center-based world coordinate system

- Top-left-based coordinate system
- Bottom-left-based coordinate system
- Object-local coordinate system
- Page-based coordinate system

center-based world coordinate system

Center-based world coordinate system is a diagram coordinate system in which the main reference origin is placed at the center of the logical world, usually at:

$(0, 0)$

or, if depth is included:

$(0, 0, 0)$

In this system, all item positions are measured relative to that center origin.

Typical axis rules are:

- positive X extends to the right
- negative X extends to the left
- positive Y extends upward
- negative Y extends downward

If a Z axis is used for stacking or depth:

- positive Z extends forward or to higher visual order
- negative Z extends backward or to lower visual order

A center-based world coordinate system is especially suitable for diagram systems that define items by their center points, such as rectangles, circles, and junction points, because it makes size, symmetry, and boundary calculations simpler and more consistent.

A clean formal definition is:

Center-based world coordinate system is a world coordinate system in which the origin is located at the center of the logical diagram space, and all item positions are defined relative to that central origin.

For this project, it matches naturally with variables such as:

- CenterX
- CenterY
- HalfWidth
- HalfHeight
- Radius

because shapes are described from their centers instead of from a corner.

JavaScript and C# Usage

Description

The system uses two main implementation layers:

- **JavaScript (JS)** for the **client-side browser layer**
- **C#** for the **server-side application layer**

JavaScript is responsible for the live behavior that the user directly sees and interacts with in the browser, such as drawing the DiagramCanvas, handling mouse and keyboard actions, updating the viewport, performing hit testing, and showing visual feedback immediately.

C# is responsible for the secure and controlled server-side logic, such as receiving requests from the browser, validating data, applying business rules, saving and loading diagrams, and communicating with the database.

This separation keeps the system organized, scalable, and easier to maintain. It ensures that fast visual interaction happens in the browser, while trusted processing and persistence happen on the server.

Purpose of Using Both JavaScript and C#

The diagram system requires two different categories of work:

1) Interactive visual behavior

This includes:

- drawing shapes
- dragging items
- resizing items
- zooming
- panning
- rotating the view or items
- showing selection states
- showing hover states
- showing resize control points
- showing snapping guides

- reacting to mouse and keyboard input immediately

2) Persistent and controlled application behavior

This includes:

- storing diagrams
- loading saved diagrams
- validating diagram data
- managing users and permissions
- applying server rules
- reading from and writing to the database
- protecting the application from invalid or malicious requests

These two responsibilities should remain separated.

The browser is best for immediate rendering and interaction.

The server is best for trusted logic, persistence, and security-sensitive operations.

Therefore:

- **JavaScript** should handle the live diagram experience in the browser.
- **C#** should handle the application services, trusted validation, and database communication on the server.

JavaScript Usage

Role of JavaScript

JavaScript is used on the client side inside the user's browser.

Its main purpose is to provide a smooth, immediate, visual, and interactive experience without requiring a full page reload for every action.

JavaScript is the correct layer for handling moment-to-moment DiagramCanvas behavior.

JavaScript Responsibilities

JavaScript should be used for:

- rendering the DiagramCanvas
- drawing the background, grid, axes, and origin marker
- converting between world space, viewport space, and screen space

- handling user interaction events
- performing hit testing
- updating temporary client-side state
- updating the visible result immediately
- preparing diagram data to be sent to the server when needed

Mouse actions handled in JavaScript

The following actions should normally be handled in JavaScript:

- left click
- double left click
- right click
- mouse hover
- Ctrl + mouse wheel up/down
- left click then drag shape
- left click then resize shape
- click on resize control points
- click on junction points
- click on canvas background
- context-menu-related mouse actions if used

Keyboard actions handled in JavaScript

The following actions should normally be handled in JavaScript:

- Up Arrow
- Down Arrow
- Left Arrow
- Right Arrow
- Ctrl key combinations related to zoom or tool behavior
- Shift or Alt modifiers if used by the editor
- delete, escape, or selection-related keys if later supported

Interaction target detection in JavaScript

JavaScript should also detect **what part of the item the pointer is targeting**.

Examples include:

- hover over shape center
- hover over interior area
- hover over edge
- hover over perimeter
- hover over resize control point
- hover over junction point
- hover over empty canvas space

These are not all the same action. Each may produce a different response.

For example:

- hover over the center of a rectangle may mean item hover
- hover over the perimeter may mean boundary-sensitive hover
- hover over the edge-middle control point may mean resize is available
- hover over a junction point may mean connection behavior is available

This type of detection is part of **client-side hit testing**, and it should normally be implemented in JavaScript.

JavaScript and Immediate Interaction

Immediate interaction behavior should normally run in JavaScript because it requires very fast visual response.

Examples include:

- moving the mouse across the canvas
- dragging an item continuously
- resizing an item continuously
- zooming with Ctrl + mouse wheel
- showing hover highlight while the pointer moves
- checking whether the pointer is over the center, edge, perimeter, or control point
- updating selection rectangles or temporary guides

If these actions were sent to the server for every mouse movement, the system would become unnecessarily slow and heavy.

Therefore, for DiagramCanvas interaction:

JavaScript shall handle immediate mouse and keyboard actions, hit testing, temporary visual state, and live rendering updates in the browser.

JavaScript Should Not Be the Main Authority for Permanent Data

JavaScript may temporarily hold working state while the user is editing, but it should not be treated as the final trusted authority for permanent storage or security-sensitive rules.

For example:

- JavaScript may allow a rectangle to move visually on screen
- JavaScript may calculate temporary coordinates during drag
- JavaScript may show temporary hover or selection states

But:

- the final save operation should be verified by the server
- permanent data validation should be enforced by the server
- database writes should not be performed directly from browser JavaScript

Why JavaScript Is Suitable Here

JavaScript is suitable for the browser layer because it:

- runs directly in the browser
- reacts immediately to user input
- is well suited for canvas, SVG, and DOM rendering
- supports live diagram editing behavior
- can perform fast hit testing and visual updates
- can update the visible interface without waiting for the server

Examples of JavaScript Usage in This Project

Examples include:

- drawing a rectangle on the canvas
- recalculating screen position during pan and zoom
- dragging a circle with the mouse
- resizing a rectangle using control points

- highlighting junction points during hover
- detecting whether the pointer is over the center or perimeter of a shape
- responding to arrow keys for item movement
- rotating the current viewport visually
- showing live values for world position and screen position
- sending a save request to the server after the user clicks Save

C# Usage

Role of C#

C# is used on the server side.

Its main purpose is to provide structured application logic, trusted validation, secure processing, and communication with the database.

C# is not normally responsible for per-mouse-move browser interaction. Instead, it is responsible for controlled backend operations.

C# Responsibilities

C# should be used for:

- receiving requests from the browser
- validating incoming diagram data
- enforcing business rules
- saving diagrams to the database
- loading diagrams from the database
- updating existing diagrams
- deleting diagrams when allowed
- managing user identity and access rules
- applying permission checks
- generating server responses
- logging important operations
- protecting the system from invalid or malicious input

C# as the Trusted Application Layer

The server should be considered the trusted layer of the system.

Even if JavaScript performs client-side checks for usability, the server must still validate critical data again.

Examples:

- JavaScript may check whether a required field is empty
- C# must also check it before storing the data
- JavaScript may send item coordinates
- C# must verify that the request format is valid before writing to the database
- JavaScript may indicate that an item was moved or resized
- C# must still validate the save request before persisting it

Why C# Is Suitable Here

C# is suitable for the server layer because it:

- supports structured backend development
- works well with APIs
- integrates well with databases
- is strong for business logic and validation
- provides a clear separation between frontend and backend responsibilities
- is suitable for trusted server-side processing

Examples of C# Usage in This Project

Examples include:

- saving a diagram record
- loading all items of a diagram by diagram ID
- validating whether a user has access to edit a diagram
- storing rectangle, circle, and line data
- returning diagram JSON to the browser
- recording audit logs for save operations

Communication Between JavaScript and C#

JavaScript and C# should communicate through clear server endpoints or APIs.

Typical flow:

1. the user performs an action in the browser
2. JavaScript updates the visual state immediately when appropriate
3. JavaScript decides whether a server call is needed
4. JavaScript sends a request to the server when required
5. C# receives the request
6. C# validates the request
7. C# reads from or writes to the database
8. C# returns a response
9. JavaScript updates the UI based on that response

Example: Save Diagram

- JavaScript collects the current diagram data from the browser
- JavaScript sends the diagram data to the server
- C# receives the diagram data
- C# validates the structure and business rules
- C# stores it in the database
- C# returns success or failure
- JavaScript shows the result to the user

So the practical architecture is:

Browser UI (JavaScript) $\rightleftharpoons$ Server API (C#) $\rightleftharpoons$ Database

Recommended Responsibility Split

JavaScript should handle

- visual rendering
- user interaction
- temporary client state
- coordinate conversion for display and interaction
- hit testing

- drag behavior
- resize behavior
- zoom behavior
- pan behavior
- hover behavior
- selection behavior
- keyboard-driven movement
- target detection such as center, edge, perimeter, or control point
- non-secure convenience validation
- calling backend APIs

C# should handle

- server APIs
- trusted validation
- business rules
- security checks
- user/session logic
- database reads and writes
- audit and error logging
- permanent data storage rules

Important Design Rule

The browser should control the **interactive experience**, but the server should control the **trusted application rules**.

That means:

- **JavaScript controls how the tool behaves visually and interactively**
- **C# controls what the system accepts, validates, and stores**

This is a key architectural rule for a diagram editor.

Recommended Rule for the Diagram Project

For this diagram tool, use the following principle:

JavaScript layer

Use JavaScript for all live graphical and interaction behavior on the DiagramCanvas, including:

- rendering
- hit testing
- left click
- double left click
- right click
- hover
- Ctrl + scroll up/down
- drag behavior
- resize behavior
- arrow key movement
- zooming
- panning
- rotation display
- selection behavior
- coordinate inspection
- detecting whether interaction occurs on the center, interior, edge, perimeter, control point, or junction point

C# layer

Use C# for all server-side services, including:

- diagram persistence
- validation
- database communication
- security checks
- user-based operations
- controlled retrieval of saved diagram data

Database layer

The database should not be accessed directly from browser JavaScript.

All database access should go through the server-side C# layer.

This improves:

- security
- control
- maintainability
- auditability

Short Developer Summary

JavaScript is used in the browser for rendering, interaction, hit testing, and temporary visual state.

C# is used on the server for validation, business logic, and database communication.

Immediate mouse and keyboard behavior on DiagramCanvas should normally run in JavaScript, while trusted persistence and server rules should run in C#.

Function Interaction Contract for JavaScript and C# Files

Because the system is split into separate JavaScript and C# files, every important function should be defined using one clear interaction contract. This prevents ambiguity between browser behavior, server behavior, state updates, and rendering flow.

This contract should be used for:

- JavaScript event handlers
- JavaScript processing functions
- JavaScript rendering functions
- JavaScript API-calling functions
- C# controller/API endpoint methods
- C# service methods
- C# repository/database methods

The goal is to make every function easy to understand in terms of:

- what causes it to run
- what data it receives
- what it does internally
- what it returns or changes
- what may happen next

Required definition for each function

Every important function should document the following fields:

- Trigger.
- Input.
- Validation/preconditions.
- Processing Responsibility.
- Output.
- Stat Change / Persistence Effect.
- Next Possible Call(s).
- Error Output / Failure Path.

Trigger

The Trigger defines what causes the function to run.

Examples:

- browser mouse event
- keyboard event
- call from another JavaScript function
- API request from JavaScript to C#
- internal C# service call
- database save request
- timer-based refresh
- initialization during page load

Examples:

- mousedown on a rectangle
- mousemove while dragging
- clicking Save Diagram
- JavaScript calling `/api/diagram/save`
- C# controller calling `DiagramService.SaveDiagram()`

Input

The Input defines the data the function receives.

This should include the exact values or objects expected by the function.

Examples:

- mouse screen coordinates
- keyboard key value
- current canvas state
- selected item ID
- diagram JSON payload
- validated DTO object
- diagram ID from route parameter

Examples:

- screenX, screenY
- canvasState
- selectedShapeId
- SaveDiagramRequest
- diagramId
- userId

Important rule:

Inputs should be defined clearly as **read-only incoming data**, not mixed with internal working variables.

Validation / Preconditions

Before processing starts, the function should define what must already be true.

Examples:

- canvas must be initialized
- item must exist
- item must be selectable
- drag mode must be active
- request payload must not be null
- user must be authorized
- diagram ID must be valid

This is important because many bugs happen when the trigger is correct but the system state is not ready.

Examples:

- selected rectangle exists
- PanEnabled = true
- ZoomScale > 0
- request model passes validation
- current user has edit permission

Processing Responsibility

This is the corrected version of your “processed code” idea.

This section defines what the function is responsible for doing internally.

It should describe the logic, not every code line.

Examples:

- convert screen coordinates to world coordinates
- update dragged item position
- recalculate viewport
- perform hit testing
- validate diagram structure
- apply business rules
- save data to database
- build API response
- re-render canvas

Important rule:

Processing Responsibility should describe **what the function owns logically**, not low-level implementation details.

Output

The Output defines the direct result produced by the function.

The output may be:

- a returned value
- an updated object

- a UI refresh
- an API response
- a database write result
- a success/failure flag
- a changed visual state

Examples:

- updated CenterX, CenterY
- recalculated screen coordinates
- true/false
- SaveDiagramResponse
- updated HTML canvas rendering
- error response object

Important rule:

Output should describe the **observable result**, not just “function completed.”

State Change / Persistence Effect

This section defines whether the function changes temporary state, permanent state, or both.

Examples:

- updates temporary drag state only
- updates current viewport state
- updates selected item state
- writes to database
- creates audit log
- changes in-memory canvas model
- no persistence effect

This distinction is very important in your architecture because JavaScript and C# have different responsibilities:

- JavaScript usually changes **temporary interaction state**
- C# usually changes **trusted persistent state**

Next Possible Call(s)

This is the corrected version of “next called code function.”

It should not always be singular, because the next step may branch.

Examples:

- call render function
- call hit-test function
- call API client function
- call C# service method
- call repository save method
- no further call
- branch depending on success/failure

Examples:

- `handleMouseMoveDrag()` → `updateDraggedRectangle()` → `renderCanvas()`
- `saveDiagram()` → `/api/diagram/save` → `DiagramController.Save()` → `DiagramService.SaveDiagram()`

Important rule:

Use **Next Possible Call(s)**, not **Next Called Function**, because real systems branch.

Error Output / Failure Path

Every important function should define what happens if processing fails.

Examples:

- ignore event
- cancel drag
- show validation message
- return 400 Bad Request
- return 403 Forbidden
- return 500 error
- keep previous state unchanged
- log error

This prevents unclear behavior during invalid states or failed requests.

Recommended standard template

Use this template for each important function:

Function Name

Layer: JavaScript or C#

File: file name

Purpose: short purpose statement

Trigger:

What causes this function to run

Input:

What data it receives

Validation / Preconditions:

What must already be true

Processing Responsibility:

What the function does internally

Output:

What it returns, changes, or produces

State Change / Persistence Effect:

Temporary state change, permanent save, no state change, etc.

Next Possible Call(s):

What function(s) may be called next

Error Output / Failure Path:

What happens if the function fails

Recommended architectural rule

You should also add this rule:

Function ownership rule

A function should have one primary responsibility only.

That means:

- JavaScript event functions should detect and route actions
- JavaScript processing functions should update client-side interaction state
- JavaScript rendering functions should draw visual results
- JavaScript API functions should send and receive server requests
- C# controller functions should receive requests and route them

- C# service functions should apply business rules
- C# repository functions should handle database access

This keeps the code modular and prevents mixing rendering, input handling, validation, and persistence in one place.

Recommended flow for project

For this diagram tool, the normal cross-layer flow should be written like this:

Browser-side flow

User action

- JavaScript event handler
- JavaScript validation/precheck
- JavaScript processing logic
- JavaScript state update
- JavaScript render update
- optional API request to server

Server-side flow

API request received

- C# controller/endpoint
- request validation
- service/business logic
- repository/database access
- response creation
- response returned to JavaScript

Return flow

JavaScript receives response

- update UI state if needed
- render final visible result
- show success or error

Examples

Example 1 — JavaScript drag function

Function Name: handleRectangleDragMove

Layer: JavaScript

File: drag-rectangle.js

Purpose:

Update the dragged rectangle position while the mouse moves.

Trigger:

mousemove event while drag mode is active

Input:

Current mouse screen position, drag state, selected rectangle ID, current canvas state

Validation / Preconditions:

A rectangle is selected, drag mode is active, canvas is initialized

Processing Responsibility:

Convert screen coordinates to world coordinates, compute movement delta, update rectangle center position in world space

Output:

Updated rectangle CenterX and CenterY

State Change / Persistence Effect:

Updates temporary in-browser diagram state only; no database write

Next Possible Call(s):

renderCanvas()

updateDebugPanel()

Error Output / Failure Path:

If drag state is invalid, ignore movement and keep previous rectangle position

Example 2 — JavaScript save request

Function Name: saveDiagram

Layer: JavaScript

File: diagram-api.js

Purpose:

Send the current diagram to the server for trusted validation and persistence.

Trigger:

User clicks Save

Input:

Current diagram model, user session context

Validation / Preconditions:

Diagram exists, required fields exist, user session is active

Processing Responsibility:

Serialize diagram data into request payload and send POST request to server API

Output:

HTTP request sent; later receives success or failure response

State Change / Persistence Effect:

No direct database write from JavaScript; only initiates server persistence request

Next Possible Call(s):

C# DiagramController.SaveDiagram()

On response: showSaveSuccess() or showSaveError()

Error Output / Failure Path:

Show error message and keep current client state unchanged

Example 3 — C# save endpoint

Function Name: SaveDiagram

Layer: C#

File: DiagramController.cs

Purpose:

Receive diagram save request from the browser and route it to trusted application logic.

Trigger:

HTTP POST request from JavaScript

Input:

SaveDiagramRequest

Validation / Preconditions:

Request must not be null, model must be valid, user must be authenticated and authorized

Processing Responsibility:

Validate request model, call service layer, return standardized response

Output:

SaveDiagramResponse or HTTP error response

State Change / Persistence Effect:

May lead to persistent storage after service validation

Next Possible Call(s):

DiagramService.SaveDiagram()

Error Output / Failure Path:

Return 400, 403, or 500 depending on failure type

Example 4 — C# service function

Function Name: SaveDiagram

Layer: C#

File: DiagramService.cs

Purpose:

Apply trusted business rules and persist the diagram if valid.

Trigger:

Called by controller

Input:

Validated save request DTO, user context

Validation / Preconditions:

Diagram structure valid, user allowed to modify diagram, IDs consistent

Processing Responsibility:

Apply business rules, map DTO to storage model, call repository, create save result

Output:

Save result object

State Change / Persistence Effect:

Writes diagram data to database and may create audit records

Next Possible Call(s):

DiagramRepository.Save()

Error Output / Failure Path:

Throw controlled service exception or return structured failure result

DiagramCanvas

DiagramCanvas is the root graphical workspace of the diagram model. It contains all engineering items, manages their spatial placement and visual stacking, and provides the common coordinate system in which items are rendered, selected, moved, connected, and validated.

Purpose

DiagramCanvas is the root drawing surface and logical container for all diagram items. It holds and manages all items such as rectangles, circles, lines, square junction points, and future engineering symbols.

It defines:

- the global coordinate system
- the spatial placement of all items
- the drawing order of all items
- the interaction space for selection, movement, snapping, and overlap rules

DiagramCanvas is an **infinite logical workspace**. The visible screen is only a viewport into that workspace.

Coordinate system

The DiagramCanvas shall use a **center-based world coordinate system**.

Center point

The logical center of the canvas shall be:

(0·0·0)

This point is the **origin** of the world coordinate system.

Axis directions

The recommended logical axis directions are:

- positive X = to the right
- negative X = to the left
- positive Y = upward
- negative Y = downward
- positive Z = in front / higher draw order
- negative Z = behind / lower draw order

Important recommendation

Use this logical coordinate system everywhere in the model, even if a future UI technology uses screen coordinates where Y grows downward.

That means:

- the **model** uses Y upward
- the **renderer** may convert it if needed

This keeps the geometry rules mathematically clean.

Infinite dimensions in X, Y, and Z

DiagramCanvas shall be treated as **logically unbounded** in all axes.

X dimension

The canvas has no fixed minimum or maximum X limit.

Y dimension

The canvas has no fixed minimum or maximum Y limit.

Z dimension

The canvas has no fixed minimum or maximum Z limit.

However, for practical implementation:

- X and Y should be stored using a numeric type suitable for high precision
- Z should usually be treated as **drawing order**, not physical 3D depth

Recommendation for Z

Unless true 3D behavior is required later, developers should interpret Z as:

- ZOrder
- or DrawOrder

not as real 3D geometry.

So the canvas is infinite in stacking order, but that does **not** mean it is a 3D engine.

World space and viewport

The system shall distinguish between:

World space

The infinite logical coordinate system in which all items are stored.

Viewport

The finite visible window currently shown to the user.

The viewport may move and scale, but the world coordinates of items shall remain unchanged.

Rule

Panning changes the viewport position, not the item coordinates.

Rule

Zooming changes the viewport scale, not the item coordinates.

Units

All item geometry shall be stored in **logical diagram units**, not screen pixels.

Examples:

- center positions
- half-width
- half-height
- radius
- line endpoints
- padding values

Recommendation

Do not make the diagram model depend on display resolution.

Pixels belong to rendering only.

Item placement

Every diagram item placed on the canvas shall exist in world space.

Each item shall have:

- a unique ID
- a position in the canvas coordinate system
- a set of geometry variables
- a set of appearance variables
- a draw order / Z order

Examples:

- rectangle center at (CenterX, CenterY)
- line endpoints at (P1X, P1Y) and (P2X, P2Y)
- circle center at (CenterX, CenterY)
- junction point center at (CenterX, CenterY)

Layering and Z behavior

The canvas shall support item stacking using ZOrder.

Rule

If two items overlap visually, the item with the larger ZOrder shall be drawn above the item with the smaller ZOrder.

Recommendation

Use integer values for ZOrder.

Example:

- background items: negative values
- normal items: 0
- items intentionally above others: positive values

Secondary rule

If two items have the same ZOrder, a stable secondary rule shall be used, such as:

- creation order
- explicit draw sequence
- internal item ordering

This rule must be deterministic.

Origin behavior

The canvas origin (0,0,0) is only the reference point of the world space.

It is not required that:

- an item always exists there
- the viewport always shows it
- the user always sees the center on screen

Recommendation

Treat the origin as a mathematical reference only.

Item movement

When an item moves, its world coordinates shall be updated directly.

Example:

- rectangle movement updates CenterX, CenterY
- line movement updates P1X, P1Y, P2X, P2Y
- circle movement updates CenterX, CenterY

Rule

Movement shall not be stored as screen-relative offsets.

It shall always be stored in world coordinates.

Selection and hit testing

Selection and hit testing shall be based on item geometry in world space.

The renderer may transform input from screen space to world space first, then perform selection checks.

Recommendation

All geometry checks should happen in world coordinates:

- inside rectangle
- on line
- on circle boundary
- inside padded area
- overlap detection

This keeps selection behavior consistent across zoom levels.

Snapping and overlap

All snapping and overlap rules shall be evaluated in world space.

Examples:

- rectangle padded overlap

- junction point padded overlap
- attachment to rectangle boundary
- attachment to circle boundary
- attachment to any position on a line

Rule

Padding regions are logical interaction regions, not necessarily visible render regions.

Precision and rounding

Developers shall use numeric precision suitable for geometry operations.

Recommendation

Use floating-point or decimal-compatible numeric types for:

- positions
- dimensions
- padding
- radius
- line coordinates

Rule

Do not round stored world coordinates unnecessarily.

Rounding should be limited to:

- display formatting
- snapping policies
- export rules if required

Default canvas behavior

Unless overridden, DiagramCanvas should use the following defaults:

- origin at (0,0,0)
- infinite logical X range
- infinite logical Y range
- infinite logical Z range
- no forced page boundaries
- world coordinates independent from screen pixels
- panning allowed
- zooming allowed
- item geometry stored in world space
- ZOrder used for stacking

Rectangle Resize Control Point

A **Resize Control Point** is a draggable control point defined on the boundary of a rectangle. When the user left-clicks and drags one of these points, the rectangle may be resized in the allowed direction of that point.

A rectangle shall have **8 Resize Control Points**:

Corner Resize Control Points

- TopLeftResizeControlPoint
- TopRightResizeControlPoint
- BottomRightResizeControlPoint
- BottomLeftResizeControlPoint

Edge-Middle Resize Control Points

- TopCenterResizeControlPoint
- RightCenterResizeControlPoint
- BottomCenterResizeControlPoint
- LeftCenterResizeControlPoint

Each Resize Control Point shall have its own unique identity and variable name, even though all belong to the same rectangle.

Meaning

Resize Control Points are editing points used to modify rectangle size and, depending on the implementation, rectangle position. They are not connection points and are not junction points. Their purpose is shape editing only.

Placement

Each Resize Control Point is placed on a specific geometric location of the rectangle boundary.

If the rectangle has:

- CenterX
- CenterY
- HalfWidth
- HalfHeight

and derived boundaries:

$$\begin{aligned}LeftX &= CenterX - HalfWidth \\RightX &= CenterX + HalfWidth \\TopY &= CenterY + HalfHeight \\BottomY &= CenterY - HalfHeight\end{aligned}$$

then the Resize Control Point positions are:

Corner points

$$TopLeftResizeControlPoint = (LeftX, TopY)$$

TopRightResizeControlPoint = (RightX, TopY)
BottomRightResizeControlPoint = (RightX, BottomY)
BottomLeftResizeControlPoint = (LeftX, BottomY)

Edge-middle points

TopCenterResizeControlPoint = (CenterX, TopY)
RightCenterResizeControlPoint = (RightX, CenterY)
BottomCenterResizeControlPoint = (CenterX, BottomY)
LeftCenterResizeControlPoint = (LeftX, CenterY)

Behavior

When the user left-clicks and drags a Resize Control Point, the rectangle shall resize according to the point being dragged.

Corner points

Corner Resize Control Points affect both X and Y dimensions.

- TopLeftResizeControlPoint changes left boundary and top boundary
- TopRightResizeControlPoint changes right boundary and top boundary
- BottomRightResizeControlPoint changes right boundary and bottom boundary
- BottomLeftResizeControlPoint changes left boundary and bottom boundary

Edge-middle points

Edge-middle Resize Control Points affect one dimension only.

- TopCenterResizeControlPoint changes top boundary only
- RightCenterResizeControlPoint changes right boundary only
- BottomCenterResizeControlPoint changes bottom boundary only
- LeftCenterResizeControlPoint changes left boundary only

Constraint rules

- Every Resize Control Point shall remain attached to its defined rectangle boundary position.
- Resize Control Point positions are **derived from rectangle geometry** and should not be treated as independent primary geometry.
- If the rectangle is resized, all 8 Resize Control Point positions shall be recalculated.
- If the rectangle is moved, all 8 Resize Control Point positions shall move with it.
- If RotationAngle is later supported for rectangles, Resize Control Point positions shall be rotated together with the rectangle.

Recommendation

Treat each Resize Control Point as a named logical point of the rectangle, but derive its coordinates from the rectangle's current geometry instead of storing separate unrelated position values.

Recommended developer rules

Developers should follow these rules:

Rule 1

Treat DiagramCanvas as the root container of all diagram items.

Rule 2

Store all item geometry in world coordinates relative to the canvas origin.

Rule 3

Keep the world model independent from rendering technology.

Rule 4

Use ZOrder as stacking order unless true 3D behavior is explicitly introduced later.

Rule 5

Use the canvas as logically infinite; do not impose page limits in the core model.

Rule 6

Implement viewport, zoom, and pan as view behavior, not model behavior.

Rule 7

Apply all geometry, padding, attachment, and overlap rules in world space.

Recommended short description for developers

DiagramCanvas is the infinite logical drawing surface and root coordinate space of the diagram system. It contains all diagram items, defines their world positions and stacking order, and serves as the reference space for rendering, selection, snapping, movement, and overlap validation.

- Radius must satisfy $\text{CalculatedMinimumRadius} \leq \text{Radius} \leq \text{CalculatedMaximumRadius}$ using the global Circle On Container radius-ratio variables and the current active edge length.
- Resizing the Circle On Container shall not resize the container.
- Resizing the container shall not automatically resize the Circle On Container.
- If the attached container, a parent container, or any higher nested parent container moves, the Circle On Container shall move with the affected container subtree and then recalculate its CenterX and CenterY from HostEdge and EdgeParameterT.

Recommendations

Recommended additional decisions for consistency:

- Use ZOrder instead of ZPosition unless you truly want 3D geometry.
- Keep the logical model center-based with origin (0,0,0).
- Keep Y positive upward in the model.
- Treat the visible UI as a viewport over an infinite canvas.
- Keep all geometry rules renderer-independent.

The next useful section would be a **formal DiagramCanvas specification** with fields like:

- CanvasID
- OriginX
- OriginY
- OriginZ
- ViewportCenterX
- ViewportCenterY
- ZoomScale
- DefaultZOrder
- CanvasItems[]

Suggested variables for DiagramCanvas

Identity variables

- CanvasID
- CanvasName
- CanvasDescription

Background variables

- BackgroundColor
- BackgroundType

Possible values for BackgroundType:

- SolidBackground
- TransparentBackground
- GridBackground later if needed

For version 1, this is enough:

- BackgroundColor

3) Coordinate system variables

- CoordinateSystemType
- OriginDefinition
- AxisOrientationX
- AxisOrientationY
- AxisOrientationZ

Example:

- CoordinateSystemType = CenterBasedWorld

- OriginDefinition = Center
- AxisOrientationX = RightPositive
- AxisOrientationY = UpPositive

4) World-space range variables

Because the canvas is required to support infinite dimensions, these can be logical flags:

- IsInfiniteX
- IsInfiniteY
- IsInfiniteZ

Recommended defaults:

- IsInfiniteX = True
- IsInfiniteY = True
- IsInfiniteZ = True

5) Viewport variables

These describe what part of the canvas is currently visible:

- ViewportCenterX
- ViewportCenterY
- ViewportWidth
- ViewportHeight
- ZoomScale

6) Grid variables

If canvas alignment support is required:

- GridVisible
- GridColor
- GridSpacingX
- GridSpacingY

7) Snapping variables

- SnapEnabled
- SnapToGrid
- SnapToItems
- SnapTolerance

8) Layering variables

- DefaultZOrder
- ZOrderMode

9) Interaction variables

- PanEnabled
- ZoomEnabled
- SelectionEnabled
- MultiSelectionEnabled

- DragEnabled
- ConnectionEnabled

10) Display aid variables

- ShowOriginMarker
- ShowAxes
- ShowCanvasBorder if later needed

11) Item container variables

- CanvasItems
- ItemCount

Best minimum set for version 1

For a clean first version, use these variables only:

- CanvasID
- CanvasName
- BackgroundColor
- CoordinateSystemType
- OriginDefinition
- AxisOrientationX
- AxisOrientationY
- AxisOrientationZ
- IsInfiniteX
- IsInfiniteY
- IsInfiniteZ
- ViewportCenterX
- ViewportCenterY
- ViewportWidth
- ViewportHeight
- ZoomScale
- GridVisible
- GridColor
- GridSpacingX
- GridSpacingY
- SnapEnabled
- SnapTolerance
- PanEnabled
- ZoomEnabled
- CanvasItems

Recommendation

For this project, define DiagramCanvas with these groups:

- Identity
- Background
- Coordinate System
- Infinity Rules

- Viewport
- Grid
- Snapping
- Interaction
- Items Container

And yes, definitely include:

- BackgroundColor

A clean short example would be:

DiagramCanvas

- CanvasID
- CanvasName
- BackgroundColor
- CoordinateSystemType
- OriginDefinition
- AxisOrientationX
- AxisOrientationY
- AxisOrientationZ
- IsInfiniteX
- IsInfiniteY
- IsInfiniteZ
- ViewportCenterX
- ViewportCenterY
- ViewportWidth
- ViewportHeight
- ZoomScale
- GridVisible
- GridColor
- GridSpacingX
- GridSpacingY
- SnapEnabled
- SnapTolerance
- PanEnabled
- ZoomEnabled
- CanvasItems

For DiagramCanvas

You may add:

- RotationAngle

but for **DiagramCanvas**, I do **not** recommend it in version 1.

Why:

- the canvas is usually the fixed world reference

- rotating the whole canvas makes coordinate handling, viewport logic, snapping, and input conversion more complex
- most systems keep the canvas/world stable and rotate the items, not the whole canvas

So for DiagramCanvas:

- possible: yes
- recommended now: no

2) For diagram items

For items, **yes, definitely**.

It is a very good variable for:

- rectangles
- lines
- symbols
- future engineering items

Examples:

- rotated rectangle
- angled equipment symbol
- custom shape orientation

So item definitions can include:

- RotationAngle

3) Best meaning of RotationAngle

RotationAngle is the angular orientation of an item relative to its default unrotated position.

Recommended unit

Use:

- degrees

unless you have a strong reason to use radians.

Recommended direction

Define clearly:

- positive angle = counterclockwise
- negative angle = clockwise

This is the standard mathematical convention.

4) Recommended default value

$$RotationAngle = 0^\circ$$

Meaning:

- no rotation
- item is in its default orientation

5) Best place to add it now

Recommendation:

Rectangle

Add:

- RotationAngle

Circle

Usually not needed visually for a plain circle, because a circle looks the same when rotated.
But you may still allow it if the circle later carries:

- labels
- ports
- markers
- directional symbols

Straight Line

A line already has direction from its endpoints, so a separate RotationAngle is usually unnecessary.

Square Junction Point

Usually not needed now, because it is symmetric.
But it could be added later if the symbol changes.

6) Recommendation for your specification

Use this rule:

For general diagram items

- RotationAngle is optional
- default:

$$RotationAngle = 0^\circ$$

For DiagramCanvas

Do not include it now unless you intentionally want:

- rotated whole-diagram views

- special CAD-like camera behavior

7) Clean definition

RotationAngle is a variable that defines the angular orientation of a diagram item relative to its default unrotated position, measured in degrees around the item's reference point, usually its center.

8) Recommendation

Add RotationAngle to:

- Rectangle
- future engineering symbols
- any non-symmetric item

Do **not** add it to DiagramCanvas in version 1.

A good variable list for Rectangle would then become:

- RectangleID
- CenterX
- CenterY
- HalfWidth
- HalfHeight
- RotationAngle
- ZOrder
- FillType
- FillColor
- LineType
- LineColor
- LineWidth
- PaddingXRatio
- PaddingYRatio
- PaddingColor

The Rectangle definition shall include RotationAngle when rectangle orientation is required.

Rectangle Definition

A **Rectangle** is a geometric engineering item defined by a center point, a horizontal half-size, a vertical half-size, appearance properties, padding properties, rotation, and layering order. It may be used as:

- a visible shape
- a host item for junction points
- a host item for line attachment

- a boundary item for connection logic
- a non-overlap item
- a child item inside a parent container
- a rectangle container for other items

This definition assumes a center-based world coordinate system, where:

- positive X is to the right
- positive Y is upward

Canonical stored variables

Identity

- RectangleID

Geometry

- CenterX
- CenterY
- HalfWidth
- HalfHeight
- RotationAngle

Layering

- ZOrder

Fill

- FillType
- FillColor

Line

- LineType
- LineColor
- LineWidth

Hover Padding

- HoverPaddingXRatio
- HoverPaddingYRatio

- HoverPaddingColor

Protection Padding

- ProtectionPaddingXRatio
- ProtectionPaddingYRatio
- ProtectionPaddingColor

Default values

Fill

- FillType = NoFill

Line

- LineType = SolidLine
- LineColor = Black

Rotation

- RotationAngle = 0°

Hover Padding

- HoverPaddingXRatio = 0.1
- HoverPaddingYRatio = 0.1
- HoverPaddingColor = Transparent

Protection Padding

- ProtectionPaddingXRatio = 0.2
- ProtectionPaddingYRatio = 0.2
- ProtectionPaddingColor = Transparent

Fill definition

The rectangle fill controls the inside area of the rectangle.

Allowed values:

- NoFill
- SolidFill

Rules:

- If FillType = NoFill, then FillColor is ignored.
- If FillType = SolidFill, then FillColor must exist.

Line definition

The rectangle line represents the 4 borders of the rectangle as one common border style.

Allowed values:

- NoLine
- SolidLine

Rules:

- If LineType = NoLine, then LineColor and LineWidth are ignored.
- If LineType = SolidLine, then LineColor and LineWidth must exist.

Padding definition

A rectangle shall support **two independent padding areas** outside its visible boundary.

1) Hover Padding

The **Hover Padding** is used for:

- mouse-hover detection near the rectangle perimeter
- visual highlighting of the rectangle during hover

By default, it is transparent.

Derived values:

$$\begin{aligned} \textit{HoverPaddingX} &= \textit{HoverPaddingXRatio} \cdot \textit{HalfWidth} \\ \textit{HoverPaddingY} &= \textit{HoverPaddingYRatio} \cdot \textit{HalfHeight} \end{aligned}$$

By default:

$$\begin{aligned} \textit{HoverPaddingX} &= 0.1 \cdot \textit{HalfWidth} \\ \textit{HoverPaddingY} &= 0.1 \cdot \textit{HalfHeight} \end{aligned}$$

When the mouse hovers over the rectangle perimeter region defined by the Hover Padding, the Hover Padding shall take the same color as the rectangle line:

$$\textit{HoverPaddingColor} = \textit{LineColor}$$

2) Protection Padding

The **Protection Padding** is used for:

- avoiding overlap
- spacing rules
- child-inside-container calculations
- containment validation

By default, it is transparent.

Derived values:

$$\begin{aligned} ProtectionPaddingX &= ProtectionPaddingXRatio \cdot HalfWidth \\ ProtectionPaddingY &= ProtectionPaddingYRatio \cdot HalfHeight \end{aligned}$$

By default:

$$\begin{aligned} ProtectionPaddingX &= 0.2 \cdot HalfWidth \\ ProtectionPaddingY &= 0.2 \cdot HalfHeight \end{aligned}$$

All overlap and container calculations shall use the **Protection Padding**, not the Hover Padding.

Derived visible geometry variables

The following variables shall be derived from the canonical stored variables:

$$\begin{aligned} LeftX &= CenterX - HalfWidth \\ RightX &= CenterX + HalfWidth \\ TopY &= CenterY + HalfHeight \\ BottomY &= CenterY - HalfHeight \end{aligned}$$

These derived variables shall not be treated as independent stored truths.

Derived Hover Padding boundaries

$$\begin{aligned} HoverLeftX &= LeftX - HoverPaddingX \\ HoverRightX &= RightX + HoverPaddingX \\ HoverTopY &= TopY + HoverPaddingY \\ HoverBottomY &= BottomY - HoverPaddingY \end{aligned}$$

Derived Protection Padding boundaries

$$ProtectionLeftX = LeftX - ProtectionPaddingX$$

$$\begin{aligned}
ProtectionRightX &= RightX + ProtectionPaddingX \\
ProtectionTopY &= TopY + ProtectionPaddingY \\
ProtectionBottomY &= BottomY - ProtectionPaddingY
\end{aligned}$$

These derived variables shall not be treated as independent stored truths.

Boundary definition

A point (X, Y) lies inside or on the unrotated rectangle if:

$$LeftX \leq X \leq RightX$$

and

$$BottomY \leq Y \leq TopY$$

A point lies on the rectangle boundary if at least one of the following is true:

$$X = LeftX \text{ and } BottomY \leq Y \leq TopY$$

or

$$X = RightX \text{ and } BottomY \leq Y \leq TopY$$

or

$$Y = TopY \text{ and } LeftX \leq X \leq RightX$$

or

$$Y = BottomY \text{ and } LeftX \leq X \leq RightX$$

For $RotationAngle \neq 0$, the rectangle shall be treated as rotated about its center, and visible boundary tests shall be performed after applying the rectangle rotation around $(CenterX, CenterY)$.

Hover region rule

The rectangle hover region is the band between:

- the visible rectangle boundary
- the outer Hover Padding boundary

A mouse point is considered in the hover region when it lies inside the Hover Padding outer boundary and within the intended perimeter-sensitive hover band.

When hover is active, the Hover Padding shall visually highlight using the rectangle line color.

Resize Control Points

A rectangle shall have **8 Resize Control Points**.

Corner Resize Control Points

- TopLeftResizeControlPoint
- TopRightResizeControlPoint
- BottomRightResizeControlPoint
- BottomLeftResizeControlPoint

Edge-Middle Resize Control Points

- TopCenterResizeControlPoint
- RightCenterResizeControlPoint
- BottomCenterResizeControlPoint
- LeftCenterResizeControlPoint

These points are derived from the rectangle geometry.

Corner point equations

$$\begin{aligned} \text{TopLeftResizeControlPoint} &= (\text{LeftX}, \text{TopY}) \\ \text{TopRightResizeControlPoint} &= (\text{RightX}, \text{TopY}) \\ \text{BottomRightResizeControlPoint} &= (\text{RightX}, \text{BottomY}) \\ \text{BottomLeftResizeControlPoint} &= (\text{LeftX}, \text{BottomY}) \end{aligned}$$

Edge-middle point equations

$$\begin{aligned} \text{TopCenterResizeControlPoint} &= (\text{CenterX}, \text{TopY}) \\ \text{RightCenterResizeControlPoint} &= (\text{RightX}, \text{CenterY}) \\ \text{BottomCenterResizeControlPoint} &= (\text{CenterX}, \text{BottomY}) \\ \text{LeftCenterResizeControlPoint} &= (\text{LeftX}, \text{CenterY}) \end{aligned}$$

Behavior

When the user left-clicks and drags a Resize Control Point, the rectangle shall resize according to the allowed direction of that point.

- TopLeftResizeControlPoint changes left and top boundaries
- TopRightResizeControlPoint changes right and top boundaries
- BottomRightResizeControlPoint changes right and bottom boundaries

- BottomLeftResizeControlPoint changes left and bottom boundaries
- TopCenterResizeControlPoint changes top boundary only
- RightCenterResizeControlPoint changes right boundary only
- BottomCenterResizeControlPoint changes bottom boundary only
- LeftCenterResizeControlPoint changes left boundary only

If $\text{RotationAngle} \neq 0$, the Resize Control Points shall rotate together with the rectangle around its center.

Junction point attachment

A junction point may be placed on any allowed point on the rectangle boundary.

If:

- $\text{HostItemType} = \text{Rectangle}$
- $\text{HostLocationType} = \text{RectangleBoundary}$

then the junction point center must satisfy one of the rectangle boundary equations above, or the rotated equivalent if $\text{RotationAngle} \neq 0$.

Rectangle as a container

If a rectangle acts as a **parent container**, then any child item inside it must remain fully inside its usable interior.

All child containment calculations shall use the parent's **Protection Padding** or explicitly defined inner usable boundary.

For a child item with padded boundaries:

- $\text{ChildLeftBoundaryWithPadding}$
- $\text{ChildRightBoundaryWithPadding}$
- $\text{ChildTopBoundaryWithPadding}$
- $\text{ChildBottomBoundaryWithPadding}$

the containment rules are:

$$\begin{aligned} \text{ChildLeftBoundaryWithPadding} &> \text{ParentInnerLeftX} \\ \text{ChildRightBoundaryWithPadding} &< \text{ParentInnerRightX} \\ \text{ChildTopBoundaryWithPadding} &< \text{ParentInnerTopY} \\ \text{ChildBottomBoundaryWithPadding} &> \text{ParentInnerBottomY} \end{aligned}$$

If the rectangle itself is a **child rectangle** inside another parent container, then its own Protection Padding boundaries shall be used for the containment check.

Layering

ZOrder defines the visual stacking order of the rectangle relative to other items.

Rule:

- A rectangle with higher ZOrder is drawn above a rectangle with lower ZOrder.

If two rectangles have the same ZOrder, a deterministic secondary drawing rule shall be used.

Non-overlap using Protection Padding

Let two rectangles have Protection Padding boundaries:

- Rectangle 1:
 - ProtectionLeftX_1
 - ProtectionRightX_1
 - ProtectionTopY_1
 - ProtectionBottomY_1
- Rectangle 2:
 - ProtectionLeftX_2
 - ProtectionRightX_2
 - ProtectionTopY_2
 - ProtectionBottomY_2

Then the two rectangles do **not overlap** if at least one of the following is true:

$$ProtectionRightX_1 \leq ProtectionLeftX_2$$

or

$$ProtectionRightX_2 \leq ProtectionLeftX_1$$

or

$$ProtectionTopY_1 \leq ProtectionBottomY_2$$

or

$$ProtectionTopY_2 \leq ProtectionBottomY_1$$

If even touching must be forbidden, then replace $\leq$ with $<$.

Constraints

$HalfWidth > 0$
 $HalfHeight > 0$
 $HoverPaddingXRatio \geq 0$
 $HoverPaddingYRatio \geq 0$
 $ProtectionPaddingXRatio \geq 0$
 $ProtectionPaddingYRatio \geq 0$
 $LineWidth \geq 0$

RotationAngle is measured in degrees, with:

- default 0°
- positive angle = counterclockwise

Recommendation

Use:

- CenterX, CenterY, HalfWidth, and HalfHeight as the only primary geometry variables
- RotationAngle as the rectangle orientation variable
- Hover Padding only for hover detection and highlight behavior
- Protection Padding only for overlap and container calculations

All side positions, padding boundaries, and Resize Control Point locations should be derived from the rectangle's current geometry rather than stored as unrelated independent values

Square Junction Point definition

Meaning of Junction Point

A **Square Junction Point** is a connection marker that can be placed:

- at the start of a line

- at the end of a line
- at any location on a line
- at any allowed position on a rectangle boundary
- at any allowed position on a circle boundary

It is a real item with its own identity, even if another junction point exists at exactly the same position.

So two or more Square Junction Points may:

- have the same coordinates
- have the same variables
- occupy the same visible place

but they are still different items because they have different IDs.

A Square Junction Point also has a **Junction Sensor Area** used for sensing and activation behavior.

Visual definition

A Square Junction Point is visually represented as a square whose default appearance is fully transparent.

Fill

- FillType = SolidFill
- FillColor = Transparent

Line

- LineType = SolidLine
- LineColor = Transparent
- LineWidth = defined default value

This means the junction point exists geometrically, but by default it is not visibly shown.

When its activation rule is met, the Square Junction Point shall become white.

Activated visual state

- FillColor = White
- LineColor = White

No Padding

A Square Junction Point has **no padding at all**.

So:

- no padding variables are used
- no padding equations are used
- no padding overlap rules are used

If compatibility with earlier models is needed, the correct interpretation is:

- Padding = Not Used

Junction Sensor Area

A Square Junction Point shall support a **Junction Sensor Area**.

The Junction Sensor Area is equivalent in mathematical form to padding, but it shall **not** be called padding.

Its purpose is:

- sensing approach
- activation
- highlighting logic

It is not a normal visible fill region.

Junction Sensor Area variables

- JunctionSensorAreaXRatio
- JunctionSensorAreaYRatio
- JunctionSensorAreaColor

Default values

- JunctionSensorAreaXRatio = 0.2
- JunctionSensorAreaYRatio = 0.2
- JunctionSensorAreaColor = Transparent

Junction Sensor Area equations

If the square uses:

- HalfSide

then:

$$\begin{aligned}JunctionSensorAreaX &= 0.2 \cdot HalfSide \\ JunctionSensorAreaY &= 0.2 \cdot HalfSide\end{aligned}$$

If instead rectangle-style variables are used:

- HalfWidth
- HalfHeight

then:

$$\begin{aligned}JunctionSensorAreaX &= 0.2 \cdot HalfWidth \\JunctionSensorAreaY &= 0.2 \cdot HalfHeight\end{aligned}$$

Since this is a square:

$$HalfWidth = HalfHeight$$

so both forms are equivalent.

Junction Sensor Area boundaries

If the square center is:

$$(CenterX, CenterY)$$

and its half-side is:

$$HalfSide$$

then the visible boundaries are:

$$\begin{aligned}LeftX &= CenterX - HalfSide \\RightX &= CenterX + HalfSide \\TopY &= CenterY + HalfSide \\BottomY &= CenterY - HalfSide\end{aligned}$$

and the Junction Sensor Area boundaries are:

$$\begin{aligned}JunctionSensorLeftX &= LeftX - JunctionSensorAreaX \\JunctionSensorRightX &= RightX + JunctionSensorAreaX \\JunctionSensorTopY &= TopY + JunctionSensorAreaY \\JunctionSensorBottomY &= BottomY - JunctionSensorAreaY\end{aligned}$$

By default:

- JunctionSensorAreaColor = Transparent

When the activation rule is met:

- JunctionSensorAreaColor = White

Geometry definition

Since this item is a square, the cleanest definition is to use one size variable instead of separate width and height variables.

Recommended canonical stored variables

- JunctionPointID
- CenterX
- CenterY
- HalfSide
- ZOrder
- FillType
- FillColor
- LineType
- LineColor
- LineWidth
- JunctionSensorAreaXRatio
- JunctionSensorAreaYRatio
- JunctionSensorAreaColor

Equivalent compatibility form

If compatibility with the rectangle model is needed, you may use:

- HalfWidth
- HalfHeight

with the constraint:

$$\textit{HalfWidth} = \textit{HalfHeight}$$

Derived variables

$$\begin{aligned}\textit{LeftX} &= \textit{CenterX} - \textit{HalfSide} \\ \textit{RightX} &= \textit{CenterX} + \textit{HalfSide} \\ \textit{TopY} &= \textit{CenterY} + \textit{HalfSide} \\ \textit{BottomY} &= \textit{CenterY} - \textit{HalfSide}\end{aligned}$$

and:

- JunctionSensorLeftX

- JunctionSensorRightX
- JunctionSensorTopY
- JunctionSensorBottomY

as defined above.

Placement / attachment definition

A Square Junction Point should know where it belongs.

Attachment variables

- HostItemID
- HostItemType
- HostLocationType
- HostLocationParameter

These define both the host item and the exact attachment rule.

HostItemType

Possible values:

- Line
- Rectangle
- Circle

HostLocationType

Possible values:

- LineStart
- LineEnd
- LineAnywhere
- RectangleBoundary
- CircleBoundary

HostLocationParameter

This variable defines the exact location on the host item.

For line attachment

A Square Junction Point may be placed:

- at the start of a line
- at the end of a line
- at any location along a line

Case A: start of line

If:

- `HostItemType = Line`
- `HostLocationType = LineStart`

then:

$$CenterX = P1X$$

$$CenterY = P1Y$$

Case B: end of line

If:

- `HostItemType = Line`
- `HostLocationType = LineEnd`

then:

$$CenterX = P2X$$

$$CenterY = P2Y$$

Case C: any location on a line

If:

- `HostItemType = Line`
- `HostLocationType = LineAnywhere`

then define a line parameter:

$$t \in [0,1]$$

where:

- $t = 0$ means line start
- $t = 1$ means line end

and:

$$CenterX = P1X + t(P2X - P1X)$$

$$CenterY = P1Y + t(P2Y - P1Y)$$

This is the cleanest mathematical way to place a junction point anywhere on a straight line.

For rectangle attachment

A Square Junction Point may be placed on any allowed point on a rectangle boundary.

If:

- `HostItemType = Rectangle`
- `HostLocationType = RectangleBoundary`

then the junction point center must satisfy at least one of the following:

$$CenterX = LeftX \text{ and } BottomY \leq CenterY \leq TopY$$

or

$$CenterX = RightX \text{ and } BottomY \leq CenterY \leq TopY$$

or

$$CenterY = TopY \text{ and } LeftX \leq CenterX \leq RightX$$

or

$$CenterY = BottomY \text{ and } LeftX \leq CenterX \leq RightX$$

This means the junction point lies on one of the four sides of the rectangle.

For circle attachment

A Square Junction Point may be placed on any allowed point on a circle boundary.

If:

- `HostItemType = Circle`
- `HostLocationType = CircleBoundary`

and the circle has center:

$$(C_x, C_y)$$

and radius:

$$r$$

then the junction point center must satisfy:

$$(CenterX - C_x)^2 + (CenterY - C_y)^2 = r^2$$

This places the junction point exactly on the circle boundary.

Multiple junction points at the same place

Two or more Square Junction Points may:

- have identical geometry
- have identical appearance
- have identical host item
- have identical host location

They are still distinct items if:

$$JunctionPointID_1 \neq JunctionPointID_2$$

So identity is not based on position.

This means:

- there is no uniqueness rule on (CenterX, CenterY)
- there is no uniqueness rule on host attachment
- two different junction points may occupy the same visible location

Activation rule

The default color of both:

- the Square Junction Point
- the Junction Sensor Area

shall be **Transparent**.

They shall become **White** when the following action is met:

Activation condition 1

Another junction point approaches and enters the sensor or padding area of a rectangle or a circle.

Result

When this happens for a rectangle:

- all junction points belonging to that rectangle shall change to White

When this happens for a circle:

- all junction points belonging to that circle shall change to White

Host-based activation rule

For a rectangle host:

If a triggering junction point enters the defined active sensing region of a rectangle, then for every junction point where:

- $HostItemType = Rectangle$
- $HostItemID = RectangleID$

set:

- $FillColor = White$
- $LineColor = White$
- $JunctionSensorAreaColor = White$

For a circle host:

If a triggering junction point enters the defined active sensing region of a circle, then for every junction point where:

- $HostItemType = Circle$
- $HostItemID = CircleID$

set:

- $FillColor = White$
- $LineColor = White$
- $JunctionSensorAreaColor = White$

Constraint rules

Geometry rules

$$HalfSide > 0$$

Square rule

If width and height form is used instead of $HalfSide$, then:

$$HalfWidth = HalfHeight$$

Visual rules

- FillType = SolidFill
- LineType = SolidLine
- LineWidth ≥ 0

Default visual state

- FillColor = Transparent
- LineColor = Transparent

Junction Sensor Area rules

$$\begin{aligned} JunctionSensorAreaX &= 0.2 \cdot HalfSide \\ JunctionSensorAreaY &= 0.2 \cdot HalfSide \\ JunctionSensorAreaColor &= Transparent \end{aligned}$$

Activated visual state

When the activation condition is met:

- FillColor = White
- LineColor = White
- JunctionSensorAreaColor = White

Line-host rules

If:

- HostItemType = Line
- HostLocationType = LineStart

then:

$$CenterX = P1X, CenterY = P1Y$$

If:

- HostItemType = Line
- HostLocationType = LineEnd

then:

$$CenterX = P2X, CenterY = P2Y$$

If:

- HostItemType = Line
- HostLocationType = LineAnywhere

then:

$$\begin{aligned} CenterX &= P1X + t(P2X - P1X) \\ CenterY &= P1Y + t(P2Y - P1Y) \end{aligned}$$

where:

$$0 \leq t \leq 1$$

Rectangle-host rules

If:

- HostItemType = Rectangle
- HostLocationType = RectangleBoundary

then the junction point center must lie on one rectangle side.

Circle-host rules

If:

- HostItemType = Circle
- HostLocationType = CircleBoundary

then:

$$(CenterX - C_x)^2 + (CenterY - C_y)^2 = r^2$$

Recommendation

Treat the Square Junction Point as:

- a real geometric item
- transparent by default
- host-attached by geometry
- sensor-enabled by **Junction Sensor Area**
- activated by host-based approach logic

Keep Junction Sensor Area separate from any rectangle or circle padding terminology, so developers do not confuse:

- junction sensing behavior
- rectangle hover padding
- rectangle protection padding

Circle Definition

A **Circle** is a geometric engineering item defined by a center point and a radius. It can be used as:

- a standalone visible shape
- a host item for junction points
- a host item for line attachment
- a boundary item for connection logic
- a base primitive for more advanced engineering symbols later

A circle is fully determined by:

- its center position
- its radius

Everything else should be derived from these.

Visual Definition

A circle has two appearance groups, similar to the rectangle.

Fill

Possible values:

- NoFill
- SolidFill

If:

- FillType = SolidFill

then:

- FillColor must exist

Line

This represents the circular outer border.

Possible values:

- NoLine
- SolidLine

If:

- LineType = SolidLine

then:

- LineColor must exist
- LineWidth must exist

So the circle uses the same visual logic as the rectangle, but its border is one continuous curve instead of 4 sides.

Padding

A circle shall support **two independent padding areas**, both defined radially.

These two padding areas are independent logical regions and each one has its own purpose, size, and display behavior.

1) Hover Padding

The **Hover Padding** is used for:

- mouse-hover detection near the circle perimeter
- visual highlighting of the circle during hover

By default, it is transparent.

Hover Padding variables

- HoverPaddingRadiusRatio
- HoverPaddingColor

Default values

- HoverPaddingRadiusRatio = 0.1
- HoverPaddingColor = Transparent

Derived value

$$\text{HoverPaddingRadius} = \text{HoverPaddingRadiusRatio} \cdot \text{Radius}$$

By default:

$$\text{HoverPaddingRadius} = 0.1 \cdot \text{Radius}$$

Hover padded radius

$$\textit{HoverPaddedRadius} = \textit{Radius} + \textit{HoverPaddingRadius}$$

When the mouse hovers over the circle perimeter region defined by the Hover Padding, the Hover Padding shall take the same color as the circle line:

$$\textit{HoverPaddingColor} = \textit{LineColor}$$

2) Protection Padding

The **Protection Padding** is used for:

- avoiding overlap
- spacing rules
- container calculations
- containment validation

By default, it is transparent.

Protection Padding variables

- ProtectionPaddingRadiusRatio
- ProtectionPaddingColor

Default values

- ProtectionPaddingRadiusRatio = 0.2
- ProtectionPaddingColor = Transparent

Derived value

$$\textit{ProtectionPaddingRadius} = \textit{ProtectionPaddingRadiusRatio} \cdot \textit{Radius}$$

By default:

$$\textit{ProtectionPaddingRadius} = 0.2 \cdot \textit{Radius}$$

Protection padded radius

$$\textit{ProtectionPaddedRadius} = \textit{Radius} + \textit{ProtectionPaddingRadius}$$

All overlap and container calculations shall use the **Protection Padding**, not the Hover Padding.

Geometry definition

Canonical stored variables

- CircleID
- CenterX
- CenterY
- Radius
- ZOrder
- FillType
- FillColor
- LineType
- LineColor
- LineWidth
- HoverPaddingRadiusRatio
- HoverPaddingColor
- ProtectionPaddingRadiusRatio
- ProtectionPaddingColor

Optional stored variables later

- Visible
- Locked

Derived variables

$$\begin{aligned}LeftX &= CenterX - Radius \\RightX &= CenterX + Radius \\TopY &= CenterY + Radius \\BottomY &= CenterY - Radius\end{aligned}$$

Derived Hover Padding variables

$$\begin{aligned}HoverPaddingRadius &= HoverPaddingRadiusRatio \cdot Radius \\HoverPaddedRadius &= Radius + HoverPaddingRadius \\HoverLeftX &= CenterX - HoverPaddedRadius \\HoverRightX &= CenterX + HoverPaddedRadius \\HoverTopY &= CenterY + HoverPaddedRadius \\HoverBottomY &= CenterY - HoverPaddedRadius\end{aligned}$$

Derived Protection Padding variables

$$ProtectionPaddingRadius = ProtectionPaddingRadiusRatio \cdot Radius$$

$$ProtectionPaddedRadius = Radius + ProtectionPaddingRadius$$

$$ProtectionLeftX = CenterX - ProtectionPaddedRadius$$

$$ProtectionRightX = CenterX + ProtectionPaddedRadius$$

$$ProtectionTopY = CenterY + ProtectionPaddedRadius$$

$$ProtectionBottomY = CenterY - ProtectionPaddedRadius$$

These derived values should not be treated as independent stored geometry.

Layering

The circle should support stacking order exactly like the rectangle.

Use:

- ZOrder

Rule:

- higher ZOrder is drawn above lower ZOrder

If two items have the same ZOrder, then a secondary draw-order rule will be needed.

Appearance variables

Fill group

- FillType
- FillColor

Rules:

- $FillType \in \{NoFill, SolidFill\}$
- if $FillType = SolidFill$, then $FillColor$ is required
- if $FillType = NoFill$, then $FillColor$ is ignored

Line group

- LineType
- LineColor
- LineWidth

Rules:

- $LineType \in \{NoLine, SolidLine\}$

- if LineType = SolidLine, then LineColor and LineWidth are required
- if LineType = NoLine, then LineColor and LineWidth are ignored

Boundary equation

This is the most important mathematical definition of the circle.

Let the circle center be:

$$(CenterX, CenterY)$$

and radius be:

$$Radius$$

Then a point (X, Y) lies on the circle boundary if:

$$(X - CenterX)^2 + (Y - CenterY)^2 = Radius^2$$

This is the exact equation for the circle boundary.

If Hover Padding is applied, then the hover outer boundary becomes:

$$(X - CenterX)^2 + (Y - CenterY)^2 = HoverPaddedRadius^2$$

If Protection Padding is applied, then the protection outer boundary becomes:

$$(X - CenterX)^2 + (Y - CenterY)^2 = ProtectionPaddedRadius^2$$

Inside / outside equations

These are useful later for collision checks, selection, and attachment rules.

Point inside or on circle

$$(X - CenterX)^2 + (Y - CenterY)^2 \leq Radius^2$$

Point strictly inside circle

$$(X - CenterX)^2 + (Y - CenterY)^2 < Radius^2$$

Point outside circle

$$(X - CenterX)^2 + (Y - CenterY)^2 > Radius^2$$

For the Hover Padding version, replace Radius with HoverPaddedRadius.

For the Protection Padding version, replace Radius with ProtectionPaddedRadius.

Hover region rule

The circle hover region is the band between:

- the visible circle perimeter
- the outer Hover Padding boundary

The mouse is considered in the hover region when it lies inside the Hover Padding outer boundary and within the intended perimeter-sensitive hover band.

When hover is active, the Hover Padding shall visually highlight using the same color as the circle line.

Bounding box

Every circle should have a derived bounding box.

Non-padded bounding box

- $\text{LeftX} = \text{CenterX} - \text{Radius}$
- $\text{RightX} = \text{CenterX} + \text{Radius}$
- $\text{TopY} = \text{CenterY} + \text{Radius}$
- $\text{BottomY} = \text{CenterY} - \text{Radius}$

Hover Padding bounding box

- $\text{HoverLeftX} = \text{CenterX} - \text{HoverPaddedRadius}$
- $\text{HoverRightX} = \text{CenterX} + \text{HoverPaddedRadius}$
- $\text{HoverTopY} = \text{CenterY} + \text{HoverPaddedRadius}$
- $\text{HoverBottomY} = \text{CenterY} - \text{HoverPaddedRadius}$

Protection Padding bounding box

- $\text{ProtectionLeftX} = \text{CenterX} - \text{ProtectionPaddedRadius}$
- $\text{ProtectionRightX} = \text{CenterX} + \text{ProtectionPaddedRadius}$
- $\text{ProtectionTopY} = \text{CenterY} + \text{ProtectionPaddedRadius}$
- $\text{ProtectionBottomY} = \text{CenterY} - \text{ProtectionPaddedRadius}$

These bounding boxes are useful for:

- selection

- hit testing
- rough collision checks
- zoom-to-fit
- grouping
- overlap calculations
- container calculations

Junction point attachment on a circle

If:

- `HostItemType = Circle`
- `HostLocationType = CircleBoundary`

then the junction point center must satisfy:

$$(JunctionX - CenterX)^2 + (JunctionY - CenterY)^2 = Radius^2$$

If the junction point must be placed on the Hover Padding outer boundary, then:

$$(JunctionX - CenterX)^2 + (JunctionY - CenterY)^2 = HoverPaddedRadius^2$$

If the junction point must be placed on the Protection Padding outer boundary, then:

$$(JunctionX - CenterX)^2 + (JunctionY - CenterY)^2 = ProtectionPaddedRadius^2$$

This should be decided by rule later, but the mathematical forms are clear.

Connection points

A circle may also define formal connection points.

There are two approaches.

Option A: free boundary attachment

A connection can exist at any point on the circle boundary.

This means the attachment point is continuous.

Option B: discrete connection points

You define specific points such as:

- top

- right
- bottom
- left

These are:

Top point

$$(CenterX, CenterY + Radius)$$

Right point

$$(CenterX + Radius, CenterY)$$

Bottom point

$$(CenterX, CenterY - Radius)$$

Left point

$$(CenterX - Radius, CenterY)$$

For version 1, support for both is acceptable, but starting with discrete points is simpler.

Non-overlap with another circle

Let two circles have:

- centers $(C_{x1}, C_{y1}), (C_{x2}, C_{y2})$
- **Protection padded radii** R_1, R_2

Then they do not overlap if:

$$\sqrt{(C_{x2} - C_{x1})^2 + (C_{y2} - C_{y1})^2} \geq R_1 + R_2$$

Equivalent squared form:

$$(C_{x2} - C_{x1})^2 + (C_{y2} - C_{y1})^2 \geq (R_1 + R_2)^2$$

If you do not want even touching, then use:

$$>$$

instead of:

$\geq$

Non-overlap with a rectangle

If later you need a circle and rectangle not to overlap, the standard method is:

1. find the closest point on the rectangle to the circle center
2. measure the distance from that point to the circle center
3. compare with the circle radius

Let the rectangle boundaries be:

- LeftX
- RightX
- BottomY
- TopY

Let the circle center be:

- (Cx, Cy)

Define the closest point:

$$\begin{aligned}ClosestX &= \max(LeftX, \min(Cx, RightX)) \\ ClosestY &= \max(BottomY, \min(Cy, TopY))\end{aligned}$$

Then non-overlap is:

$$(Cx - ClosestX)^2 + (Cy - ClosestY)^2 \geq Radius^2$$

For padded checks:

- use the rectangle's **Protection Padding** boundaries
- use the circle's **ProtectionPaddedRadius**

Recommended formal definition

Circle v2

Identity

- CircleID

Geometry

- CenterX
- CenterY
- Radius

Layering

- ZOrder

Fill

- FillType
- FillColor

Line

- LineType
- LineColor
- LineWidth

Hover Padding

- HoverPaddingRadiusRatio
- HoverPaddingColor

Protection Padding

- ProtectionPaddingRadiusRatio
- ProtectionPaddingColor

Constraint rules

Geometry rules

$$Radius > 0$$

$$HoverPaddingRadiusRatio \geq 0$$

$$ProtectionPaddingRadiusRatio \geq 0$$

Appearance rules

- if FillType = SolidFill, then FillColor is required
- if LineType = SolidLine, then LineColor and LineWidth are required
- LineWidth ≥ 0

Boundary rule

A point lies on the circle boundary if:

$$(X - CenterX)^2 + (Y - CenterY)^2 = Radius^2$$

Clean variable list

Circle canonical stored variables

- CenterX
- CenterY
- Radius
- ZOrder
- FillType
- FillColor
- LineType
- LineColor
- LineWidth
- HoverPaddingRadiusRatio
- HoverPaddingColor
- ProtectionPaddingRadiusRatio
- ProtectionPaddingColor

Circle derived variables

- LeftX
- RightX
- TopY
- BottomY
- HoverPaddingRadius
- HoverPaddedRadius
- HoverLeftX
- HoverRightX
- HoverTopY
- HoverBottomY
- ProtectionPaddingRadius

- ProtectionPaddedRadius
- ProtectionLeftX
- ProtectionRightX
- ProtectionTopY
- ProtectionBottomY

Circle Resize Control Point

A **Circle Resize Control Point** is a draggable control point defined on the perimeter of a circle. When the user left-clicks and drags one of these points, the circle may be resized.

A circle shall have **4 Resize Control Points** located at the intersection of the circle perimeter with the X-axis and Y-axis passing through the circle center.

Resize Control Point names

- RightResizeControlPoint
- TopResizeControlPoint
- LeftResizeControlPoint
- BottomResizeControlPoint

Each Circle Resize Control Point shall have its own unique variable name, even though all belong to the same circle.

Meaning

Circle Resize Control Points are editing points used to modify the radius of a circle. They are not connection points and are not junction points. Their purpose is shape editing only.

Because the circle is symmetric, 4 resize points are sufficient.

Geometry definition

If the circle has:

- CenterX
- CenterY
- Radius

then the circle boundary is defined by:

$$(X - CenterX)^2 + (Y - CenterY)^2 = Radius^2$$

The 4 Circle Resize Control Points are located on the perimeter where the horizontal and vertical center axes intersect the circle boundary.

Resize Control Point equations

Right Resize Control Point

$$RightResizeControlPoint = (CenterX + Radius, CenterY)$$

Top Resize Control Point

$$TopResizeControlPoint = (CenterX, CenterY + Radius)$$

Left Resize Control Point

$$LeftResizeControlPoint = (CenterX - Radius, CenterY)$$

Bottom Resize Control Point

$$BottomResizeControlPoint = (CenterX, CenterY - Radius)$$

So in your example, the points are:

- $(x + R, y)$
- $(x, y + R)$
- $(x - R, y)$
- $(x, y - R)$

Boundary verification

Each resize control point lies on the circle perimeter.

Right point

$$\begin{aligned} ((CenterX + Radius) - CenterX)^2 + (CenterY - CenterY)^2 &= Radius^2 \\ Radius^2 + 0 &= Radius^2 \end{aligned}$$

Top point

$$\begin{aligned} (CenterX - CenterX)^2 + ((CenterY + Radius) - CenterY)^2 &= Radius^2 \\ 0 + Radius^2 &= Radius^2 \end{aligned}$$

Left point

$$\begin{aligned} ((CenterX - Radius) - CenterX)^2 + (CenterY - CenterY)^2 &= Radius^2 \\ (-Radius)^2 + 0 &= Radius^2 \end{aligned}$$

Bottom point

$$(CenterX - CenterX)^2 + ((CenterY - Radius) - CenterY)^2 = Radius^2$$

$$0 + (-Radius)^2 = Radius^2$$

So all 4 points are mathematically on the perimeter.

Behavior

When the user left-clicks and drags a Circle Resize Control Point, the circle shall resize by changing its radius.

- RightResizeControlPoint changes the radius along the positive X direction
- TopResizeControlPoint changes the radius along the positive Y direction
- LeftResizeControlPoint changes the radius along the negative X direction
- BottomResizeControlPoint changes the radius along the negative Y direction

Since the shape is a circle, resizing from any of the 4 points changes the same variable:

$$Radius$$

and the circle must remain circular.

That means:

- there is no separate width
- there is no separate height
- the result must always keep one common radius

Radius equations during resize

If the dragged resize point moves to a new point (X_d, Y_d) , then the new radius shall be the distance from the circle center to that dragged point:

$$NewRadius = \sqrt{(X_d - CenterX)^2 + (Y_d - CenterY)^2}$$

If drag is constrained to the original axis of the selected point, use these axis-specific forms.

Dragging right point

$$NewRadius = X_d - CenterX$$

with constraint:

$$X_d \geq CenterX$$

Dragging top point

$$NewRadius = Y_d - CenterY$$

with constraint:

$$Y_d \geq CenterY$$

Dragging left point

$$NewRadius = CenterX - X_d$$

with constraint:

$$X_d \leq CenterX$$

Dragging bottom point

$$NewRadius = CenterY - Y_d$$

with constraint:

$$Y_d \leq CenterY$$

These axis-constrained equations are usually better for predictable editing behavior.

Derived circle boundaries

If needed, the circle boundaries are:

$$\begin{aligned} LeftX &= CenterX - Radius \\ RightX &= CenterX + Radius \\ TopY &= CenterY + Radius \\ BottomY &= CenterY - Radius \end{aligned}$$

So the 4 resize control points may also be written as:

$$\begin{aligned} RightResizeControlPoint &= (RightX, CenterY) \\ TopResizeControlPoint &= (CenterX, TopY) \\ LeftResizeControlPoint &= (LeftX, CenterY) \\ BottomResizeControlPoint &= (CenterX, BottomY) \end{aligned}$$

Constraint rules

- Every Circle Resize Control Point shall remain attached to its defined circle perimeter position.
- Circle Resize Control Point positions are derived from circle geometry and shall not be treated as independent primary geometry.
- If the circle radius changes, all 4 Circle Resize Control Point positions shall be recalculated.
- If the circle center moves, all 4 Circle Resize Control Point positions shall move with it.

- The circle must always preserve:

$$Radius > 0$$

- If a minimum circle size is required, then:

$$Radius \geq Radius_{min}$$

- If RotationAngle is later introduced for a circle with directional content, these 4 points may rotate visually with the circle content. For a plain circle, rotation has no visible geometric effect.

Recommendation

Treat each Circle Resize Control Point as a named logical point of the circle, but derive its coordinates from the circle's current center and radius instead of storing unrelated independent coordinates.

For implementation clarity, use these exact variable names:

- RightResizeControlPointX, RightResizeControlPointY
- TopResizeControlPointX, TopResizeControlPointY
- LeftResizeControlPointX, LeftResizeControlPointY
- BottomResizeControlPointX, BottomResizeControlPointY

But mathematically, each point should remain derived from:

- CenterX
- CenterY
- Radius

Rectangle Container

A **Rectangle Container** is a special type of **Rectangle** that works as a container for other diagram items inside its interior area.

A Rectangle Container inherits the same main rectangle concepts:

- center point
- horizontal half-size
- vertical half-size
- fill properties
- line properties

- layering order
- Hover Padding
- Protection Padding

A Rectangle Container may contain:

- a normal child rectangle
- a nested rectangle container
- a child circle

A child item must always remain fully inside its immediate parent container.

The child item, including its **Protection Padding** region, must:

- never go outside the parent container
- never touch the parent container border line
- never overlap the parent container border line

A nested rectangle container may itself contain other items. Therefore, containers may exist in multiple nested levels.

Each child item has exactly one immediate parent container. A parent container may itself be a child of another parent container.

If containment is enforced correctly between every child and its immediate parent, then the full multi-level containment hierarchy remains valid.

Parent Rectangle Container Definition

A **Parent Rectangle Container** is the immediate rectangle container that constrains a child item.

Canonical stored variables

Identity

- ParentContainerID

Geometry

- ParentCenterX
- ParentCenterY
- ParentHalfWidth
- ParentHalfHeight
- ParentRotationAngle

Layering

- ParentZOrder

Fill

- ParentFillType
- ParentFillColor

Line

- ParentLineType
- ParentLineColor
- ParentLineWidth

Hover Padding

- ParentHoverPaddingXRatio
- ParentHoverPaddingYRatio
- ParentHoverPaddingColor

Protection Padding

- ParentProtectionPaddingXRatio
- ParentProtectionPaddingYRatio
- ParentProtectionPaddingColor

Default values

Fill

- ParentFillType = NoFill

Line

- ParentLineType = SolidLine
- ParentLineColor = Black

Rotation

- ParentRotationAngle = 0°

Hover Padding

- ParentHoverPaddingXRatio = 0.1
- ParentHoverPaddingYRatio = 0.1
- ParentHoverPaddingColor = Transparent

Protection Padding

- ParentProtectionPaddingXRatio = 0.2

- $\text{ParentProtectionPaddingYRatio} = 0.2$
- $\text{ParentProtectionPaddingColor} = \text{Transparent}$

Derived parent boundaries

Visible boundaries

$$\begin{aligned}\text{ParentLeftX} &= \text{ParentCenterX} - \text{ParentHalfWidth} \\ \text{ParentRightX} &= \text{ParentCenterX} + \text{ParentHalfWidth} \\ \text{ParentTopY} &= \text{ParentCenterY} + \text{ParentHalfHeight} \\ \text{ParentBottomY} &= \text{ParentCenterY} - \text{ParentHalfHeight}\end{aligned}$$

Hover Padding values

$$\begin{aligned}\text{ParentHoverPaddingX} &= \text{ParentHoverPaddingXRatio} \cdot \text{ParentHalfWidth} \\ \text{ParentHoverPaddingY} &= \text{ParentHoverPaddingYRatio} \cdot \text{ParentHalfHeight}\end{aligned}$$

Hover Padding boundaries

$$\begin{aligned}\text{ParentHoverLeftX} &= \text{ParentLeftX} - \text{ParentHoverPaddingX} \\ \text{ParentHoverRightX} &= \text{ParentRightX} + \text{ParentHoverPaddingX} \\ \text{ParentHoverTopY} &= \text{ParentTopY} + \text{ParentHoverPaddingY} \\ \text{ParentHoverBottomY} &= \text{ParentBottomY} - \text{ParentHoverPaddingY}\end{aligned}$$

Protection Padding values

$$\begin{aligned}\text{ParentProtectionPaddingX} &= \text{ParentProtectionPaddingXRatio} \cdot \text{ParentHalfWidth} \\ \text{ParentProtectionPaddingY} &= \text{ParentProtectionPaddingYRatio} \cdot \text{ParentHalfHeight}\end{aligned}$$

Protection Padding boundaries

$$\begin{aligned}\text{ParentProtectionLeftX} &= \text{ParentLeftX} - \text{ParentProtectionPaddingX} \\ \text{ParentProtectionRightX} &= \text{ParentRightX} + \text{ParentProtectionPaddingX} \\ \text{ParentProtectionTopY} &= \text{ParentTopY} + \text{ParentProtectionPaddingY} \\ \text{ParentProtectionBottomY} &= \text{ParentBottomY} - \text{ParentProtectionPaddingY}\end{aligned}$$

These derived values shall not be treated as independent stored truths.

Recommended internal usable boundary

For long-term flexibility, the parent container's usable interior should be defined separately from its visible border.

So later you may define:

- ParentInnerLeftX
- ParentInnerRightX
- ParentInnerTopY
- ParentInnerBottomY

In version 1, these may simply equal the parent visible boundaries:

$$\begin{aligned} \textit{ParentInnerLeftX} &= \textit{ParentLeftX} \\ \textit{ParentInnerRightX} &= \textit{ParentRightX} \\ \textit{ParentInnerTopY} &= \textit{ParentTopY} \\ \textit{ParentInnerBottomY} &= \textit{ParentBottomY} \end{aligned}$$

All child-containment equations shall preferably use the **inner boundaries**.

Parent container hover rule

The Rectangle Container Hover Padding is used for mouse-hover detection and highlight behavior near the container perimeter.

When the mouse hovers over the container perimeter region defined by the Hover Padding:

$$\textit{ParentHoverPaddingColor} = \textit{ParentLineColor}$$

By default, it remains transparent.

Child Rectangle Definition

A **Child Rectangle** is a normal rectangle placed inside a parent rectangle container.

Canonical stored variables

- ChildRectangleID
- ChildCenterX
- ChildCenterY
- ChildHalfWidth
- ChildHalfHeight
- ChildProtectionPaddingX
- ChildProtectionPaddingY
- ChildZOrder
- ParentContainerID

Derived visible boundaries

$$\begin{aligned}ChildLeftX &= ChildCenterX - ChildHalfWidth \\ChildRightX &= ChildCenterX + ChildHalfWidth \\ChildTopY &= ChildCenterY + ChildHalfHeight \\ChildBottomY &= ChildCenterY - ChildHalfHeight\end{aligned}$$

Derived Protection Padding boundaries

$$\begin{aligned}ChildLeftPadX &= ChildLeftX - ChildProtectionPaddingX \\ChildRightPadX &= ChildRightX + ChildProtectionPaddingX \\ChildTopPadY &= ChildTopY + ChildProtectionPaddingY \\ChildBottomPadY &= ChildBottomY - ChildProtectionPaddingY\end{aligned}$$

Containment equations for Child Rectangle

For the child rectangle to remain fully inside the parent container without touching the border line:

$$\begin{aligned}ChildLeftPadX &> ParentInnerLeftX \\ChildRightPadX &< ParentInnerRightX \\ChildTopPadY &< ParentInnerTopY \\ChildBottomPadY &> ParentInnerBottomY\end{aligned}$$

These are the 4 fundamental containment equations.

Center-position range for Child Rectangle

$$\begin{aligned}ParentInnerLeftX + ChildHalfWidth + ChildProtectionPaddingX &< ChildCenterX \\&< ParentInnerRightX - ChildHalfWidth - ChildProtectionPaddingX \\ParentInnerBottomY + ChildHalfHeight + ChildProtectionPaddingY &< ChildCenterY \\&< ParentInnerTopY - ChildHalfHeight - ChildProtectionPaddingY\end{aligned}$$

Directional movement equations for Child Rectangle

Move right

$$ChildRightPadX + \Delta X < ParentInnerRightX$$

So:

$$\Delta X < ParentInnerRightX - ChildRightPadX$$

Move left

$$ChildLeftPadX - \Delta X > ParentInnerLeftX$$

So:

$$\Delta X < ChildLeftPadX - ParentInnerLeftX$$

Move upward

$$ChildTopPadY + \Delta Y < ParentInnerTopY$$

So:

$$\Delta Y < ParentInnerTopY - ChildTopPadY$$

Move downward

$$ChildBottomPadY - \Delta Y > ParentInnerBottomY$$

So:

$$\Delta Y < ChildBottomPadY - ParentInnerBottomY$$

Nested Rectangle Container Definition

A Nested Rectangle Container is a rectangle container that exists inside another parent rectangle container.

It has two roles at the same time:

- it is a child item relative to its immediate parent
- it is a parent container relative to its own children

Canonical stored variables

- NestedContainerID
- NestedContainerCenterX
- NestedContainerCenterY
- NestedContainerHalfWidth
- NestedContainerHalfHeight
- NestedContainerProtectionPaddingX
- NestedContainerProtectionPaddingY
- NestedContainerZOrder
- ParentContainerID

Derived visible boundaries

$$\begin{aligned}
ChildContainerLeftX &= ChildContainerCenterX - ChildContainerHalfWidth \\
ChildContainerRightX &= ChildContainerCenterX + ChildContainerHalfWidth \\
ChildContainerTopY &= ChildContainerCenterY + ChildContainerHalfHeight \\
ChildContainerBottomY &= ChildContainerCenterY - ChildContainerHalfHeight
\end{aligned}$$

Derived Protection Padding boundaries

$$\begin{aligned}
ChildContainerLeftPadX &= ChildContainerLeftX - ChildContainerProtectionPaddingX \\
ChildContainerRightPadX &= ChildContainerRightX + ChildContainerProtectionPaddingX \\
ChildContainerTopPadY &= ChildContainerTopY + ChildContainerProtectionPaddingY \\
ChildContainerBottomPadY &= ChildContainerBottomY - ChildContainerProtectionPaddingY
\end{aligned}$$

Containment equations for Nested Rectangle Container

Since the nested container is still a child item, it must satisfy the same containment equations:

$$\begin{aligned}
ChildContainerLeftPadX &> ParentInnerLeftX \\
ChildContainerRightPadX &< ParentInnerRightX \\
ChildContainerTopPadY &< ParentInnerTopY \\
ChildContainerBottomPadY &> ParentInnerBottomY
\end{aligned}$$

Center-position range for Nested Rectangle Container

$$\begin{aligned}
&ParentInnerLeftX + ChildContainerHalfWidth + ChildContainerProtectionPaddingX \\
&< ChildContainerCenterX \\
&< ParentInnerRightX - ChildContainerHalfWidth \\
&- ChildContainerProtectionPaddingX \\
&ParentInnerBottomY + ChildContainerHalfHeight + ChildContainerProtectionPaddingY \\
&< ChildContainerCenterY \\
&< ParentInnerTopY - ChildContainerHalfHeight \\
&- ChildContainerProtectionPaddingY
\end{aligned}$$

Directional movement equations for Nested Rectangle Container

Move right

$$\begin{aligned}
ChildContainerRightPadX + \Delta X &< ParentInnerRightX \\
\Delta X &< ParentInnerRightX - ChildContainerRightPadX
\end{aligned}$$

Move left

$$\begin{aligned}
ChildContainerLeftPadX - \Delta X &> ParentInnerLeftX \\
\Delta X &< ChildContainerLeftPadX - ParentInnerLeftX
\end{aligned}$$

Move upward

$$ChildContainerTopPadY + \Delta Y < ParentInnerTopY$$

$$\Delta Y < ParentInnerTopY - ChildContainerTopPadY$$

Move downward

$$ChildContainerBottomPadY - \Delta Y > ParentInnerBottomY$$

$$\Delta Y < ChildContainerBottomPadY - ParentInnerBottomY$$

Important rule for nested containers

A nested rectangle container becomes the immediate parent of its own children.

So for any deeper nesting level, the same containment rules are reused with:

- the nested container acting as parent
- its own child acting as child

This means no new mathematics is required for multiple levels.

Child Circle Definition

A **Child Circle** is a circle placed inside a parent rectangle container.

Canonical stored variables

- ChildCircleID
- ChildCircleCenterX
- ChildCircleCenterY
- ChildCircleRadius
- ChildCircleProtectionPadding
- ChildCircleZOrder
- ParentContainerID

Derived visible boundaries

$$ChildCircleLeftX = ChildCircleCenterX - ChildCircleRadius$$

$$ChildCircleRightX = ChildCircleCenterX + ChildCircleRadius$$

$$ChildCircleTopY = ChildCircleCenterY + ChildCircleRadius$$

$$ChildCircleBottomY = ChildCircleCenterY - ChildCircleRadius$$

Derived Protection Padding boundaries

$$ChildCircleLeftPadX = ChildCircleCenterX - ChildCircleRadius - ChildCircleProtectionPadding$$

$$\begin{aligned}
ChildCircleRightPadX &= ChildCircleCenterX + ChildCircleRadius + ChildCircleProtectionPadding \\
ChildCircleTopPadY &= ChildCircleCenterY + ChildCircleRadius + ChildCircleProtectionPadding \\
ChildCircleBottomPadY &= ChildCircleCenterY - ChildCircleRadius - ChildCircleProtectionPadding
\end{aligned}$$

Containment equations for Child Circle

For the child circle to remain fully inside the parent rectangle container without touching the border line:

$$\begin{aligned}
ChildCircleLeftPadX &> ParentInnerLeftX \\
ChildCircleRightPadX &< ParentInnerRightX \\
ChildCircleTopPadY &< ParentInnerTopY \\
ChildCircleBottomPadY &> ParentInnerBottomY
\end{aligned}$$

Center-position range for Child Circle

$$\begin{aligned}
ParentInnerLeftX + ChildCircleRadius + ChildCircleProtectionPadding &< ChildCircleCenterX \\
&< ParentInnerRightX - ChildCircleRadius - ChildCircleProtectionPadding \\
ParentInnerBottomY + ChildCircleRadius + ChildCircleProtectionPadding &< ChildCircleCenterY \\
&< ParentInnerTopY - ChildCircleRadius - ChildCircleProtectionPadding
\end{aligned}$$

Directional movement equations for Child Circle

Move right

$$\begin{aligned}
ChildCircleRightPadX + \Delta X &< ParentInnerRightX \\
\Delta X &< ParentInnerRightX - ChildCircleRightPadX
\end{aligned}$$

Move left

$$\begin{aligned}
ChildCircleLeftPadX - \Delta X &> ParentInnerLeftX \\
\Delta X &< ChildCircleLeftPadX - ParentInnerLeftX
\end{aligned}$$

Move upward

$$\begin{aligned}
ChildCircleTopPadY + \Delta Y &< ParentInnerTopY \\
\Delta Y &< ParentInnerTopY - ChildCircleTopPadY
\end{aligned}$$

Move downward

$$\begin{aligned}
ChildCircleBottomPadY - \Delta Y &> ParentInnerBottomY \\
\Delta Y &< ChildCircleBottomPadY - ParentInnerBottomY
\end{aligned}$$

General containment rule

Any child item inside a rectangle container must satisfy this general rule:

Left side rule

$$ChildLeftBoundaryWithProtectionPadding > ParentInnerLeftX$$

Right side rule

$$ChildRightBoundaryWithProtectionPadding < ParentInnerRightX$$

Top side rule

$$ChildTopBoundaryWithProtectionPadding < ParentInnerTopY$$

Bottom side rule

$$ChildBottomBoundaryWithProtectionPadding > ParentInnerBottomY$$

This rule applies to:

- child rectangle
- nested rectangle container
- child circle

Only the child boundary formulas change according to item type.

General clamped movement equations

If a child is dragged to a target center position, the safest method is to clamp the target center so the child remains valid.

Child Rectangle clamped X

$$\begin{aligned} ClampedChildCenterX \\ &= \min (ParentInnerRightX - ChildHalfWidth \\ &\quad - ChildProtectionPaddingX, \max (ParentInnerLeftX + ChildHalfWidth \\ &\quad + ChildProtectionPaddingX, TargetChildCenterX)) \end{aligned}$$

Child Rectangle clamped Y

$$\begin{aligned} ClampedChildCenterY \\ &= \min (ParentInnerTopY - ChildHalfHeight \\ &\quad - ChildProtectionPaddingY, \max (ParentInnerBottomY + ChildHalfHeight \\ &\quad + ChildProtectionPaddingY, TargetChildCenterY)) \end{aligned}$$

Child Circle clamped X

ClampedChildCircleCenterX

$$= \min (ParentInnerRightX - ChildCircleRadius \\ - ChildCircleProtectionPadding, \max (ParentInnerLeftX + ChildCircleRadius \\ + ChildCircleProtectionPadding, TargetChildCircleCenterX))$$

Child Circle clamped Y

ClampedChildCircleCenterY

$$= \min (ParentInnerTopY - ChildCircleRadius \\ - ChildCircleProtectionPadding, \max (ParentInnerBottomY + ChildCircleRadius \\ + ChildCircleProtectionPadding, TargetChildCircleCenterY))$$

The same form applies to nested rectangle containers by replacing child rectangle values with nested container values.

Strict no-touch rule

Because the child must not:

- get out of the parent
- overlap the parent border line
- even touch the border line

the inequalities must remain strict:

- $>$
- $<$

If developers need a safe implementation margin, use a very small positive value:

$$\varepsilon > 0$$

Then the rules become, for example:

$$\begin{aligned} ChildLeftPadX &\geq ParentInnerLeftX + \varepsilon \\ ChildRightPadX &\leq ParentInnerRightX - \varepsilon \\ ChildTopPadY &\leq ParentInnerTopY - \varepsilon \\ ChildBottomPadY &\geq ParentInnerBottomY + \varepsilon \end{aligned}$$

This is often safer in real software because of numeric precision.

Multiple nesting levels

Multiple levels of containers do not require new equations.

The same single-child / single-parent rules are enough if they are applied recursively.

Example:

- Parent Container A contains Nested Container B
- Nested Container B contains Child Rectangle C

Then:

- B must satisfy containment rules relative to A
- C must satisfy containment rules relative to B

If both are valid, then C is indirectly valid inside A as well.

So the containment system scales naturally to any nesting depth.

Recommendations

Recommendation 1

Always compare child **Protection Padding** boundaries against parent inner boundaries.

Recommendation 2

Do not use center-only equations for containment. Use boundary equations.

Recommendation 3

Treat a nested rectangle container exactly like a child rectangle for containment, then additionally let it act as a parent for its own children.

Recommendation 4

For circles inside containers, convert the circle to Protection-Padded left/right/top/bottom boundaries before applying the same rectangle-container checks.

Recommendation 5

Use clamped target positions during dragging instead of allowing an invalid move and correcting it afterward.

Recommendation 6

Treat the Rectangle Container Hover Padding exactly like the normal Rectangle Hover Padding:

- 10% of half-width and half-height
- transparent by default
- takes the same color as the container line during hover

Recommendation 7

Use the Rectangle Container Protection Padding for:

- overlap avoidance
- spacing rules
- child-inside-container calculations
- containment validation

Clarified Circle On Container type separation

The project shall treat Container and Circle On Container as two different geometry/device concepts. A Container remains a rectangle-container geometry. A Circle On Container is a separate circle-shaped geometry/device that is different from a normal Circle, but it inherits all normal Circle features except where the container-edge center restriction and drop rules override normal behavior.

- Normal Container: rectangle-based container, such as Region, VPC, Subnet, Availability Zone, or another business-defined container.
- Normal Circle: normal circular geometry that may be free on DiagramCanvas or handled by ordinary circle/container rules.
- Circle On Container: separate edge device whose center must lie on the edge of an allowed rectangle container.
- Circle On Container shall have its own item type, validation logic, save/load variables, and renderer behavior.

Circle On Container drop rule

When the user drags a Circle On Container geometry from the Shapes Panel, it can be dropped only on a valid container edge. Drop eligibility begins when any part of the dragged circle enters or overlaps the Hover Padding area of a candidate container. The system shall not wait until the circle center reaches the edge. After a candidate container is detected, the drop is accepted only if the CSV relationship rule allows that device type on that container type. The accepted center shall then be projected or snapped onto the nearest valid container edge.

CSV relationship definition for allowed edge devices

Allowed Circle On Container placement shall be controlled by a CSV file. The CSV file shall have exactly two columns: device on container edge and container.

device on container edge	container
Internet Gateway	VPC

The example means Internet Gateway can be placed on the edge of a VPC container. Internet Gateway cannot be placed on the edge of a Region container unless the CSV explicitly includes Internet Gateway,Region.

Recommended CSV file name: device-on-container-edge-rules.csv.

CSV validation rule: a Circle On Container drop is allowed only if a CSV row exists where device on container edge equals the dragged Circle On Container device type and container equals the target container type.

Layering and container-border occlusion rule

Circle On Container shall always be rendered above its parent rectangle container. This ensures the user sees the circle clearly and does not see the rectangle container line passing through the circle.

Required ZOrder rule: $\text{CircleOnContainerZOrder} > \text{ParentContainerZOrder}$.

Recommended default: $\text{CircleOnContainerZOrder} = \text{ParentContainerZOrder} + 1$.

Higher ZOrder alone is not sufficient if the circle fill is transparent. Therefore, the renderer shall enforce either an opaque fill for Circle On Container or a border-masking rule that suppresses the parent container border segment covered by the circle. The default project rule is that the parent container border line shall not be visible through Circle On Container.

Final summary

A **Rectangle Container** is a special type of Rectangle that constrains:

- Child Rectangle
- Nested Rectangle Container
- Child Circle

It supports the same dual-padding model already used for Rectangle:

- **Hover Padding** = 10%, transparent by default, used for hover highlight
- **Protection Padding** = 20%, transparent by default, used for overlap and containment

Fundamental 4-direction containment equations

$$\begin{aligned} \text{ChildLeftBoundaryWithProtectionPadding} &> \text{ParentInnerLeftX} \\ \text{ChildRightBoundaryWithProtectionPadding} &< \text{ParentInnerRightX} \\ \text{ChildTopBoundaryWithProtectionPadding} &< \text{ParentInnerTopY} \\ \text{ChildBottomBoundaryWithProtectionPadding} &> \text{ParentInnerBottomY} \end{aligned}$$

Movement limits

- right movement limited by parent right boundary
- left movement limited by parent left boundary
- upward movement limited by parent top boundary
- downward movement limited by parent bottom boundary

Multiple nesting

Handled by applying the same immediate-parent rules recursively

Circle On Container Definition

A Circle On Container is a separate geometric item whose visible shape is still a circle, but whose center point is constrained to lie on the boundary edge of an allowed rectangle container. It is not treated as a normal child circle fully contained inside the rectangle container. It inherits the normal Circle features for appearance, radius, hover padding, Protection Padding, resize controls, selection, ZOrder, and save/load behavior unless this section explicitly overrides them. Its special restriction is positional: its center is constrained to the container boundary and may move across any valid edge of the same allowed container according to nearest-edge projection, EdgeParameterT, and deterministic corner-transition rules.

Meaning and purpose

This geometry represents a circle that belongs logically to a rectangle container boundary. It may be used later for ports, boundary markers, external/internal service points, connection anchors, interface points, or any engineering object that must travel along the edge of a container.

The key difference from a normal child circle is that the center of this circle is on the container boundary. Therefore, the existing child-circle containment rule shall not be reused without modification, because that rule requires the whole circle and its Protection Padding to remain strictly inside the container.

Visual definition

The visible geometry is still a circle. It uses the same circle appearance concepts already defined for Circle Definition: fill, line, line color, line width, hover padding, protection padding, and ZOrder. The special behavior is not the circle shape itself; the special behavior is the center-position constraint relative to the rectangle container edge.

Default visual behavior should follow the normal circle defaults unless a project-specific symbol style is later introduced.

- FillType = NoFill or SolidFill.
- LineType = SolidLine unless intentionally hidden.
- Center point must remain on one valid edge of the allowed rectangle container at all times.
- The circle may extend partly inside and partly outside the parent container depending on the selected placement policy.

Geometry definition

Recommended canonical stored variables

Variable name	Description	Recommended value
CircleOnContainerID	Unique identity of the Circle On Container.	GUID/UUID string

ParentContainerID	Rectangle container that owns the Circle On Container.	Valid ContainerID
HostEdge	Current active container edge on which the circle center lies. This value may change during movement because the circle can move on any edge of the same allowed container.	Top, Right, Bottom, or Left
EdgeParameterT	Normalized movement parameter along the current active edge.	0.0 to 1.0
CenterX	Derived or persisted center X in world space.	Calculated from parent edge and EdgeParameterT
CenterY	Derived or persisted center Y in world space.	Calculated from parent edge and EdgeParameterT
Radius	Circle radius.	Positive numeric value
ZOrder	Stacking order of the Circle On Container.	Integer
FillType	Circle fill behavior.	NoFill or SolidFill
FillColor	Fill color when fill is used.	Hex color or Transparent
LineType	Circle border behavior.	NoLine or SolidLine
LineColor	Circle line color.	Hex color
LineWidth	Circle line width.	Positive numeric value
HoverPaddingRadiusRatio	Radial hover-padding ratio.	0.1
ProtectionPaddingRadiusRatio	Radial protection-padding ratio.	0.2
MovementEnabled	Whether the circle can move along the edge.	True
CornerTransitionEnabled	Whether movement may change HostEdge at a corner. Required True for this project. This project requires it to be enabled so the circle can move on any edge.	True by default

EdgePlacementPolicy	Defines whether the circle is centered on the border, inside-biased, outside-biased, or custom-offset.	CenterOnBoundary
RadiusResizeEnabled	Whether the user may resize the Circle On Container radius.	True
CircleOnContainerMinimumRadiusRatio	Global lower radius ratio applied to the current active edge length.	0.10
CircleOnContainerMaximumRadiusRatio	Global upper radius ratio applied to the current active edge length.	0.20
CurrentActiveEdgeLength	Derived length of the current active edge. Top/Bottom use container width; Right/Left use container height.	Derived, not stored as independent truth
CalculatedMinimumRadius	Derived minimum allowed radius for the current active edge.	CurrentActiveEdgeLength * 0.10
CalculatedMaximumRadius	Derived maximum allowed radius for the current active edge.	CurrentActiveEdgeLength * 0.20

Global Variables — Circle On Container radius constraints

These values belong to the global-variable configuration of the diagram tool. They shall not be hard-coded inside a single Circle On Container item. The Circle On Container item stores its current Radius, while the allowed minimum and maximum radius are calculated from the current active edge length using the global ratios.

Global variable name	Description	Recommended value
CircleOnContainerMinimumRadiusRatio	Minimum allowed Circle On Container radius as a ratio of the current active edge length.	0.10
CircleOnContainerMaximumRadiusRatio	Maximum allowed Circle On Container radius as a ratio	0.20

	of the current active edge length.	
CircleOnContainerMinimumAbsoluteRadius	Optional absolute lower radius limit used as an additional safety clamp if the project requires one.	Business-defined or 0 when unused
CircleOnContainerMaximumAbsoluteRadius	Optional absolute upper radius limit used as an additional safety clamp if the project requires one.	Business-defined or NULL when unused
CircleOnContainerResizeEnabled	Enables user resizing of Circle On Container items.	True
CircleOnContainerContainerResizeInvalidRadiusPolicy	Defines what happens when a container resize would make an existing Circle On Container radius invalid without changing the circle size.	RejectContainerResize by default

Recommended derivation rule

The authoritative attachment identity should be ParentContainerID + HostEdge + EdgeParameterT. CenterX and CenterY may be saved for faster reconstruction, but they shall be recalculated from the parent container geometry before rendering, validation, and persistence acceptance.

Placement / attachment definition

Let the parent rectangle container have visible or selected attachment boundaries:

$\text{ParentLeftX} = \text{ParentCenterX} - \text{ParentHalfWidth}$

$\text{ParentRightX} = \text{ParentCenterX} + \text{ParentHalfWidth}$

$\text{ParentTopY} = \text{ParentCenterY} + \text{ParentHalfHeight}$

$\text{ParentBottomY} = \text{ParentCenterY} - \text{ParentHalfHeight}$

The Circle On Container center must satisfy one valid container-edge equation at a time. During movement, the active edge may change at corners or by nearest-edge projection, so the circle can move on any edge of the same allowed container.

- Top edge: $\text{CenterY} = \text{ParentTopY}$ and $\text{ParentLeftX} \leq \text{CenterX} \leq \text{ParentRightX}$.
- Right edge: $\text{CenterX} = \text{ParentRightX}$ and $\text{ParentBottomY} \leq \text{CenterY} \leq \text{ParentTopY}$.
- Bottom edge: $\text{CenterY} = \text{ParentBottomY}$ and $\text{ParentLeftX} \leq \text{CenterX} \leq \text{ParentRightX}$.
- Left edge: $\text{CenterX} = \text{ParentLeftX}$ and $\text{ParentBottomY} \leq \text{CenterY} \leq \text{ParentTopY}$.

For version 1, use the parent visible boundary as the attachment boundary. If a later requirement states that the circle center must move on the parent inner boundary or Protection Padding boundary, then the same equations apply after replacing the parent visible boundaries with the selected boundary set.

Movement along rectangle container edge

Movement shall be controlled by the current active HostEdge and EdgeParameterT, where $t = 0$ means the start of the current active edge and $t = 1$ means the end of the current active edge. The recommended orientation is clockwise around the rectangle container: Top edge left to right, Right edge top to bottom, Bottom edge right to left, and Left edge bottom to top. If the project prefers a different direction, it must be defined once and used consistently for all edges.

- Top edge: $\text{CenterX} = \text{ParentLeftX} + t * (\text{ParentRightX} - \text{ParentLeftX})$, $\text{CenterY} = \text{ParentTopY}$.
- Right edge: $\text{CenterX} = \text{ParentRightX}$, $\text{CenterY} = \text{ParentTopY} - t * (\text{ParentTopY} - \text{ParentBottomY})$.
- Bottom edge: $\text{CenterX} = \text{ParentRightX} - t * (\text{ParentRightX} - \text{ParentLeftX})$, $\text{CenterY} = \text{ParentBottomY}$.
- Left edge: $\text{CenterX} = \text{ParentLeftX}$, $\text{CenterY} = \text{ParentBottomY} + t * (\text{ParentTopY} - \text{ParentBottomY})$.

During mouse drag, the pointer shall be projected onto the nearest valid edge of the allowed parent container, converted into a candidate HostEdge and EdgeParameterT, clamped to the valid range for that edge, and then converted back into CenterX and CenterY.

Mandatory any-edge movement rule: the Circle On Container shall not be locked to the first edge on which it was dropped. After placement, the user may move it along the full boundary of the same allowed container, across top, right, bottom, and left edges, using deterministic corner-transition behavior.

Resize behavior and radius constraints

The Circle On Container shall support resize behavior using the same radius-based concept as a normal Circle. Resizing changes only the Circle On Container Radius. It shall not resize the attached container, and it shall not move or resize any parent container.

The radius shall be constrained by global variables. Let CurrentActiveEdgeLength be the length of the edge on which the center currently lies. If HostEdge is Top or Bottom, $\text{CurrentActiveEdgeLength} = \text{ParentRightX} - \text{ParentLeftX}$. If HostEdge is Right or Left, $\text{CurrentActiveEdgeLength} = \text{ParentTopY} - \text{ParentBottomY}$.

$\text{CalculatedMinimumRadius} = \text{CurrentActiveEdgeLength} * \text{CircleOnContainerMinimumRadiusRatio}$.
Recommended default: $\text{CircleOnContainerMinimumRadiusRatio} = 0.10$.

$\text{CalculatedMaximumRadius} = \text{CurrentActiveEdgeLength} * \text{CircleOnContainerMaximumRadiusRatio}$.
Recommended default: $\text{CircleOnContainerMaximumRadiusRatio} = 0.20$.

A resize candidate is valid only if $\text{CalculatedMinimumRadius} \leq \text{CandidateRadius} \leq \text{CalculatedMaximumRadius}$. During live resize, the UI may clamp CandidateRadius to the nearest valid boundary, but the committed saved Radius must always remain inside the allowed range.

The Circle On Container size is position-related to the container edge but not scale-related to the container size. If the user resizes the container, the Circle On Container Radius shall not automatically change. If the user resizes the Circle On Container, the parent container shall not automatically change.

If a container resize changes the active edge length so that an existing Circle On Container Radius becomes invalid, the default behavior shall be to reject the container resize and restore the previous valid container geometry. The system shall not silently resize the Circle On Container as a side effect of container resizing unless a later explicit policy overrides this rule.

Protection Padding and containment interpretation

The Circle On Container shall keep normal Circle Protection Padding behavior. $\text{ProtectionPaddingRadiusRatio}$, $\text{ProtectionPaddingRadius}$, $\text{ProtectionPaddedRadius}$, and the protection bounding box are calculated the same way as a normal Circle, and shall be used for overlap/touch validation.

This geometry requires a special containment interpretation. Because the center lies on the container boundary, the circle cannot be validated as a normal child circle that must remain fully inside the parent. Instead, the document shall distinguish between edge-attachment validity and overlap validity.

- Attachment validity checks whether CenterX and CenterY lie on one valid edge of the allowed parent container.
- Sibling overlap validity checks whether the circle Protection Padding conflicts with sibling geometries or sibling Circle On Containers according to the selected strict no-touch rule.
- Parent containment validity for normal child circles shall not be applied directly to this geometry unless the project later defines an inside-biased placement policy.
- If $\text{EdgePlacementPolicy} = \text{CenterOnBoundary}$, the circle is allowed to extend across the parent boundary line by design.

Constraint rules

- ParentContainerID must reference a valid rectangle container.
- HostEdge must be one of Top, Right, Bottom, or Left.
- $0.0 \leq \text{EdgeParameterT} \leq 1.0$.
- $\text{Radius} > 0$.
- CenterX and CenterY must match the current active HostEdge equation after recalculation.
- Corner transition is required for this project because the circle can move on any edge of the same allowed container.

- Crossing a corner must update HostEdge and EdgeParameterT deterministically so movement continues on the adjacent edge.
- The Circle On Container shall move with its parent container when the parent container moves.
- If the parent container is resized, CenterX and CenterY shall be recalculated from HostEdge and EdgeParameterT.

Recommendations

- Treat this as a separate geometry behavior, not as a normal child circle.
- Use ParentContainerID, current active HostEdge, and EdgeParameterT as the cleanest attachment model.
- Recalculate CenterX and CenterY from the parent container before rendering and saving.
- Do not apply normal child-circle containment unless an explicit inside-only placement policy is later required.
- Keep the movement calculation in JavaScript for live interaction, and revalidate the committed state in C# before database save.

Final summary

The Circle On Container is a circle geometry whose center is constrained to the boundary of a parent rectangle container. It can move on any edge of that same allowed container. At any instant, its position is represented by the current active HostEdge plus a normalized EdgeParameterT value. This geometry is related to containers, circles, and connection behavior, but it is not the same as a normal child circle inside a container. Its special rule is boundary-edge attachment: the circle center belongs to the parent container boundary and is recalculated whenever the parent container moves or resizes.

Mathematical Calculations

Purpose and scope

Recommended JavaScript files for this section: No dedicated JavaScript file. This section is architectural guidance only.

This section is the dedicated mathematical and geometry-calculation chapter for the diagram tool. It groups together the calculations that control coordinate conversion, hover detection, selection activation, move validation, resize validation, overlap checks, containment checks, parent-container expansion, render-rate estimation, and Protection Padding logic. The purpose of this chapter is to give developers one clear home for all calculation rules before those rules are routed into milestone-specific JavaScript files.

Coordinate conversion calculations

Recommended JavaScript files for this section: world-to-screen.js; screen-to-world.js; viewport-math.js.

All geometry calculations shall treat world space as the authoritative mathematical space of the diagram. Screen space and viewport values are temporary view or input values only. Rendering follows World Space -> Viewport -> Screen Space, while pointer-based interaction follows Screen Space -> Viewport -> World Space. Panning changes viewport state only. Zooming changes viewport scale only. Stored geometry values such as CenterX, CenterY, HalfWidth, HalfHeight, and Radius shall remain world-space values.

Recommended derived viewport boundaries:

$$\text{ViewportLeftX} = \text{ViewportCenterX} - (\text{ViewportWidth} / 2)$$
$$\text{ViewportRightX} = \text{ViewportCenterX} + (\text{ViewportWidth} / 2)$$
$$\text{ViewportTopY} = \text{ViewportCenterY} + (\text{ViewportHeight} / 2)$$
$$\text{ViewportBottomY} = \text{ViewportCenterY} - (\text{ViewportHeight} / 2)$$

World-to-screen conversion:

$$\text{ScreenX} = ((\text{WorldX} - \text{ViewportLeftX}) / \text{ViewportWidth}) * \text{ScreenWidthPx}$$
$$\text{ScreenY} = \text{ScreenHeightPx} - (((\text{WorldY} - \text{ViewportBottomY}) / \text{ViewportHeight}) * \text{ScreenHeightPx})$$

Screen-to-world conversion:

$$\text{WorldX} = \text{ViewportLeftX} + (\text{ScreenX} / \text{ScreenWidthPx}) * \text{ViewportWidth}$$
$$\text{WorldY} = \text{ViewportBottomY} + ((\text{ScreenHeightPx} - \text{ScreenY}) / \text{ScreenHeightPx}) * \text{ViewportHeight}$$

Zoom rule:

$$\text{ViewportWidth} = \text{BaseViewportWidth} / \text{ZoomScale}$$
$$\text{ViewportHeight} = \text{BaseViewportHeight} / \text{ZoomScale}$$

Pan rule: update only ViewportCenterX and ViewportCenterY. Do not change stored world coordinates during pan.

Hover, selection, and visual activation calculations

Recommended JavaScript files for this section: geometry-hover-activation.js; geometry-selection.js; resize-control-point-hit-test.js. Use geometry-hover-activation.js for hover-state activation and hover-band calculations, geometry-selection.js for Left Click Down selection activation, and resize-control-point-hit-test.js for deciding whether resize has priority over move.

A geometry becomes targetable for interaction when the pointer enters its Hover Padding region. At that moment, the Hover Padding changes from Transparent to the geometry line color, and the predefined transparent resize rectangles become White. This is a hover-state calculation only; the geometry is not yet selected.

Selection happens when the user performs Left Click Down on the Hover Padding region. From that point, the geometry becomes the active selected geometry. Two editing branches are then available. The first branch is move. The second branch is resize. If Left Click Down happens on a visible resize rectangle, the action is resize. Otherwise, if Left Click Down happens in the valid selected-geometry move region, the action is move. Keyboard-arrow movement also requires the geometry to already be selected.

Resize recalculation rule: whenever any geometry is resized, all derived interaction and validation geometry shall be recalculated immediately from the updated canonical geometry. This includes at minimum visible boundaries, Hover Padding boundaries, Protection Padding boundaries, resize control point positions, and any geometry-specific helper boundaries. This rule applies to Rectangle, Circle, and Rectangle Container.

Rectangle hover outer boundaries:

$$\text{RectHoverLeftX} = \text{CenterX} - \text{HalfWidth} - \text{HoverPaddingX}$$
$$\text{RectHoverRightX} = \text{CenterX} + \text{HalfWidth} + \text{HoverPaddingX}$$
$$\text{RectHoverTopY} = \text{CenterY} + \text{HalfHeight} + \text{HoverPaddingY}$$
$$\text{RectHoverBottomY} = \text{CenterY} - \text{HalfHeight} - \text{HoverPaddingY}$$

Pointer is inside the rectangle hover outer region when $\text{PointerX} \geq \text{RectHoverLeftX}$ and $\text{PointerX} \leq \text{RectHoverRightX}$ and $\text{PointerY} \leq \text{RectHoverTopY}$ and $\text{PointerY} \geq \text{RectHoverBottomY}$.

Pointer is in the rectangle hover band when it is inside the hover outer region and not strictly inside the visible rectangle interior.

Circle hover band equations:

$$\text{DistanceSquared} = (\text{PointerX} - \text{CenterX})^2 + (\text{PointerY} - \text{CenterY})^2$$
$$\text{VisibleRadiusSquared} = \text{Radius}^2$$
$$\text{HoverOuterRadiusSquared} = \text{HoverPaddedRadius}^2$$

Pointer is in the circle hover band when $\text{DistanceSquared} \geq \text{VisibleRadiusSquared}$ and $\text{DistanceSquared} \leq \text{HoverOuterRadiusSquared}$.

Generic resize-rectangle hit test:

$$\text{ResizeRectLeftX} = \text{ResizePointX} - \text{ResizeRectHalfWidth}$$
$$\text{ResizeRectRightX} = \text{ResizePointX} + \text{ResizeRectHalfWidth}$$
$$\text{ResizeRectTopY} = \text{ResizePointY} + \text{ResizeRectHalfHeight}$$
$$\text{ResizeRectBottomY} = \text{ResizePointY} - \text{ResizeRectHalfHeight}$$

Pointer is on a resize rectangle when PointerX and PointerY both lie inside those resize-rectangle boundaries.

Interaction priority rule: if the pointer is on any resize rectangle, action = resize; otherwise, if the pointer is in Hover Padding and the geometry is already selected, action = move; otherwise Left Click Down on Hover Padding performs selection.

Move-update calculations and MaxMoveValidationRate

Recommended JavaScript files for this section: move-update-scheduler.js; detect-render-rate.js; compute-rectangle-candidate-center-on-move.js; compute-circle-candidate-center-on-move.js.

Relationship rule for this section: detect-render-rate.js estimates EstimatedRenderHz. move-update-scheduler.js receives EstimatedRenderHz together with MaxMoveValidationRate and calculates

EffectiveMoveValidationRate. The effective processed move-validation rate shall not exceed MaxMoveValidationRate.

JavaScript file	Responsibility	Output	Used by
detect-render-rate.js	Estimate active client render or frame rate using requestAnimationFrame timing.	EstimatedRenderHz	move-update-scheduler.js
move-update-scheduler.js	Calculate and apply the effective processed move-validation rate for move actions.	EffectiveMoveValidationRate	rectangle move, circle move, child move, container move, and container-subtree move validation files

Input to this section includes EstimatedRenderHz produced by detect-render-rate.js.

Move validation shall not run on every raw physical mouse change, and it shall not run only on large fixed delays such as one second or half a second. Instead, move validation shall run on a controlled processed move-update cycle. The controlling variable for this cycle is MaxMoveValidationRate, which defines the maximum number of processed move-validation updates per second during any move action.

The processed move-update count for one action may be estimated as: ProcessedMoveUpdatesApprox = DragDurationSeconds * MaxMoveValidationRate. This variable applies to rectangle move, circle move, child move inside a container, container move, and container-subtree move. It does not represent raw hardware movement count.

Let S = number of sibling geometries in the same immediate parent container, excluding the moved geometry.

$\text{SiblingOverlapChecksApprox} = \text{ProcessedMoveUpdatesApprox} * S$

$\text{ParentContainmentChecksApprox} = \text{ProcessedMoveUpdatesApprox} * 1$

$\text{TotalDirectValidationChecksApprox} = \text{ProcessedMoveUpdatesApprox} * (S + 1)$

If total children in that same immediate parent container = N including the moved geometry, then $S = N - 1$ and $\text{TotalDirectValidationChecksApprox} = \text{ProcessedMoveUpdatesApprox} * N$.

$\text{EffectiveMoveValidationRate} = \min(\text{EstimatedRenderHz}, \text{MaxMoveValidationRate})$

Parent-container expansion calculations

Recommended JavaScript files for this section: resize-parent-container.js.

When a child geometry is inserted or resized inside a parent rectangle container and there is not enough valid internal space, the parent container may be expanded around the child geometry. A valid parent-expansion strategy is to compare the current usable parent boundaries against the child Protection Padding boundaries and grow the parent enough to keep the child fully valid without touching the parent border. After the parent grows, the parent center and half-sizes are recalculated from the updated

boundaries. If the parent is itself a child container, the same containment logic may be validated recursively upward.

Recommended parent-expansion equations:

$$\text{NewParentInnerLeftX} = \min(\text{CurrentParentInnerLeftX}, \text{ChildProtectionLeftX} - \text{Epsilon})$$
$$\text{NewParentInnerRightX} = \max(\text{CurrentParentInnerRightX}, \text{ChildProtectionRightX} + \text{Epsilon})$$
$$\text{NewParentInnerTopY} = \max(\text{CurrentParentInnerTopY}, \text{ChildProtectionTopY} + \text{Epsilon})$$
$$\text{NewParentInnerBottomY} = \min(\text{CurrentParentInnerBottomY}, \text{ChildProtectionBottomY} - \text{Epsilon})$$

Derived parent geometry after expansion:

$$\text{NewParentCenterX} = (\text{NewParentInnerLeftX} + \text{NewParentInnerRightX}) / 2$$
$$\text{NewParentCenterY} = (\text{NewParentInnerBottomY} + \text{NewParentInnerTopY}) / 2$$
$$\text{NewParentHalfWidth} = (\text{NewParentInnerRightX} - \text{NewParentInnerLeftX}) / 2$$
$$\text{NewParentHalfHeight} = (\text{NewParentInnerTopY} - \text{NewParentInnerBottomY}) / 2$$

Render-rate detection calculation

Recommended JavaScript files for this section: detect-render-rate.js.

Output of this section is EstimatedRenderHz, which shall be consumed by move-update-scheduler.js when calculating EffectiveMoveValidationRate.

JavaScript file	Responsibility	Output	Consumed by
detect-render-rate.js	Estimate active render interval and derive EstimatedRenderHz.	EstimatedRenderHz	move-update-scheduler.js

The client may estimate the active render interval by measuring requestAnimationFrame timing. A practical estimate is: $\text{EstimatedRenderHz} = 1000 / \text{AverageFrameMilliseconds}$. This estimate may be used as a performance helper when choosing a safe processed update rate, but it shall not replace MaxMoveValidationRate as the project-defined cap for move validation.

A helper file such as detect-render-rate.js may run at application start, estimate the active render interval, and optionally save the estimate locally for the current user and browser. This locally saved value is helper data only. The application shall continue working correctly if browser storage is blocked, cleared, unavailable, or denied.

Recommended render-rate equations:

$$\text{AverageFrameMilliseconds} = \text{SumFrameDeltaMilliseconds} / \text{FrameSampleCount}$$
$$\text{EstimatedRenderHz} = 1000 / \text{AverageFrameMilliseconds}$$
$$\text{EffectiveMoveValidationRate} = \min(\text{EstimatedRenderHz}, \text{MaxMoveValidationRate})$$

The helper file detect-render-rate.js should run once at application start, use requestAnimationFrame timing to estimate the active render interval, and keep the estimate local to the current user and browser only.

Circle On Container edge-placement calculations

Recommended JavaScript files for this section: circle-on-container-drop-detection.js; validate-circle-on-container-csv-rule.js; project-circle-on-container-to-edge.js; calculate-circle-on-container-center.js; render-circle-on-container-occlusion.js.

This subsection defines the mathematical calculations for the separate Circle On Container geometry. It reuses the normal Circle equations for radius, hover padding, Protection Padding, resize, and appearance, but adds center-on-container-edge constraints, CSV relationship validation, hover-area drop detection, nearest-edge projection, ZOrder enforcement, and border occlusion.

$\text{ParentLeftX} = \text{ParentCenterX} - \text{ParentHalfWidth}$

$\text{ParentRightX} = \text{ParentCenterX} + \text{ParentHalfWidth}$

$\text{ParentTopY} = \text{ParentCenterY} + \text{ParentHalfHeight}$

$\text{ParentBottomY} = \text{ParentCenterY} - \text{ParentHalfHeight}$

Top edge center: $\text{CenterY} = \text{ParentTopY}$ and $\text{ParentLeftX} \leq \text{CenterX} \leq \text{ParentRightX}$

Right edge center: $\text{CenterX} = \text{ParentRightX}$ and $\text{ParentBottomY} \leq \text{CenterY} \leq \text{ParentTopY}$

Bottom edge center: $\text{CenterY} = \text{ParentBottomY}$ and $\text{ParentLeftX} \leq \text{CenterX} \leq \text{ParentRightX}$

Left edge center: $\text{CenterX} = \text{ParentLeftX}$ and $\text{ParentBottomY} \leq \text{CenterY} \leq \text{ParentTopY}$

EdgeParameterT equations

Top edge: $t = (\text{CenterX} - \text{ParentLeftX}) / (\text{ParentRightX} - \text{ParentLeftX})$

Right edge: $t = (\text{ParentTopY} - \text{CenterY}) / (\text{ParentTopY} - \text{ParentBottomY})$

Bottom edge: $t = (\text{ParentRightX} - \text{CenterX}) / (\text{ParentRightX} - \text{ParentLeftX})$

Left edge: $t = (\text{CenterY} - \text{ParentBottomY}) / (\text{ParentTopY} - \text{ParentBottomY})$

Clamp rule for the current active edge: $\text{EdgeParameterT} = \text{clamp}(t, 0.0, 1.0)$. If movement reaches an edge endpoint, HostEdge shall update to the adjacent edge according to the defined clockwise or counterclockwise corner-transition rule, so the circle can continue moving on any edge of the same allowed container.

Drop eligibility from container Hover Padding

$\text{DraggedCircleLeftX} = \text{DragCenterX} - \text{Radius}$

$\text{DraggedCircleRightX} = \text{DragCenterX} + \text{Radius}$

$\text{DraggedCircleTopY} = \text{DragCenterY} + \text{Radius}$

$\text{DraggedCircleBottomY} = \text{DragCenterY} - \text{Radius}$

$\text{ContainerHoverLeftX} = \text{ParentCenterX} - \text{ParentHalfWidth} - \text{ParentHoverPaddingX}$

$\text{ContainerHoverRightX} = \text{ParentCenterX} + \text{ParentHalfWidth} + \text{ParentHoverPaddingX}$

$\text{ContainerHoverTopY} = \text{ParentCenterY} + \text{ParentHalfHeight} + \text{ParentHoverPaddingY}$

$\text{ContainerHoverBottomY} = \text{ParentCenterY} - \text{ParentHalfHeight} - \text{ParentHoverPaddingY}$

Hover-area overlap exists when: $\text{DraggedCircleRightX} \geq \text{ContainerHoverLeftX}$ AND $\text{DraggedCircleLeftX} \leq \text{ContainerHoverRightX}$ AND $\text{DraggedCircleTopY} \geq \text{ContainerHoverBottomY}$ AND $\text{DraggedCircleBottomY} \leq \text{ContainerHoverTopY}$.

CSV relationship validation

Let $D = \text{DeviceOnContainerEdgeType}$ and $C = \text{ParentContainerType}$. The drop is allowed only if the CSV contains a row where $\text{CsvRow}["\text{device on container edge}"] = D$ AND $\text{CsvRow}["\text{container}"] = C$.

Nearest-edge projection

$\text{DistanceToTop} = \text{abs}(\text{DragCenterY} - \text{ParentTopY})$. $\text{DistanceToRight} = \text{abs}(\text{DragCenterX} - \text{ParentRightX})$. $\text{DistanceToBottom} = \text{abs}(\text{DragCenterY} - \text{ParentBottomY})$. $\text{DistanceToLeft} = \text{abs}(\text{DragCenterX} - \text{ParentLeftX})$. HostEdge equals the nearest valid edge, using deterministic tie-breaking at corners. HostEdge is therefore the current active edge, not a permanent restriction to a single edge.

Any-edge projection rule: during drag, calculate candidate projections for all four parent-container edges, select the nearest valid projection, then update HostEdge and EdgeParameterT together. The user must be able to move the Circle On Container across top, right, bottom, and left edges of the same allowed container.

ZOrder and border occlusion

Required layering equation: $\text{CircleOnContainerZOrder} > \text{ParentContainerZOrder}$. Recommended default: $\text{CircleOnContainerZOrder} = \text{ParentContainerZOrder} + 1$. If BorderOcclusionPolicy = OpaqueFill, render the opaque Circle On Container fill above the parent container line. If BorderOcclusionPolicy = MaskContainerBorderSegment, suppress or mask the parent container border segment covered by the circle radius.

Circle On Container radius-resize calculations

CurrentActiveEdgeLength calculation: if HostEdge is Top or Bottom, $\text{CurrentActiveEdgeLength} = \text{ParentRightX} - \text{ParentLeftX}$. If HostEdge is Right or Left, $\text{CurrentActiveEdgeLength} = \text{ParentTopY} - \text{ParentBottomY}$.

$\text{MinimumRadius} = \text{CurrentActiveEdgeLength} * \text{CircleOnContainerMinimumRadiusRatio}$. With the recommended default, $\text{MinimumRadius} = \text{CurrentActiveEdgeLength} * 0.10$.

$\text{MaximumRadius} = \text{CurrentActiveEdgeLength} * \text{CircleOnContainerMaximumRadiusRatio}$. With the recommended default, $\text{MaximumRadius} = \text{CurrentActiveEdgeLength} * 0.20$.

CandidateRadius is valid only when $\text{MinimumRadius} \leq \text{CandidateRadius} \leq \text{MaximumRadius}$. The live resize handler may compute $\text{AppliedRadius} = \text{clamp}(\text{CandidateRadius}, \text{MinimumRadius}, \text{MaximumRadius})$, but the committed saved Radius must never be outside the accepted range.

$\text{ProtectionPaddingRadius} = \text{Radius} * \text{ProtectionPaddingRadiusRatio}$. $\text{ProtectionPaddedRadius} = \text{Radius} + \text{ProtectionPaddingRadius}$. These equations remain the same as the normal Circle equations.

Container-resize independence calculation

Container resize shall recalculate the Circle On Container center from the new parent boundaries, HostEdge, and EdgeParameterT, but shall not automatically change Radius.

After a candidate container resize, recompute `NewActiveEdgeLength` and then recompute `NewMinimumRadius` and `NewMaximumRadius`. The existing `CircleOnContainerRadius` remains valid only if `NewMinimumRadius <= ExistingRadius <= NewMaximumRadius`.

Default rejection rule: if `ExistingRadius` is outside the new allowed range, reject the candidate container resize and restore the previous valid container geometry. Do not silently scale or clamp the `Circle On Container` radius as a side effect of resizing the container.

Hierarchy movement calculation for Circle On Container

If the attached container moves by `DeltaX` and `DeltaY`, the `Circle On Container` center moves by the same translation, or equivalently is recalculated from the moved parent boundaries using the same `HostEdge` and `EdgeParameterT`.

If a parent container or any higher nested parent container moves, the attached container moves as part of that subtree. Therefore, every `Circle On Container` attached to any descendant container edge shall also move with the same subtree movement and then recalculate `CenterX` and `CenterY` from its host container geometry.

The `Circle On Container` shall be included in the moved subtree descendant collection for movement, previous-valid-position restoration, rendering, and save/load recalculation.

Protection Padding, Overlap, Containment, and Persistence Calculations

Recommended JavaScript files for this section: `recalculate-rectangle-protection-padding.js`; `recalculate-circle-protection-padding.js`; `rectangle-overlap-on-move.js`; `circle-overlap-on-move.js`; `rectangle-circle-overlap-on-move.js`; `rectangle-overlap-on-resize.js`; `circle-overlap-on-resize.js`; `rectangle-circle-overlap-on-resize.js`.

Recommended resize-derived-geometry JavaScript files for this section: `recalculate-rectangle-derived-geometry-on-resize.js`; `recalculate-circle-derived-geometry-on-resize.js`; `recalculate-container-derived-geometry-on-resize.js`. These files shall run immediately after any accepted resize so that hover, padding, control points, and containment helper boundaries are recalculated before further validation or rendering.

Additional JavaScript files for containment, recovery, and persistence handoff: `validate-child-containment.js`; `resize-parent-container.js`; `restore-previous-valid-position.js`; `apply-validated-center.js`; `update-previous-valid-position.js`; `build-diagram-save-payload.js`; `diagram-api.js`.

This subsection is the authoritative equation set and validation rule set for Protection Padding derivation, overlap validation, containment validation, move-time candidate validation, resize-time candidate validation, previous-valid-position restore, clamped movement, parent-container expansion, and persistence-time recalculation.

Purpose

Protection Padding is the mathematical safety region used to prevent forbidden overlap and forbidden touching between diagram geometries. It is not a visual hover region. Its purpose is to enforce spacing rules, overlap rules, and containment rules during geometry insertion, movement, and later resizing. The source file defines Protection Padding as the region used for overlap avoidance, spacing rules, child-inside-container calculations, and containment validation, and states that overlap and container calculations shall use Protection Padding rather than Hover Padding.

Protection Padding shall be used to prevent overlap in all of the following cases:

- rectangle against rectangle
- circle against circle
- rectangle against circle
- child geometry against parent container boundary
- child geometry against sibling geometries inside the same immediate parent container

Circle On Container against sibling geometries and other Circle On Container items using the Circle ProtectionPaddedRadius

All Protection Padding calculations shall be performed in world space. The visible screen location of an item is not the authoritative source for validation. The authoritative source is the item's world-space geometry, especially its center-based canonical variables and the derived Protection Padding boundaries. The file explicitly states that geometry, selection, hit testing, snapping, and overlap validation should happen in world space, and that DiagramCanvas is an infinite logical workspace whose stored item coordinates remain in world space even when the viewport changes.

Main architectural rule

The system shall follow the same responsibility split already defined for the project.

JavaScript responsibilities

JavaScript shall be responsible for:

- screen-to-world conversion
- drag, drop, and move interaction
- temporary interaction state
- immediate Protection Padding calculations
- live validation during insert or move
- preview of accepted or rejected placement
- immediate render updates on DiagramCanvas

The file states that JavaScript is responsible for rendering, hit testing, drag behavior, resize behavior, coordinate conversion, temporary visual state, and immediate interaction in the browser.

C# responsibilities

C# shall be responsible for:

- trusted validation of committed diagram state
- server-side business rules
- JSON and database persistence flow control
- API endpoints

- database read and write operations
- final validation before permanent save

The file states that C# is the trusted server-side layer responsible for validation, business logic, persistence, and database communication.

Database rule

The database shall never be written directly from browser JavaScript. All permanent saves shall go through the C# layer. The file explicitly requires that database access should not happen directly from browser JavaScript.

World-space rule

All geometry positions shall be stored in a center-based world coordinate system. When a geometry moves, its world coordinates shall be updated directly. Protection Padding checks shall use world-space centers, world-space dimensions, and world-space derived boundaries. Panning and zooming change only the viewport and screen presentation; they do not change stored geometry coordinates. This follows the file's model for World Space, Viewport, and DiagramCanvas item movement.

Trigger conditions

Protection Padding calculations shall be triggered only when a geometry may create or change a spatial conflict.

Insert trigger

The Protection Padding calculation shall be triggered when a new rectangle or circle is inserted into DiagramCanvas.

Typical examples:

- dropping a rectangle from the left panel into the canvas
- dropping a circle from the left panel into the canvas
- creating a geometry by another insertion tool in the future

Move trigger

The Protection Padding calculation shall be triggered when an existing rectangle or circle is moved to a candidate new center position.

Typical examples:

- dragging a rectangle on the free canvas
- dragging a circle inside a container
- keyboard movement of a selected item

Optional later resize trigger

Although this section is focused on insert and move, the same recalculation rule shall also be triggered whenever a resize changes geometry dimensions. A resize shall immediately recalculate visible geometry, Hover Padding boundaries, Protection Padding boundaries, resize control point positions, and any

container/helper boundaries derived from the updated canonical geometry. Protection Padding validation shall then use those recalculated derived values.

Canonical geometry and derived Protection Padding geometry

The canonical stored variables are the primary truth. Protection Padding boundaries shall be derived from those canonical variables and shall not be treated as unrelated independent stored truths. The file states this rule for rectangles, circles, and container-related boundaries.

Rectangle Protection Padding model

Canonical rectangle variables

A rectangle shall use these canonical geometry variables as the primary truth:

- RectangleID
- CenterX
- CenterY
- HalfWidth
- HalfHeight
- RotationAngle
- ProtectionPaddingXRatio
- ProtectionPaddingYRatio

Recommended default values:

- ProtectionPaddingXRatio = 0.2
- ProtectionPaddingYRatio = 0.2

The file defines these rectangle variables and default protection padding ratios.

Derived rectangle Protection Padding values

For rectangle Protection Padding, derive:

- ProtectionPaddingX = HalfWidth × ProtectionPaddingXRatio
- ProtectionPaddingY = HalfHeight × ProtectionPaddingYRatio

Then derive the rectangle Protection Padding boundaries:

- ProtectionLeftX = CenterX - HalfWidth - ProtectionPaddingX
- ProtectionRightX = CenterX + HalfWidth + ProtectionPaddingX
- ProtectionTopY = CenterY + HalfHeight + ProtectionPaddingY
- ProtectionBottomY = CenterY - HalfHeight - ProtectionPaddingY

These are the boundaries that shall be used for overlap and containment validation. The file defines the rectangle's derived Protection Padding boundaries and states that overlap and container calculations shall use Protection Padding.

Circle Protection Padding model

7.1 Canonical circle variables

A circle shall use these canonical geometry variables as the primary truth:

- CircleID
- CenterX
- CenterY
- Radius
- ProtectionPaddingRadiusRatio

Recommended default value:

- ProtectionPaddingRadiusRatio = 0.2

The file defines the circle's Protection Padding variables and default ratio.

7.2 Derived circle Protection Padding values

For circle Protection Padding, derive:

- ProtectionPaddingRadius = Radius × ProtectionPaddingRadiusRatio
- ProtectionPaddedRadius = Radius + ProtectionPaddingRadius

Then derive the circle Protection Padding bounding box if needed:

- ProtectionLeftX = CenterX - ProtectionPaddedRadius
- ProtectionRightX = CenterX + ProtectionPaddedRadius
- ProtectionTopY = CenterY + ProtectionPaddedRadius
- ProtectionBottomY = CenterY - ProtectionPaddedRadius

The circle's main Protection Padding truth is the ProtectionPaddedRadius, while the protection bounding box is used as a helper for fast checks and mixed-shape calculations. This follows the file's circle geometry and Protection Padding definitions.

Overlap validation rules

Rectangle against rectangle

Two rectangles shall be considered non-overlapping only if their Protection Padding regions are separated.

Let rectangle 1 have:

- ProtectionLeftX_1
- ProtectionRightX_1
- ProtectionTopY_1
- ProtectionBottomY_1

Let rectangle 2 have:

- ProtectionLeftX_2
- ProtectionRightX_2
- ProtectionTopY_2
- ProtectionBottomY_2

Then rectangle 1 and rectangle 2 do not overlap if at least one of the following is true:

- ProtectionRightX_1 <= ProtectionLeftX_2
- ProtectionRightX_2 <= ProtectionLeftX_1
- ProtectionTopY_1 <= ProtectionBottomY_2
- ProtectionTopY_2 <= ProtectionBottomY_1

If even touching must be forbidden, replace <= with <. The file states this rectangle non-overlap rule directly.

Circle against circle

Two circles shall be considered non-overlapping only if the distance between their centers is large enough to separate their Protection Padded Radii.

Let:

- $dx = \text{CenterX_2} - \text{CenterX_1}$
- $dy = \text{CenterY_2} - \text{CenterY_1}$
- $d^2 = dx^2 + dy^2$

Then two circles do not overlap if:

- $d^2 \geq (\text{ProtectionPaddedRadius_1} + \text{ProtectionPaddedRadius_2})^2$

If even touching must be forbidden, use:

- $d^2 > (\text{ProtectionPaddedRadius_1} + \text{ProtectionPaddedRadius_2})^2$

The file states this circle non-overlap rule directly.

Rectangle against circle

For rectangle-circle validation, use the rectangle Protection Padding boundaries and the circle ProtectionPaddedRadius.

Let the rectangle Protection Padding boundaries be:

- RectProtectionLeftX
- RectProtectionRightX
- RectProtectionTopY
- RectProtectionBottomY

Let the circle candidate center be:

- CircleCenterX
- CircleCenterY

Find the closest point on the rectangle Protection Padding region to the circle center:

- ClosestX = clamp(CircleCenterX, RectProtectionLeftX, RectProtectionRightX)
- ClosestY = clamp(CircleCenterY, RectProtectionBottomY, RectProtectionTopY)

Then compute:

- $dx = \text{CircleCenterX} - \text{ClosestX}$
- $dy = \text{CircleCenterY} - \text{ClosestY}$
- $d^2 = dx^2 + dy^2$

The rectangle and circle do not overlap if:

- $d^2 \geq \text{ProtectionPaddedRadius}^2$

If even touching must be forbidden, use:

- $d^2 > \text{ProtectionPaddedRadius}^2$

The file defines rectangle-circle non-overlap using the closest-point method and explicitly says to use rectangle Protection Padding boundaries and the circle's ProtectionPaddedRadius for padded checks.

Validation when geometry is not inside a container

If a moved or inserted geometry is not within a parent container, then there is no container-containment check. DiagramCanvas is logically infinite, so the geometry is not validated against canvas boundaries. It is validated only against the other geometries already present on the canvas. The file defines DiagramCanvas as logically unbounded and states that all overlap rules are evaluated in world space.

Free-canvas validation scope

If the candidate geometry is on free DiagramCanvas, the direct validation set is:

- all other rectangles on the canvas
- all other circles on the canvas

If there are N total geometries on the canvas including the moved or inserted one, then the direct overlap check count is typically:

- N - 1

because the candidate geometry is checked against all other geometries except itself.

Free-canvas validation flow

The recommended validation flow for free-canvas insertion or movement is:

1. Get the candidate world-space center.
2. Recalculate the candidate Protection Padding geometry.
3. Compare the candidate geometry against all other geometries on the canvas.
4. If no conflict exists, accept the position.
5. If a conflict exists, reject the position or resolve to the nearest valid position according to the selected UI rule.

Validation when geometry is inside a container

If a moved or inserted geometry is within a parent container, then validation has two direct scopes:

- the candidate geometry against the other geometries that share the same immediate parent container
- the candidate geometry against the immediate parent container itself

A child item shall always remain fully inside its immediate parent container. The child item, including its Protection Padding region, shall never go outside the parent boundary, never touch the parent boundary, and never overlap the parent boundary. The file defines this rule for rectangle containers and child items.

Immediate-parent rule

Each child geometry has exactly one immediate parent container. Nested containment shall be enforced through immediate-parent rules. A child should normally be validated directly against its immediate parent, not against every ancestor independently. The file explicitly states that each child item has exactly one immediate parent container and that multiple nesting levels are handled by applying immediate-parent rules recursively.

Validation count inside a container

If there are N items inside the same immediate parent container including the moved geometry, then the normal direct validation set is:

- N - 1 sibling overlap checks
- 1 immediate-parent containment check

So the usual direct validation count is:

- N direct checks total

This is the correct interpretation for a normal moved child geometry.

Child container case

If the moved geometry is itself a nested rectangle container, then it is still validated as a child relative to its own immediate parent container. Its internal children move with it as part of the same container system. The moved nested container is therefore validated primarily against:

- its siblings in its immediate parent
- its immediate parent container boundary

The file states that a nested rectangle container is a child relative to its immediate parent and a parent relative to its own children.

Container containment equations

Child rectangle inside container

A child rectangle shall remain valid only if its Protection Padding boundaries remain fully inside the parent inner usable boundary.

Let the parent usable interior be:

- ParentInnerLeftX
- ParentInnerRightX
- ParentInnerTopY
- ParentInnerBottomY

Then the child rectangle is valid only if:

- ChildProtectionLeftX > ParentInnerLeftX
- ChildProtectionRightX < ParentInnerRightX
- ChildProtectionTopY < ParentInnerTopY
- ChildProtectionBottomY > ParentInnerBottomY

If strict no-touch is not required, these may be relaxed later, but the current rule is strict no-touch. The file defines strict no-touch and child containment using padded boundaries.

Child circle inside container

Convert the child circle into Protection Padding boundaries and apply the same containment logic:

- ChildProtectionLeftX > ParentInnerLeftX
- ChildProtectionRightX < ParentInnerRightX
- ChildProtectionTopY < ParentInnerTopY
- ChildProtectionBottomY > ParentInnerBottomY

The file explicitly recommends converting circles into Protection-Padded left/right/top/bottom boundaries before applying the same container checks.

Insertion without a parent container

When a new geometry is inserted on free DiagramCanvas, there is no container limitation. Only overlap with existing geometries must be validated.

Required steps

1. Convert the insertion point from screen space to world space.
2. Create a candidate geometry using canonical center-based values.
3. Recalculate the candidate Protection Padding geometry.
4. Validate the candidate geometry against all other geometries on the free canvas.
5. Accept the insertion if no conflict exists.
6. If a conflict exists, reject the insertion or move it to the nearest valid position according to the chosen UI policy.

Insertion within a container

When a new geometry is inserted inside a parent container, the following must be validated:

- it must not overlap sibling geometries
- it must fit inside the parent container using its Protection Padding boundaries

If there is not enough space inside the container, the parent container shall be resized.

Required steps

1. Convert the insertion point from screen space to world space.
2. Create the candidate child geometry.
3. Recalculate the child Protection Padding geometry.
4. Validate the child geometry against sibling geometries in the same immediate parent container.
5. Validate the child geometry against the parent container's inner usable boundary.
6. If the child does not fit, resize the parent container.
7. Recalculate the parent container's derived geometry.
8. If the parent container is itself a child of another parent container, validate it recursively upward.
9. Commit the final accepted geometry positions.

Parent resizing rule

When there is not enough space inside the immediate parent container, the parent container shall be expanded enough to contain the child geometry's Protection Padding region.

A valid parent-resize strategy is:

- $\text{NewParentInnerLeftX} = \min(\text{CurrentParentInnerLeftX}, \text{ChildProtectionLeftX} - \text{Epsilon})$
- $\text{NewParentInnerRightX} = \max(\text{CurrentParentInnerRightX}, \text{ChildProtectionRightX} + \text{Epsilon})$
- $\text{NewParentInnerTopY} = \max(\text{CurrentParentInnerTopY}, \text{ChildProtectionTopY} + \text{Epsilon})$
- $\text{NewParentInnerBottomY} = \min(\text{CurrentParentInnerBottomY}, \text{ChildProtectionBottomY} - \text{Epsilon})$

Then derive the new parent center and half-sizes from those updated inner boundaries.

This preserves the inserted child position while allowing the parent to grow around it.

Movement on free canvas

When an existing geometry moves on free DiagramCanvas, the movement validation shall be:

- candidate center generation
- Protection Padding recalculation
- overlap validation against all other geometries on the free canvas
- acceptance or rejection of the new center

There is no container containment check in this case.

Overlap return rule during mouse movement

If a geometry is being moved by mouse and its candidate moved position overlaps another geometry, then the moving geometry shall be returned immediately to its previous valid position.

This means:

- the system shall store the last valid world-space center before applying each new mouse-move update
- each mouse-move event shall calculate a candidate center
- the candidate center shall be validated against the other geometries on the canvas
- if overlap is detected, the move shall be rejected and the geometry shall be placed back at the stored previous valid center

This added project rule applies specifically to overlap caused by mouse movement.

Movement inside a container

When an existing geometry moves inside a container, the movement validation shall be:

- candidate center generation
- Protection Padding recalculation
- sibling overlap validation within the same immediate parent

- containment validation against the immediate parent
- acceptance, rejection, or clamping to nearest valid position

Recommended rule

For contained movement, use clamped movement if the product behavior should keep the item movable but never invalid. The file recommends clamped target positions during dragging as the safest method for contained items.

That means:

- compute candidate center
- compute candidate Protection Padding boundaries
- clamp the center to remain valid inside the parent
- validate against siblings
- accept the clamped valid center if possible
- otherwise preserve the previous valid center

Overlap return rule during mouse movement inside a container

If a geometry is being moved by mouse inside a container and the candidate moved position overlaps another sibling geometry, then the moving geometry shall be returned immediately to its previous valid position.

This means:

- containment against the immediate parent may still use clamping if desired
- but overlap against another geometry shall use revert-to-previous-valid-position behavior
- the previous valid center shall be the last accepted world-space center that satisfied both containment and non-overlap rules

So, during mouse movement inside a container:

- if the failure is caused by overlap with another geometry, return to previous valid position
- if the failure is caused only by parent-boundary containment, clamping may be used according to the selected UI rule
- if both fail and no valid clamped result exists, return to previous valid position

Large moving rectangle container against smaller container

This rule covers the case where a very large rectangle container is being moved and its candidate position suddenly overlaps a smaller rectangle container or another smaller geometry. The validation shall not depend on the visible size of the moved container. It shall depend only on the candidate Protection Padding boundaries in world space.

Let A be the moving rectangle container and let B be an existing sibling rectangle container or rectangle geometry. Calculate A at the candidate moved position, not at its old position.

Candidate Protection Padding boundaries for moving container A:

```
A_Left   = A_CenterX_candidate - A_HalfWidth  - A_ProtectionPaddingX
A_Right  = A_CenterX_candidate + A_HalfWidth  + A_ProtectionPaddingX
A_Top    = A_CenterY_candidate + A_HalfHeight + A_ProtectionPaddingY
A_Bottom = A_CenterY_candidate - A_HalfHeight - A_ProtectionPaddingY
```

Protection Padding boundaries for existing container or geometry B:

```
B_Left   = B_CenterX - B_HalfWidth  - B_ProtectionPaddingX
B_Right  = B_CenterX + B_HalfWidth  + B_ProtectionPaddingX
B_Top    = B_CenterY + B_HalfHeight + B_ProtectionPaddingY
B_Bottom = B_CenterY - B_HalfHeight - B_ProtectionPaddingY
```

Non-overlap is valid only if at least one separating condition is true:

```
A_Right <= B_Left
OR B_Right <= A_Left
OR A_Top <= B_Bottom
OR B_Top <= A_Bottom
```

Therefore, the candidate move is an overlap violation if this expression is true:

```
A_Right > B_Left
AND A_Left < B_Right
AND A_Top > B_Bottom
AND A_Bottom < B_Top
```

Strict no-touch version:

If touching is also forbidden, the valid non-overlap condition shall use strict separation:

```
A_Right < B_Left
OR B_Right < A_Left
OR A_Top < B_Bottom
OR B_Top < A_Bottom
```

Any case that does not satisfy one of these strict separation conditions shall be treated as overlap/touch violation.

Required validation scope:

If A is inside a parent container, validate A against all sibling containers and sibling geometries inside the same immediate parent container. If A is on the free DiagramCanvas, validate A against all other root-level geometries. Do not validate the moved container only against its own descendants, because descendants move with the container subtree.

Required rejection behavior:

If the candidate Protection Padding region of A overlaps or touches B under the strict no-touch rule, reject the candidate move and restore A to its previous valid world-space position. The previous valid position must be the last accepted position where A satisfied both parent containment and sibling/root non-overlap validation.

Previous valid position rule

To support the overlap return behavior, the system shall maintain previous valid position variables for every movable geometry.

Recommended variables:

- PreviousValidCenterX
- PreviousValidCenterY

Recommended update rule:

- whenever a geometry reaches a valid accepted position, update PreviousValidCenterX and PreviousValidCenterY
- whenever a mouse-move event produces an overlapping candidate center, restore CenterX and CenterY from PreviousValidCenterX and PreviousValidCenterY

This rule applies to:

- rectangles
- circles
- child geometries inside containers
- free-canvas geometries

Client-side and server-side execution rule

Better to run on client side

The following calculations are better on the client side because they are interaction-critical and must respond immediately:

- screen-to-world conversion
- candidate center calculation
- candidate Protection Padding recalculation
- rectangle-rectangle overlap checks
- circle-circle overlap checks
- rectangle-circle overlap checks
- immediate container-fit checks

- immediate parent-resize preview
- live clamped movement
- live drag and drop validation
- previous-valid-position restore during mouse movement
- render updates

The file assigns immediate drag, move, resize, hit testing, and rendering behavior to JavaScript.

Better to run on server side

The following must run on the server side because they are trusted and persistent:

- final committed diagram validation
- server-side business rules
- user permission checks
- payload validation
- final persistence validation
- database save
- database load
- audit logging

The file assigns these responsibilities to C#.

Combined rule

The best design is:

- JavaScript performs immediate validation and interaction-time calculations
- C# performs trusted re-validation before final save

This gives good user experience and good system trustworthiness.

Required save behavior after calculation finishes

After insert or move calculations finish and the final accepted positions are known, the committed diagram state shall be saved to:

- JSON
- database

The saved state shall contain the committed world-space geometry, not temporary interaction-only values. The file's save flow and recalculation-before-save model explicitly distinguish committed diagram state from temporary interaction state.

What shall be saved

The save operation shall include:

Rectangle data

- RectangleID
- CenterX
- CenterY
- HalfWidth
- HalfHeight
- RotationAngle
- ZOrder
- appearance values
- Protection Padding ratios
- parent relationship if applicable

Circle data

- CircleID
- CenterX
- CenterY
- Radius
- ZOrder
- appearance values
- Protection Padding ratio
- parent relationship if applicable

Diagram data

- DiagramID
- DiagramVersion
- DiagramName
- canvas state as needed
- item collection
- optionally viewport values if view restoration is desired

What shall not be saved

The save operation shall not persist temporary interaction-only state such as:

- current hover-only state
- temporary drag preview
- temporary resize preview before commit
- temporary mouse position
- temporary invalid candidate positions

The file's save design distinguishes committed state from temporary interaction state and keeps persistence under trusted server control.

Recalculation before save

Before any save operation is accepted, the system shall recalculate all required derived values from the canonical geometry.

For rectangles, this includes at least:

- visible boundaries
- Protection Padding boundaries
- resize control point positions

For circles, this includes at least:

- ProtectionPaddedRadius
- protection bounding box

The save operation shall use the recalculated committed state, not stale derived values. The file states that derived geometry is recalculated from canonical values and that resize control points are derived from the rectangle geometry rather than stored independently.

Required event-to-function routing

Insert on free canvas

User inserts geometry

→ screenToWorld
→ createCandidateGeometry
→ recalculateProtectionPaddingGeometry
→ validateFreeCanvasPlacement
→ checkRectangleRectangleNonOverlap / checkCircleCircleNonOverlap /
checkRectangleCircleNonOverlap
→ acceptOrResolvePlacement
→ insertCanvasItem

- renderCanvas
- markDiagramDirty

Insert inside container

User inserts geometry inside a container

- screenToWorld
- createCandidateChildGeometry
- recalculateProtectionPaddingGeometry
- validateSiblingNonOverlap
- validateChildContainment
- resizeParentContainerIfNeeded
- recalculateParentDerivedGeometry
- validateParentRecursivelyIfNeeded
- insertCanvasItem
- renderCanvas
- markDiagramDirty

Move on free canvas

User moves geometry on free canvas

- screenToWorld
- computeCandidateCenter
- recalculateProtectionPaddingGeometry
- validateAgainstAllOtherCanvasGeometries
- if overlap then restorePreviousValidCenter
- else applyValidatedCenter
- updatePreviousValidCenter
- renderCanvas
- markDiagramDirty

Move inside container

User moves geometry inside a container

- screenToWorld
- computeCandidateCenter
- recalculateProtectionPaddingGeometry
- validateAgainstSiblingGeometries
- if sibling overlap then restorePreviousValidCenter
- else validateAgainstImmediateParentContainer
- clampOrRejectCandidateCenterIfNeeded
- applyValidatedCenter
- updatePreviousValidCenter
- renderCanvas
- markDiagramDirty

Save committed diagram

User clicks Save

- recalculateAllDerivedGeometry
- validateCommittedDiagramState
- buildDiagramJsonObject
- serializeDiagramToJson

→ saveDiagramJsonFile
→ buildDiagramSavePayload
→ callSaveDiagramApi
→ DiagramController.SaveDiagram
→ DiagramService.SaveDiagram
→ DiagramRepository.SaveDiagram
→ handleSaveCommitResult

This flow matches the document's general function-contract style and cross-layer flow: browser-side processing, optional API request, then controller, service, repository, and response.

Function contract rule

Every function related to Protection Padding validation and persistence shall be documented using the same standard contract structure defined in the file:

- Function Name
- Layer
- File
- Purpose
- Trigger
- Input
- Validation / Preconditions
- Processing Responsibility
- Output
- State Change / Persistence Effect
- Next Possible Call(s)
- Error Output / Failure Path

The file explicitly defines this function contract structure for important functions.

Recommended function list

The following functions are recommended for this scope.

JavaScript functions

- screenToWorld
- createCandidateRectangle
- createCandidateCircle

- recalculateRectangleProtectionPaddingGeometry
- recalculateCircleProtectionPaddingGeometry
- validateRectangleRectangleNonOverlap
- validateCircleCircleNonOverlap
- validateRectangleCircleNonOverlap
- validateFreeCanvasPlacement
- validateSiblingNonOverlap
- validateChildContainment
- resizeParentContainerIfNeeded
- computeCandidateCenter
- applyValidatedCenter
- restorePreviousValidCenter
- updatePreviousValidCenter
- renderCanvas
- markDiagramDirty
- buildDiagramJsonObject
- serializeDiagramToJson
- saveDiagramJsonFile
- buildDiagramSavePayload
- callSaveDiagramApi

C# functions

- DiagramController.SaveDiagram
- DiagramController.LoadDiagram
- DiagramService.SaveDiagram
- DiagramService.LoadDiagram
- DiagramRepository.SaveDiagram
- DiagramRepository.LoadDiagramByld

Final implementation rule

Protection Padding shall be treated as the authoritative mathematical region for overlap prevention and containment validation. The visible geometry alone is not sufficient. Hover Padding shall not be used for these calculations. All final accepted positions shall be generated from world-space calculations, and all committed positions shall be saved to both JSON and database through the approved persistence flow. The file's rectangle, circle, container, and DiagramCanvas sections all reinforce that Protection Padding is the correct region for overlap and containment logic, and that these checks are world-space rules rather than screen-space rules.

Different Keyboard and Mouse actions

Purpose

The diagram tool shall support multiple mouse and keyboard actions on DiagramCanvas and on diagram items. These actions are not all equivalent. The system shall distinguish both:

- the **user action type**
- the **interaction target**

For example, hovering over the center of a shape is not the same as hovering over its edge, perimeter, resize control point, junction point, or empty canvas. The source file already states that the tool includes actions such as left click, double left click, right click, hover, Ctrl + mouse wheel up/down, left click then drag shape, left click then resize, and keyboard arrow actions, and it explicitly says that target detection must distinguish between center, edge, perimeter, control point, junction point, and empty canvas.

Main interaction rule

Every interaction shall be interpreted using this model:

User Action + Interaction Target + Current Tool/State = Resulting Behavior

That means the same mouse or keyboard action may produce different results depending on:

- whether the pointer is over empty canvas or over an item
- whether the pointer is over item center, interior, edge, perimeter, resize control point, or junction point
- whether an item is already selected
- whether drag mode, resize mode, pan mode, or another tool mode is active

This is consistent with the file's interaction model, which states that JavaScript should detect what part of the item the pointer is targeting and respond accordingly.

Architectural ownership rule

JavaScript side

JavaScript shall handle immediate interaction behavior, including:

- left click
- double left click
- right click
- hover
- Ctrl + mouse wheel up/down
- left click then drag shape
- left click then resize shape
- click on resize control points
- click on junction points
- click on canvas background
- keyboard arrow actions
- Ctrl, Shift, Alt, Delete, Escape, and selection-related keys if enabled

The source file explicitly assigns these actions to JavaScript as part of client-side live interaction handling.

C# side

C# shall not handle per-mouse-move or per-keypress canvas interaction. C# shall only handle trusted validation, persistence, API calls, and database communication when those actions lead to committed save/load operations. The file states this separation clearly.

Interaction target categories

The system shall identify the exact interaction target before applying action logic.

Canvas-level targets

- empty canvas background
- canvas grid
- canvas axes
- origin marker
- viewport/background area

Shape-level targets

- shape center
- shape interior area
- shape edge
- shape perimeter
- hover padding region
- protection padding region for validation only, not normal hover visualization

Editing targets

- rectangle resize control point
- circle resize control point
- junction point
- line start point
- line end point
- line body
- connection point
- container boundary

The file explicitly mentions target detection such as center, interior, edge, perimeter, control point, junction point, and empty canvas. It also defines resize control points for rectangles and circles, and attachment/interaction points for junction points and lines.

Core mouse actions

Left click

Left click shall be used for primary selection and activation behavior.

Typical uses:

- select a shape
- select a container
- select a junction point
- activate a resize control point
- activate a tool-specific object
- place focus on canvas

- clear selection when clicking empty canvas, if that UI rule is enabled

Double left click

Double left click shall be reserved for secondary edit or open behavior.

Recommended uses:

- open text or label editing

The source file lists double left click as a supported JavaScript interaction.

Right click

Right click shall be used for context-sensitive actions.

Recommended uses:

- open context menu for shape
- open context menu for canvas
- show item-specific actions
- show container-specific actions
- show alignment or arrangement options in the future

The source file lists right click and context-menu-related actions as supported JavaScript interaction behavior.

Mouse hover

Mouse hover shall be used to detect pointer presence and target category without committing selection or movement.

Hover shall be distinguishable by target type, including:

- hover over shape center
- hover over interior
- hover over edge
- hover over perimeter
- hover over resize control point
- hover over junction point
- hover over empty canvas

The source file explicitly says these are not the same action and should be distinguished.

Ctrl + mouse wheel up/down

Ctrl + mouse wheel up/down shall be used for zoom behavior or tool-specific zoom-related behavior.

Typical uses:

- zoom in
- zoom out
- zoom around cursor
- optionally change zoom sensitivity or alternate editor mode in future

The source file explicitly lists Ctrl + mouse wheel up/down as a handled JavaScript interaction.

Left click then drag shape

Left click then drag on a selected shape shall be used for movement of the shape.

Typical uses:

- move rectangle
- move circle
- move child geometry inside a container
- move a container
- drag a library item toward the canvas during creation

The source file explicitly lists left click then drag shape as a JavaScript-handled action and defines movement in world space.

Left click then resize

Left click then drag on a valid resize control point shall be used for resizing behavior.

Typical uses:

- resize rectangle from one of 8 rectangle resize control points
- resize circle from one of 4 circle resize control points

The file defines these resize control points and their direction-specific behavior.

Core keyboard actions

Arrow keys

The four keyboard arrow keys shall be supported for directional movement.

- Up Arrow
- Down Arrow
- Left Arrow
- Right Arrow

Typical uses:

- move selected shape
- move selected container
- move selected child item inside a container
- nudge a selected object by a small step

The source file explicitly lists the four arrow keys as handled JavaScript actions.

Ctrl combinations

Ctrl combinations may be used for zoom or tool-specific behavior.

Recommended uses:

- Ctrl + Mouse Wheel for zoom
- Ctrl + Click for additive selection later
- Ctrl + Drag for alternate drag rule later
- Ctrl + C, Ctrl + V, Ctrl + X for copy/paste/cut later
- Ctrl + Z, Ctrl + Y for undo/redo later

The file explicitly mentions Ctrl combinations related to zoom or tool behavior. The copy/paste and undo/redo examples here are recommended future actions, not currently mandated by the file.

Shift and Alt modifiers

Shift and Alt modifiers may be used for alternate editing behavior.

Recommended uses:

- Shift + Arrow for larger movement step
- Shift + Drag for axis-constrained movement later
- Shift + Resize for proportion-preserving resize later
- Alt + Drag for duplicate-on-drag later
- Alt + Click for alternate tool behavior later

The file explicitly states that Shift or Alt modifiers may be used by the editor. The specific examples here are recommended future behaviors.

Delete key

Delete shall be reserved for deletion of selected items if deletion is enabled.

Recommended uses:

- delete selected shape

- delete selected line
- delete selected junction point
- delete selected container

The file explicitly lists delete as a possible later supported key.

Escape key

Escape shall be reserved for canceling the current temporary action.

Recommended uses:

- cancel drag
- cancel resize
- cancel current tool
- clear hover-only temporary state
- close context menu

The file explicitly lists escape as a possible later supported key.

Interaction result depends on target

The same action shall produce different results depending on what part of the system is targeted.

Hover over center

Hover over the center of a shape may indicate:

- object-level hover
- object identification
- movement readiness
- center-based selection readiness

Hover over interior

Hover over the interior of a shape may indicate:

- shape selection target
- body drag readiness
- item-level hover state

Hover over edge or perimeter

Hover over the edge or perimeter may indicate:

- boundary-sensitive hover
- hover padding activation
- edge-specific selection behavior
- connection readiness for future line attachment
- resize preparation if edge-middle resize points are nearby

The file explicitly distinguishes perimeter-sensitive hover from center hover.

Hover over resize control point

Hover over a resize control point may indicate:

- resize availability
- resize cursor change
- activation of direction-specific resize behavior

The source file explicitly mentions hover and click behavior related to resize control points.

Hover over junction point

Hover over a junction point may indicate:

- connection behavior readiness
- host-attachment behavior
- activation/highlight logic for connection-related work

The file explicitly mentions clicking on junction points and treating them as distinct interaction targets.

Hover over empty canvas

Hover over empty canvas may indicate:

- no item target
- pan-ready background
- selection-clear target
- diagram-level tool operation target

Recommended missing mouse actions to include

The following actions are strongly recommended to be added explicitly to the document, because they are commonly needed in a diagram editor even if they are not all listed in the current source text.

Mouse down

MouseDown should be treated as the true action-start event for:

- drag start
- resize start
- pan start
- context menu preparation
- selection start

Mouse move

MouseMove should be treated as the continuous-update event for:

- hover updates
- drag updates
- resize updates
- live snapping preview
- live target detection
- live cursor update

Mouse up

MouseUp should be treated as the commit/end event for:

- drag end
- resize end
- click completion
- drop completion
- selection box completion

Click on canvas background

This is already implied in the file, and it should be written explicitly in the document as a separate action because it usually has different logic from clicking on an item.

Recommended uses:

- clear selection
- start pan mode
- place focus on canvas
- end temporary edit mode

The source file explicitly lists clicking on canvas background as a JavaScript action.

Drag over canvas

This is useful for:

- library-item drag preview
- drop-target validation
- valid/invalid drop indication

The file's milestone flow for drag from Shapes Panel to DiagramCanvas strongly implies drag-over and drop-target handling.

Drop on canvas

This is useful for:

- create rectangle or circle from left panel
- create item in world space
- convert screen position to world position
- insert new item into canvas model

This is part of the drag-and-drop milestone already described in the file.

Pan action

Pan should be written explicitly as a separate action, because the file already defines panning as a viewport behavior and not a model movement.

Recommended uses:

- drag empty canvas background
- middle-mouse drag if later supported
- spacebar + drag if later supported

Selection box / marquee drag

Recommended future action:

- click and drag on empty canvas to create a selection rectangle
- multi-select several items at once

This is not explicitly mandated in the file, but it is a useful missing action for a diagram editor.

Middle click / mouse wheel click

Recommended future action:

- middle-click pan
- quick viewport centering

- alternate tool behavior

This is a recommendation, not a file-mandated requirement.

Home / End

Recommended future uses:

- jump viewport to origin
- jump to first or last selected object
- reset camera position

Page Up / Page Down

Recommended future uses:

- change Z order
- move selection visually backward/forward
- cycle between layers

Copy / cut / paste shortcuts

Recommended future uses:

- Ctrl + C
- Ctrl + X
- Ctrl + V

Undo / redo shortcuts

Recommended future uses:

- Ctrl + Z
- Ctrl + Y

Recommended priority order for interaction targets

When the pointer is over more than one possible target, the system should use a deterministic priority order.

Recommended priority:

1. active resize control point
2. junction point
3. line endpoint
4. line body
5. shape perimeter / edge

6. shape interior / center
7. container boundary
8. empty canvas background

This avoids ambiguity and produces predictable editing behavior. The file already emphasizes that target type matters, so a priority rule is a useful formal addition.

Recommended action-state categories

For implementation clarity, each action should belong to one of these categories:

Detection actions

Used only to detect or prepare behavior.

Examples:

- hover
- mouse move without button press
- target hit test
- cursor update

Selection actions

Used to select, activate, or focus an object.

Examples:

- left click on shape
- click on resize control point
- click on junction point
- click on canvas background

Transformation actions

Used to change geometry or viewport.

Examples:

- drag shape
- resize shape
- pan viewport
- zoom viewport
- keyboard nudge

Context actions

Used to open menus or alternate commands.

Examples:

- right click
- context menu open
- alternate click modifier actions

Commit / cancel actions

Used to finish or cancel temporary interaction.

Examples:

- mouse up
- Enter
- Escape

Recommended section rule for developers

Every important action handler should be documented using the function contract already defined in the file:

- Trigger
- Input
- Validation / Preconditions
- Processing Responsibility
- Output
- State Change / Persistence Effect
- Next Possible Call(s)
- Error Output / Failure Path

The file explicitly requires this contract structure for important functions.

Short formal summary

The diagram system shall support multiple mouse and keyboard actions, and each action shall be interpreted according to both the action type and the exact interaction target. Hover over center, edge, perimeter, resize control point, junction point, and empty canvas shall be treated as different cases. JavaScript shall handle immediate interaction behavior, hit testing, and visual updates, while C# shall

handle only trusted validation, persistence, and database-related operations. In addition to the currently listed actions, the document should also define mouse down, mouse move, mouse up, drag-over, drop, pan, selection box, Enter, Escape, Tab, Spacebar, copy/paste, and undo/redo as explicit actions or future-ready actions.

Development phases

Important Rules

Each section is having the following:

- Table that is Listing variables, indicating its description, and recommended values.
- Tables indicating Database tables, type, and purpose
- Table having file names of CSS, JS and C#, indicating its description.
 - o In this table, For JS and CSS, indicate the following:
 - Trigger
 - Input
 - Validation / Preconditions
 - Processing Responsibility
 - Output
 - State Change / Persistence Effect
 - Next Possible Call(s)
 - Error Output / Failure Path

Global milestone save architecture. All milestone persistence in this project shall use exactly two shared JavaScript save entry files only: save-to-json.js and save-to-db.js. These two files are the only approved top-level save entry points exposed to the UI and test flow.

Superseding rule. Any milestone-specific save filenames previously mentioned in this document are replaced as top-level save entry points by save-to-json.js and save-to-db.js. A milestone may still use helper functions or helper modules internally, but the final user-invoked save action shall route through these same two shared files.

Milestone save extension rule. When the developer starts a new milestone, the developer shall update the same save-to-json.js and save-to-db.js files instead of creating new save-entry files. Each update shall add support for the newly introduced committed variables of the current milestone while preserving correct save behavior for every committed variable already approved in previous milestones.

Milestone save acceptance rule. The current milestone save test shall check only the committed variables introduced by that milestone. In addition to that focused test, regression validation shall confirm that all committed variables from all previously completed milestones are still saved correctly. This rule prevents overlap, duplicated save scope, and ambiguity about which milestone owns which saved variables.

Single save UI rule. The HTML page shall include one Save button placed at the top-right of the page. When the user presses Save, a drop-down menu shall appear with exactly these options: Save to database, Save to JSON, and Save to DB and JSON.

Save action routing rule. Save to database shall call save-to-db.js only. Save to JSON shall call save-to-json.js only. Save to DB and JSON shall execute both shared save files in one controlled flow. The UI may use existing page JavaScript for click binding, but the final save actions shall still route only to these two shared save files.

Per-milestone prerequisite rule. The prerequisites of every milestone shall explicitly require verification that save-to-json.js and save-to-db.js are already saving the correct committed variable set for that milestone stage.

Milestone 1 — Shapes Panel Developer Plan

Overview

Milestone 1 defines the **Shapes Panel** that appears on the left side of the diagram tool.

This milestone comes before DiagramCanvas Milestone 1 because it provides the user-facing library from which shape items, nodes, or symbols are selected before they are dragged into the diagram workspace. Milestone 1 focuses on the **shape library panel**, while Milestone 1 focuses on the **DiagramCanvas** itself.

The Shapes Panel is a collapsible, searchable, resizable, category-driven library panel that allows the user to:

- open and close the panel
- search available shapes
- browse many shape categories
- select one category at a time
- display the matching library items for the selected category
- drag a library item toward the diagram canvas

A core design rule for this milestone is:

- **JavaScript** is responsible for panel rendering, interaction handling, selection behavior, item loading, search filtering, resizing behavior, and drag-start behavior
- **C#** is responsible only for shared model definitions, persistence contracts, or future server-side loading rules if required later
- For Milestone 1, the main implementation is primarily on the **JavaScript side**

Another important rule for this milestone is:

- every major function introduced in this milestone should be documented using this contract structure:
 - **Trigger**
 - **Input**
 - **Validation / Preconditions**

- **Processing Responsibility**
- **Output**
- **State Change / Persistence Effect**
- **Next Possible Call(s)**
- **Error Output / Failure Path**

Shapes Panel terminology

Main developer names

The following names are recommended for the UI parts in this milestone:

Whole left panel

- ShapesPanel

Panel header

- ShapesPanelHeader

Panel title

- ShapesPanelTitle

Open / close arrow

- ShapesPanelToggleButton

Search input

- ShapesSearchInput

Category area container

- ShapeCategoryList

One category entry

- ShapeCategoryItem

Currently selected category entry

- ActiveShapeCategoryItem

Resizable separator between category area and items box

- ShapesPanelSplitter

Items box container

- ShapeLibraryItemsContainer

One icon/item inside the items box

- DraggableShapeLibraryItem

Diagram workspace on the right

- DiagramCanvas

Important meaning of category area

The category area is **not limited to four fixed entries**.

The current diagram shows only these current categories:

- Blocks With Perspective
- Computers and Monitors - 3D
- Rack Mounted Servers
- Connectors

However, this area shall be treated as a **scalable category list** that may contain many more categories in the future.

So the category area should be designed as a **dynamic shape category list**, not as four hard-coded tabs.

Important meaning of selected category

The currently selected category is the **ActiveShapeCategoryItem**.

In the current example:

- RackMountedServers is the active shape category

That active category determines which items appear in the ShapeLibraryItemsContainer.

Example:

- if the active category is RackMountedServers, then the items box shows server-related icons such as web server, file server, application server, and similar items
- if the active category changes to BlocksWithPerspective, then the items box must update and show a different item set

Mandatory existing JavaScript before starting Milestone 1

Only JavaScript files that must already exist and work correctly before this milestone starts are listed here.

Shared save prerequisite. Milestone 1 shall create and wire the two shared persistence entry files save-to-json.js and save-to-db.js as the only approved top-level save files. If Milestone 1 introduces committed variables, its save acceptance test shall check only the Milestone 1 committed variables. Later milestones shall extend these same two files instead of creating new save-entry files.

JavaScript file that must already exist	Description	Reason this JavaScript must already be ready and working correctly
---	-------------	--

None from previous milestones	Milestone 1 is the first implementation milestone and establishes the initial JavaScript baseline.	No earlier milestone JavaScript dependency exists.
-------------------------------	--	--

Mathematical Calculation Rules for Milestone 1

This milestone is mainly a user-interface milestone. Its calculations control panel layout, splitter movement, item filtering, and drag-start handoff. These calculations shall not create committed diagram geometry and shall not write directly to the database.

Primary JavaScript files for this calculation scope: shapes-panel-splitter.js, shapes-search.js, drag-shape-library-item.js, shape-library-drag-payload.js, update-shapes-panel-layout.js.

Panel layout calculation

$\text{AvailablePanelContentHeight} = \text{ShapesPanelHeight} - \text{ShapesPanelHeaderHeight} - \text{ShapesSearchHeight} - \text{PanelPaddingTop} - \text{PanelPaddingBottom}$.

$\text{CategoryAreaHeight} + \text{SplitterHeight} + \text{ItemsAreaHeight} = \text{AvailablePanelContentHeight}$.

SplitterPosition shall be clamped so that $\text{CategoryAreaHeight} \geq \text{MinCategoryAreaHeight}$ and $\text{ItemsAreaHeight} \geq \text{MinItemsAreaHeight}$.

Search calculation

$\text{FilteredVisibleItems} = \text{ShapeLibraryItems}$ where item name, category name, tags, or searchable metadata contain SearchQuery according to the selected SearchMode.

If SearchQuery is empty, FilteredVisibleItems shall equal the full item list of the active category.

Drag-start handoff calculation

DragStartScreenX and DragStartScreenY are screen-space values. Milestone 1 shall build a drag payload containing DraggedLibraryItemID, DraggedLibraryItemType, source category, and initial screen position. The payload shall not yet calculate final world-space placement; final placement belongs to the DiagramCanvas drop milestone.

Milestone 1 phases

Milestone 1 shall be implemented in 3 phases.

Phase 1.1 — Define Shapes Panel structure and visible behavior

Goal

Define the Shapes Panel structure, core variables, visual layout, and basic open/close behavior.

Each important panel variable should produce a visible or measurable effect.

Scope

This phase is only about the **Shapes Panel UI structure**, not about actual canvas placement logic.

This phase is about:

- panel layout
- header and title
- open / close behavior
- search input placement
- category area placement
- items box placement
- splitter placement
- initial panel styling
- initial state variables

Variables to define

At minimum:

Identity

- ShapesPanelID
- ShapesPanelName

Visibility and state

- IsShapesPanelVisible
- IsShapesPanelCollapsed

Header

- ShapesPanelTitleText

Search

- SearchPlaceholderText
- SearchQuery

Category list

- CategoryListVisible
- ActiveShapeCategoryId
- ActiveShapeCategoryName

Splitter

- SplitterEnabled
- CategoryAreaHeight
- ItemsAreaHeight

Items area

- ItemsBoxVisible

Interaction flags

- CategorySelectionEnabled
- DragFromLibraryEnabled
- SearchEnabled

Required visible behaviors

Each important variable should have a tangible effect when changed.

Examples:

- changing IsShapesPanelCollapsed collapses or expands the panel
- changing ShapesPanelTitleText updates the visible title
- changing SearchPlaceholderText updates the search input placeholder
- changing CategoryListVisible shows or hides the category list
- changing CategoryAreaHeight changes the visible height of the category area
- changing ItemsAreaHeight changes the visible height of the items area
- changing ItemsBoxVisible shows or hides the item area

Function contract deliverables

The following major JavaScript functions or modules should be defined in this phase:

- initializeShapesPanelState
- renderShapesPanel
- renderShapesPanelHeader
- renderShapesSearchInput
- renderShapeCategoryList
- renderShapeLibraryItemsContainer
- renderShapesPanelSplitter
- toggleShapesPanel
- updateShapesPanelLayout

If a shared model is introduced, it may also include:

- ShapesPanelModel
- ShapeCategoryModel

- ShapeLibraryItemModel

Phase 1.1 responsibility split

JavaScript responsibilities

- initialize Shapes Panel state
- render panel layout
- render category area and items area
- render splitter
- handle open/close behavior
- update UI state when panel visibility changes

C# responsibilities

- define shared model types only if needed
- do not own panel rendering or interaction behavior

Output

This phase should produce:

- a formal Shapes Panel variable list
- a JavaScript state definition for the panel
- an optional C# shared model definition if needed
- a visible left-side Shapes Panel
- a working open/close behavior
- a visible search box
- a visible category list region
- a visible items box region
- a visible splitter between the two regions

Validation / Preconditions

Before Phase 1.1 is considered valid:

- the Shapes Panel must render successfully
- the panel must support expanded and collapsed states
- the category region and items region must both be visually distinct
- the splitter must be positioned correctly between the two regions

Error / failure expectations

Examples of failure behavior:

- if the Shapes Panel state fails to initialize, rendering must not proceed
- if collapse state is invalid, preserve the previous valid state
- if layout calculation fails, keep the previous valid panel layout
- if the splitter state is invalid, fall back to default region heights

Success criteria

This phase is successful if:

- the Shapes Panel is visible and structurally correct
- the panel can open and close correctly
- the header, search box, category region, items region, and splitter are visible
- developers can verify one consistent panel structure and state definition
- the main Phase 1.1 functions are documented with clear function contracts

Phase 1.2 — Add category selection and dynamic item loading

Goal

Allow the user to select one shape category at a time and load the correct item set into the items box based on the active category.

Scope

This phase is about:

- multiple categories in the category list
- active category state
- visual highlighting of the selected category
- switching item sets when the selected category changes
- support for future growth beyond the currently shown categories

Current example categories

The current known categories are:

- Blocks With Perspective
- Computers and Monitors - 3D
- Rack Mounted Servers
- Connectors

These are examples only. The system must support more categories later.

Required behavior

The category list must behave as a **single-select category library**.

That means:

- only one ShapeCategoryItem is active at a time
- the selected item becomes the ActiveShapeCategoryItem
- the ShapeLibraryItemsContainer updates to show only the items for the active category

Example:

- selecting RackMountedServers shows items such as web server, file server, application server, and similar server nodes
- selecting BlocksWithPerspective shows a different group of items
- selecting Connectors shows connector-related items

Variables to define

At minimum:

Category collection

- ShapeCategories[]

Active category

- ActiveShapeCategoryId
- ActiveShapeCategoryName

Category item state

- IsCategorySelected
- CategoryDisplayOrder

Item source state

- VisibleShapeLibraryItems[]
- FilteredShapeLibraryItems[]

Required visible behaviors

Examples:

- selecting a category changes the selected visual style
- the previously active category returns to non-selected style
- the items box refreshes immediately when the active category changes

- empty categories show an empty-state message if no items exist

Function contract deliverables

The following major JavaScript functions or modules should be defined in this phase:

- loadShapeCategories
- renderShapeCategoryItem
- selectShapeCategory
- setActiveShapeCategory
- loadShapeLibraryItemsByCategory
- renderShapeLibraryItems
- clearPreviousActiveCategoryState
- showEmptyItemsState

Optional helper functions:

- sortShapeCategories
- getCategoryItems
- refreshItemsBox

Required event-to-function routing

Category selection flow

click on ShapeCategoryItem
 → selectShapeCategory
 → setActiveShapeCategory
 → loadShapeLibraryItemsByCategory
 → renderShapeLibraryItems

Initial load flow

Shapes Panel initialized
 → loadShapeCategories
 → setDefaultActiveShapeCategory
 → loadShapeLibraryItemsByCategory
 → renderShapeLibraryItems

State rule

During this phase:

- category selection updates **temporary JavaScript UI state**
- changing the active category does not create or save anything on the canvas
- C# is not required for local category switching unless categories are loaded from a backend later

Output

This phase should produce:

- a working category list
- a clearly highlighted active category
- dynamic loading of items based on selected category
- support for many future categories
- clear function contracts for category selection and item loading

Validation / Preconditions

Before category switching is accepted:

- the Shapes Panel must already be rendered
- the category list must exist
- the selected category must have a valid identifier
- item loading must support categories with zero, one, or many items

Error / failure expectations

Examples of failure behavior:

- if a category ID is invalid, keep the previous active category
- if item loading fails, show a controlled empty/error state in the items box
- if the active category is missing at startup, use a fallback default category
- if duplicate category activation occurs, preserve only one active category

Success criteria

This phase is successful if:

- the user can select one category at a time
- the active category is visually clear
- the items box changes correctly when the category changes
- the system supports future category expansion
- the event-to-function routing is documented clearly

Phase 1.3 — Add search, splitter resize, and drag-start from library to canvas

Goal

Complete the Shapes Panel interaction by adding:

- search behavior
- splitter resize behavior
- drag-start behavior for library items that will later be dropped on the DiagramCanvas

Scope

This phase is about:

- filtering visible categories and/or library items through search
- resizing the category area and items area using the splitter
- dragging a library item from the items box toward the canvas
- preparing the handoff into future DiagramCanvas behavior

This phase does **not** yet require full diagram-item persistence rules. It only requires correct library drag-start behavior and handoff preparation.

Search behavior

The search system should be designed clearly.

Recommended initial rule:

- the search input filters the **visible library items**
- category filtering may be added later if needed

Optional future rule:

- support searching both categories and items with explicit search modes

Splitter behavior

The ShapesPanelSplitter sits between:

- the ShapeCategoryList
- the ShapeLibraryItemsContainer

Dragging the splitter changes the height of these two regions.

Examples:

- moving the splitter upward increases the items area and reduces the category area
- moving the splitter downward increases the category area and reduces the items area

Drag-start behavior

A DraggableShapeLibraryItem in the items box is a library source item.

When the user presses and drags it:

- the system begins a drag operation
- the drag payload identifies which shape item is being dragged
- the item can then be handed over to the future DiagramCanvas drop logic

Important rule:

- the original library item in the panel is not removed
- dragging from the library should create a **new placed item later**, not move the source library icon itself

Variables to define

At minimum:

Search

- SearchQuery
- FilteredVisibleItems[]
- SearchMode

Splitter

- IsSplitterDragging
- SplitterPosition
- MinCategoryAreaHeight
- MinItemsAreaHeight

Drag state

- IsLibraryItemDragging
- DraggedLibraryItemId
- DraggedLibraryItemType
- DragPreviewVisible

Function contract deliverables

The following major JavaScript functions or modules should be defined in this phase:

- handleShapesSearchInput
- filterVisibleShapeLibraryItems
- beginSplitterDrag
- updateSplitterDrag
- endSplitterDrag

- beginLibraryItemDrag
- updateLibraryItemDragPreview
- endLibraryItemDrag
- buildLibraryItemDragPayload
- handoffDragToCanvas

Optional helper functions:

- filterCategories
- filterItems
- clampSplitterPosition
- createDragPreview

Required event-to-function routing

Search flow

input in ShapesSearchInput
 → handleShapesSearchInput
 → filterVisibleShapeLibraryItems
 → renderShapeLibraryItems

Splitter flow

mousedown on ShapesPanelSplitter
 → beginSplitterDrag
 → mousemove
 → updateSplitterDrag
 → updateShapesPanelLayout
 → mouseup
 → endSplitterDrag

Drag-start flow

mousedown on DraggableShapeLibraryItem
 → beginLibraryItemDrag
 → buildLibraryItemDragPayload
 → updateLibraryItemDragPreview
 → handoffDragToCanvas

State rule

During this phase:

- search, splitter drag, and library drag-start update **temporary JavaScript interaction state**
- drag-start from the library does not yet require database writes
- the library item remains in the source list

- the future canvas placement should create a new instance, not move the source library item

Output

This phase should produce:

- working search input behavior
- working splitter resizing behavior
- working drag-start behavior from the library
- a clear payload handoff path toward the DiagramCanvas
- clear function contracts for search, resizing, and drag-start modules

Validation / Preconditions

Before Phase 1.3 is considered valid:

- the Shapes Panel must already support active category switching
- visible items must already load correctly
- splitter bounds must be defined
- drag-start must identify a valid library item

Error / failure expectations

Examples of failure behavior:

- if search filtering fails, show the previous valid item set
- if splitter drag exceeds allowed limits, clamp to the nearest valid position
- if a library item drag starts with invalid item data, cancel the drag
- if canvas handoff is unavailable, preserve drag-source state and fail safely
- if search returns no results, show a controlled empty-state message

Success criteria

This phase is successful if:

- the user can search the visible library items
- the splitter can resize the upper and lower regions safely
- a library item can begin a drag operation correctly
- the drag operation keeps the source library intact
- the drag handoff path to DiagramCanvas is clearly defined
- the panel is ready to connect to Milestone 1

Final result after Milestone 1

Milestone 1 persistence note

Milestone 1 introduces no committed diagram variables that must be persisted as milestone data. Its persistence deliverables are structural only: the top-right Save button, the drop-down options Save to database / Save to JSON / Save to DB and JSON, and the two shared top-level save entry files save-to-json.js and save-to-db.js. Shapes Panel temporary UI state is not milestone-required saved data.

Save test criteria for Milestone 1

Milestone 1 passes only if the Save UI exists, each menu choice routes to the correct shared save entry file, and no panel-only temporary state is treated as required committed persistence data.

At the end of Milestone 1, the developer should have:

- a fully defined ShapesPanel
- a collapsible left-side panel
- a scalable category list that supports many categories
- a clearly defined ActiveShapeCategoryItem
- dynamic item loading in the ShapeLibraryItemsContainer
- search input behavior
- splitter resize behavior
- drag-start behavior for DraggableShapeLibraryItem
- a clear handoff path from the shape library toward the future DiagramCanvas

Coding files structure for Milestone 1

For this milestone, the following structure is recommended.

HTML

- 1 file extension to the existing main page shell, or the same index.html

CSS

Recommended:

- shapes-panel.css
- shapes-panel-header.css
- shape-category-list.css
- shape-library-items.css
- shapes-panel-splitter.css

- shapes-search.css

A more compact version is also acceptable if fewer files are required.

JavaScript

Recommended modules:

- shapes-panel-state.js
- render-shapes-panel.js
- toggle-shapes-panel.js
- shape-categories.js
- select-shape-category.js
- load-shape-library-items.js
- render-shape-library-items.js
- shapes-search.js
- shapes-panel-splitter.js
- drag-shape-library-item.js
- shape-library-drag-payload.js

Optional C# structure

Only if needed later:

- ShapeCategoryDto.cs
- ShapeLibraryItemDto.cs
- ShapesPanelModel.cs

Do not move panel rendering, category selection, search, splitter drag, or library drag-start into C#.

Best folder layout for Milestone 1

```
project/
├─ index.html
├─ css/
│   ├─ shapes-panel.css
│   ├─ shapes-panel-header.css
│   ├─ shape-category-list.css
│   ├─ shape-library-items.css
│   ├─ shapes-panel-splitter.css
│   └─ shapes-search.css
└─ js/
```



```
| ├── shapes-panel-state.js
| ├── render-shapes-panel.js
| ├── toggle-shapes-panel.js
| ├── shape-categories.js
| ├── select-shape-category.js
| ├── load-shape-library-items.js
| ├── render-shape-library-items.js
| ├── shapes-search.js
| ├── shapes-panel-splitter.js
| ├── drag-shape-library-item.js
| └── shape-library-drag-payload.js
└── csharp/
    ├── ShapeCategoryDto.cs
    ├── ShapeLibraryItemDto.cs
    └── ShapesPanelModel.cs
└── assets/
    └── shape-library/
```

Milestone 2 — DiagramCanvas Developer Plan

Overview

Milestone 2 defines the foundational behavior of **DiagramCanvas** before advanced diagram-item milestones continue.

The purpose of this milestone is to establish one correct and durable canvas model and then make that model persistable through:

- JSON file save/load
- database save/load
- loading multiple diagrams from JSON
- loading multiple diagrams from the database

This milestone must clearly prove and preserve the relationship between:

- **World Space**
- **Viewport**
- **Screen Space**

For this project, the three concepts shall be interpreted as follows:

- **World Space** is the authoritative logical coordinate space of the diagram.
- **Viewport** is the visible window into world space.

- **Screen Space** is the display/input space used for rendering and mouse interaction.

The recommended developer thinking model for this milestone is:

World Space → Viewport → Screen Space

and for pointer input:

Screen Space → Viewport → World Space

A core architectural rule for this milestone is:

- **JavaScript** handles live rendering, coordinate conversion, pan, zoom, test-object drag, JSON file creation/opening, and client-side diagram switching.
- **C#** handles trusted validation, database save/load, multi-diagram listing from the database, and persistence orchestration.
- **C# does not need to read diagram data from a JSON file in order to save to the database.**
- **JavaScript gets the variable values from the in-memory diagram model.**
- **JavaScript saves JSON directly.**
- **JavaScript sends the committed diagram payload to C# for database save.**

Another important rule for this milestone is:

Every major function introduced in a phase should be documented using this contract structure:

- Trigger
- Input
- Validation / Preconditions
- Processing Responsibility
- Output
- State Change / Persistence Effect
- Next Possible Call(s)
- Error Output / Failure Path

Prerequisites

Mandatory existing JavaScript before starting Milestone 2

Only JavaScript files that must already exist and work correctly before this milestone starts are listed here.

Shared save prerequisite. Before starting Milestone 2, the team shall verify that save-to-json.js and save-to-db.js are already saving all committed variables approved in Milestones 1 to 1 correctly. When Milestone 2 introduces new committed variables, the developer shall extend only these same two files so

the current milestone save test checks the Milestone 2 variables, while regression validation confirms that all previous milestone variables are still saved correctly.

JavaScript file that must already exist	Description	Reason this JavaScript must already be ready and working correctly
None from previous milestones	Milestone 2 establishes the first DiagramCanvas, rendering, coordinate-conversion, and persistence-ready client baseline.	No earlier milestone JavaScript dependency is required before starting Milestone 2.

Before Milestone 2 implementation starts, the following prerequisites must be confirmed.

Required confirmations

1. Database connection must be confirmed

- the database server is reachable
- the application connection string is valid
- the application can successfully open a database connection
- read and write permissions are available for the required schema

Minimum readiness rule

Milestone 2 should not be considered ready to begin until the team confirms at least this:

- **database is connected**

DiagramCanvas variables for Milestone 2

Only variables that shall be treated as committed and persisted in both **JSON** and **database** are included here.

Variable name	Description	Recommended values
CanvasID	Unique identity of the DiagramCanvas.	GUID/UUID string
CanvasName	Human-readable name of the canvas.	MainCanvas or business-defined name
BackgroundColor	Background color of the canvas.	Hex color such as #FFFFFF
CoordinateSystemType	Defines the coordinate-system model used by the canvas.	CenterBasedWorld
OriginDefinition	Defines where the logical origin is defined.	Center
AxisOrientationX	Defines the positive direction of the X axis.	RightPositive

Variable name	Description	Recommended values
AxisOrientationY	Defines the positive direction of the Y axis.	UpPositive
AxisOrientationZ	Defines the positive direction or interpretation of the Z axis.	FrontPositive
IsInfiniteX	Indicates whether the canvas is logically unbounded in X.	True
IsInfiniteY	Indicates whether the canvas is logically unbounded in Y.	True
IsInfiniteZ	Indicates whether the canvas is logically unbounded in Z.	True
ViewportCenterX	Current viewport center X in world space.	0 by default
ViewportCenterY	Current viewport center Y in world space.	0 by default
ViewportWidth	Width of visible world-space area.	Positive numeric value such as 1000
ViewportHeight	Height of visible world-space area.	Positive numeric value such as 800
ZoomScale	Current zoom factor of the viewport.	Positive numeric value such as 1.0
GridVisible	Indicates whether grid is shown.	True
GridColor	Grid line color.	Hex color such as #D0D0D0
GridSpacingX	Horizontal spacing between grid lines.	Positive numeric value such as 20
GridSpacingY	Vertical spacing between grid lines.	Positive numeric value such as 20
ShowOriginMarker	Indicates whether origin marker is shown.	True
ShowAxes	Indicates whether axes are shown.	True
PanEnabled	Indicates whether panning is allowed.	True
ZoomEnabled	Indicates whether zooming is allowed.	True

Canonical persistence rule

The variables listed above are the stable Milestone 2 DiagramCanvas variables and shall be:

- saved to JSON
- saved to the database
- loaded from JSON
- loaded from the database
- used to rebuild the same DiagramCanvas state on reopen

Recommended database table structure

Logical database tables

Database table	Purpose
Diagrams	Main diagram identity and summary table
DiagramCanvas	Current persistent DiagramCanvas configuration/state
DiagramPayloads	Optional versioned full payload snapshot table

Table details

Table — Diagrams

Column	Type	Purpose
DiagramID	uniqueidentifier or varchar(64)	Permanent identity of the diagram
DiagramName	nvarchar(200)	Human-readable diagram name
DiagramVersion	int	Current committed version
CanvasID	uniqueidentifier or varchar(64)	Linked canvas identity
CreatedAt	datetime	Creation timestamp
UpdatedAt	datetime	Last update timestamp
IsDeleted	bit	Optional soft-delete flag
CreatedBy	nvarchar(200) nullable	Optional creator identity
UpdatedBy	nvarchar(200) nullable	Optional updater identity

Table — DiagramCanvas

Column	Type	Purpose
CanvasID	uniqueidentifier or varchar(64)	Primary key for canvas
DiagramID	uniqueidentifier or varchar(64)	Foreign key to Diagrams
CanvasName	nvarchar(200)	Human-readable canvas name
BackgroundColor	nvarchar(50)	Canvas background color
CoordinateSystemType	nvarchar(100)	Example: CenterBasedWorld
OriginDefinition	nvarchar(100)	Example: Center
AxisOrientationX	nvarchar(100)	Example: RightPositive
AxisOrientationY	nvarchar(100)	Example: UpPositive
AxisOrientationZ	nvarchar(100)	Example: FrontPositive
IsInfiniteX	bit	Infinite X flag
IsInfiniteY	bit	Infinite Y flag
IsInfiniteZ	bit	Infinite Z flag
ViewportCenterX	float or decimal(18,6)	Viewport center X
ViewportCenterY	float or decimal(18,6)	Viewport center Y
ViewportWidth	float or decimal(18,6)	Viewport width
ViewportHeight	float or decimal(18,6)	Viewport height
ZoomScale	float or decimal(18,6)	Zoom factor
GridVisible	bit	Grid visibility
GridColor	nvarchar(50)	Grid color
GridSpacingX	float or decimal(18,6)	Grid spacing X
GridSpacingY	float or decimal(18,6)	Grid spacing Y
ShowOriginMarker	bit	Origin marker visibility
ShowAxes	bit	Axes visibility
PanEnabled	bit	Panning enabled flag

Column	Type	Purpose
ZoomEnabled	bit	Zoom enabled flag

Table — DiagramPayloads optional but recommended

Column	Type	Purpose
PayloadID	uniqueidentifier or varchar(64)	Primary key
DiagramID	uniqueidentifier or varchar(64)	Foreign key to Diagrams
DiagramVersion	int	Version of the saved payload
PayloadJson	nvarchar(max) or JSON-native type	Full committed diagram payload snapshot
SavedAt	datetime	Save timestamp

Recommended relationship rule

- Diagrams = summary and identity
- DiagramCanvas = current persistent canvas state
- DiagramPayloads = optional versioned full payload snapshot

Recommended primary implementation

For Milestone 2, the minimum practical database implementation is:

- Diagrams
- DiagramCanvas

Add DiagramPayloads for easier audit, version snapshotting, or full payload restore later.

Recommended files for Milestone 2

The table below includes CSS, JS, and C# files. The trigger column now states the explicit trigger and the practical call count for one action or one request.

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
base.css	CSS	Base page styling and reset rules.	Page load, and whenever the owning component visual state is redrawn. Call count: stylesheet parsed once per page load; browser reapplies styles automatically.	Global HTML structure, root theme values	Main HTML shell exists	Apply baseline layout, reset spacing, standardize typography and root sizing	Consistent visual baseline	No persistence effect	Browser layout/render	Broken layout if selectors mismatch

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
canvas.css	CSS	DiagramCanvas contains styling.	Page load, and whenever the owning component visual state is redrawn. Call count: stylesheet parsed once per page load; browser reapplies styles automatically.	Canvas container classes/IDs	Canvas container exists	Style canvas region, dimensions, positioning, overflow, borders/background layer support	Correct visible canvas region	No persistence effect	render-canvas.js	Canvas may render with incorrect size/layout

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
controls.css	CSS	Styling for control buttons and toolbar.	Page load, and whenever the owning component visual state is redrawn. Call count: stylesheet parsed once per page load; browser reapplies styles automatically.	Toolbar DOM structure	Controls container exists	Style buttons, zoom controls, save/open controls, list controls	Readable controls UI	No persistence effect	User interaction via JS modules	Controls may appear misaligned or inconsistent

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
debug-panel.css	CSS	Styling for debug/status panel.	Page load, and whenever the owning component visual state is redrawn. Call count: stylesheet parsed once per page load; browser reapplies styles automatically.	Debug panel DOM structure	Debug panel exists	Style debug rows, labels, values, spacing	Readable diagnostic panel	No persistence effect	debug-panel.js	Debug panel may be hard to read

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
diagram-list.css	CSS	Styling for single/multi-diagram list UI.	Page load, and whenever the owning component visual state is redrawn. Call count: stylesheet parsed once per page load; browser reapplies styles automatically.	Diagram list DOM structure	Diagram list container exists	Style list rows, selected states, hover states, section layout	Readable selectable diagram list	No persistence effect	multi-diagram-loader.js	Diagram list may be visually unclear
main.js	JS	Starts the application and wires modules together.	Page load after DOM is ready. Call count: once per page load.	Module references, initial DOM readiness	DOM loaded, required modules available	Initialize app, bootstrap canvas state, attach handlers	Application startup completed	Initializes client-side state	loadCanvasConfig, initializeCanvasState, renderCanvas	Startup failure, controlled initialization stop

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
canvas-config.js	JS	Holds default canvas configuration values.	App startup or settings reset. Call count: once per initialization run unless configuration is reloaded.	Static config definitions	Default values exist and are valid	Provide default DiagramCanvas values	Config object	No immediate persistence; source for initial state	canvas-state.js, initializeCanvasState	Invalid config object returned
canvas-state.js	JS	Holds live state of the active diagram.	App startup, load action, or state mutation request. Call count: as needed per state initialization or state update.	Default config, loaded diagram data, UI actions	Required canvas fields available	Maintain in-memory active diagram/canvas state	Updated current state object	Client-side active state changes; persisted only when save is called	render-canvas.js, save/load modules	Invalid or incomplete state object

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
DiagramCanvas.js	JS	Defines the client-side DiagramCanvas object / model.	App startup or diagram load. Call count: once per active canvas model build or rebuild.	Canvas values	Required fields exist	Normalize/build DiagramCanvas model	Structured DiagramCanvas object	In-memory model creation/update	canvas-state.js, save modules	Invalid model creation

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
render-canvas.js	JS	Main DiagramCanvas renders coordinator. It also works with the refresh-rate detector so the client can estimate active screen refresh behavior (Hz) and choose a safe render/update cadence.	App startup, canvas state change, pan, zoom, move, resize, or diagram switch. Call count: once per processed render-frame update; active frame cadence follows request AnimationFrame and may be sampled to estimate screen refresh rate (Hz).	Active canvas state; visual layers; optional estimated refresh-rate profile	Canvas state is valid; visual layers are initialized	Coordinate full render flow for background, grid, axes, origin, item layers, and debug/UI refresh hooks	Full canvas redraw	No direct persistence effect	render-background.js, render-grid.js, render-axes.js, render-origin.js, render-image.js	Render stops safely on invalid state

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
render-background.js	JS	Renders canvas background.	Owning component render request. Call count: once per render update of the owning component.	Background color, canvas dimensions	Valid color and target canvas exist	Draw/apply background styling	Visible background	No persistence effect	render-canvas.js completion	Background omitted or defaulted
render-grid.js	JS	Renders canvas grid.	Owning component render request. Call count: once per render update of the owning component.	Grid settings, viewport, zoom	Grid settings valid	Draw grid lines/steps according to current view	Visible or hidden grid	No persistence effect	render-canvas.js completion	Grid skipped if invalid config

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
render-axes.js	JS	Renders axes.	Owning component render request. Call count: once per render update of the owning component.	Axes flag, coordinate system, viewport	Coordinate system valid	Draw X/Y axes based on current world/view mapping	Visible axes when enabled	No persistence effect	render-canvas.js completion	Axes hidden or skipped on error
render-origin.js	JS	Renders origin marker.	Owning component render request. Call count: once per render update of the owning component.	Origin flag, coordinate conversions	Origin render enabled and conversion valid	Draw origin marker in correct screen location	Visible origin marker when enabled	No persistence effect	render-canvas.js completion	Origin marker skipped on conversion failure

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
render-image.js	JS	Renders the Milestone 2 test image/object.	Owning component render request. Call count: once per render update of the owning component.	Test object world position, canvas state	Test object exists	Convert object position and render it	Visible test object	No persistence unless diagram save is called	render-canvas.js completion	Test object hidden/skipped
world-to-screen.js	JS	Converts world coordinates to screen coordinates.	Rendering or pointer-conversion request. Call count: zero to many times per render/update cycle, depending on the number of points that must be converted.	World point, viewport, zoom	Valid viewport and zoom	Apply world→view→screen conversion	Screen-space coordinates	No persistence effect	render-*, debug-panel.js	Return failure or null conversion result

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
screen-to-world.js	JS	Converts screen coordinates to world coordinates.	Rendering or pointer-conversion request. Call count: zero to many times per render/update cycle, depending on the number of points that must be converted.	Screen point, viewport, zoom	Valid viewport and zoom	Apply screen→view→world conversion	World-space coordinates	No persistence effect	drag-image.js, pan.js	Conversion failure, ignore action safely
viewport-math.js	JS	Handles viewport-related calculations.	Pan, zoom, or viewport rebuild request. Call count: as needed per viewport update.	Viewport values, pan delta, zoom changes	Current viewport valid	Compute updated viewport state	New viewport values	Changes in-memory state only until save	canvas-state.js, render-canvas.js	Reject invalid viewport result

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
pan.js	JS	Handles panning behavior.	Left Click Down on canvas background, followed by processed pan updates while the button remains active. Call count: once per processed pan update until Left Click Up.	Mouse movement, current viewport, PanEnabled	Pan enabled, background hit valid	Compute new viewport center from drag delta	Updated viewport center	Changes active client state; persisted only on save	viewport-math.js, render-canvas.js, debug-panel.js	Ignore pan or restore previous valid viewport

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
zoom.js	JS	Handles zoom behavior.	Ctrl + mouse wheel or zoom control command. Call count: once per wheel step or zoom control action.	Zoom intent, current zoom, ZoomEnabled	Zoom enabled, valid current zoom	Compute new zoom scale and possibly viewport adjustment	Updated ZoomScale	Changes active client state; persisted only on save	viewport-math.js, render-canvas.js, debug-panel.js	Reject invalid zoom and keep prior value

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
drag-image.js	JS	Handles dragging of the test object.	Move action request. Call count: once per processed move update while the move action remains active; move-validation cadence is limited by MaxMoveValidationRate rather than raw mouse hardware reports.	Mouse events, selected test object, conversion results	Test object hit valid, conversion valid	Update test object world position during drag	Updated test object position	Changes client-side active state; persisted only if diagram save includes test object	screen-to-world.js, render-canvas.js, debug-panel.js	Cancel drag or preserve last valid position

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
debug-panel.js	JS	Displays debug information.	Initial render, state change Call count: as needed by the owning action.	Canvas values, viewport values, test object values	Debug panel exists	Render diagnostic values clearly	Visible debug/status panel	No persistence effect	Render completion	Controlled debug display failure
build-diagram-json.js	JS	Builds committed JSON object directly from in-memory diagram.	Build/serialize request from the owning action, usually save or create. Call count: once per build action.	Active diagram model and committed canvas values	Active diagram valid, required persisted fields exist	Extract committed values and build payload object	JSON-ready diagram object	No file/DB change yet; prepares persistence payload	save-diagram-json.js, diagram-api.js	Save preparation failure
save-diagram-json.js	JS	Saves a JSON file.	User chooses the owning save command. Call count: once per save action.	JSON-ready diagram object, file name	Diagram object valid	Serialize and trigger browser file download	Downloadable JSON file	Creates file output only	User download completion	File generation/download failure

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
diagram-api.js	JS	Calls save/load/list APIs.	JavaScript chooses the backend persistence path. Call count: once per API request.	Diagram payload, Diagram ID, list request	API endpoint available, request data valid	Send HTTP requests and handle responses	API response objects	No direct DB change by itself; initiates server persistence	DiagramCanvasController.cs endpoints, applyLoadedDiagramData	API request failure, response validation failure

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
detect-render-rate.js	JS	Estimates active screen refresh rate (Hz) from requestAnimationFrame timing and can optionally cache a lightweight client profile locally for later startup reuse.	App startup, performance-settings reset, or explicit client-profile refresh command. Call count: normally once per page load or once per explicit refresh run.	requestAnimationFrame timestamp; optional local client-profile storage key	Browser supports requestAnimationFrame; tab is active long enough to collect samples	Measure frame intervals, estimate refresh rate (Hz), derive average frame interval (ms), and expose a safe input for move/render throttling rules.	Estimated refresh rate profile for the active client	Optional local client-profile update only; no database persistence required	render-canvas.js; move-validation scheduler; optional local-profile save helper	Fallback to default render assumptions if timing samples are insufficient

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
DiagramCanvasDto.cs	C#	DTO used by API layer for diagram/canvas data.	Created or mapped during an API save/load/list path. Call count: once per request/response mapping that uses this type.	Request payload from JS or repository output	Required fields exist, model binding succeeded	Carry transferable trusted data between layers	DTO object	No direct persistence effect	DiagramCanvasService.cs	Model-binding or validation failure
DiagramCanvasModel.cs	C#	Server-side model for canvas data.	Created or mapped during an API save/load/list path. Call count: once per request/response mapping that uses this type.	DTO or repository data	Valid mapped fields	Represent trusted server-side canvas/diagram model	Typed server-side model	No direct persistence effect	DiagramCanvasService.cs, DiagramCanvasRepository.cs	Mapping/model construction failure

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
DiagramSummaryDto.cs	C#	Summary DTO for multi-diagram list screens.	Created or mapped during an API save/load/list path. Call count: once per request/response mapping that uses this type.	DB result rows	Required summary fields exist	Map lightweight list data for client consumption	Diagram summary collection	No direct persistence effect	DiagramCanvasController.ListDiagrams	Summary mapping failure

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
DiagramCanvasController.cs	C#	API controller for save/load/list actions.	Only when the DB/API persistence path is enabled and JavaScript sends an HTTP save/load request. Call count: once per API request.	Request body, route/query params	Request valid, auth/routing satisfied	Route request to service layer and return response	HTTP/API response	No direct DB change except through service/repository flow	DiagramCanvasService.cs	Return validation/HTTP error response

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
DiagramCanvasService.cs	C#	Trusted validation and persistence orchestration layer.	Only when the DB/API persistence path is enabled and the controller routes a save/load request. Call count: once per routed service request.	DTOs, request context	DTO valid, business rules satisfied	Validate, map, orchestrate save/load/list flow	Save/load/list result object	Triggers trusted persistence through repository	DiagramCanvasRepository.cs	Return controlled service failure

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
DiagramCanvasRepository.cs	C#	Database access layer.	Only when the DB/API persistence path is enabled and the service performs a DB operation. Call count: once per DB save/load/list operation step.	Persistence model, Diagram ID, list criteria	DB connected, SQL/data access ready	Insert, update, load, and list diagram records	DB rows/result objects	Direct database persistence effect	Service return flow	DB read/write failure

Important rules

1. **World Space is authoritative.**
2. **Viewport is view state, not item geometry.**
3. **Screen Space is input/render space only.**
4. **JavaScript gets variable values from the in-memory diagram model.**
5. **JavaScript saves JSON directly from the in-memory diagram model.**
6. **JavaScript sends the committed diagram payload to C# for DB save.**
7. **C# validates and saves to the database.**
8. **C# does not need to read diagram data from a JSON file in order to save to the database.**
9. **Only committed canvas variables shall be persisted.**
10. **Temporary runtime-only values shall not be persisted.**
11. **Only one diagram is active in DiagramCanvas at a time in this milestone.**

12. Multi-diagram loading means collection/list/select/open behavior, not merge behavior.

Mathematical Calculation Rules for Milestone 2

Milestone 2 calculations define the DiagramCanvas coordinate foundation. All persistent geometry shall use world space. Viewport and screen-space values are view/input helpers only.

Primary JavaScript files for this calculation scope: world-to-screen.js, screen-to-world.js, viewport-math.js, pan.js, zoom.js, drag-image.js, detect-render-rate.js.

Viewport boundaries

$\text{ViewportLeftX} = \text{ViewportCenterX} - (\text{ViewportWidth} / 2).$

$\text{ViewportRightX} = \text{ViewportCenterX} + (\text{ViewportWidth} / 2).$

$\text{ViewportTopY} = \text{ViewportCenterY} + (\text{ViewportHeight} / 2).$

$\text{ViewportBottomY} = \text{ViewportCenterY} - (\text{ViewportHeight} / 2).$

Coordinate conversion

$\text{ScreenX} = ((\text{WorldX} - \text{ViewportLeftX}) / \text{ViewportWidth}) * \text{ScreenWidthPx}.$

$\text{ScreenY} = \text{ScreenHeightPx} - (((\text{WorldY} - \text{ViewportBottomY}) / \text{ViewportHeight}) * \text{ScreenHeightPx}).$

$\text{WorldX} = \text{ViewportLeftX} + (\text{ScreenX} / \text{ScreenWidthPx}) * \text{ViewportWidth}.$

$\text{WorldY} = \text{ViewportBottomY} + ((\text{ScreenHeightPx} - \text{ScreenY}) / \text{ScreenHeightPx}) * \text{ViewportHeight}.$

Pan and zoom

Pan updates ViewportCenterX and ViewportCenterY only. Pan shall not change item CenterX or CenterY.

$\text{ViewportWidth} = \text{BaseViewportWidth} / \text{ZoomScale}.$ $\text{ViewportHeight} = \text{BaseViewportHeight} / \text{ZoomScale}.$

$\text{EffectiveMoveValidationRate} = \min(\text{EstimatedRenderHz}, \text{MaxMoveValidationRate}).$

Phase 2.1 — Define DiagramCanvas variables and bind them to visible behavior

For more information, please refer to section [Coordinate conversion calculations \(place in this document\)](#) and section [Render-rate detection calculation \(place in this document\)](#).

Primary JavaScript files for this phase: world-to-screen.js; screen-to-world.js; viewport-math.js; detect-render-rate.js.

Goal

Define the persistent DiagramCanvas variables, expose them in JavaScript for live visual behavior, and mirror them in C# for trusted save/load and database persistence.

Scope

This phase is about:

- defining the DiagramCanvas variable model
- creating the initial JavaScript canvas state
- rendering background, grid, axes, and origin marker
- showing a debug panel
- confirming which values are committed and persisted

Variables to define

At minimum:

- CanvasID
- CanvasName
- BackgroundColor
- CoordinateSystemType
- OriginDefinition
- AxisOrientationX
- AxisOrientationY
- AxisOrientationZ
- IsInfiniteX
- IsInfiniteY
- IsInfiniteZ
- ViewportCenterX
- ViewportCenterY
- ViewportWidth
- ViewportHeight
- ZoomScale
- GridVisible
- GridColor
- GridSpacingX
- GridSpacingY
- ShowOriginMarker
- ShowAxes

- PanEnabled
- ZoomEnabled

Required visible behaviors

- changing BackgroundColor changes the background
- changing GridVisible shows or hides the grid
- changing GridColor changes the grid color
- changing GridSpacingX and GridSpacingY changes grid density
- changing ShowOriginMarker shows or hides the origin marker
- changing ShowAxes shows or hides axes
- changing viewport values changes visible world area
- changing ZoomScale changes visible magnification

Function contract deliverables

- initializeCanvasState
- loadCanvasConfig
- renderCanvas
- renderBackground
- renderGrid
- renderAxes
- renderOrigin
- updateDebugPanel

C# structures:

- DiagramCanvasDto
- DiagramCanvasModel

Phase 2.1 responsibility split

JavaScript

- initialize and render canvas state
- display live visual results
- display debug information

C#

- define DTO/model structure

- prepare validation and persistence structure
- no live rendering logic

Output

- persisted variable list
- initial live DiagramCanvas
- visible background/grid/axes/origin
- debug panel

Validation / Preconditions

- canvas initializes successfully
- zoom value is valid
- viewport dimensions are positive
- coordinate interpretation is stable

Error / failure expectations

- invalid zoom value must be rejected
- invalid viewport values must fail safely
- missing required variable must block valid initialization

Success criteria

- all variables are clearly defined
- visual changes match variable changes
- canvas behavior is consistent

Phase 2.2 — Drag a simple picture and validate world space, viewport, and screen space

For more information, please refer to section [Coordinate conversion calculations](#) ([place in this document](#)) and section [Render-rate detection calculation](#) ([place in this document](#)).

Primary JavaScript files for this phase: world-to-screen.js; screen-to-world.js; viewport-math.js; detect-render-rate.js.

Goal

Use one test image/object to prove correct drag, pan, and zoom behavior while preserving the correct relationship between world space, viewport, and screen space.

Scope

This phase is about:

- rendering one test object
- dragging it
- panning the viewport
- zooming the viewport
- observing the effect on world, viewport, and screen behavior

Required actions

- place a simple image on the canvas
- drag the image
- pan the viewport
- zoom in/out
- inspect resulting behavior

Required observations

During drag

- object world position changes
- object screen position changes
- viewport stays the same

During pan

- viewport changes
- object world position stays the same
- object screen position changes

During zoom

- zoom changes
- viewport visible extent changes
- object world position stays the same
- object screen behavior changes accordingly

Function contract deliverables

- renderTestImage
- hitTestTestImage
- beginImageDrag

- updateImageDrag
- endImageDrag
- beginPan
- updatePan
- endPan
- applyZoom
- worldToScreen
- screenToWorld
- updateDebugPanel

Event-to-function routing

Drag

mousedown on test image
 → hitTestTestImage
 → beginImageDrag
 → mousemove
 → updateImageDrag
 → renderCanvas

Pan

mousedown on canvas background
 → beginPan
 → mousemove
 → updatePan
 → renderCanvas

Zoom

Ctrl + mouse wheel
 → applyZoom
 → renderCanvas

Output

- draggable test object
- working pan
- working zoom
- visible proof of correct coordinate behavior

Validation / Preconditions

- canvas already initialized
- test object exists

- conversion functions work
- pan and zoom are enabled

Error / failure expectations

- drag must not start on invalid hit
- invalid coordinate conversion must stop movement safely
- invalid zoom input must preserve previous valid zoom

Success criteria

- drag works smoothly
- pan changes viewport only
- zoom changes visible scale only
- coordinate model is clearly proven correct

Phase 2.3 — Save the active diagram to JSON and reopen it

Exact variables to save in this phase

Exact committed Milestone 2 save variables: CanvasID, CanvasName, BackgroundColor, CoordinateSystemType, OriginDefinition, AxisOrientationX, AxisOrientationY, AxisOrientationZ, IsInfiniteX, IsInfiniteY, IsInfiniteZ, ViewportCenterX, ViewportCenterY, ViewportWidth, ViewportHeight, ZoomScale, GridVisible, GridColor, GridSpacingX, GridSpacingY, ShowOriginMarker, ShowAxes, PanEnabled, ZoomEnabled.

Save regression rule for this phase

Milestone 2 is the first committed persistence milestone. No earlier milestone save-variable regression set exists, but the phase still fails if save-to-json.js writes temporary runtime-only values as if they were committed diagram truth.

Save test criteria for this phase

Verify that save-to-json.js writes every listed Milestone 2 variable into the committed JSON payload, that reopen restores the same DiagramCanvas state, and that temporary runtime-only values are excluded.

Goal

Save the active diagram to a JSON file and reopen it later so the active DiagramCanvas is rebuilt correctly.

Scope

This phase is about:

- getting committed values from the in-memory diagram model in JavaScript
- building a JSON object
- saving a JSON file

- reopening a JSON file
- rebuilding the active diagram from JSON

Important architecture rule

For the JSON path:

- JavaScript gets the values directly from the active diagram model
- JavaScript builds the JSON object
- JavaScript saves the JSON file
- JavaScript opens and parses the JSON file
- JavaScript rebuilds the diagram from JSON

Variables to define

- DiagramID
- DiagramVersion
- DiagramName
- DiagramJsonObject
- DiagramJsonText
- SavedJsonFileName
- OpenedJsonFileName

Function contract deliverables

- buildDiagramJsonObject
- serializeDiagramToJson
- saveDiagramJsonFile
- openDiagramJsonFile
- parseDiagramJsonFile
- validateDiagramJsonStructure
- rebuildDiagramFromJson
- clearCurrentDiagramBeforeOpen

Output

- valid JSON save path
- valid JSON reopen path
- correct rebuild of the active DiagramCanvas

Validation / Preconditions

- active diagram is valid
- required persisted canvas values exist
- JSON structure is valid

Error / failure expectations

- invalid JSON must not load
- incomplete JSON must fail safely
- save failure must not produce false success

Success criteria

- JSON file is created correctly
- reopened JSON reconstructs the same canvas state

Phase 2.4 — Save the active diagram to the database and reopen it

Exact variables to save in this phase

Exact committed Milestone 2 save variables: CanvasID, CanvasName, BackgroundColor, CoordinateSystemType, OriginDefinition, AxisOrientationX, AxisOrientationY, AxisOrientationZ, IsInfiniteX, IsInfiniteY, IsInfiniteZ, ViewportCenterX, ViewportCenterY, ViewportWidth, ViewportHeight, ZoomScale, GridVisible, GridColor, GridSpacingX, GridSpacingY, ShowOriginMarker, ShowAxes, PanEnabled, ZoomEnabled.

Save regression rule for this phase

Milestone 2 is the first committed persistence milestone. No earlier milestone save-variable regression set exists, but the phase still fails if save-to-db.js writes temporary runtime-only values as if they were committed diagram truth.

Save test criteria for this phase

Verify that save-to-db.js writes every listed Milestone 2 variable to the trusted database model, that load-by-DiagramID restores the same DiagramCanvas state, and that temporary runtime-only values are excluded.

Goal

Save the active diagram to the database and reopen it later by DiagramID.

Scope

This phase is about:

- JavaScript building a committed save payload from the active in-memory diagram model
- sending that payload to C#
- C# validating it

- C# writing it to the database
- reloading the same diagram from the database

Important architecture rule

For the database path:

- JavaScript gets committed values from the active diagram model
- JavaScript builds the save payload
- JavaScript sends the payload to the API
- C# validates the payload
- C# maps it to DB persistence structures
- C# writes to the database

Variables to define

- DiagramID
- DiagramVersion
- DiagramSavePayload
- LoadedDiagramID
- LoadedDiagramData
- LastDatabaseSaveResult

Function contract deliverables

JavaScript

- buildDiagramSavePayload
- callSaveDiagramApi
- callLoadDiagramApi
- applyLoadedDiagramData

C#

- DiagramCanvasController.SaveDiagram
- DiagramCanvasController.LoadDiagram
- DiagramCanvasService.SaveDiagram
- DiagramCanvasService.LoadDiagram
- DiagramCanvasRepository.SaveDiagram
- DiagramCanvasRepository.LoadDiagramById

Output

- working DB save
- working DB reopen
- trusted persistence of committed canvas state

Validation / Preconditions

- database connection is confirmed
- API is reachable
- payload is valid
- DiagramID is valid

Error / failure expectations

- DB save failure must be reported clearly
- invalid ID must fail safely
- incomplete DB response must not partially rebuild a corrupted canvas

Success criteria

- save to DB works
- reopen by DiagramID works
- loaded diagram matches the saved state

Phase 2.5 — Load multiple diagrams from JSON

Goal

Load multiple diagram JSON files, validate them as a collection, display them as a selectable list, and open one selected diagram into the active DiagramCanvas.

Scope

This phase is about:

- selecting multiple JSON files
- parsing and validating them
- building a diagram list
- selecting one diagram as active
- switching between loaded diagrams

Important rule

Multiple JSON diagrams are treated as a collection, not a merged single diagram.

Variables to define

- LoadedJsonDiagramCollection
- LoadedJsonDiagramCount
- ActiveDiagramID
- ActiveDiagramName
- SelectedDiagramSummary
- DiagramListVisible

Function contract deliverables

- openMultipleDiagramJsonFiles
- parseMultipleDiagramJsonFiles
- validateDiagramJsonCollection
- buildDiagramSummaryList
- renderDiagramList
- setActiveDiagramFromJsonCollection
- applyLoadedDiagramData

Output

- multiple JSON diagrams can be opened
- diagram list UI exists
- one selected diagram becomes active

Validation / Preconditions

- each JSON file must be valid
- required persisted values must exist
- active diagram switching must be clean

Error / failure expectations

- invalid files must be reported clearly
- partial success must be shown clearly
- failed activation must preserve previous active diagram

Success criteria

- multiple JSON diagrams load correctly

- user can switch between them reliably

Phase 2.6 — Load multiple diagrams from the database

Goal

Retrieve multiple saved diagrams from the database, display them as a selectable list, and load the selected one into the active DiagramCanvas.

Scope

This phase is about:

- listing diagrams from DB
- showing diagram summaries
- selecting one diagram
- loading it into the active canvas
- switching between saved diagrams

Important rule

Multiple DB diagrams are treated as a collection, not a merged canvas.

Variables to define

- DatabaseDiagramSummaryList
- DatabaseDiagramCount
- SelectedDatabaseDiagramID
- SelectedDatabaseDiagramSummary
- ActiveDiagramID
- DiagramListVisible

Function contract deliverables

JavaScript

- callListDiagramsApi
- renderDiagramList
- selectDatabaseDiagram
- callLoadDiagramApi
- applyLoadedDiagramData

C#

- DiagramCanvasController.ListDiagrams
- DiagramCanvasService.ListDiagrams
- DiagramCanvasRepository.ListDiagrams
- DiagramSummaryDto

Output

- multiple DB diagrams can be listed
- one selected DB diagram can become active
- active diagram switching works

Validation / Preconditions

- DB is connected
- list API works
- summary rows contain valid DiagramID

Error / failure expectations

- list failure must be reported clearly
- invalid summary rows must not be selectable
- failed load must preserve current active diagram

Success criteria

- multiple DB diagrams are listed correctly
- selected saved diagram loads correctly
- switching works reliably

Final result after Milestone 2

At the end of Milestone 2, the developer should have:

- a clearly defined persistent DiagramCanvas model
- correct visual canvas behavior
- correct proof of world space, viewport, and screen space
- working pan and zoom
- working test-object drag
- direct JSON save/load handled by JavaScript
- trusted DB save/load handled through C#

- multiple-diagram loading from JSON
- multiple-diagram loading from DB
- one active diagram loaded into DiagramCanvas at a time

The developer should also have a clear separation between:

- canonical persisted canvas variables
- committed diagram state
- temporary runtime-only state

Recommended implementation order

Use this order:

1. Define persistent DiagramCanvas variables and render them visibly
2. Prove drag, pan, and zoom
3. Save/reopen active diagram in JSON
4. Save/reopen active diagram in DB
5. Load multiple diagrams from JSON
6. Load multiple diagrams from DB

Milestone 3 — Rectangle and Circle Definition Developer Plan

Overview

Milestone 3 defines the formal Rectangle, Circle, and Container host-geometry rules that all later interaction, connection, containment, SVG, and persistence milestones depend on.

The purpose of this milestone is to establish one correct and durable shape-definition model for both item types and then make that model persistable through:

JSON file save/load

database save/load

shape reconstruction from JSON

shape reconstruction from the database

This milestone must clearly define and preserve the relationship between:

Rectangle canonical stored variables

Circle canonical stored variables

default values

derived geometry rules

persisted versus derived values

For this project, the source-item model shall be interpreted as follows:

Devices are non-container items. A Device shall use Rectangle or Circle as its authoritative host geometry depending on rendering rules.

Containers are container items. A Container shall use Rectangle Container geometry and may contain Devices or nested Containers.

The recommended developer thinking model for this milestone is:

Canonical stored variables → Derived geometry → Rendered result

and for persistence:

Canonical stored variables → JSON/API payload → trusted save/load

A core architectural rule for this milestone is:

JavaScript handles in-memory shape definition, default creation, client-side reconstruction, JSON file creation/opening, and visible rendering.

C# handles trusted validation, database save/load, and persistence orchestration.

Top-level Devices and top-level Containers belong directly to DiagramCanvas root. Their ParentContainerID shall be NULL.

JavaScript gets the variable values from the committed in-memory shape model.

JavaScript saves JSON directly.

JavaScript sends the committed shape-definition payload to C# for database save.

Another important rule for this milestone is:

Every major function introduced in a phase should be documented using this contract structure:

Trigger

Input

Validation / Preconditions

Processing Responsibility

Output

State Change / Persistence Effect

Next Possible Call(s)

Error Output / Failure Path

Prerequisites

Mandatory existing JavaScript before starting Milestone 3

Only JavaScript files that must already exist and work correctly before this milestone starts are listed here.

Shared save prerequisite. Before starting Milestone 3, the team shall verify that save-to-json.js and save-to-db.js are already saving all committed variables approved in Milestones 1 to 2 correctly. When Milestone 3 introduces new committed variables, the developer shall extend only these same two files so the current milestone save test checks the Milestone 3 variables, while regression validation confirms that all previous milestone variables are still saved correctly.

JavaScript file that must already exist	Description	Reason this JavaScript must already be ready and working correctly
render-canvas.js	Renders the active DiagramCanvas and visible item result.	Milestone 3 shape definitions must be proven on a working canvas render path.
world-to-screen.js	Converts world coordinates to screen coordinates.	Rectangle and Circle definitions must be shown correctly in screen space.
screen-to-world.js	Converts screen coordinates to world coordinates.	Reverse conversion must already be correct before later interaction and validation are layered on top.
canvas-state.js	Maintains the active diagram state in memory.	Shape-definition modules need a stable place to hold committed canvas and item state.

Before Milestone 3 implementation starts, the following prerequisites must be confirmed.

Required confirmations

Milestone 2 DiagramCanvas model is already working

world-space, viewport, and screen-space conversion is already proven correct

the application can already render shapes on DiagramCanvas

the JSON save/open path is available if Phase 4.4 will be implemented immediately

database connection is confirmed if Phase 4.5 will be implemented immediately

Minimum readiness rule

Milestone 3 should not be considered ready to begin until the team confirms at least this:

Milestone 2 is complete

database is connected

Milestone 3 variables for Devices, Containers, Rectangles, and Circles

Only variables that shall be treated as committed and persisted in both JSON and database are included here.

Variable name	Description	Recommended values
DeviceID	Unique identity of the Device item.	GUID/UUID string
DeviceName	Human-readable name of the Device.	Business-defined name such as EC2_X1
DeviceType	Logical device classification used by rendering and reporting.	EC2 / NATGateway / RouteTable / Router / Switch / Lambda / Business-defined type
DeviceStatus	Current business/device state.	Active / Inactive / Planned / Error
DeviceDescription	Human-readable description of the Device.	Short business description
DeviceHostGeometryType	Host geometry used by the Device before SVG attachment.	Rectangle or Circle
DeviceParentContainerID	Immediate parent container of the Device. NULL means the Device belongs directly to DiagramCanvas root.	Valid ContainerID or NULL
ContainerID	Unique identity of the Container item.	GUID/UUID string
ContainerName	Human-readable name of the Container.	Business-defined name such as VPC_A or Region_1
ContainerType	Logical container classification.	Region / VPC / VNet / AvailabilityZone / Subnet / Business-defined container type
ContainerStatus	Current container state.	Active / Inactive / Planned / Error
ContainerDescription	Human-readable description of the Container.	Short business description
ContainerParentContainerID	Immediate parent container of the Container. NULL means the Container belongs directly to DiagramCanvas root.	Valid ContainerID or NULL
ShapeType	Identifies whether the host geometry is a rectangle or circle.	Rectangle or Circle

Variable name	Description	Recommended values
RectangleID	Unique identity of the rectangle host geometry.	GUID/UUID string
RectangleCenterX	Rectangle center X in world space.	0 by default
RectangleCenterY	Rectangle center Y in world space.	0 by default
RectangleHalfWidth	Half of the rectangle width.	Positive numeric value such as 40
RectangleHalfHeight	Half of the rectangle height.	Positive numeric value such as 25
RectangleRotationAngle	Rectangle orientation angle.	0 by default
RectangleZOrder	Rectangle stacking order.	Integer such as 0
RectangleFillType	Rectangle fill behavior.	NoFill or SolidFill
RectangleFillColor	Rectangle fill color when fill is used.	Hex color such as #FFFFFF
RectangleLineType	Rectangle border behavior.	NoLine or SolidLine
RectangleLineColor	Rectangle border color.	Hex color such as #000000
RectangleLineWidth	Rectangle border width.	Positive numeric value such as 1
RectangleHoverPaddingXRatio	Horizontal hover padding ratio for rectangle.	0.1
RectangleHoverPaddingYRatio	Vertical hover padding ratio for rectangle.	0.1
RectangleHoverPaddingColor	Rectangle hover padding color.	Transparent by default
RectangleProtectionPaddingXRatio	Horizontal protection padding ratio for rectangle.	0.2
RectangleProtectionPaddingYRatio	Vertical protection padding ratio for rectangle.	0.2
RectangleProtectionPaddingColor	Rectangle protection padding color.	Transparent by default
CircleID	Unique identity of the circle host geometry.	GUID/UUID string

Variable name	Description	Recommended values
CircleCenterX	Circle center X in world space.	0 by default
CircleCenterY	Circle center Y in world space.	0 by default
CircleRadius	Authoritative circle size value.	Positive numeric value such as 25
CircleZOrder	Circle stacking order.	Integer such as 0
CircleFillType	Circle fill behavior.	NoFill or SolidFill
CircleFillColor	Circle fill color when fill is used.	Hex color such as #FFFFFF
CircleLineType	Circle border behavior.	NoLine or SolidLine
CircleLineColor	Circle border color.	Hex color such as #000000
CircleLineWidth	Circle border width.	Positive numeric value such as 1
CircleHoverPaddingRadiusRatio	Radial hover padding ratio for circle.	0.1
CircleHoverPaddingColor	Circle hover padding color.	Transparent by default
CircleProtectionPaddingRadiusRatio	Radial protection padding ratio for circle.	0.2
CircleProtectionPaddingColor	Circle protection padding color.	Transparent by default

Canonical persistence rule

The variables listed above are the stable Milestone 3 shape-definition variables and shall be:

saved to JSON

saved to the database

loaded from JSON

loaded from the database

used to rebuild the same Rectangle and Circle state on reopen

Recommended database table structure

Logical database tables

Database table	Purpose
Diagrams	Main diagram identity and summary table

Database table	Purpose
Devices	Persistent non-container items rendered through Rectangle or Circle host geometry
Containers	Persistent container items rendered as Rectangle Containers
DiagramShapePayloads	Optional versioned full shape payload snapshot table

Table details

Table — Diagrams

Column	Type	Purpose
DiagramID	uniqueidentifier or varchar(64)	Permanent identity of the diagram
DiagramName	nvarchar(200)	Human-readable diagram name
DiagramVersion	int	Current committed version
CanvasID	uniqueidentifier or varchar(64)	Linked canvas identity
CreatedAt	datetime	Creation timestamp
UpdatedAt	datetime	Last update timestamp
IsDeleted	bit	Optional soft-delete flag
CreatedBy	nvarchar(200) nullable	Optional creator identity
UpdatedBy	nvarchar(200) nullable	Optional updater identity

Table — Devices

Column	Type	Purpose
DeviceID	uniqueidentifier or varchar(64)	Primary key for Device
DiagramID	uniqueidentifier or varchar(64)	Foreign key to Diagrams
DeviceName	nvarchar(200)	Human-readable Device name
DeviceType	nvarchar(100)	Business/device classification
DeviceStatus	nvarchar(50)	Current device state
DeviceDescription	nvarchar(500) nullable	Optional readable description

Column	Type	Purpose
HostGeometryType	nvarchar(50)	Rectangle or Circle
CenterX	float or decimal(18,6)	Device center X in world space
CenterY	float or decimal(18,6)	Device center Y in world space
HalfWidth	float or decimal(18,6) nullable	Rectangle half width when HostGeometryType = Rectangle
HalfHeight	float or decimal(18,6) nullable	Rectangle half height when HostGeometryType = Rectangle
Radius	float or decimal(18,6) nullable	Circle radius when HostGeometryType = Circle
RotationAngle	float or decimal(18,6) nullable	Rectangle rotation angle when HostGeometryType = Rectangle
ZOrder	int	Device stacking order
FillType	nvarchar(50)	Host fill type
FillColor	nvarchar(50)	Host fill color
LineType	nvarchar(50)	Host line type
LineColor	nvarchar(50)	Host line color
LineWidth	float or decimal(18,6)	Host line width
HoverPaddingXRatio	float or decimal(18,6) nullable	Rectangle hover padding X ratio
HoverPaddingYRatio	float or decimal(18,6) nullable	Rectangle hover padding Y ratio
HoverPaddingRadiusRatio	float or decimal(18,6) nullable	Circle hover padding ratio
ProtectionPaddingXRatio	float or decimal(18,6) nullable	Rectangle protection padding X ratio
ProtectionPaddingYRatio	float or decimal(18,6) nullable	Rectangle protection padding Y ratio
ProtectionPaddingRadiusRatio	float or decimal(18,6) nullable	Circle protection padding ratio
ParentContainerID	uniqueidentifier or varchar(64) nullable	Immediate parent container. NULL means the Device belongs directly to DiagramCanvas root.

Column	Type	Purpose
CreatedAt	datetime	Creation timestamp
UpdatedAt	datetime	Last update timestamp

Table — Containers

Column	Type	Purpose
ContainerID	uniqueidentifier or varchar(64)	Primary key for Container
DiagramID	uniqueidentifier or varchar(64)	Foreign key to Diagrams
ContainerName	nvarchar(200)	Human-readable Container name
ContainerType	nvarchar(100)	Business/container classification
ContainerStatus	nvarchar(50)	Current container state
ContainerDescription	nvarchar(500) nullable	Optional readable description
CenterX	float or decimal(18,6)	Container center X in world space
CenterY	float or decimal(18,6)	Container center Y in world space
HalfWidth	float or decimal(18,6)	Container half width
HalfHeight	float or decimal(18,6)	Container half height
RotationAngle	float or decimal(18,6)	Container rotation angle
ZOrder	int	Container stacking order
FillType	nvarchar(50)	Container fill type
FillColor	nvarchar(50)	Container fill color
LineType	nvarchar(50)	Container line type
LineColor	nvarchar(50)	Container line color
LineWidth	float or decimal(18,6)	Container line width
HoverPaddingXRatio	float or decimal(18,6)	Container hover padding X ratio
HoverPaddingYRatio	float or decimal(18,6)	Container hover padding Y ratio

Column	Type	Purpose
ProtectionPaddingXRatio	float or decimal(18,6)	Container protection padding X ratio
ProtectionPaddingYRatio	float or decimal(18,6)	Container protection padding Y ratio
ParentContainerID	uniqueidentifier or varchar(64) nullable	Immediate parent container. NULL means the Container belongs directly to DiagramCanvas root.
CreatedAt	datetime	Creation timestamp
UpdatedAt	datetime	Last update timestamp

Table — DiagramShapePayloads optional but recommended

Column	Type	Purpose
PayloadID	uniqueidentifier or varchar(64)	Primary key
DiagramID	uniqueidentifier or varchar(64)	Foreign key to Diagrams
DiagramVersion	int	Version of the saved payload
PayloadJson	nvarchar(max) or JSON-native type	Full committed rectangle/circle payload snapshot
SavedAt	datetime	Save timestamp

Recommended relationship rule

Devices = persistent non-container items rendered through Rectangle or Circle host geometry

Containers = persistent container items rendered as Rectangle Containers

DiagramShapePayloads = optional versioned full shape payload snapshot

DiagramShapePayloads = optional versioned full shape payload snapshot

Recommended primary implementation

For Milestone 3, the minimum practical database implementation is:

Devices

Containers

Add DiagramShapePayloads for easier audit, version snapshotting, or full shape payload restore later.

Add DiagramShapePayloads for easier audit, version snapshotting, or full payload restore later.

Recommended files for Milestone 3

The table below includes CSS, JS, and C# files. The trigger column now states the explicit trigger and the practical call count for one action or one request.

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
shape-base.css	CSS	Base shape styling and shared visual tokens.	Page load, and whenever the owning component visual state is redrawn. Call count: stylesheet parsed once per page load; browser reapplies styles automatically.	Global HTML structure, root theme values	Main HTML shell exists	Apply baseline shape-related styling and common tokens	Consistent visual baseline for shapes	No persistence effect	Browser layout/render	Broken layout if selectors mismatch

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
rectangle.css	CSS	Rectangle visual defaults and render classes.	Page load, and whenever the owning component visual state is redrawn. Call count: stylesheet parsed once per page load; browser reapplies styles automatically.	Rectangle DOM/canvas classes	Rectangle render target exists	Style rectangle stroke, fill, selection, and padding visuals	Correct visible rectangle styling	No persistence effect	render-rectangle.js	Rectangle may render with incorrect appearance

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
circle.css	CSS	Circle visual defaults and render classes.	Page load, and whenever the owning component visual state is redrawn. Call count: stylesheet parsed once per page load; browser reapplies styles automatically.	Circle DOM/canvas classes	Circle render target exists	Style circle stroke, fill, selection, and padding visuals	Correct visible circle styling	No persistence effect	render-circle.js	Circle may render with incorrect appearance

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
shape-state.js	JS	Holds live in-memory rectangle and circle definition state.	App startup, load action, or state mutation request. Call count: as needed per state initialization or state update.	Default config, loaded diagram data	Required shape fields available	Maintain active rectangle and circle state objects	Updated current shape state	Client-side active state changes; persisted only when save is called	create-shape-models.js, render-rectangle.js, render-circle.js	Invalid or incomplete state object
shape-definition-config.js	JS	Holds default rectangle and circle configuration values.	App startup or settings reset. Call count: once per initialization run unless configuration is reloaded.	Static config definitions	Default values exist and are valid	Provide canonical default values for both shapes	Config object	No immediate persistence; source for initial state	shape-state.js, create-shape-models.js	Invalid config object returned

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
create-shape-models.js	JS	Builds normalized rectangle and circle models.	App startup, load actions, new shape creation Call count: as needed by the owning action.	Rectangle values, circle values	Required canonical fields exist	Normalize and build structured shape-definition models	Structured Rectangle and Circle objects	In-memory model creation/update	shape-state.js, derive-shape-geometry.js	Invalid model creation

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
derive-shape-geometry.js	JS	Recalculates derived rectangle and circle geometry.	Canonical shape values change, shapes are loaded, or a save/rebuild path needs fresh derived geometry. Call count: once per affected shape update or reconstruction step.	Canonical shape values	Canonical values are valid	Calculate visible boundaries, hover padding, protection padding, and helper geometry	Derived geometry object	Updates derived in-memory values only	render-rectangle.js, render-circle.js, build-shape-json.js	Reject invalid geometry result

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
render-rectangle.js	JS	Renders rectangle items from canonical + derived values.	Owning component render request. Call count: once per render update of the owning component.	Active rectangle state, derived geometry	Rectangle state is valid	Draw rectangle visual result on Diagram Canvas	Visible rectangle output	No direct persistence effect	render-selection-state.js, update-debug-panel.js	Render stops safely on invalid rectangle state
render-circle.js	JS	Renders circle items from canonical + derived values.	Owning component render request. Call count: once per render update of the owning component.	Active circle state, derived geometry	Circle state is valid	Draw circle visual result on Diagram Canvas	Visible circle output	No direct persistence effect	render-selection-state.js, update-debug-panel.js	Render stops safely on invalid circle state

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
render-selection-state.js	JS	Shows active/selected state for rectangle and circle definitions.	App startup, load action, or state mutation request. Call count: as needed per state initialization or state update.	Active shape identity and visual flags	A valid active shape exists	Apply visible selected-state styling	Visible active-shape feedback	No persistence effect	render-rectangle.js, render-circle.js	Selection styling skipped safely
build-shape-json.js	JS	Builds committed JSON object directly from in-memory rectangle and circle definitions.	Build/serialize request from the owning action, usually save or create. Call count: once per build action.	Active diagram model and committed shape values	Active shape model valid, required persisted fields exist	Extract committed values and build payload object	JSON-ready shape - definition object	No file/DB change yet; prepares persistence payload	save-shape-json.js, shape-api.js	Save preparation failure

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
save-shape-json.js	JS	Saves a JSON file containing committed rectangle and circle definitions.	User chooses the owning save command. Call count: once per save action.	JSON-ready shape-definition object, file name	Shape-definition object valid	Serialize and trigger browser file download	Downloadable JSON file	Creates file output only	User download completion	File generation/download failure
open-shape-json.js	JS	Opens one or more JSON files from browser file picker.	User chooses the owning open command. Call count: once per open action.	Selected file(s)	Browser file selection succeeded	Read raw file content	Raw JSON text/file content	No persistence effect	parse-shape-json.js	Open/read failure

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
parse-shape-json.js	JS	Parses and validates shape-definition JSON files.	Raw file or payload text is available for parsing. Call count: once per opened file or payload parse request.	Raw JSON text	Non-empty valid JSON text	Parse JSON and validate required structure	Valid shape-definition object(s) or error info	No direct persistence effect	apply-loaded-shapes.js, shape-state.js	Parse/validation failure
apply-loaded-shapes.js	JS	Applies loaded rectangle and circle definitions into active canvas state.	After JSON load or DB load Call count: as needed by the owning action.	Validated loaded shape data	Loaded structure is valid	Replace current in-memory shape-definition state and trigger render	Updated active shape state	Client-side active state change s; persisted only when save is called	render-rectangle.js, render-circle.js	Activation failure preserves previous valid state

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
shape-api.js	JS	Calls save/load APIs for rectangle and circle definitions.	JavaScript chooses the backend persistence path. Call count: once per API request.	Shape payload, DiagramID	API endpoint available, request data valid	Send HTTP requests and handle responses	API response objects	No direct DB change by itself; initiates server persistence	ShapeDefinition Controller.cs endpoints, apply-loaded-shapes.js	API request failure or response validation failure
update-debug-panel.js	JS	Displays current canonical and derived rectangle/circle values.	Initial render, state changes Call count: as needed by the owning action.	Shape values, derived geometry	Debug panel exists	Render diagnostic values clearly	Visible debug/status panel	No persistence effect	Render completion	Controlled debug display failure

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
ShapeDefinitionDto.cs	C#	DTO used by API layer for rectangle/circle definition data.	Created or mapped during an API save/load/list path. Call count: once per request/response mapping that uses this type.	Request payload from JS or repository output	Required fields exist, model binding succeeded	Carry transferable trusted data between layers	DTO object	No direct persistence effect	ShapeDefinitionService.cs	Model-binding or validation failure

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
RectangleDto.cs	C#	DTO for rectangle-specific canonical values.	Created or mapped during an API save/load/list path. Call count: once per request/response mapping that uses this type.	Rectangle payload	Rectangle payload valid	Carry rectangle data between layers	Rectangle DTO object	No direct persistence effect	ShapeDefinitionService.cs	Rectangle DTO validation failure

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
CircleDto.cs	C#	DTO for circle - specific canonical values.	Created or mapped during an API save/load/list path. Call count: once per request/response mapping that uses this type.	Circle payload	Circle payload valid	Carry circle data between layers	Circle DTO object	No direct persistence effect	ShapeDefinitionService.cs	Circle DTO validation failure

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
ShapeDefinitionModel.cs	C#	Server-side model for trusted shape-definition data.	Created or mapped during an API save/load/list path. Call count: once per request/response mapping that uses this type.	DTO or repository data	Valid mapped fields	Represent trusted server-side rectangle/circle model	Typed server-side model	No direct persistence effect	ShapeDefinitionService.cs, ShapeDefinitionRepository.cs	Mapping/model construction failure

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
ShapeDefinitionController.cs	C#	API controller for shape definition save/load actions.	Only when the DB/API persistence path is enabled and JavaScript sends an HTTP save/load request. Call count: once per API request.	Request body, route/query params	Request valid, auth/routing satisfied	Route request to service layer and return response	HTTP/API response	No direct DB change except through service/repository flow	ShapeDefinitionService.cs	Return validation/HTTP error response

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
ShapeDefinitionService.cs	C#	Trusted validation and persistence orchestration layer for rectangle/circle definitions.	Only when the DB/API persistence path is enabled and the controller routes a save/load request. Call count: once per routed service request.	DTOs, request context	DTO valid, business rules satisfied	Validate, map, orchestrate save/load flow	Save/load result object	Triggers trusted persistence through repository	ShapeDefinitionRepository.cs	Return controlled service failure

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
ShapeDefinitionRepository.cs	C#	Data base access layer for rectangle/circle definition records.	Only when the DB/API persistence path is enabled and the service performs a DB operation. Call count: once per DB save/load/list operation step.	Persistence model, DiagramID	DB connected, SQL/data accesses ready	Insert, update, load, and list shape-definition records	DB rows/result objects	Direct database persistence effect	Service return flow	DB read/write failure

Important rules

Devices are non-container items and shall be rendered through Rectangle or Circle host geometry.

Containers are container items and shall be rendered as Rectangle Containers in this version.

ParentContainerID = NULL means the item belongs directly to DiagramCanvas root rather than to a visible parent container.

Canonical rectangle, circle, and container values are authoritative.

Rectangle uses center-based world coordinates with HalfWidth and HalfHeight as primary size values.

Circle uses center-based world coordinates with Radius as the primary size value.

Hover padding and protection padding configuration values are part of the committed host definitions.

Derived boundaries, helper boxes, and control-point positions are not independent stored truths.

JavaScript gets variable values from the in-memory host model.

JavaScript saves JSON directly from the in-memory host model.

JavaScript sends the committed host-definition payload to C# for DB save.

C# validates and saves to the database. Only committed host-definition values shall be persisted; temporary runtime-only interaction states shall not be persisted.

Mathematical Calculation Rules for Milestone 3

Milestone 3 calculations derive rectangle, circle, and container helper geometry from canonical stored variables. Derived boundaries, padding regions, and resize control point positions shall not be treated as independent persisted truth.

Primary JavaScript files for this calculation scope: `derive-shape-geometry.js`, `derive-rectangle-geometry.js`, `derive-circle-geometry.js`, `render-rectangle.js`, `render-circle.js`.

Rectangle derived geometry

$\text{RectangleLeftX} = \text{RectangleCenterX} - \text{RectangleHalfWidth}$. $\text{RectangleRightX} = \text{RectangleCenterX} + \text{RectangleHalfWidth}$.

$\text{RectangleTopY} = \text{RectangleCenterY} + \text{RectangleHalfHeight}$. $\text{RectangleBottomY} = \text{RectangleCenterY} - \text{RectangleHalfHeight}$.

$\text{RectangleProtectionPaddingX} = \text{RectangleHalfWidth} * \text{RectangleProtectionPaddingXRatio}$.
 $\text{RectangleProtectionPaddingY} = \text{RectangleHalfHeight} * \text{RectangleProtectionPaddingYRatio}$.

$\text{RectangleProtectionLeftX} = \text{RectangleCenterX} - \text{RectangleHalfWidth} - \text{RectangleProtectionPaddingX}$.
 $\text{RectangleProtectionRightX} = \text{RectangleCenterX} + \text{RectangleHalfWidth} + \text{RectangleProtectionPaddingX}$.

$\text{RectangleProtectionTopY} = \text{RectangleCenterY} + \text{RectangleHalfHeight} + \text{RectangleProtectionPaddingY}$.
 $\text{RectangleProtectionBottomY} = \text{RectangleCenterY} - \text{RectangleHalfHeight} - \text{RectangleProtectionPaddingY}$.

Circle derived geometry

$\text{CircleProtectionPaddingRadius} = \text{CircleRadius} * \text{CircleProtectionPaddingRadiusRatio}$.

$\text{CircleProtectionPaddedRadius} = \text{CircleRadius} + \text{CircleProtectionPaddingRadius}$.

$\text{CircleProtectionLeftX} = \text{CircleCenterX} - \text{CircleProtectionPaddedRadius}$. $\text{CircleProtectionRightX} = \text{CircleCenterX} + \text{CircleProtectionPaddedRadius}$.

$\text{CircleProtectionTopY} = \text{CircleCenterY} + \text{CircleProtectionPaddedRadius}$. $\text{CircleProtectionBottomY} = \text{CircleCenterY} - \text{CircleProtectionPaddedRadius}$.

Persistence rule

Save canonical values and configured ratios only. Recalculate derived values after load, after resize, and before committed save validation.

Phase 3.1 — Define Rectangle canonical variables and bind them to visible behavior

Goal

Define the canonical Rectangle variables, defaults, and derived-geometry rules, then bind those values to a visible rendered rectangle on `DiagramCanvas`.

Scope

This phase is about:

defining the Rectangle variable model

creating the initial JavaScript rectangle state

deriving visible, hover, and protection boundaries

rendering one correct visible rectangle from the canonical definition

confirming which rectangle values are committed and persisted

Variables to define

At minimum:

RectangleID

CenterX

CenterY

HalfWidth

HalfHeight

RotationAngle

ZOrder

FillType

FillColor

LineType

LineColor

LineWidth

HoverPaddingXRatio

HoverPaddingYRatio

HoverPaddingColor

ProtectionPaddingXRatio

ProtectionPaddingYRatio

ProtectionPaddingColor

Required visible behaviors

changing CenterX and CenterY changes the rectangle position

changing HalfWidth and HalfHeight changes the rectangle size

changing RotationAngle changes the rectangle orientation

changing FillType and FillColor changes the interior appearance

changing LineType, LineColor, and LineWidth changes the border appearance

changing hover/protection ratios changes the derived rectangle regions

Function contract deliverables

loadRectangleDefaults

createRectangleModel

deriveRectangleGeometry

renderRectangle

updateDebugPanel

C# structures:

RectangleDto

ShapeDefinitionModel

Phase 3.1 responsibility split

JavaScript

initialize and render rectangle state

calculate derived rectangle geometry

display live visual results

C#

define DTO/model structure

prepare validation and persistence structure

no live rendering logic

Output

persisted Rectangle variable list

initial live rectangle on DiagramCanvas

correct derived rectangle boundaries

debug panel for rectangle values

Validation / Preconditions

rectangle canonical variables exist

half-width and half-height are positive

rotation value is valid

derived geometry calculations are stable

Error / failure expectations

invalid half-size values must be rejected

invalid rotation values must fail safely

missing required Rectangle variable must block valid initialization

Success criteria

all rectangle variables are clearly defined

visual changes match variable changes

rectangle behavior is consistent

Phase 3.2 — Define Circle canonical variables and bind them to visible behavior

Goal

Define the canonical Circle variables, defaults, and derived-geometry rules, then bind those values to a visible rendered circle on DiagramCanvas.

Scope

This phase is about:

defining the Circle variable model

creating the initial JavaScript circle state

deriving visible, hover, and protection boundaries

rendering one correct visible circle from the canonical definition

confirming which circle values are committed and persisted

Variables to define

At minimum:

CircleID

CenterX

CenterY

Radius

ZOrder

FillType

FillColor

LineType

LineColor

LineWidth

HoverPaddingRadiusRatio

HoverPaddingColor

ProtectionPaddingRadiusRatio

ProtectionPaddingColor

Required visible behaviors

changing CenterX and CenterY changes the circle position

changing Radius changes the circle size

changing FillType and FillColor changes the interior appearance

changing LineType, LineColor, and LineWidth changes the border appearance

changing hover/protection ratios changes the derived circle regions

Function contract deliverables

loadCircleDefaults

createCircleModel

deriveCircleGeometry

renderCircle

updateDebugPanel

C# structures:

CircleDto

ShapeDefinitionModel

Phase 3.2 responsibility split

JavaScript

initialize and render circle state

calculate derived circle geometry

display live visual results

C#

define DTO/model structure

prepare validation and persistence structure

no live rendering logic

Output

persisted Circle variable list

initial live circle on DiagramCanvas

correct derived circle boundaries

debug panel for circle values

Validation / Preconditions

circle canonical variables exist

radius is positive

derived geometry calculations are stable

Error / failure expectations

invalid radius values must be rejected

missing required Circle variable must block valid initialization

invalid derived circle geometry must fail safely

Success criteria

all circle variables are clearly defined

visual changes match variable changes

circle behavior is consistent

Phase 3.3 — Build the active shape-definition model and validate derived geometry

Goal

Combine the Rectangle and Circle definitions into one active DiagramCanvas shape-definition model and validate that canonical and derived values remain consistent.

Scope

This phase is about:

holding Rectangle and Circle items together in one active item collection

tracking item type and item identity clearly

recalculating derived geometry before render or save

showing that canonical values remain the only authoritative source

Variables to define

ActiveShapeCollection

ActiveShapeCount

ActiveRectangleID

ActiveCircleID

LastDerivedGeometryResult

Function contract deliverables

buildActiveShapeCollection

recalculateAllDerivedShapeGeometry

validateCommittedRectangleState

validateCommittedCircleState

validateCommittedShapeCollection

renderRectangle

renderCircle

Output

working active shape-definition collection

consistent derived-geometry recalculation

clear canonical-versus-derived separation

Validation / Preconditions

Rectangle and Circle model creation already works

shape identities are valid

derived-geometry rules are callable

Error / failure expectations

invalid shape type must be rejected

if a derived recalculation fails, preserve the previous valid shape state

if collection activation fails, do not partially activate inconsistent shape data

Success criteria

rectangle and circle can exist together in one active model

derived geometry can be recalculated safely for both shapes

the committed model remains stable and predictable

Phase 3.4 — Save the active shape definitions to JSON and reopen them

Exact variables to save in this phase

Exact committed Milestone 3 save variables: Device fields = DeviceID, DeviceName, DeviceType, DeviceStatus, DeviceDescription, DeviceHostGeometryType, DeviceParentContainerID. Container fields = ContainerID, ContainerName, ContainerType, ContainerStatus, ContainerDescription, ContainerParentContainerID. Rectangle-host fields = ShapeType, RectangleID, RectangleCenterX, RectangleCenterY, RectangleHalfWidth, RectangleHalfHeight, RectangleRotationAngle, RectangleZOrder, RectangleFillType, RectangleFillColor, RectangleLineType, RectangleLineColor, RectangleLineWidth, RectangleHoverPaddingXRatio, RectangleHoverPaddingYRatio, RectangleHoverPaddingColor, RectangleProtectionPaddingXRatio, RectangleProtectionPaddingYRatio, RectangleProtectionPaddingColor. Circle-host fields = CircleID, CircleCenterX, CircleCenterY, CircleRadius, CircleZOrder, CircleFillType, CircleFillColor, CircleLineType, CircleLineColor, CircleLineWidth, CircleHoverPaddingRadiusRatio, CircleHoverPaddingColor, CircleProtectionPaddingRadiusRatio, CircleProtectionPaddingColor.

Save regression rule for this phase

All approved Milestone 2 DiagramCanvas variables shall already remain saved correctly. This phase passes only if save-to-json.js adds the Milestone 3 committed shape-definition variables without breaking the Milestone 2 save set.

Save test criteria for this phase

Verify that the JSON payload contains every listed Milestone 3 variable together with the still-required Milestone 2 variables, and that reopen reconstructs the same committed Rectangle and Circle state.

Goal

Save the active Rectangle and Circle definitions to a JSON file and reopen them later so the same committed shape state is rebuilt correctly.

Scope

This phase is about:

getting committed values from the in-memory shape model in JavaScript

building a JSON object

saving a JSON file

reopening a JSON file

rebuilding the active Rectangle and Circle definitions from JSON

Important architecture rule

For the JSON path:

JavaScript gets the values directly from the active shape model

JavaScript builds the JSON object

JavaScript saves the JSON file

JavaScript opens and parses the JSON file

JavaScript rebuilds the shapes from JSON

Variables to define

DiagramID

DiagramVersion

ShapeDefinitionJsonObject

ShapeDefinitionJsonText

SavedJsonFileName

OpenedJsonFileName

Function contract deliverables

buildShapeDefinitionJsonObject

serializeShapeDefinitionToJson

saveShapeDefinitionJsonFile

openShapeDefinitionJsonFile

parseShapeDefinitionJsonFile

validateShapeDefinitionJsonStructure

rebuildShapeDefinitionsFromJson

clearCurrentShapeStateBeforeOpen

Output

valid JSON save path

valid JSON reopen path

correct rebuild of Rectangle and Circle definitions

Validation / Preconditions

active shape-definition model is valid

required persisted variables exist

JSON structure is valid

Error / failure expectations

invalid JSON must not load

incomplete JSON must fail safely

save failure must not produce false success

Success criteria

JSON file is created correctly

reopened JSON reconstructs the same committed shape state

Phase 3.5 — Save the active shape definitions to the database and reopen them

Exact variables to save in this phase

Exact committed Milestone 3 save variables: Device fields = DeviceID, DeviceName, DeviceType, DeviceStatus, DeviceDescription, DeviceHostGeometryType, DeviceParentContainerID. Container fields = ContainerID, ContainerName, ContainerType, ContainerStatus, ContainerDescription, ContainerParentContainerID. Rectangle-host fields = ShapeType, RectangleID, RectangleCenterX, RectangleCenterY, RectangleHalfWidth, RectangleHalfHeight, RectangleRotationAngle, RectangleZOrder, RectangleFillType, RectangleFillColor, RectangleLineType, RectangleLineColor, RectangleLineWidth, RectangleHoverPaddingXRatio, RectangleHoverPaddingYRatio, RectangleHoverPaddingColor, RectangleProtectionPaddingXRatio, RectangleProtectionPaddingYRatio, RectangleProtectionPaddingColor. Circle-host fields = CircleID, CircleCenterX, CircleCenterY, CircleRadius, CircleZOrder, CircleFillType, CircleFillColor, CircleLineType, CircleLineColor, CircleLineWidth, CircleHoverPaddingRadiusRatio, CircleHoverPaddingColor, CircleProtectionPaddingRadiusRatio, CircleProtectionPaddingColor.

Save regression rule for this phase

All approved Milestone 2 DiagramCanvas variables shall already remain saved correctly. This phase passes only if save-to-db.js adds the Milestone 3 committed shape-definition variables without breaking the Milestone 2 save set.

Save test criteria for this phase

Verify that the database payload/model contains every listed Milestone 3 variable together with the still-required Milestone 2 variables, and that reload reconstructs the same committed Rectangle and Circle state.

Goal

Save the active Rectangle and Circle definitions to the database and reopen them later by DiagramID.

Scope

This phase is about:

JavaScript building a committed save payload from the active in-memory shape model

sending that payload to C#

C# validating it

C# writing it to the database

reloading the same diagram shapes from the database

Important architecture rule

For the database path:

JavaScript gets committed values from the active shape model

JavaScript builds the save payload

JavaScript sends the payload to the API

C# validates the payload

C# maps it to DB persistence structures

C# writes to the database

Variables to define

DiagramID

DiagramVersion

ShapeDefinitionSavePayload

LoadedDiagramID

LoadedShapeDefinitionData

LastDatabaseSaveResult

Function contract deliverables

JavaScript

buildShapeDefinitionSavePayload

callSaveShapeDefinitionApi

callLoadShapeDefinitionApi

applyLoadedShapeDefinitionData

C#

ShapeDefinitionController.SaveShapeDefinitions

ShapeDefinitionController.LoadShapeDefinitions

ShapeDefinitionService.SaveShapeDefinitions

ShapeDefinitionService.LoadShapeDefinitions

ShapeDefinitionRepository.SaveShapeDefinitions

ShapeDefinitionRepository.LoadShapeDefinitionsByDiagramId

Output

working DB save

working DB reopen

trusted persistence of committed Rectangle and Circle definitions

Validation / Preconditions

database connection is confirmed

API is reachable

payload is valid

DiagramID is valid

Error / failure expectations

DB save failure must be reported clearly

invalid ID must fail safely

incomplete DB response must not partially rebuild corrupted shape state

Success criteria

save to DB works

reopen by DiagramID works

loaded shapes match the saved committed state

Final result after Milestone 3

At the end of Milestone 3, the developer should have:

a clearly defined persistent Rectangle model

a clearly defined persistent Circle model

correct canonical-versus-derived geometry rules

working visible rendering for both shapes

direct JSON save/load handled by JavaScript

trusted DB save/load handled through C#

a stable shape-definition foundation for later milestones such as:

drag and drop creation

selection and hover behavior

move and resize behavior

connections

containment

persistence

The developer should also have a clear separation between:

canonical persisted shape variables

derived geometry values

committed shape state

temporary runtime-only interaction state

Recommended implementation order

Use this order:

Define and render Rectangle canonical variables

Define and render Circle canonical variables

Build the active shape-definition collection and derived-geometry validation

Save/reopen committed shape definitions in JSON

Save/reopen committed shape definitions in DB

Milestone 4 — Drag and Drop Rectangle or Circle from Left Panel into DiagramCanvas

Overview

Milestone 4 connects the Shapes Panel with DiagramCanvas and turns a left-panel library drag action into the creation of a real Rectangle or Circle item inside the active diagram.

This milestone is the first point where the system stops dealing only with panel-side source icons, temporary previews, or Milestone 2 test objects, and starts dealing with committed canvas items that belong to the diagram model.

The drop begins in Screen Space, but the new item must be created in World Space. The correct developer rule remains: Screen Space -> Viewport -> World Space for input, and World Space -> Viewport -> Screen Space for rendering. Pan and zoom may change the visible result, but they must not corrupt the stored world coordinates of the new item.

JavaScript handles drag-start, drag preview, canvas drop-target detection, screen-to-world conversion, item creation, immediate rendering, and active-item selection. JavaScript may also save JSON directly from the in-memory diagram model. C# handles trusted validation, API orchestration, and database persistence when Milestone 4 reaches file or database save phases. C# does not need to read JSON in order to save to the database.

Prerequisites

Mandatory existing JavaScript before starting Milestone 4

Only JavaScript files that must already exist and work correctly before this milestone starts are listed here.

Shared save prerequisite. Before starting Milestone 4, the team shall verify that save-to-json.js and save-to-db.js are already saving all committed variables approved in Milestones 1 to 3 correctly. Milestone 4 does not introduce new permanent persistence field names beyond those already approved in Milestone 2 and Milestone 3. The developer shall therefore continue using the same two shared save files while keeping any temporary drag-and-drop values as internal implementation details rather than milestone-defined committed variables.

JavaScript file that must already exist	Description	Reason this JavaScript must already be ready and working correctly
---	-------------	--

shapes-panel-state.js	Maintains the Shapes Panel state, selected category, and library data.	Milestone 4 begins drag from the left panel, so the panel state must already be stable.
render-shapes-panel.js	Renders the Shapes Panel and its visible UI structure.	The drag source items must already exist visually before drag-to-canvas can begin.
drag-shape-library-item.js	Starts drag from a library item in the Shapes Panel.	Milestone 4 depends on a stable left-panel drag-start path.
shape-library-drag-payload.js	Builds the payload for the dragged library item.	Canvas drop logic needs a normalized drag payload before item creation can happen.
screen-to-world.js	Converts drop screen coordinates to world coordinates.	Dropped shapes must be created in world space, not in screen pixels.
render-canvas.js	Renders real items on the DiagramCanvas.	The newly dropped Rectangle or Circle must become visible immediately after creation.

Before Milestone 4 implementation starts, the following prerequisites should be confirmed.

Required confirmations: Shapes Panel Milestone 1 is already working with draggable RectangleLibraryItem and CircleLibraryItem source items; DiagramCanvas Milestone 2 is already working with render flow, active canvas state, and CanvasItems collection; screenToWorld and worldToScreen are already validated; default Rectangle and Circle definitions are already agreed; unique item-ID generation is available; and JSON/database save plumbing is available if Phases 3.4 and 3.5 will be implemented.

Minimum readiness rule: Milestone 4 should not be considered ready to begin until the team confirms at least this: the left panel can start a valid drag, DiagramCanvas can be detected as a drop target, and screen-to-world conversion is already correct.

Milestone 4 variables

No dedicated Milestone 4 variable table is required. Milestone 4 shall rely on the already approved Milestone 2 DiagramCanvas variables and Milestone 3 committed Device/Container/Rectangle/Circle variables. Any temporary drag-and-drop values used during implementation are internal working details only and shall not be treated as required milestone variables or required persistence fields.

Recommended database table structure

Milestone 4 extends the Milestone 2 persistence design by adding normalized item tables for real canvas shapes. The Diagrams and DiagramCanvas tables from Milestone 2 continue to exist; Milestone 4 adds item-oriented tables for creation, save, reopen, and later editing milestones.

Database table	Purpose
Diagrams	Main diagram identity and summary table carried forward from Milestone 2
DiagramCanvas	Current persistent DiagramCanvas configuration/state carried forward from Milestone 2
CanvasItems	Common item registry for all real canvas items inside a diagram
Rectangles	Rectangle-specific geometry and appearance table

Database table	Purpose
Circles	Circle-specific geometry and appearance table
DiagramPayloads	Optional versioned full payload snapshot table

Recommended relationship rule: Diagrams = summary and identity; DiagramCanvas = persistent canvas/view state; CanvasItems = common item registry; Rectangles and Circles = shape-specific subtype tables; DiagramPayloads = optional full committed payload snapshot.

Table - CanvasItems

Column	Type	Purpose
ItemID	uniqueidentifier or varchar(64)	Primary key for the real canvas item
DiagramID	uniqueidentifier or varchar(64)	Foreign key to Diagrams
CanvasID	uniqueidentifier or varchar(64)	Foreign key to DiagramCanvas
ItemType	nvarchar(50)	Rectangle or Circle
ZOrder	int	Committed draw order of the item
ParentContainerID	uniqueidentifier or varchar(64) nullable	Optional parent container relationship for later milestones
CreatedAt	datetime	Creation timestamp
UpdatedAt	datetime	Last update timestamp
CreatedBy	nvarchar(200) nullable	Optional creator identity
IsDeleted	bit	Optional soft-delete flag

Table - Rectangles

Column	Type	Purpose
RectangleID	uniqueidentifier or varchar(64)	Primary key for the rectangle record
ItemID	uniqueidentifier or varchar(64)	Foreign key to CanvasItems
CenterX	float or decimal(18,6)	World-space center X
CenterY	float or decimal(18,6)	World-space center Y
HalfWidth	float or decimal(18,6)	Half width of the rectangle
HalfHeight	float or decimal(18,6)	Half height of the rectangle
RotationAngle	float or decimal(18,6)	Rectangle rotation angle in degrees
FillType	nvarchar(50)	Example: NoFill or SolidFill
FillColor	nvarchar(50)	Rectangle fill color
LineType	nvarchar(50)	Example: SolidLine

Column	Type	Purpose
LineColor	nvarchar(50)	Rectangle line color
LineWidth	float or decimal(18,6)	Rectangle line width
HoverPaddingXRatio	float or decimal(18,6)	Hover padding ratio on X
HoverPaddingYRatio	float or decimal(18,6)	Hover padding ratio on Y
HoverPaddingColor	nvarchar(50)	Hover padding display color
ProtectionPaddingXRatio	float or decimal(18,6)	Protection padding ratio on X
ProtectionPaddingYRatio	float or decimal(18,6)	Protection padding ratio on Y
ProtectionPaddingColor	nvarchar(50)	Protection padding display color

Table - Circles

Column	Type	Purpose
CircleID	uniqueidentifier or varchar(64)	Primary key for the circle record
ItemID	uniqueidentifier or varchar(64)	Foreign key to CanvasItems
CenterX	float or decimal(18,6)	World-space center X
CenterY	float or decimal(18,6)	World-space center Y
Radius	float or decimal(18,6)	Committed circle radius
FillType	nvarchar(50)	Example: NoFill or SolidFill
FillColor	nvarchar(50)	Circle fill color
LineType	nvarchar(50)	Example: SolidLine
LineColor	nvarchar(50)	Circle line color
LineWidth	float or decimal(18,6)	Circle line width
HoverPaddingRadiusRatio	float or decimal(18,6)	Hover padding ratio
HoverPaddingColor	nvarchar(50)	Hover padding display color
ProtectionPaddingRadiusRatio	float or decimal(18,6)	Protection padding ratio
ProtectionPaddingColor	nvarchar(50)	Protection padding display color

Table - DiagramPayloads optional but recommended

Column	Type	Purpose
PayloadID	uniqueidentifier or varchar(64)	Primary key
DiagramID	uniqueidentifier or varchar(64)	Foreign key to Diagrams

Column	Type	Purpose
DiagramVersion	int	Version of the saved payload
PayloadJson	nvarchar(max) or JSON-native type	Full committed diagram payload snapshot
SavedAt	datetime	Save timestamp

Recommended files for Milestone 4

The table below mirrors the Milestone 2 file-definition style and includes CSS, JavaScript, and C# files relevant to drag-start, drop, item creation, activation, and save/reopen flows. The trigger column now states the explicit trigger and the practical call count for one action or one request.

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
shape-library-items.css	CSS	Visual styling for left-panel draggable shape items.	Page load, and whenever the owning component visual state is redrawn. Call count: stylesheet parsed once per page load; browser reapplies styles automatically.	Shape-library DOM structure	Library item container exists	Style rectangle/circle source items, drag affordance, hover states	Readable draggable library items	No persistence effect	drag-shape-to-canvas.js	Misaligned or unclear source-item styling

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
drag-preview.css	CSS	Visual styling for drag preview and valid/invalid drop states.	Page load, and whenever the owning component visual state is redrawn. Call count: stylesheets parsed once per page load; browser reapplies styles automatically.	Drag-preview DOM/canvas layer	Preview layer exists	Style preview opacity, cursor feedback, allowed/disallowed state	Readable drag preview	No persistence effect	drop-preview.js	Preview may be visually unclear

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
canvas-drop.css	CSS	Styling for canvas drop-target highlight behavior.	Page load, and whenever the owning component visual state is redrawn. Call count: stylesheet parsed once per page load; browser reapplies styles automatically.	Canvas container classes/IDs	Canvas container exists	Style drop-target border/background feedback	Visible drop-target feedback	No persistence effect	canvas-drop.js	Canvas drop state may be hard to recognize

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
canvas-items.css	CSS	Styling rules for real dropped rectangle and circle items.	Page load, and whenever the owning component visual state is redrawn. Call count: stylesheet parsed once per page load; browser reapplies styles automatically.	Canvas item classes or render rules	Canvas item render path exists	Apply baseline item styling, line/fill visuals, selection-friendly appearance	Correct visible item styling	No persistence effect	render-canvas.js	Items may render with wrong appearance

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
selection.css	CSS	Styling for active-item selection state after drop.	Page load, and whenever the owning component visual state is redrawn. Call count: stylesheet parsed once per page load; browser reapplies styles automatically.	Selected item classes/state	Selection state exists	Style active-item outline/highlight state	Readable active-item state	No persistence effect	canvas-item-selection.js	Selection may be visually weak or inconsistent

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
drag-shape-to-canvas.js	JS	Starts drag from the left panel and maintains drag state.	Left Click Down on a draggable library shape, followed by processed drag updates while Left Click remains active. Call count: once per processed drag update until Left Click Up.	Source-item metadata, pointer events	Source item valid, no conflicting mode active	Begin library drag, capture shape type, maintain drag state	Updated drag state	Temporary client-side drag state only	shape-drop-payload.js, drop-preview.js, canvas-drop.js	Cancel drag and clear state safely
shape-drop-payload.js	JS	Builds the drag/drop payload used between the panel and canvas.	Drag-start Call count: as needed by the owning action.	Shape type, source ID, defaults	Recognized shape type exists	Build normalized payload for rectangle or circle drop	ShapeDropPayload object	No persistence effect	canvas-drop.js, create-rectangle-on-drop.js, create-circle-on-drop.js	Return null/invalid payload result

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
drop-preview.js	JS	Shows, updates, and hides the drag preview.	Processed drag update while a library shape drag is active. Call count: once per processed drag update until drop or cancel.	Drag state, pointer position, drop-allowed flag	Preview layer exists, drag active	Render visual preview and allowed/disallowed feedback	Updated preview layer	Temporary preview state only	canvas-drop.js, render-canvas.js	Hide preview and preserve prior UI state
canvas-drop.js	JS	Detects valid canvas drop targets and handles mouse-up drop.	Processed drag update over Diagram Canvas and final Left Click Up for drop evaluation. Call count: once per processed drag update, plus one final drop decision at Left Click Up.	Pointer position, drag payload, canvas bounds	Canvas exists, drag payload valid	Detect drop target, route drop, convert final position handling	Drop accepted/rejected result	Temporary drag state becomes committed creation request	screen-to-world.js, create-rectangle-on-drop.js, create-circle-on-drop.js	Reject drop and clear drag state safely

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
create-rectangle-on-drop.js	JS	Creates a real rectangle item from the accepted drop.	Accepted rectangle drop on Diagram Canvas. Call count: once per accepted rectangle drop action.	DropWorld X/Y, rectangle defaults, payload	Drop valid, defaults available	Build committed rectangle model using world coordinates	Rectangle item object	Adds committed rectangle to in-memory diagram state	canvas-item-factory.js, canvas-state.js, render-canvas.js	Cancel creation and preserve previous valid state
create-circle-on-drop.js	JS	Creates a real circle item from the accepted drop.	Accepted circle drop on Diagram Canvas. Call count: once per accepted circle drop action.	DropWorld X/Y, circle defaults, payload	Drop valid, defaults available	Build committed circle model using world coordinates	Circle item object	Adds committed circle to in-memory diagram state	canvas-item-factory.js, canvas-state.js, render-canvas.js	Cancel creation and preserve previous valid state
canvas-item-factory.js	JS	Central factory for new canvas items and ID assignment.	Rectangle/circle creation request Call count: as needed by the owning action.	Shape type, defaults, converted drop point	Valid shape type and ID generator available	Assign ID, apply defaults, build normalized item model	New canvas item instance	Creates committed item object in client memory	canvas-state.js, canvas-item-selection.js	Return failure and stop insertion

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
canvas-state.js	JS	Maintains active diagram item collection and counts.	App startup, load action, or state mutation request. Call count: as needed per state initialization or state update.	New item object, active diagram state	Active diagram exists	Insert item into CanvasItems, update item count, expose current active item data	Updated client-side state object	Client-side committed state changes; persisted only on save	render-canvas.js, build-diagram-json.js	Reject invalid state mutation
canvas-item-selection.js	JS	Activates the newly dropped item and clears drag state.	Successful item creation or selection-change request after drop/open. Call count: once per activation action.	New item ID/type, current drag state	Created item exists	Set ActiveCanvasItem, clear drag state, show selection state	Updated selection/activation state	Client-side active-item state only	render-canvas.js, debug-panel.js	Preserve item but clear selection safely

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
screen-to-world.js	JS	Converts drop screen coordinates to world coordinates.	Rendering or pointer-conversion request. Call count: zero to many times per render/update cycle, depending on the number of points that must be converted.	DropScreenX/Y, viewport, zoom	Viewport and zoom valid	Apply Screen Space -> Viewport -> World Space conversion	DropWorldX/Y	No persistence effect	create-rectangle-on-drop.js, create-circle-on-drop.js	Cancel drop on conversion failure

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
render-canvas.js	JS	Renders real rectangle/circle items after drop and reopen.	App startup, canvas state change, pan, zoom, move, resize, or diagram switch. Call count: once per processed render-frame update; the active frame cadence may be sampled to estimate screen refresh rate (Hz).	Active canvas state, CanvasItems	Canvas state valid	Re-render canvas items, active state, and view result	Updated canvas image	No direct persistence effect	debug-panel.js	Render stops safely on invalid state

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
build-diagram-json.js	JS	Builds committed diagram JSON including dropped shapes.	Build/serialize request from the owning action, usually save or create. Call count: once per build action.	Active diagram model and CanvasItems	Committed diagram state valid	Serialize rectangles/circles from in-memory model	JSON-ready diagram object	No file/DB change yet; prepares persistence payload	save-diagram-json.js, diagram-api.js	Return controlled serialization failure
save-diagram-json.js	JS	Saves committed diagram JSON to a file.	User chooses the owning save command. Call count: once per save action.	JSON-ready diagram object, file name	Diagram JSON valid	Serialize and trigger file save/download	Saved JSON file	Creates file output only	open-diagram-json.js	Report file-save failure clearly
open-diagram-json.js	JS	Opens a saved diagram JSON file and rebuilds the diagram.	User chooses the owning open command. Call count: once per open action.	Selected JSON file	File selected, JSON text readable	Read, parse, validate, and apply saved diagram state	Loaded diagram object	Replaces active in-memory diagram state	parse-diagram-json.js, render-canvas.js	Reject malformed or incomplete file

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
diagram-api.js	JS	Calls trusted save/load APIs for database persistence.	JavaScript chooses the backend persistence path. Call count: once per API request.	Diagram payload, DiagramID	API endpoint reachable, payload valid	Send save/load request and handle response	API response objects	No direct DB write by itself; initiates server persistence	DiagramCanvasController.cs	Report API failure clearly
RectangleDto.cs	C#	DTO carrying trusted rectangle data across API/service layers.	Created or mapped during an API save/load/list path. Call count: once per request/response mapping that uses this type.	Rectangle payload fields	Model binding succeeded, fields valid	Transfer rectangle data between layers	DTO object	No direct persistence effect	DiagramCanvasService.cs	Validation or mapping failure

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
CircleDto.cs	C#	DTO carrying trusted circle data across API/service layers.	Created or mapped during an API save/load /list path. Call count: once per request/response mapping that uses this type.	Circle payload fields	Model binding succeeded, fields valid	Transfer circle data between layers	DTO object	No direct persistence effect	DiagramCanvasService.cs	Validation or mapping failure
CanvasItemDto.cs	C#	Common item DTO for rectangle/circle identity and registry data.	Created or mapped during an API save/load /list path. Call count: once per request/response mapping that uses this type.	Item ID, type, diagram links, ZOrder	Common item fields valid	Carry normalized canvas item data	DTO object	No direct persistence effect	DiagramCanvasService.cs	Validation or mapping failure

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
DiagramCanvasController.cs	C#	API controller for save/load actions involving dropped items.	Only when the DB/API persistence path is enabled and JavaScript sends an HTTP save/load request. Call count: once per API request.	Request body, DiagramID	Request valid, routing/auth satisfied	Route request to service layer and return response	HTTP/API response	No direct DB change except through service/repository flow	DiagramCanvasService.cs	Return controlled HTTP error response
DiagramCanvasService.cs	C#	Trusted validation and persistence orchestration layer.	Only when the DB/API persistence path is enabled and the controller routes a save/load request. Call count: once per routed service request.	Rectangle/ Circle DTOs, request context	Business rules and payload structure valid	Validate, map, and orchestrate save/load of real canvas items	Save/load result object	Triggers trusted persistence through repository	DiagramCanvasRepository.cs	Return controlled service failure

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
DiagramCanvasRepository.cs	C#	Database access layer for diagrams and item tables.	Only when the DB/API persistence path is enabled and the service performs a DB operation. Call count: once per DB save/load/list operation step.	Persistence models, DiagramID	DB connected, SQL/data access ready	Insert, update, and load Diagrams, CanvasItems, Rectangles, and Circles	DB rows/result objects	Direct database persistence effect	Service return flow	DB read/write failure

Important rules

Rule 1 - The source library item remains unchanged. Dragging from the left panel must create a new canvas item instance; it must not move or remove the original source item.

Rule 2 - The new shape must be created in World Space. The final drop is detected in Screen Space, then converted through the viewport into world coordinates before the new item is created.

Rule 3 - Rectangle and Circle creation shall follow the formal item definitions already defined in this document, including geometry, appearance, padding, and ZOrder fields.

Rule 4 - Use defaults for first placement. The first implementation should focus on correct drag-start, correct drop, correct world placement, correct item creation, and correct rendering.

Rule 5 - Milestone 4 is primarily about creation and reopen. It is not the milestone for full move/resize editing, advanced snapping, container logic, or connection logic.

Rule 6 - JavaScript gets values from the in-memory diagram model, saves JSON directly from that model, and sends the committed payload to C# for database save.

Rule 7 - Only committed rectangle/circle data shall be persisted. Temporary drag preview, pointer location, or hover-only states shall not be persisted.

Rule 8 - Only one diagram is active in DiagramCanvas at a time for this milestone, and the newly created item becomes the active canvas item after a successful drop.

Mathematical Calculation Rules for Milestone 4

Milestone 4 calculations convert a left-panel drag/drop action into a real world-space rectangle or circle item on DiagramCanvas. The source library item is never moved or removed; the drop creates a new diagram item instance.

Primary JavaScript files for this calculation scope: drag-shape-library-item.js, shape-library-drag-payload.js, canvas-drop-target.js, create-dropped-geometry.js, screen-to-world.js, derive-shape-geometry.js.

Drop coordinate calculation

$\text{DropWorldPoint} = \text{screenToWorld}(\text{DropScreenX}, \text{DropScreenY}, \text{ActiveViewport})$.

For a dropped rectangle: $\text{RectangleCenterX} = \text{DropWorldPoint.X}$ and $\text{RectangleCenterY} = \text{DropWorldPoint.Y}$.

For a dropped circle: $\text{CircleCenterX} = \text{DropWorldPoint.X}$ and $\text{CircleCenterY} = \text{DropWorldPoint.Y}$.

Default geometry calculation

Dropped rectangle half-size values shall come from the selected library item defaults or from the global Rectangle variables if the library item does not override them.

Dropped circle Radius shall come from the selected library item defaults or from the global Circle variables if the library item does not override it.

Creation validation calculation

After creating the candidate geometry, the system shall recalculate visible boundaries, Hover Padding boundaries, Protection Padding boundaries, and default resize control point positions before rendering or saving.

If future containment rules are active at drop time, the candidate geometry shall be validated using the same Protection Padding and parent-containment rules defined in later milestones.

Phase 4.1 - Start dragging Rectangle or Circle from the left panel and show drag preview over the canvas

Goal: Allow the user to begin dragging a rectangle or circle from the left panel and show valid visual drag feedback while moving toward or over DiagramCanvas.

Scope: Detect drag-start from the shape library, build the drag payload, maintain temporary drag state, detect when the pointer is over the canvas, and show valid-drop or invalid-drop feedback. This phase does not yet create a real canvas item.

Variables to define: No dedicated Milestone 4 variable list is required for this phase. The developer may use internal temporary drag-state values as implementation details, but they shall not be treated as required milestone variables or persistence fields.

Required visible behaviors: Starting drag on rectangle begins rectangle drag state; starting drag on circle begins circle drag state; moving over DiagramCanvas shows valid drop feedback; moving away shows neutral or invalid state; the left-panel source item remains unchanged.

Function contract deliverables: beginLibraryShapeDrag, buildShapeDropPayload, updateLibraryShapeDrag, showShapeDragPreview, hideShapeDragPreview, detectCanvasDropTarget, setDropAllowedState, cancelLibraryShapeDrag.

Required event-to-function routing: Mouse down on RectangleLibraryItem or CircleLibraryItem -> beginLibraryShapeDrag -> buildShapeDropPayload -> showShapeDragPreview; mouse move during drag -> updateLibraryShapeDrag -> detectCanvasDropTarget -> setDropAllowedState -> showShapeDragPreview.

State rule: Only temporary JavaScript drag state is updated in this phase. No real Rectangle or Circle is inserted into CanvasItems and no C# call is required.

Output: Working rectangle drag-start, working circle drag-start, visible drag preview, valid/invalid drop feedback, and a clear payload structure for later drop creation.

Validation / Preconditions: The left panel already supports draggable source items, DiagramCanvas already exists as a visible drop target, and the dragged shape type is recognized as Rectangle or Circle.

Error / failure expectations: If shape type is invalid, cancel the drag; if drag state becomes inconsistent, clear preview and reset state; if canvas drop target cannot be identified, mark drop as not allowed; if source metadata is missing, do not start drag.

Success criteria: The user can begin dragging rectangle or circle from the left panel, drag preview behaves correctly, DiagramCanvas is recognized as a drop target, and the source library item remains unchanged.

Phase 4.2 - Drop the Rectangle or Circle into DiagramCanvas and create a real item in world space

Goal: When the user drops a rectangle or circle onto DiagramCanvas, create a real item in the active world model at the correct world-space position.

Scope: Accept the drop on DiagramCanvas, convert the drop location from Screen Space to World Space, apply default shape values, create a real Rectangle or Circle item, insert it into CanvasItems, and render it immediately.

Variables to define: No dedicated Milestone 4 variable list is required for this phase. The developer may use internal drop-processing values as implementation details, but the real created item shall be expressed through the already approved committed Milestone 3 Rectangle/Circle and Device/Container fields together with the Milestone 2 canvas context.

Required visible behaviors: Dropping rectangle creates a visible rectangle; dropping circle creates a visible circle; the created center is stored in world coordinates; pan and zoom do not change the committed stored placement.

Function contract deliverables: handleCanvasDrop, convertDropScreenToWorld, createRectangleOnDrop, createCircleOnDrop, assignNewCanvasItemId, insertCanvasItem, renderCanvas, updateDebugPanel.

Required event-to-function routing: Mouse up over DiagramCanvas while dragging rectangle -> handleCanvasDrop -> convertDropScreenToWorld -> createRectangleOnDrop -> insertCanvasItem -> renderCanvas -> updateDebugPanel; the circle flow follows the same pattern with createCircleOnDrop.

State rule: The drop creates a new real canvas item in JavaScript state. The new item becomes part of CanvasItems. No database write is required yet.

Output: Working drop support for rectangle and circle, correct screen-to-world conversion, insertion of the new item into CanvasItems, and immediate rendering of the created item.

Validation / Preconditions: DiagramCanvas already supports screen-to-world conversion, the drag payload is valid, the drop occurs inside the canvas drop target, and rectangle/circle default definitions already exist.

Error / failure expectations: If the drop occurs outside the canvas, do not create an item; if conversion fails, cancel the drop; if shape type is missing, do not create any item; if item insertion fails, do not partially render inconsistent state; if ID generation fails, preserve the previous valid canvas state.

Success criteria: Dropping rectangle creates a real rectangle in DiagramCanvas, dropping circle creates a real circle in DiagramCanvas, the created item center is stored in World Space, and the new shape appears immediately after drop.

Phase 4.3 - Activate the new item, show post-drop state, and prepare for later item editing milestones

Goal: After a successful drop, make the new item the active canvas item and leave DiagramCanvas in a stable post-drop state ready for later move, resize, and edit milestones.

Scope: Select the newly created item, clear drag-preview state, show post-drop selection state, update debug information, and prepare the canvas for the next interaction.

Variables to define: No dedicated Milestone 4 variable list is required for this phase. The developer may use internal post-drop selection values as implementation details, but they shall not be treated as required milestone variables or persistence fields.

Required visible behaviors: The new item becomes the active item; selection state is shown; drag preview disappears; debug output may show item type, item ID, world position, and current viewport values; the left panel remains ready for another drag-and-drop operation.

Function contract deliverables: setActiveCanvasItem, clearLibraryDragState, showCanvasItemSelectionState, updateDebugPanelForActiveItem, prepareCanvasForNextInteraction.

Required event-to-function routing: Successful item creation -> setActiveCanvasItem -> clearLibraryDragState -> showCanvasItemSelectionState -> updateDebugPanelForActiveItem -> prepareCanvasForNextInteraction.

State rule: The created item remains in JavaScript canvas state and becomes the active selected item. No persistence is required yet.

Output: A selected newly created Rectangle or Circle, a cleared drag state, a stable post-drop canvas state, and a clean handoff into future editing milestones.

Validation / Preconditions: Item creation already succeeds, active-item state supports Rectangle and Circle, drag preview can be cleared safely, and the debug panel can show new-item data.

Error / failure expectations: If the new item cannot be activated, keep the item visible but mark selection state as unavailable; if debug update fails, preserve the created item; if drag state is not cleared properly, reset it before allowing another drag.

Success criteria: The new Rectangle or Circle becomes the active canvas item after drop, drag state is cleared cleanly, and DiagramCanvas remains ready for another drag-and-drop operation.

Phase 4.4 - Save the diagram to a JSON file and reopen it

Exact variables to save in this phase

Milestone 4 introduces no new permanent persistence field names beyond the already approved Milestone 2 DiagramCanvas variables and Milestone 3 committed Device/Container/Rectangle/Circle variables. The exact save requirement in this phase is therefore: save every Milestone 2 variable, and save the full committed Milestone 3 item record for each shape that was successfully created by drag and drop.

Do not treat any temporary Milestone 4 drag-and-drop runtime values used by the implementation as milestone-required persisted save fields. Save only the approved Milestone 2 DiagramCanvas variables and the committed Milestone 3 Device/Container/Rectangle/Circle fields for each successfully created item.

Save regression rule for this phase

All approved Milestone 2 and Milestone 3 committed variables shall already remain saved correctly. This phase passes only if save-to-json.js persists the committed created-item state without promoting temporary drag-and-drop working variables into the canonical saved truth.

Save test criteria for this phase

Verify that each successfully dropped item is saved through the approved Milestone 2 and Milestone 3 committed fields, that any temporary drag-and-drop runtime values used by the implementation are excluded as milestone-required saved data, and that reopen restores the created items in the same world-space positions and sizes.

Goal: Save the current diagram into a JSON file after the user presses Save, and later reopen that file through File -> Open so the diagram is reconstructed correctly.

Scope: Build committed JSON from the in-memory diagram model, save it as a file, reopen a previously saved JSON file, and rebuild the active diagram from the saved content.

Important save rule: The JSON file shall save the committed diagram model, not temporary interaction-only states. Save real canvas items, world coordinates, geometry, appearance values, ZOrder, and optional viewport restoration values. Do not save drag preview, temporary pointer location, or hover-only state.

Variables to define: DiagramID, DiagramVersion, DiagramName, DiagramJsonObject, DiagramJsonText, SavedJsonFileName, OpenedJsonFileName, LoadedDiagramJsonObject.

Function contract deliverables: buildDiagramJsonObject, serializeDiagramToJson, saveDiagramJsonFile, openDiagramJsonFile, parseDiagramJsonFile, validateDiagramJsonStructure, rebuildDiagramFromJson, clearCurrentDiagramBeforeOpen, renderCanvas, updateDebugPanel.

Required event-to-function routing: Click Save button -> buildDiagramJsonObject -> serializeDiagramToJson -> saveDiagramJsonFile; File -> Open -> openDiagramJsonFile -> parseDiagramJsonFile -> validateDiagramJsonStructure -> clearCurrentDiagramBeforeOpen -> rebuildDiagramFromJson -> renderCanvas -> updateDebugPanel.

State rule: Saving to JSON stores the committed diagram state. Opening a JSON file replaces or reloads the current in-memory diagram state. Item positions remain based on stored world coordinates.

Output: A valid JSON save path, a valid JSON reopen path, and correct reconstruction of rectangles and circles from JSON.

Validation / Preconditions: The active diagram is valid, all real canvas items have valid IDs and geometry, and the JSON structure contains the minimum required fields.

Error / failure expectations: If serialization fails, no file is produced; if the JSON structure is invalid, the file must not be loaded; if one item cannot be reconstructed, load must fail safely or report which item failed; incomplete files must be rejected.

Success criteria: Pressing Save produces a valid JSON file of the diagram; the file contains the real committed diagram state; the user can reopen it through File -> Open; and rectangles/circles are reconstructed in the correct places.

Phase 4.5 - Save the diagram to the database and reopen it

Exact variables to save in this phase

Milestone 4 introduces no new permanent persistence field names beyond the already approved Milestone 2 DiagramCanvas variables and Milestone 3 committed Device/Container/Rectangle/Circle variables. The exact save requirement in this phase is therefore: save every Milestone 2 variable, and save the full committed Milestone 3 item record for each shape that was successfully created by drag and drop.

Do not treat any temporary Milestone 4 drag-and-drop runtime values used by the implementation as milestone-required persisted save fields. Save only the approved Milestone 2 DiagramCanvas variables and the committed Milestone 3 Device/Container/Rectangle/Circle fields for each successfully created item.

Save regression rule for this phase

All approved Milestone 2 and Milestone 3 committed variables shall already remain saved correctly. This phase passes only if save-to-db.js persists the committed created-item state without promoting temporary drag-and-drop working variables into the canonical database truth.

Save test criteria for this phase

Verify that each successfully dropped item is saved through the approved Milestone 2 and Milestone 3 committed fields, that any temporary drag-and-drop runtime values used by the implementation are excluded as milestone-required saved data, and that database reopen restores the created items in the same world-space positions and sizes.

Goal: Save the current diagram into the database under a valid DiagramID and later reopen the same diagram from the database so it appears exactly as it was saved.

Scope: JavaScript builds the committed diagram payload, sends it to the C# API, C# validates the request, writes the diagram and item tables to the database, and later reloads the same diagram by DiagramID.

Important save rule: JavaScript shall not write directly to the database. JavaScript builds the committed payload and sends it to C#. C# validates the payload, maps it to persistence structures, writes it to the database, and returns success or failure.

Variables to define: DiagramID, DiagramVersion, DiagramSavePayload, LoadedDiagramID, LoadedDiagramData, LastDatabaseSaveResult.

Function contract deliverables: JavaScript: buildDiagramSavePayload, callSaveDiagramApi, callLoadDiagramApi, applyLoadedDiagramData, renderCanvas. C#: DiagramCanvasController.SaveDiagram, DiagramCanvasController.LoadDiagram, DiagramCanvasService.SaveDiagram, DiagramCanvasService.LoadDiagram, DiagramCanvasRepository.SaveDiagram, DiagramCanvasRepository.LoadDiagramById.

Required event-to-function routing: Click Save button -> buildDiagramSavePayload -> callSaveDiagramApi -> DiagramCanvasController.SaveDiagram -> DiagramCanvasService.SaveDiagram -> DiagramCanvasRepository.SaveDiagram; Open Diagram by DiagramID -> callLoadDiagramApi -> DiagramCanvasController.LoadDiagram -> DiagramCanvasService.LoadDiagram -> DiagramCanvasRepository.LoadDiagramById -> applyLoadedDiagramData -> renderCanvas.

State rule: Only the committed diagram model is saved. Temporary UI-only states are not persisted. Reopening by DiagramID must reconstruct the same committed diagram state.

Output: A working database save path, a working load-by-DiagramID path, trusted persistence of rectangle and circle items, and correct reconstruction from the database.

Validation / Preconditions: DiagramID is valid, the diagram payload is structurally valid, all required item data exists, the API is reachable, and the server validates the diagram before database write.

Error / failure expectations: If API save fails, the user receives a clear failure result; invalid DiagramID fails safely; database write failure must not show false success; incomplete load data must not partially rebuild a corrupted diagram.

Success criteria: Pressing Save stores the diagram in the database using a valid DiagramID; the user can reopen the saved diagram by DiagramID; and the reopened diagram restores rectangles and circles with the same geometry, appearance, and logical placement as before save.

Final result after Milestone 4

At the end of Milestone 4, the developer should have:

drag-start of rectangle and circle from the left panel

drag preview and canvas drop-target feedback

correct drop behavior into DiagramCanvas

real Rectangle and Circle item creation in World Space

insertion of those items into the CanvasItems model

automatic activation/selection of the new item after drop

JSON save/open support for the committed diagram state

trusted database save/load support through the C# layer

a clean foundation for future milestones such as move, resize, hover, snapping, and containment

Recommended implementation order

Use this order:

Confirm prerequisites and reuse Milestone 1 drag source items plus Milestone 2 screen-to-world conversion.

Implement temporary drag state, drag preview, and canvas drop-target detection.

Implement accepted drop handling and create real Rectangle/Circle items in World Space.

Activate the newly created item and clear drag state cleanly.

Save and reopen the committed diagram through JSON.

Save and reopen the committed diagram through the database using C# validation and persistence.

After creation is stable, add JSON save/reopen, then database save/reopen.

Then implement world-space drop creation and active-item state.

Confirm the left-panel drag source and canvas drop-target behavior first.

Recommended implementation order

Milestone 5 — Rectangle Interaction Behavior on DiagramCanvas

Overview

Milestone 5 defines the interaction behavior of a **Rectangle** after it has already been created on the DiagramCanvas.

This milestone focuses on **user actions applied to an existing rectangle**, including:

- hover over rectangle perimeter and hover padding
- left click on rectangle
- selection state
- drag / move rectangle on canvas
- movement constraints
- resize control point hover behavior
- resize control point selection
- resizing from each of the 8 rectangle resize control points

This milestone is the first milestone that turns the rectangle from a static canvas item into an **interactive editable item**.

A core design rule for this milestone is:

- **JavaScript** is responsible for hit testing, hover detection, selection handling, drag movement, resize behavior, visual feedback, temporary interaction state, coordinate conversion, and immediate rendering
- **C#** is responsible only for trusted validation, persistence, and future server-side save/load rules if required
- for Milestone 5, the main implementation remains primarily on the **JavaScript side**

Another important rule for this milestone is:

- all interaction geometry checks should occur in **world space**
- screen-space mouse input should first be converted to world space
- rectangle movement and resize behavior must respect:
 - no-overlap rules
 - parent-container containment rules if a parent exists
 - strict no-touch rules if enabled
- hover feedback must use **Hover Padding**
- overlap and containment validation must use **Protection Padding**

This milestone should use the same function-contract structure already defined for this project:

- **Trigger**
- **Input**
- **Validation / Preconditions**

- Processing Responsibility
- Output
- State Change / Persistence Effect
- Next Possible Call(s)
- Error Output / Failure Path

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
rectangle-interaction.css	CSS	Style sheet for rectangle hover padding activation, selection state, White resize-point visibility, and move/resize feedback.	Page load, and whenever the owning component visual state is redrawn. Call count: stylesheet parsed once per page load; browser reapplies styles automatically.	Rectangle DOM/canvas classes and visual state flags	Rectangle render surface exists	Applies visual rules for hover padding, selected outline, and resize-point display	Consistent Rectangle interaction styling	No persistence effect	render-canvas.js; rectangle-hover.js	Rectangle visuals may fall back to default styling
rectangle-hover.js	JS	Detects Rectangle move over	Pointer move over	Pointer screen position;	Canvas initialized; Rectangle	Detect whether	Updated RectangleHoverState	Temporary client-side state	showRectangleHoverPadding; renderCanvas	Clear hover safely if hit

rectangle-
selection.js

J
S

Handles
Rectangle
angle
selection
when the
user per-
forms Left

Left Click
Down on
Rectangle
Hover
Padding
or empty
canvas.
Call
count:
once per
selection
action.

Pointer
position;
current
active
item;
Rectangle
hover
result;
selection
state

Canv
as
initiali-
zed;
hit-
test
availa-
ble

Selects
the
clicked
Rectang-
le or
clears
selectio-
n
accordin-
g to UI
rule

Updated
Selected
Rectangl-
eId and
ActiveR-
ectangle
Id

Tempo-
rary
state
chang-
e until
save

renderCanvas;
updateDebugPanel

Prese-
rve
prior
select-
ion if
select-
ion
updat-
e fails

angle
e
Hover
Padding
entry
/exit,
turns
Hover
e
Pad-
ding
to
the
Rect-
angle
e
line
color
, and
sho-
ws
the
trans-
pare-
nt
resiz-
e
recta-
ngle
s in
White.
e.

canvas.
Call
count:
once per
processed
hover
update
while the
pointer is
moving.

canvas
state;
Rectangle
geometry;
Hover
Padding
rules;
resize-
point
visibility
state

ngle
geom-
etry
exists

er the
point
er is
inside
Recta-
ngle
Hover
Paddi-
ng,
updat-
e
hover
state,
chan-
ge
Hover
Paddi-
ng
color
to the
Recta-
ngle
line
color,
and
expos-
e
White
resize-
-point
visibili-
ty
while
hover-
ed.

chang-
e only

testin-
g fails

Click
Down on
Rectangle
Hover
Padding,
and
clear
selection
when
rules
require it.

rectangle-
drag.js

J
S

Moves
the
selected
Rectangle
angle by
mouse
drag
under
movement
constraints
and
previous-
valid
-

Left Click
Down on
the
selected
Rectangle
move
region,
followed
by
processed
move
updates
while Left
Click
remains
active.
Call
count:
once per
processed
move
update,
limited by
MaxMove

Current
pointer
position;
selected
Rectangle
; viewport
conversion;
MaxMove
Validation
Rate;
previous
valid
Rectangle
position;
sibling/parent
context if
contained

Rectangle
is selected;
drag allowed;
conversion valid

Computes
candidate move,
validates
overlap/
containment,
applies
accepted center

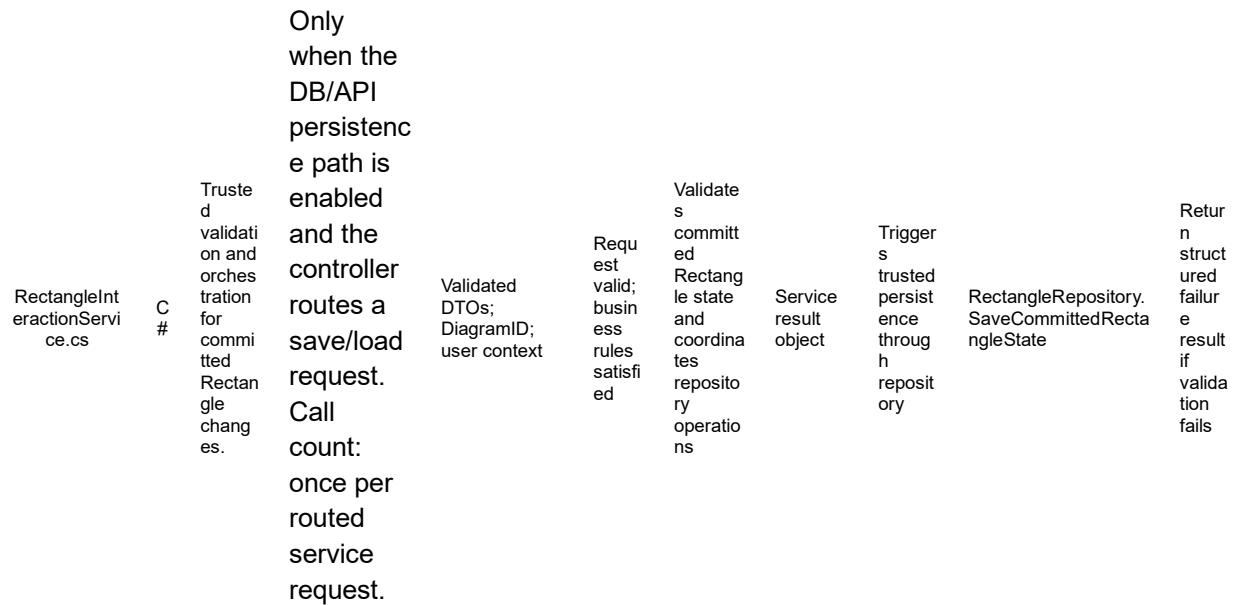
Updated
CenterX
and
CenterY

Committed in-
memory
geometry
changes;
persisted only
on
save

recalculateRectangle
DerivedGeometry;
renderCanvas

Restore
previous
valid
center on
invalid
move

rectangle-context-menu.js	J S	Validation rules .	Opens the Rectangle-specific context menu for Right Click actions on a Rectangle target.	Right Click on a Rectangle target. Call count: once per Right Click menu request.	Pointer position; Rectangle target identity; current selection state	Pointer is on a valid Rectangle target	Resolve the Rectangle target and open the correct context menu for that Rectangle.	Visible Rectangle context menu	Temporary UI state only	Follow-up command handlers chosen from the menu	Ignore the request if no valid Rectangle target is found
RectangleInteractionController.cs	C #	API controller for committed Rectangle save/load operations.	Only when the DB/API persistence path is enabled and JavaScript sends an HTTP save/load request. Call count: once per API request.	Rectangle payload; DiagramID; user/request context	Request model valid; routing available	Routes committed rectangle-related diagram requests to service layer	HTTP response object	No direct persistence effect except via service/repository flow	RectangleInteractionService.SaveDiagram		Return controlled validation or server error response



The table below follows the Milestone 2 file-definition style and includes CSS, JavaScript, and C# files relevant to Rectangle Interaction Behavior on DiagramCanvas. For move-related rows, call count is based on processed updates limited by MaxMoveValidationRate, not raw hardware mouse reports.

Recommended files for Milestone 5

Database table	Type	Purpose
Rectangles	Core	Stores committed Rectangle canonical geometry and appearance values.
CanvasItems	Core	Stores item identity, type, DiagramID, CanvasID, and Z-order linkage for all canvas items.
DiagramPayloads	Optional snapshot	Stores committed full-diagram payload snapshots for reopen and audit.
RectangleChangeLog	Optional audit	Stores committed rectangle move/resize history if auditability is needed later.

The table below lists the main database tables used or extended by Milestone 5.

Recommended database table structure

Variable name	Description	Recommended values
ActiveRectangleId	Current selected Rectangle item on the canvas.	Valid RectangleId / GUID
IsRectangleHovered	Indicates whether a Rectangle hover region is active.	True / False
HoveredRectangleId	Rectangle currently under hover hit testing.	Valid RectangleId or null
IsRectangleSelected	Indicates whether a Rectangle is the active selected item.	True / False
SelectedRectangleId	Current selected Rectangle identity.	Valid RectangleId or null
IsRectangleDragging	Indicates active Rectangle movement state.	True / False
IsRectangleResizing	Indicates active Rectangle resize state.	True / False
ActiveResizeControlPoint	Current resize control point being dragged.	TopLeft / TopCenter / TopRight / RightCenter / BottomRight / BottomCenter / BottomLeft / LeftCenter

CandidateCenterX	Candidate Rectangle center X during move or resize.	Numeric world-space value
CandidateCenterY	Candidate Rectangle center Y during move or resize.	Numeric world-space value
CandidateHalfWidth	Candidate Rectangle half-width during resize.	Positive numeric value
CandidateHalfHeight	Candidate Rectangle half-height during resize.	Positive numeric value
PreviousValidCenterX	Last accepted Rectangle center X.	Numeric world-space value
PreviousValidCenterY	Last accepted Rectangle center Y.	Numeric world-space value

The table below includes the main Milestone 5 working variables.

Milestone 5 variables

Minimum readiness rule: Milestone 5 should not be considered ready to begin until the team confirms at least this: an existing Rectangle can be selected on DiagramCanvas and its canonical world-space geometry can be recalculated safely.

Required confirmations: Milestone 3 Rectangle definition is complete; Milestone 4 drag-and-drop creation is already working for Rectangle items; screenToWorld and worldToScreen remain stable; Protection Padding calculation is available for movement and resize validation; ActiveCanvasItem state and CanvasItems collection are already working; and JSON/database save plumbing is available for the later persistence phases.

Before Milestone 5 implementation starts, the following prerequisites should be confirmed.

Prerequisites

Mandatory existing JavaScript before starting Milestone 5

Only JavaScript files that must already exist and work correctly before this milestone starts are listed here.

Shared save prerequisite. Before starting Milestone 5, the team shall verify that save-to-json.js and save-to-db.js are already saving all committed variables approved in Milestones 1 to 4 correctly. When Milestone 5 introduces new committed variables, the developer shall extend only these same two files so the current milestone save test checks the Milestone 5 variables, while regression validation confirms that all previous milestone variables are still saved correctly.

JavaScript file that must already exist	Description	Reason this JavaScript must already be ready and working correctly
create-rectangle-on-drop.js	Creates a committed Rectangle item after a valid canvas drop.	Milestone 5 can only add Rectangle interaction after real Rectangle items already exist on the canvas.
canvas-item-factory.js	Assigns item IDs and builds normalized canvas items.	Rectangle interaction depends on stable item identity and normalized item creation.
canvas-item-selection.js	Activates the created canvas item and manages active item state.	Rectangle hover, selection, move, and resize need a stable active-item path.
derive-shape-geometry.js	Recalculates derived shape boundaries from canonical geometry.	Rectangle interaction uses derived boundaries, padding, and control-point geometry.
render-rectangle.js	Renders rectangle items from canonical and derived values.	Rectangle interaction must update a visible rectangle on every accepted state change.

screen-to-world.js	Converts pointer positions into world coordinates.	Rectangle move and resize calculations must work in world space.
--------------------	--	--

Milestone 5 terminology

Main developer names

The following names are recommended for this milestone:

Rectangle item

- CanvasRectangleItem

Rectangle state

- RectangleHoverState
- RectangleSelectedState
- RectangleDragState
- RectangleResizeState

Hover regions

- RectangleVisibleBoundary
- RectangleHoverPaddingRegion
- RectangleProtectionPaddingRegion

Resize control points

- TopLeftResizeControlPoint
- TopCenterResizeControlPoint
- TopRightResizeControlPoint
- RightCenterResizeControlPoint
- BottomRightResizeControlPoint
- BottomCenterResizeControlPoint
- BottomLeftResizeControlPoint
- LeftCenterResizeControlPoint

Selected resize point

- ActiveResizeControlPoint

Movement and resize validation

- validateRectangleMove
- validateRectangleResize

- clampRectangleMoveWithinParent
- checkRectangleOverlap
- recalculateRectangleDerivedGeometry

Main rectangle interaction collection

- ActiveCanvasRectangle
- SelectedCanvasRectangle
- HoveredCanvasRectangle

Important milestone rules

Rule 1 — Hover Padding is for hover and hover-like highlight behavior

The rectangle's **Hover Padding** is the region used for:

- perimeter-sensitive hover detection
- hover highlight behavior
- optional selected highlight behavior if you choose to reuse the same visual style

By default, the Hover Padding is transparent.

When hover is active near the rectangle perimeter, the Hover Padding shall take the same color as the rectangle line.

Rule 2 — Protection Padding is for movement and resize validation

The rectangle's **Protection Padding** shall be used for:

- non-overlap rules
- spacing rules
- container containment checks
- movement validation
- resize validation

It shall not be used as the main hover-highlight region.

Rule 3 — Selection state should be separate from hover state

A left click on the rectangle should make it the active selected rectangle.

The system shall define:

- IsRectangleHovered
- IsRectangleSelected

as separate logical states.

You may still choose the visual rule that selected state also keeps the hover padding highlighted in the rectangle line color, but logically selection and hover should not be treated as the same state.

Rule 4 — Resize control points are derived, not independent

The rectangle has **8 resize control points**, and their positions must always be derived from rectangle geometry.

They are not independent stored primary geometry.

If the rectangle moves or resizes, all 8 control point positions must be recalculated.

Rule 5 — Movement must be validated before commit

When the user drags a selected rectangle, the system should:

- calculate the candidate new world position
- validate that candidate position against:
 - overlap rules
 - parent-container rules if any
- accept the move only if valid, or clamp it to the nearest valid position if clamping is the chosen policy

Your document already recommends clamped movement for containment.

Rule 6 — Resizing must be direction-specific

Each resize control point has its own allowed resize behavior:

- corner control points change both X and Y boundaries
- edge-middle control points change only one boundary

This is already defined in your Rectangle Resize Control Point section and should be preserved exactly.

Mathematical Calculation Rules for Milestone 5

Milestone 5 calculations control rectangle hover, selection, move, resize, derived-geometry recalculation, and committed rectangle persistence. All calculations shall be performed in world space after converting pointer input from screen space.

Primary JavaScript files for this calculation scope: rectangle-hover-hit-test.js, rectangle-selection.js, compute-rectangle-candidate-center-on-move.js, rectangle-resize-control-point-hit-test.js, resize-rectangle.js, recalculate-rectangle-derived-geometry-on-resize.js, rectangle-overlap-on-move.js, restore-previous-valid-position.js.

Rectangle hover and selection

Pointer is inside the rectangle hover outer region when $\text{PointerX} \geq \text{RectHoverLeftX}$ and $\text{PointerX} \leq \text{RectHoverRightX}$ and $\text{PointerY} \leq \text{RectHoverTopY}$ and $\text{PointerY} \geq \text{RectHoverBottomY}$.

Pointer is in the rectangle hover band when it is inside the hover outer region and not strictly inside the visible rectangle interior.

Move calculation

$\text{CandidateCenterX} = \text{PreviousCenterX} + \text{DeltaWorldX}$. $\text{CandidateCenterY} = \text{PreviousCenterY} + \text{DeltaWorldY}$.

The candidate center shall be accepted only after Protection Padding non-overlap validation passes. If validation fails, CenterX and CenterY shall be restored from PreviousValidCenterX and PreviousValidCenterY.

Resize calculation

Each rectangle resize control point updates only the boundaries it owns. Corner control points update one X boundary and one Y boundary. Edge-middle control points update one boundary only.

After an accepted resize: $\text{NewCenterX} = (\text{NewLeftX} + \text{NewRightX}) / 2$; $\text{NewCenterY} = (\text{NewBottomY} + \text{NewTopY}) / 2$; $\text{NewHalfWidth} = (\text{NewRightX} - \text{NewLeftX}) / 2$; $\text{NewHalfHeight} = (\text{NewTopY} - \text{NewBottomY}) / 2$.

Milestone 5 phases

Milestone 5 shall be implemented in 4 phases.

Phase 5.1 — Hover detection and rectangle selection behavior

For more information, please refer to section [Hover, selection, and visual activation calculations](#) ([place in this document](#)).

Primary JavaScript files for this phase: geometry-hover-activation.js; geometry-selection.js; resize-control-point-hit-test.js.

Goal

Define rectangle hover behavior, hover padding color behavior, perimeter-sensitive hit testing, resize control point hover visibility, and rectangle selection by left click.

Scope

This phase is about:

- detecting mouse hover over rectangle perimeter / hover padding region
- changing Hover Padding color from transparent to rectangle line color
- changing resize control point colors from transparent to white during valid hover
- selecting the rectangle by left click
- maintaining a clear selected state

This phase is not yet about moving or resizing.

Required user actions

- hover over rectangle perimeter or hover padding region
- hover over resize control points
- left click on rectangle
- left click on hover-sensitive rectangle region
- left click away from rectangle to clear selection if desired by your UI rule

Required visible behaviors

Examples:

- when the pointer enters the rectangle hover region, the Hover Padding changes from transparent to the rectangle line color
- when the pointer enters the rectangle hover region, the resize control points become visible in white
- when the rectangle is left-clicked, it becomes the active selected rectangle
- when selected, the rectangle may keep the same highlighted Hover Padding color
- if another rectangle is selected later, the previous selected rectangle loses selected state unless multi-selection is added in a future milestone

Variables to define

At minimum:

Hover state

- IsRectangleHovered
- HoveredRectangleId

Selection state

- IsRectangleSelected
- SelectedRectangleId

Visual state

- CurrentHoverPaddingColor
- AreResizeControlPointsVisible
- ResizeControlPointColor

Function contract deliverables

The following JavaScript functions or modules should be defined in this phase:

- hitTestRectangleHoverRegion

- hitTestRectanglePerimeter
- hitTestRectangleResizeControlPoints
- setRectangleHoverState
- clearRectangleHoverState
- showRectangleHoverPadding
- hideRectangleHoverPadding
- showRectangleResizeControlPoints
- hideRectangleResizeControlPoints
- selectRectangleOnLeftClick
- clearRectangleSelection
- setActiveCanvasRectangle

Required event-to-function routing

Hover flow

mousemove over canvas

→ screenToWorld
 → hitTestRectangleHoverRegion
 → setRectangleHoverState
 → showRectangleHoverPadding
 → showRectangleResizeControlPoints
 → renderCanvas

Left-click selection flow

left click on rectangle or valid hover region

→ screenToWorld
 → hitTestRectangleHoverRegion
 → selectRectangleOnLeftClick
 → setActiveCanvasRectangle
 → renderCanvas

State rule

During this phase:

- only rectangle hover and selection state is updated
- no movement occurs yet
- no resize occurs yet
- no persistence is required

Output

This phase should produce:

- correct rectangle hover behavior
- correct hover padding highlight behavior
- correct white resize control point visibility during hover
- correct rectangle selection on left click

Validation / Preconditions

Before Phase 5.1 is considered valid:

- the rectangle must already exist on the canvas
- rectangle world geometry must be available
- hover padding region must already be derivable from rectangle geometry
- resize control point positions must already be derivable from rectangle geometry

Error / failure expectations

Examples of failure behavior:

- if rectangle geometry is invalid, hover must not activate
- if control point geometry cannot be derived, keep them hidden
- if click does not hit a valid rectangle region, selection must not change unless clear-selection-on-empty-space is enabled
- if hover state becomes inconsistent, clear hover safely

Success criteria

This phase is successful if:

- hovering over the rectangle activates hover padding correctly
- hovering over the rectangle makes resize control points visible in white
- left click selects the rectangle correctly
- hover state and selected state are both clearly defined

Phase 5.2 — Drag / move the selected rectangle on the canvas under movement constraints

For more information, please refer to section [Move-update calculations and MaxMoveValidationRate \(place in this document\)](#) and section [Protection Padding, Overlap, Containment, and Persistence Calculations \(place in this document\)](#).

Primary JavaScript files for this phase: move-update-scheduler.js; detect-render-rate.js; compute-rectangle-candidate-center-on-move.js; recalculate-rectangle-protection-padding.js; rectangle-overlap-on-move.js; validate-child-containment.js; restore-previous-valid-position.js; apply-validated-center.js; update-previous-valid-position.js.

Goal

Allow the selected rectangle to be dragged and moved on the canvas while respecting overlap and parent-container constraints.

Scope

This phase is about:

- starting a rectangle drag
- updating rectangle position during mouse movement
- validating movement using Protection Padding
- enforcing no-overlap rules
- enforcing parent-container containment if a parent exists
- applying clamped movement if required

Required user actions

- left click on rectangle to select
- left click and drag selected rectangle
- release mouse to finish movement

Important movement rule

Movement should be allowed only after the rectangle is selected.

Recommended sequence:

- hover rectangle
- left click rectangle to select
- left click and drag selected rectangle to move it

This keeps the interaction model cleaner.

Required visible behaviors

Examples:

- dragging updates rectangle world position continuously
- hover padding and resize control point visibility remain visually consistent during drag if desired
- the rectangle must not overlap forbidden regions

- the rectangle must not leave its parent container if one exists
- the rectangle must not touch the parent boundary if strict no-touch is enabled

Variables to define

At minimum:

Drag state

- IsRectangleDragging
- DraggedRectangleId
- DragStartWorldX
- DragStartWorldY
- RectangleDragOffsetX
- RectangleDragOffsetY

Candidate movement

- CandidateCenterX
- CandidateCenterY
- ValidatedCenterX
- ValidatedCenterY

Function contract deliverables

The following JavaScript functions or modules should be defined in this phase:

- beginRectangleDrag
- updateRectangleDrag
- endRectangleDrag
- computeCandidateRectanglePosition
- validateRectangleMove
- checkRectangleOverlapUsingProtectionPadding
- clampRectangleMoveWithinParent
- applyValidatedRectangleMove
- recalculateRectangleDerivedGeometry
- renderCanvas

Required event-to-function routing

Rectangle move flow

left click and drag on selected rectangle
→ beginRectangleDrag
→ mousemove
→ screenToWorld
→ computeCandidateRectanglePosition
→ validateRectangleMove
→ checkRectangleOverlapUsingProtectionPadding
→ clampRectangleMoveWithinParent
→ applyValidatedRectangleMove
→ recalculateRectangleDerivedGeometry
→ renderCanvas

State rule

During this phase:

- rectangle center changes in world coordinates
- viewport does not change
- the rectangle remains a real canvas item
- no database write is required yet

Output

This phase should produce:

- correct rectangle movement on the canvas
- movement based on world coordinates
- no-overlap enforcement
- parent-container enforcement when applicable
- recalculated derived rectangle boundaries after every accepted move

Validation / Preconditions

Before Phase 5.2 is considered valid:

- the rectangle must already be selectable
- drag must only start on a selected rectangle
- Protection Padding boundaries must be derivable
- parent-container information must be known if the rectangle is a child item

Error / failure expectations

Examples of failure behavior:

- if candidate move is invalid, reject or clamp it according to the chosen policy
- if overlap detection fails, preserve the previous valid rectangle position

- if container boundaries are missing for a child rectangle, movement must fail safely
- if drag state becomes inconsistent, cancel drag and preserve the previous valid rectangle position

Success criteria

This phase is successful if:

- the selected rectangle can be dragged smoothly
- movement updates rectangle world coordinates correctly
- no invalid overlap is allowed
- the rectangle stays within the allowed parent area if it has a parent
- derived geometry updates correctly after movement

Phase 5.3 — Resize control point hover and 8-direction rectangle resize behavior

For more information, please refer to section [Hover, selection, and visual activation calculations \(place in this document\)](#) and section [Protection Padding, Overlap, Containment, and Persistence Calculations \(place in this document\)](#).

Primary JavaScript files for this phase: geometry-hover-activation.js; geometry-selection.js; resize-control-point-hit-test.js; recalculate-rectangle-derived-geometry-on-resize.js; recalculate-rectangle-protection-padding.js; rectangle-overlap-on-resize.js; validate-child-containment.js; restore-previous-valid-position.js; apply-validated-center.js.

Goal

Allow the user to hover over, select, and drag any of the 8 rectangle resize control points, and resize the rectangle according to the selected control point behavior.

Scope

This phase is about:

- hover detection on resize control points
- left click on a resize control point
- direction-specific resize behavior
- recalculation of rectangle geometry after resize
- resize validation against overlap and parent-container rules

Whenever rectangle resize changes HalfWidth, HalfHeight, or boundary positions, recalculate-rectangle-derived-geometry-on-resize.js shall run before overlap validation or rendering so that Hover Padding, Protection Padding, and all 8 rectangle resize control points are refreshed from the updated canonical geometry.

Required user actions

- hover over one of the 8 resize control points
- left click on a resize control point
- drag the selected resize control point
- release mouse to finish the resize

Required visible behaviors

Examples:

- hovering over rectangle already shows the control points in white
- clicking one control point makes it the active resize point
- dragging that control point resizes the rectangle according to its direction rule
- the rectangle must remain valid during resize
- the control points must move with the updated rectangle boundaries

8 resize behaviors

These shall follow your existing rectangle rules:

Corner control points

- TopLeftResizeControlPoint changes left and top boundaries
- TopRightResizeControlPoint changes right and top boundaries
- BottomRightResizeControlPoint changes right and bottom boundaries
- BottomLeftResizeControlPoint changes left and bottom boundaries

Edge-middle control points

- TopCenterResizeControlPoint changes top boundary only
- RightCenterResizeControlPoint changes right boundary only
- BottomCenterResizeControlPoint changes bottom boundary only
- LeftCenterResizeControlPoint changes left boundary only

Variables to define

At minimum:

Resize state

- IsRectangleResizing
- ResizedRectangleId
- ActiveResizeControlPoint

- ResizeStartWorldX
- ResizeStartWorldY

Candidate resize

- CandidateHalfWidth
- CandidateHalfHeight
- CandidateCenterX
- CandidateCenterY

Function contract deliverables

For your super-micro structure, Recommendation:

Coordinator functions

- beginRectangleResize
- updateRectangleResize
- endRectangleResize
- validateRectangleResize
- applyValidatedRectangleResize
- recalculateRectangleDerivedGeometry

8 direction-specific resize handlers

- resize-rectangle-top-left.js
- resize-rectangle-top-center.js
- resize-rectangle-top-right.js
- resize-rectangle-right-center.js
- resize-rectangle-bottom-right.js
- resize-rectangle-bottom-center.js
- resize-rectangle-bottom-left.js
- resize-rectangle-left-center.js

Yes — for your super-micro design, **8 separate JS files** for the 8 control-point resize behaviors is reasonable.

Required event-to-function routing

Generic resize flow

left click on resize control point
→ beginRectangleResize
→ setActiveResizeControlPoint
→ mousemove
→ screenToWorld
→ direction-specific resize handler
→ validateRectangleResize
→ applyValidatedRectangleResize
→ recalculateRectangleDerivedGeometry
→ renderCanvas

State rule

During this phase:

- resize changes rectangle geometry in world coordinates
- resize does not change the viewport
- resize must respect overlap and parent-container rules if enabled
- no database write is required yet

Output

This phase should produce:

- correct resize control point activation
- correct 8-direction resize behavior
- recalculated rectangle boundaries after resize
- recalculated resize control point positions after resize
- valid resizing under current rectangle rules

Validation / Preconditions

Before Phase 5.3 is considered valid:

- resize control points must already be derivable
- rectangle must already support hover and selection
- minimum rectangle size rules must be defined
- overlap and parent-container validation rules must be callable during resize

Error / failure expectations

Examples of failure behavior:

- if the active resize control point is invalid, cancel resize
- if a candidate resize creates invalid geometry, reject or clamp it

- if a candidate resize breaks parent containment, reject or clamp it
- if a candidate resize causes forbidden overlap, reject or clamp it
- if a resize handler fails, preserve the previous valid rectangle geometry

Success criteria

This phase is successful if:

- the user can resize from any of the 8 control points
- each control point affects only the correct boundaries
- derived geometry and control point positions are updated correctly
- invalid resize operations are safely rejected or clamped

Phase 5.4 — Finalize rectangle interaction state and prepare for future milestones

Goal

Stabilize the rectangle interaction model after hover, selection, move, and resize are implemented, and prepare the rectangle for future milestones.

Scope

This phase is about:

- final rectangle post-action state
- selection persistence
- hover exit behavior
- drag end behavior
- resize end behavior
- future-ready rectangle interaction contract

Required visible behaviors

Examples:

- after moving or resizing, the rectangle remains selected
- hover styling may disappear when the mouse leaves if not selected
- if selected styling and hover styling are intentionally shared, that rule must be documented clearly
- resize control points may remain visible while selected, even if the mouse is not hovering, if that is your UI rule

Variables to define

At minimum:

- IsRectangleInteractionActive
- LastRectangleActionType
- LastRectangleActionResult
- KeepResizeControlPointsVisibleWhenSelected
- KeepHoverPaddingVisibleWhenSelected

Function contract deliverables

The following JavaScript functions or modules should be defined in this phase:

- finalizeRectangleHoverState
- finalizeRectangleSelectionState
- finalizeRectangleDragState
- finalizeRectangleResizeState
- prepareRectangleForNextAction
- updateDebugPanelForRectangle

Output

This phase should produce:

- a stable rectangle interaction model
- predictable post-hover, post-drag, and post-resize behavior
- a clear ready state for future rectangle-related milestones

Success criteria

This phase is successful if:

- hover, selection, move, and resize work together consistently
- the rectangle remains in a valid interaction state after each action
- the rectangle is ready for future advanced behaviors

Phase 5.5 — Save rectangle-related committed diagram changes to a JSON file

Exact variables to save in this phase

Milestone 5 introduces no new permanent persistence field names. The exact save requirement in this phase is: keep saving all approved Milestone 2 DiagramCanvas variables, and save the committed Rectangle-host record with its updated values after selection, move, or resize. The committed rectangle

fields that must persist are: RectangleID, RectangleCenterX, RectangleCenterY, RectangleHalfWidth, RectangleHalfHeight, RectangleRotationAngle, RectangleZOrder, RectangleFillType, RectangleFillColor, RectangleLineType, RectangleLineColor, RectangleLineWidth, RectangleHoverPaddingXRatio, RectangleHoverPaddingYRatio, RectangleHoverPaddingColor, RectangleProtectionPaddingXRatio, RectangleProtectionPaddingYRatio, RectangleProtectionPaddingColor, plus the owning Device or Container identity fields that were already approved in Milestone 3.

Do not save Milestone 5 runtime-only interaction variables as milestone-required persistence data: ActiveRectangleId, IsRectangleHovered, HoveredRectangleId, IsRectangleSelected, SelectedRectangleId, IsRectangleDragging, IsRectangleResizing, ActiveResizeControlPoint, CandidateCenterX, CandidateCenterY, CandidateHalfWidth, CandidateHalfHeight, PreviousValidCenterX, PreviousValidCenterY.

Save regression rule for this phase

All approved Milestone 2, Milestone 3, and Milestone 4 committed variables shall already remain saved correctly. This phase passes only if save-to-json.js stores the updated committed rectangle state while keeping the previous approved save set intact.

Save test criteria for this phase

Verify that JSON save captures the updated committed rectangle values listed above, that the runtime-only Milestone 5 variables listed above are not treated as required saved data, and that reopen restores the rectangle exactly as committed after move or resize.

Goal

Allow the user to save the current diagram, including committed rectangle interaction changes, into a JSON file after pressing the **Save** button, and later reopen that JSON file through **File** → **Open** so that the diagram is restored correctly.

Scope

This phase is about saving the diagram after committed rectangle actions such as:

- rectangle creation
- rectangle movement
- rectangle resize
- committed rectangle property changes if later supported

This phase is not about saving temporary rectangle interaction-only states.

Important save rule

The JSON file shall save the committed rectangle state inside the diagram model.

That includes things such as:

- RectangleID
- CenterX
- CenterY

- HalfWidth
- HalfHeight
- RotationAngle
- ZOrder
- fill and line values
- hover padding configuration values
- protection padding configuration values
- parent container relationship if one exists

It shall not save temporary interaction-only states such as:

- currently hovered rectangle
- temporary hover padding color caused only by hover
- temporary drag state before mouse release
- temporary resize state before mouse release

Variables to define

At minimum:

- DiagramID
- DiagramVersion
- RectangleSaveJsonObject
- DiagramJsonObject
- OpenedJsonFileName
- LoadedDiagramJsonObject

Function contract deliverables

The following functions or modules should be defined in this phase:

- buildDiagramJsonObject
- buildRectangleJsonData
- serializeDiagramToJson
- saveDiagramJsonFile
- openDiagramJsonFile
- parseDiagramJsonFile
- validateDiagramJsonStructure

- rebuildDiagramFromJson
- rebuildRectanglesFromJson
- renderCanvas

Required event-to-function routing

Save-to-file flow

click Save button

→ buildRectangleJsonData
 → buildDiagramJsonObject
 → serializeDiagramToJson
 → saveDiagramJsonFile

Open-from-file flow

File → Open

→ openDiagramJsonFile
 → parseDiagramJsonFile
 → validateDiagramJsonStructure
 → rebuildDiagramFromJson
 → rebuildRectanglesFromJson
 → renderCanvas

Output

This phase should produce:

- a valid JSON file containing the diagram and rectangle state
- correct reconstruction of rectangles after file open
- preservation of committed rectangle position and size

Validation / Preconditions

Before save or open is accepted:

- committed rectangle geometry must be valid
- resize and move operations must already be committed before save
- the JSON structure must contain the required rectangle fields

Error / failure expectations

Examples of failure behavior:

- if rectangle serialization fails, the save must fail safely
- if rectangle geometry is incomplete in JSON, the file must not be loaded as a valid full diagram
- if one rectangle cannot be reconstructed, the load operation must fail safely or report the failure clearly

Success criteria

This phase is successful if:

- pressing **Save** produces a valid diagram JSON file
- committed rectangle changes are present in the file
- the user can use **File** → **Open** to reopen the JSON file
- the diagram is restored correctly after open
- rectangles reappear in the correct positions and sizes as saved

Phase 5.6 — Save rectangle-related committed diagram changes to the database

Exact variables to save in this phase

Milestone 5 introduces no new permanent persistence field names. The exact save requirement in this phase is: keep saving all approved Milestone 2 DiagramCanvas variables, and save the committed Rectangle-host record with its updated values after selection, move, or resize. The committed rectangle fields that must persist are: RectangleID, RectangleCenterX, RectangleCenterY, RectangleHalfWidth, RectangleHalfHeight, RectangleRotationAngle, RectangleZOrder, RectangleFillType, RectangleFillColor, RectangleLineType, RectangleLineColor, RectangleLineWidth, RectangleHoverPaddingXRatio, RectangleHoverPaddingYRatio, RectangleHoverPaddingColor, RectangleProtectionPaddingXRatio, RectangleProtectionPaddingYRatio, RectangleProtectionPaddingColor, plus the owning Device or Container identity fields that were already approved in Milestone 3.

Do not save Milestone 5 runtime-only interaction variables as milestone-required persistence data: ActiveRectangleId, IsRectangleHovered, HoveredRectangleId, IsRectangleSelected, SelectedRectangleId, IsRectangleDragging, IsRectangleResizing, ActiveResizeControlPoint, CandidateCenterX, CandidateCenterY, CandidateHalfWidth, CandidateHalfHeight, PreviousValidCenterX, PreviousValidCenterY.

Save regression rule for this phase

All approved Milestone 2, Milestone 3, and Milestone 4 committed variables shall already remain saved correctly. This phase passes only if save-to-db.js stores the updated committed rectangle state while keeping the previous approved save set intact.

Save test criteria for this phase

Verify that database save captures the updated committed rectangle values listed above, that the runtime-only Milestone 5 variables listed above are not treated as required saved data, and that reload restores the rectangle exactly as committed after move or resize.

Goal

Allow the user to save the current diagram, including committed rectangle interaction changes, into the database by DiagramID, and later reopen the same saved diagram from the database so that it appears exactly as it was saved.

Scope

This phase is about saving the diagram after committed rectangle changes such as:

- rectangle move completion
- rectangle resize completion
- committed rectangle property changes if later supported

Important save rule

The system shall save **only committed rectangle state**, not temporary interaction-only state.

That means the database save shall include:

- final committed rectangle geometry
- final rectangle appearance values
- final parent-container relation if applicable
- final diagram item collection state

It shall not persist:

- temporary hover-only visual state
- temporary drag-preview state
- temporary resize-preview state before commit

Recommended persistence identity

Use the same permanent diagram identity:

- DiagramID = GUID / UUID string

Recommended additional persistence fields:

- DiagramVersion
- UpdatedAt
- UpdatedBy if user identity exists later

Variables to define

At minimum:

- DiagramID
- DiagramVersion
- RectangleUpdatePayload
- DiagramSavePayload
- LoadedDiagramID
- LoadedRectangleData

- LastDatabaseSaveResult

Function contract deliverables

The following functions or modules should be defined in this phase:

JavaScript

- buildRectangleUpdatePayload
- buildDiagramSavePayload
- callSaveDiagramApi
- callLoadDiagramApi
- applyLoadedDiagramData
- renderCanvas

C#

- DiagramController.SaveDiagram
- DiagramController.LoadDiagram
- DiagramService.SaveDiagram
- DiagramService.LoadDiagram
- DiagramRepository.SaveDiagram
- DiagramRepository.LoadDiagramById

Required event-to-function routing

Save-to-database flow

rectangle move completed or rectangle resize completed

→ buildRectangleUpdatePayload
 → buildDiagramSavePayload
 → callSaveDiagramApi
 → DiagramController.SaveDiagram
 → DiagramService.SaveDiagram
 → DiagramRepository.SaveDiagram

Open-from-database flow

Open Diagram by DiagramID

→ callLoadDiagramApi
 → DiagramController.LoadDiagram
 → DiagramService.LoadDiagram
 → DiagramRepository.LoadDiagramById
 → applyLoadedDiagramData
 → renderCanvas

State rule

During this phase:

- committed rectangle changes become part of the trusted saved diagram state
- the diagram is stored by DiagramID
- reopening by DiagramID must rebuild the same saved rectangle state

Output

This phase should produce:

- a working database save path for rectangle-related diagram changes
- a working database load path by DiagramID
- correct persistence of committed rectangle movement and resize results
- correct reconstruction of the saved diagram

Validation / Preconditions

Before save or load is accepted:

- committed rectangle geometry must be valid
- DiagramID must be valid
- payload structure must be valid
- server-side validation must pass before database write

Error / failure expectations

Examples of failure behavior:

- if rectangle update payload is invalid, save must fail safely
- if database save fails, the UI must not show false success
- if load-by-DiagramID fails, the current diagram must remain unchanged unless the user explicitly replaces it
- if returned rectangle data is incomplete, reconstruction must fail safely

Success criteria

This phase is successful if:

- pressing **Save** stores the diagram in the database
- committed rectangle changes are saved under a valid DiagramID
- the user can reopen the saved diagram from the database
- the reopened diagram appears as it was saved

- rectangle position, size, and committed appearance values are restored correctly

Phase 5.7 — Recalculate committed rectangle geometry and persist the diagram to both JSON and database

Exact variables to save in this phase

Milestone 5 introduces no new permanent persistence field names. The exact save requirement in this phase is: keep saving all approved Milestone 2 DiagramCanvas variables, and save the committed Rectangle-host record with its updated values after selection, move, or resize. The committed rectangle fields that must persist are: RectangleID, RectangleCenterX, RectangleCenterY, RectangleHalfWidth, RectangleHalfHeight, RectangleRotationAngle, RectangleZOrder, RectangleFillType, RectangleFillColor, RectangleLineType, RectangleLineColor, RectangleLineWidth, RectangleHoverPaddingXRatio, RectangleHoverPaddingYRatio, RectangleHoverPaddingColor, RectangleProtectionPaddingXRatio, RectangleProtectionPaddingYRatio, RectangleProtectionPaddingColor, plus the owning Device or Container identity fields that were already approved in Milestone 3.

Do not save Milestone 5 runtime-only interaction variables as milestone-required persistence data: ActiveRectangleId, IsRectangleHovered, HoveredRectangleId, IsRectangleSelected, SelectedRectangleId, IsRectangleDragging, IsRectangleResizing, ActiveResizeControlPoint, CandidateCenterX, CandidateCenterY, CandidateHalfWidth, CandidateHalfHeight, PreviousValidCenterX, PreviousValidCenterY.

Save regression rule for this phase

All approved Milestones 2 through 5 committed variables shall already remain saved correctly in both persistence paths. This phase passes only if the same recalculated committed rectangle state is written by both shared save files without reintroducing any runtime-only Milestone 5 variables as persistent truth.

Save test criteria for this phase

Verify that save-to-json.js and save-to-db.js both persist the same recalculated committed rectangle values, that the runtime-only Milestone 5 variables listed above are excluded, and that reopen from either source yields the same rectangle geometry and appearance.

Goal

Ensure that whenever the user presses the **Save** button after committed rectangle changes, the system first recalculates all required rectangle-derived values, validates the rectangle and diagram state, and then persists the committed diagram to:

- a JSON file
- the database

This phase guarantees that the saved data represents the **final correct committed rectangle state**, not incomplete or outdated geometry.

Scope

This phase is about:

- recalculating rectangle-derived geometry before save

- validating committed rectangle state
- rebuilding the final diagram save model
- saving the diagram to JSON
- saving the diagram to the database
- ensuring that reopening from either source restores the same diagram state

This phase applies after committed rectangle actions such as:

- rectangle creation
- rectangle movement
- rectangle resize
- committed rectangle property changes if supported later

This phase is **not** for temporary interaction-only states.

Important recalculation rule

Before any save operation is accepted, the system shall recalculate all required derived rectangle values from the rectangle's canonical stored values.

For a rectangle, the canonical stored values remain the primary truth, such as:

- RectangleID
- CenterX
- CenterY
- HalfWidth
- HalfHeight
- RotationAngle
- ZOrder
- fill values
- line values
- hover padding ratios
- protection padding ratios

From these canonical values, the system shall recalculate derived values such as:

Visible boundaries

- LeftX
- RightX

- TopY
- BottomY

Hover Padding boundaries

- HoverLeftX
- HoverRightX
- HoverTopY
- HoverBottomY

Protection Padding boundaries

- ProtectionLeftX
- ProtectionRightX
- ProtectionTopY
- ProtectionBottomY

Resize control point positions

- TopLeftResizeControlPoint
- TopCenterResizeControlPoint
- TopRightResizeControlPoint
- RightCenterResizeControlPoint
- BottomRightResizeControlPoint
- BottomCenterResizeControlPoint
- BottomLeftResizeControlPoint
- LeftCenterResizeControlPoint

These values must be recalculated whenever rectangle geometry has changed, and must be correct before JSON save or database save.

Important persistence rule

The Save button in this phase shall persist only the **committed diagram state**.

That means the system shall save:

- final rectangle geometry
- final rectangle appearance
- final diagram item collection
- final world-space positions

- final parent-container relationship if applicable

It shall not save temporary interaction-only states such as:

- current hover-only state
- temporary drag preview
- temporary resize-preview state before commit
- temporary mouse position

Recommended persistence identity

Use the same permanent diagram identity:

- DiagramID = GUID / UUID string

Recommended example:

- DiagramID = 9f3f6f8e-8b13-4ff2-8ed1-5d9b16d77c31

Recommended additional fields:

- DiagramVersion
- DiagramName
- CreatedAt
- UpdatedAt

Best rule:

- DiagramID is created once and remains stable
- DiagramVersion increments on each committed save

Variables to define

At minimum:

Diagram identity

- DiagramID
- DiagramVersion
- DiagramName

Rectangle recalculation state

- NeedsRectangleRecalculation
- LastRecalculatedRectangleId
- RectangleRecalculationResult

Final save model

- DiagramJsonObject
- DiagramJsonText
- DiagramSavePayload

Save results

- JsonSaveResult
- DatabaseSaveResult
- LastSaveCommitResult

Function contract deliverables

The following functions or modules should be defined in this phase:

JavaScript

- recalculateRectangleDerivedGeometry
- recalculateAllRectangleDerivedGeometry
- validateCommittedRectangleState
- validateCommittedDiagramState
- buildDiagramJsonObject
- serializeDiagramToJson
- saveDiagramJsonFile
- buildDiagramSavePayload
- callSaveDiagramApi
- handleSaveCommitResult

C#

- DiagramController.SaveDiagram
- DiagramService.SaveDiagram
- DiagramRepository.SaveDiagram

Optional helper functions:

- incrementDiagramVersion
- markDiagramAsCommitted
- buildRectanglePersistenceModel

Required event-to-function routing

Save-commit flow

click Save button

- recalculateAllRectangleDerivedGeometry
- validateCommittedRectangleState
- validateCommittedDiagramState
- incrementDiagramVersion
- buildDiagramJsonObject
- serializeDiagramToJson
- saveDiagramJsonFile
- buildDiagramSavePayload
- callSaveDiagramApi
- DiagramController.SaveDiagram
- DiagramService.SaveDiagram
- DiagramRepository.SaveDiagram
- handleSaveCommitResult

Open-from-JSON validation flow

File → Open

- openDiagramJsonFile
- parseDiagramJsonFile
- validateDiagramJsonStructure
- rebuildDiagramFromJson
- recalculateAllRectangleDerivedGeometry
- renderCanvas

Open-from-database validation flow

Open Diagram by DiagramID

- callLoadDiagramApi
- DiagramController.LoadDiagram
- DiagramService.LoadDiagram
- DiagramRepository.LoadDiagramById
- applyLoadedDiagramData
- recalculateAllRectangleDerivedGeometry
- renderCanvas

State rule

During this phase:

- recalculation must occur before persistence
- JSON save and database save both use the committed recalculated diagram model
- rectangle geometry remains stored in world coordinates
- reopening from JSON or database must reconstruct the diagram from the saved committed state

Output

This phase should produce:

- recalculated rectangle-derived geometry before save

- a valid JSON file containing the committed diagram state
- a valid database save containing the same committed diagram state
- a stable save pipeline that supports reopening from either source

Validation / Preconditions

Before the save operation is accepted:

- all committed rectangles must have valid canonical geometry
- all derived rectangle values must be recalculable
- no required rectangle data may be missing
- the diagram model must be structurally valid
- DiagramID must be valid
- database save must go only through the C# layer

Error / failure expectations

Examples of failure behavior:

- if rectangle recalculation fails, the save operation must stop
- if rectangle geometry is invalid, neither JSON nor database save should proceed
- if JSON serialization fails, the system must report file-save failure clearly
- if database save fails, the system must report database-save failure clearly
- if one persistence target succeeds and the other fails, the result must clearly show partial success or failure
- if reopening from JSON or database reveals invalid data, reconstruction must fail safely instead of silently loading corrupted state

Success criteria

This phase is successful if:

- pressing **Save** first recalculates all required rectangle-derived values
- the recalculated committed diagram is saved correctly to JSON
- the same committed diagram is saved correctly to the database
- the user can use **File** → **Open** to reopen the JSON file
- the user can reopen the same diagram from the database by DiagramID
- the reopened diagram matches the saved diagram logically and visually
- rectangles appear again with the correct:

- world position
- size
- appearance
- parent-container relationship if applicable
- committed interaction result

Recommendation

This phase should be treated as the **final rectangle commit-and-persist phase** of Milestone 5.

It is the phase that guarantees:

- the rectangle model is recalculated correctly
- the saved JSON is based on valid committed state
- the saved database record is based on the same valid committed state
- reopening from either source restores the diagram as saved

Final result after Milestone 5

At the end of Milestone 5, the developer should have:

- rectangle hover behavior
- rectangle selection behavior
- hover padding highlight behavior
- resize control point hover visibility behavior
- rectangle drag / move behavior
- movement validation under overlap and parent-container rules
- full 8-control-point rectangle resize behavior
- stable rectangle interaction state after hover, move, and resize

This milestone makes the rectangle a fully interactive editable diagram item.

After interaction behavior is stable, add JSON save/reopen, then database save/reopen, then the final recalculation-before-save flow.

Implement hover and selection first, then add movement, then add resize behavior.

Confirm Milestone 3 shape-definition rules and Milestone 4 Rectangle creation are stable.

Recommended implementation order

Recommended coding files structure additions for Milestone 5

JavaScript

Recommended new modules:

- geometry-hover-activation.js
- geometry-selection.js
- resize-control-point-hit-test.js
- move-update-scheduler.js
- detect-render-rate.js
- compute-rectangle-candidate-center-on-move.js
- recalculate-rectangle-protection-padding.js
- rectangle-overlap-on-move.js
- rectangle-overlap-on-resize.js
- validate-child-containment.js
- restore-previous-valid-position.js
- apply-validated-center.js
- update-previous-valid-position.js
- recalculate-rectangle-derived-geometry-on-resize.js
- build-diagram-save-payload.js
- diagram-api.js
-
-

Optional C# structure

Only if needed later:

- RectangleInteractionDto.cs
- RectangleUpdateDto.cs

Do not move hover, selection, drag, resize, or per-mouse-move validation into C#.

More ideas you can define in Milestone 5

These are good additions you can either include now or mark as optional subfeatures:

1. Hover priority rules

Define priority if the mouse is over more than one region:

- resize control point has highest priority
- perimeter hover next
- rectangle interior next
- empty canvas last

2. Interior vs perimeter behavior

Define whether:

- hover over interior behaves differently from hover over perimeter
- only perimeter activates hover padding
- interior click selects without showing perimeter hover
This is useful because you already mentioned “hover over center is different than hover over edge or perimeter.”

3. Minimum rectangle size

Define minimum allowed:

- MinHalfWidth
- MinHalfHeight
to prevent collapse or inverted geometry

4. Selected-state visuals

Define whether selected rectangle keeps:

- hover padding visible
- resize control points visible
- stronger outline color
- selection glow
This should be explicit

5. Cursor behavior

Define cursor changes for:

- rectangle hover
- drag-ready

- each resize direction
For example:
- diagonal resize cursor
- horizontal resize cursor
- vertical resize cursor

6. Click-on-empty-canvas behavior

Define whether clicking empty canvas:

- clears rectangle selection
- keeps current selection
- finalizes current rectangle action

7. ZOrder interaction

Define whether selecting or dragging a rectangle:

- leaves ZOrder unchanged
- temporarily brings it visually to front
- changes only render emphasis, not actual stored ZOrder

8. Snap-to-grid during move or resize

If snap is enabled, define whether:

- rectangle center snaps during drag
- resized boundaries snap during resize
- snapping occurs before or after overlap/container validation

9. Undo/redo readiness

Even if not fully implemented yet, define action records for:

- move rectangle
- resize rectangle
- select rectangle

10. Debug information

Add rectangle-specific debug output such as:

- rectangle ID
- center X/Y
- half width / half height

- selected resize point
- hover state
- selected state
- parent container ID

11. Double-click behavior

Possible future rule:

- double click rectangle opens properties panel
- double click rectangle toggles a label or editor

12. Keyboard movement

Since your document already mentions arrow-key actions, you can later define:

- move selected rectangle by small step with arrow keys
- larger step with Shift + arrows

13. Rotate-ready architecture

Even if rectangle rotation is not actively edited in this milestone, you can prepare the interaction state so rotation can be added later without breaking the structure.

My strongest recommendation is this:

For Milestone 5, explicitly define **three separate visual/logic states**:

- HoverState
- SelectedState
- ResizeState

Milestone 6 — Circle Interaction Behavior on DiagramCanvas

Overview

Milestone 6 defines the complete circle interaction behavior on DiagramCanvas after circle host geometry has already been defined in Milestone 3 and after drag/drop creation has already been introduced in Milestone 4. This milestone is limited to circle interaction: hover, selection, movement, resize, derived-geometry recalculation, committed persistence, and clean event handoff for later connection logic.

Milestone 6 shall not create connection lines, connection menus, connection lookup logic, connection database rows, or junction-point route calculations. Those responsibilities belong to Milestone 7.

Prerequisites

Milestone 2 DiagramCanvas rendering, world-space conversion, viewport behavior, and save architecture are complete.

Milestone 3 circle canonical variables and derived circle geometry are defined.

Milestone 4 can create or place a circle item on DiagramCanvas.

The shared save-to-json.js and save-to-db.js files already persist committed variables from previous milestones.

Circle interaction must run in JavaScript for immediate browser response; C# remains responsible only for trusted validation and persistence.

Milestone 6 committed variables

Milestone 6 does not introduce a new business entity. It commits interaction changes to the existing Circle host geometry variables. The following variables are the committed circle state affected by this milestone:

Variable	Purpose	Persistence rule
CircleID	Unique identity of the circle geometry.	Persisted
CenterX	Circle center X in world space.	Persisted after accepted move
CenterY	Circle center Y in world space.	Persisted after accepted move
Radius	Circle authoritative size value.	Persisted after accepted resize
ZOrder	Circle stacking order.	Persisted
FillType / FillColor	Circle fill behavior.	Persisted
LineType / LineColor / LineWidth	Circle border behavior.	Persisted
HoverPaddingRadiusRatio / HoverPaddingColor	Circle hover interaction region.	Persisted as configuration
ProtectionPaddingRadiusRatio / ProtectionPaddingColor	Circle protection region for overlap and containment.	Persisted as configuration
ParentContainerID	Immediate parent container if the circle is inside a container; NULL when on root canvas.	Persisted when applicable

Milestone 6 runtime-only variables

The following are temporary interaction variables and shall not be saved as committed diagram truth:

IsCircleHovered
IsCircleSelected
IsCircleDragging
IsCircleResizing
ActiveCircleResizeControlPoint
CurrentMouseX / CurrentMouseY
TemporaryCandidateCenterX / TemporaryCandidateCenterY
TemporaryCandidateRadius
InvalidCandidateCircleGeometry
DragPreviewVisible
ResizePreviewVisible

Recommended files for Milestone 6

File name	Type	Primary responsibility
circle-hover-selection.js	JS	Detect hover/selection over circle body, perimeter, hover padding, and resize control points.
circle-move.js	JS	Handle accepted circle movement in world space and reject invalid candidates.
circle-resize.js	JS	Resize the circle from its four resize control points while preserving a single Radius value.
circle-derived-geometry.js	JS	Recalculate visible boundaries, hover radius, protection radius, protection bounding box, and resize control points.
circle-interaction-state.js	JS	Maintain temporary hover, drag, resize, and selected state.
circle-events.js	JS	Emit clean circle-moved and circle-resized events after accepted committed geometry changes.

save-to-json.js	JS	Shared save entry file extended to include committed Milestone 6 circle state.
save-to-db.js	JS	Shared DB save entry file extended to send committed Milestone 6 circle state to C#.
CircleDto.cs	C#	Trusted DTO for committed circle state.
ShapeDefinitionService.cs	C#	Trusted validation and persistence orchestration for circle state.

Important Milestone 6 rules

Circle geometry shall remain center-based: CenterX, CenterY, and Radius are the authoritative geometry variables.

A circle resize shall always preserve one common Radius value. Separate width and height values are not allowed for a true circle.

Every accepted move or resize shall immediately recalculate derived circle geometry before rendering and before persistence.

Protection Padding is used for overlap and containment validation. Hover Padding is used only for hover detection and visual activation.

Saving shall persist committed circle values only. Temporary hover, drag, resize, and invalid-candidate state shall not be saved.

Circle move and resize completion events shall be exposed so Milestone 7 can later recalculate attached junction points and connection lines.

Mathematical Calculation Rules for Milestone 6

Milestone 6 calculations control circle hover, selection, move, resize, derived-geometry recalculation, and committed circle persistence. This milestone does not create connection lines or connection lookup behavior.

Primary JavaScript files for this calculation scope: circle-hover-hit-test.js, circle-selection.js, compute-circle-candidate-center-on-move.js, circle-resize-control-point-hit-test.js, resize-circle.js, recalculate-circle-derived-geometry-on-resize.js, circle-overlap-on-move.js, restore-previous-valid-position.js.

Circle hover and selection

$\text{DistanceSquared} = (\text{PointerX} - \text{CircleCenterX})^2 + (\text{PointerY} - \text{CircleCenterY})^2$.

Pointer is in the circle hover band when $\text{Radius}^2 \leq \text{DistanceSquared} \leq \text{HoverPaddedRadius}^2$.

Circle move calculation

$\text{CandidateCenterX} = \text{PreviousCenterX} + \text{DeltaWorldX}$. $\text{CandidateCenterY} = \text{PreviousCenterY} + \text{DeltaWorldY}$.

The candidate center shall be accepted only after Protection Padding non-overlap and containment validation pass. If validation fails, the circle shall be restored to the previous valid center.

Circle resize calculation

RightResizeControlPoint = (CenterX + Radius, CenterY). TopResizeControlPoint = (CenterX, CenterY + Radius). LeftResizeControlPoint = (CenterX - Radius, CenterY). BottomResizeControlPoint = (CenterX, CenterY - Radius).

When a resize control point is dragged, NewRadius = distance(Center, DraggedWorldPoint), or the axis-constrained equivalent for the selected control point. NewRadius must remain positive and must satisfy any minimum radius rule.

After any accepted circle resize, recalculate Radius, protection padded radius, protection bounding box, hover region, and resize control point positions before rendering or saving.

Milestone 6 phases

Phase 6.1 — Circle hover detection and selection behavior

Detect when the pointer enters the circle hover band, the circle body, or a circle resize control point. A left-click on a valid circle target selects the circle. Hover state is temporary and shall not be persisted.

Success criteria: circle hover and selection are visually clear, stable across zoom levels, and based on world-space hit testing.

Phase 6.2 — Move the selected circle on DiagramCanvas

Move the selected circle by updating CenterX and CenterY in world space. Movement shall validate against applicable overlap and containment rules, then either apply the candidate center or restore the previous valid center.

Success criteria: accepted circle movement updates CenterX and CenterY only, recalculates protection geometry, renders immediately, and marks the diagram dirty.

Phase 6.3 — Resize the circle from four resize control points

The circle shall support RightResizeControlPoint, TopResizeControlPoint, LeftResizeControlPoint, and BottomResizeControlPoint. Dragging any resize control point changes Radius while keeping the circle circular.

Success criteria: accepted resize updates Radius, recalculates all circle derived geometry, preserves minimum radius, and exposes a circle-resized event for later Milestone 7 connection recalculation.

Phase 6.4 — Finalize circle interaction state and event handoff

When a move or resize completes, temporary interaction state shall be cleared and one clean completion event shall be emitted. These events do not create connections by themselves; they only allow later milestones to respond.

Event	Trigger	Expected later use
-------	---------	--------------------

circle-moved	Accepted circle move completed.	Milestone 7 may recalculate attached connection lines.
circle-resized	Accepted circle resize completed.	Milestone 7 may recalculate all junction points attached to the circle.

Phase 6.5 — Save circle-related committed changes to JSON

Exact variables to save in this phase: CircleID, CenterX, CenterY, Radius, ZOrder, FillType, FillColor, LineType, LineColor, LineWidth, HoverPaddingRadiusRatio, HoverPaddingColor, ProtectionPaddingRadiusRatio, ProtectionPaddingColor, ParentContainerID when applicable.

Save test criteria: save-to-json.js shall write the exact committed circle variable set and shall exclude temporary hover/drag/resize state.

Phase 6.6 — Save circle-related committed changes to the database

Exact variables to save in this phase are the same committed circle variables used in the JSON path. JavaScript shall send the committed circle payload to C#, and C# shall validate and persist it through the trusted backend path.

Save test criteria: save-to-db.js and the C# persistence path shall store and reload the committed circle state without persisting temporary interaction-only variables.

Phase 6.7 — Recalculate committed circle geometry and persist to both JSON and database

Before a committed save is accepted, the system shall recalculate ProtectionPaddedRadius, protection bounding box, visible bounding box, hover radius, and resize control point positions from canonical circle variables.

Success criteria: saved and reopened circle state produces the same visible circle, same protection geometry, and same interaction geometry.

Final result after Milestone 6

Circle hover, selection, movement, and resize are working.

Circle geometry remains center-based and radius-based.

Derived circle geometry recalculates after every accepted change.

Circle move and circle resize completion hooks exist for later connection recalculation.

JSON and database persistence include only committed circle state.

Recommended implementation order

Implement hover and selection.

Implement movement and previous-valid-center behavior.

Implement resize from four control points.

Implement derived-geometry recalculation after accepted changes.

Expose circle-moved and circle-resized events.

Extend shared JSON and DB save files for committed circle state.

Milestone 7 — Connecting Two Geometric Items Using Junction Points and Lines

Overview

Milestone 7 defines the complete connection behavior between two existing connectable geometries on DiagramCanvas. It covers Connect-To mode, connection-type lookup, connection-option display, junction-point calculation, route calculation, connection styling, connection labels, resize/move recalculation, JSON persistence, and database persistence.

The milestone assumes that rectangles, circles, containers, movement, resize behavior, and committed save/load foundations already exist. Milestone 7 does not create the source or destination geometries. It connects geometries that are already present in the active diagram model.

Goal

The goal of Milestone 7 is to let the user create, render, save, load, and recalculate a connection between two existing geometries using real source and destination Square Junction Point endpoints, while using route points for one-line and multi-link line geometry.

The milestone must also support connection-type selection from CSV or database, default connection styling from Global Variables -> Connections, and database-driven rendering when connection rows already exist.

Scope

Milestone 7 includes the following work:

- right-click target detection on an existing connectable geometry
- geometry context menu display with a Connect To menu item
- Connect-To mode activation and action locking
- left-click selection of the second geometry as the target
- CSV-first and database-fallback connection-type lookup
- connection options menu display after the second geometry is selected
- connection default style retrieval from Global Variables -> Connections
- source and destination Square Junction Point calculation and creation
- single-line shortest-path connection calculation
- two-line, three-line, and four-line route point calculation
- source junction text, destination junction text, and middle-line text placement

- connection recalculation when connected geometries move or resize
- JSON and database persistence of actual connections and connection details
- database-driven reload or refresh of existing connections.

Milestone 7 excludes the following work unless a later milestone explicitly adds it:

- creating new devices or containers from the Shapes Panel
- turning intermediate route bend points into real editable junction-point items
- automatic obstacle-avoiding routing around unrelated geometries
- connection animation, simulation, bandwidth calculation, or live network state
- changing Global Variables defaults retroactively on old connections without an explicit apply-to-existing command.

Prerequisites

Before Milestone 7 starts, the following must already be available and verified:

Prerequisite	Required state	Reason
Milestone 2 DiagramCanvas	World space, viewport, screen space, pan, zoom, and active canvas state are working.	Connection geometry must be calculated in world space and rendered through the existing canvas pipeline.
Milestone 3 host geometry	Rectangle, Circle, Container, and their canonical variables are defined.	Connection endpoints attach to host boundaries.
Milestone 4 placement	Existing diagram items can be placed on DiagramCanvas.	Milestone 7 connects existing items, not library source icons.
Milestone 5 rectangle interaction	Rectangle hover, selection, movement, resize control points, and committed geometry updates work.	Rectangle connections must update after movement or affected resize.
Milestone 6 circle interaction	Circle hover, selection, movement, resize control points, and committed geometry updates work.	Circle connections must update after movement or resize.
Square Junction Point model	Square Junction Point identity, host attachment, sensor area, and activation behavior are defined.	Milestone 7 uses real junction points as connection endpoints.
Shared save files	save-to-json.js and save-to-db.js already save all committed variables from earlier milestones.	Milestone 7 extends the same save files for connection data.
Connection lookup source	connection_type_lookup.csv is available or the database lookup table is available.	The connection options menu depends on allowed connection types.
Database/API readiness	Database connection, API endpoints, and repository pattern	Actual connections must be loadable from database rows.

	are available if DB save/load is included in the phase.	
Global Variables window	Existing tabs such as Canvas background, Rectangle, and Circle are available.	Milestone 7 adds a Connections tab to the same window.

Milestone 7 terminology

Term	Meaning
Source geometry	The first geometry selected by right-click before entering Connect-To mode.
Target geometry	The second geometry selected by left click while Connect-To mode is active.
Connect-To mode	Temporary interaction mode used to select a second geometry and choose a connection type.
Connection type	Business or visual meaning of the connection, such as IPsecTunnel, AWSVPCPeerLink, OneLine, TwoLines, ThreeLines, or FourLines.
Connection route type	The route calculation family, such as ShortestPathSingleLine, MultiLinkDelta, or BusinessDefined.
Real Square Junction Point	A real endpoint item with its own ID. Milestone 7 creates or restores one source junction point and one destination junction point.
Derived route point	An intermediate bend point calculated for multi-link rendering. It is not a real Square Junction Point by default.
Source junction text	Text displayed near the source junction point.
Destination junction text	Text displayed near the destination junction point.
Middle-line text	Text displayed around the midpoint of the rendered connection polyline.
Host location parameter	A normalized parameter that allows a junction point to be rebuilt on a rectangle edge or circle boundary after load or resize.

Important milestone rules

- All connection calculations shall run in world space. Screen coordinates must be converted to world coordinates before calculation.
- Connect-To mode shall disable unrelated diagram actions while it is active.
- The only allowed actions during Connect-To mode are selecting a valid second geometry, cancelling the mode, or closing the connection workflow.
- The actual line route calculation shall not run until the user selects a connection option from the menu shown after the second geometry is selected.
- CSV lookup has priority over database lookup. If CSV is unavailable or has no matching row, the database lookup table shall be used.
- Connection-type lookup is bidirectional unless the lookup row explicitly says it is directional.

- A Rectangle to Circle lookup row shall also support Circle to Rectangle without requiring a duplicate row.
- Milestone 7 creates or restores exactly two real Square Junction Point endpoints for a normal connection: one source endpoint and one destination endpoint.
- Intermediate multi-link route bend points are derived route points by default, not real Square Junction Point items.
- Connection defaults from Global Variables -> Connections are copied into the actual connection detail record when the connection is created.
- Changing a global connection default later shall not silently change old saved connections unless a future explicit apply-to-existing command is added.
- DiagramConnections identifies what is connected to what. DiagramConnectionDetails stores route, style, junction, and label information.
- ConnectionID is the common key between DiagramConnections and DiagramConnectionDetails.
- All committed connection variables shall be added to save-to-json.js and save-to-db.js. Temporary interaction state shall not be saved as committed truth.

Connect-To workflow

1. right-click target detection
2. opening the geometry context menu
3. showing Connect To as a menu item
4. entering Connect-To mode
5. Connect-To mode disables all other diagram actions, including drag/drop of new diagrams or devices, moving geometries, resizing unrelated geometries, deleting geometries, pasting, duplicating, or starting another tool action
6. left click selecting the second geometry as the target
7. check connection types from either CSV or database; priority is CSV first, and if CSV is not available or does not contain a match, use database
8. display the information retrieved from CSV or database in a menu once the second geometry has been selected
9. the actual line calculation happens in the next phase after the user selects one connection option.

Connection-type lookup source

Column	Meaning	Example
SourceDeviceType	The source device, geometry, or business type used for lookup.	Rectangle, Circle, AWS VPC
DestinationDeviceType	The destination device, geometry, or business type used for lookup.	Rectangle, Circle, AWS VPC

PossibleConnections	Allowed connection options displayed in the menu.	One line OR Two lines OR Three lines OR Four lines
IsDirectional	Optional flag. If false or missing, the row is bidirectional.	False

SourceDeviceType	DestinationDeviceType	PossibleConnections
Rectangle	Rectangle	One line OR Two lines OR Three lines OR Four lines
Rectangle	Circle	One line OR Two lines OR Three lines OR Four lines
AWS VPC	AWS VPC	IPsec tunnel OR VPC Peer link

Bidirectional lookup rule: if the exact source-to-destination lookup is not found, the system shall check the swapped destination-to-source form unless the row is explicitly directional. Therefore, the table does not need a separate Circle to Rectangle row when Rectangle to Circle already exists.

Global Variables — Connections tab

The Global Variables window shall include a tab named Connections. Existing tabs such as Canvas background, Rectangle, and Circle shall remain unchanged. The Connections tab shall display the default visual line properties of each supported connection type.

Connection type	Default line type	Default color	Default thickness
IPsec tunnel	DottedLine	Orange	Thin
AWS VPC peer link	SolidLine	Black	Thick

The right-side properties panel of the Connections tab shall show the editable default properties for each connection type. For IPsec tunnel, the defaults are dotted line, orange color, and thin line. For AWS VPC peer link, the defaults are solid line, black color, and thick line.

Milestone 7 committed variables

Variable	Purpose	Persistence rule
ConnectionID	Unique identity of the connection.	Save to JSON and database.
DiagramID	Diagram that owns the connection.	Save to JSON and database.
SourceItemID	ID of the first connected geometry or item.	Save to JSON and database.
SourceItemKind	Type of the source item, such as Rectangle, Circle, Device, Container, or AWSVPC.	Save to JSON and database.
DestinationItemID	ID of the second connected geometry or item.	Save to JSON and database.

DestinationItemKind	Type of the destination item.	Save to JSON and database.
ConnectionType	Selected connection meaning, such as IPsecTunnel, AWSVPCPeerLink, OneLine, TwoLines, ThreeLines, or FourLines.	Save to JSON and database.
ConnectionRouteType	Route calculation type, such as ShortestPathSingleLine, MultiLinkDelta, or BusinessDefined.	Save to JSON and database.
SourceJunctionPointID	Real source Square Junction Point endpoint ID.	Save to JSON and database when created/restored.
DestinationJunctionPointID	Real destination Square Junction Point endpoint ID.	Save to JSON and database when created/restored.
LineType	SolidLine, DottedLine, DashedLine, or NoLine.	Save to JSON and database.
LineColor	Connection line color.	Save to JSON and database.
LineWidth	Numeric connection line width.	Save to JSON and database.
ThicknessName	Readable thickness name such as Thin, Medium, or Thick.	Save to JSON and database when used.
SourceJunctionText	Text displayed near source junction point.	Save to JSON and database.
DestinationJunctionText	Text displayed near destination junction point.	Save to JSON and database.
MiddleLineText	Text displayed at the middle of the connection route.	Save to JSON and database.
IsDirectional	Indicates whether direction matters for this connection.	Save to JSON and database.
IsDeleted	Soft-delete flag for connection visibility.	Save to JSON and database.

Temporary variables such as IsConnectToModeActive, HoveredConnectionOption, CurrentMouseX, CurrentMouseY, DragPreviewVisible, and temporary invalid route candidates shall not be saved as committed connection data.

Recommended database table structure

Table — ConnectionTypeLookup

Column	Type	Purpose
SourceDeviceType	nvarchar(100)	Source item type used for connection option lookup.
DestinationDeviceType	nvarchar(100)	Destination item type used for connection option lookup.

PossibleConnections	nvarchar(max)	Menu values separated by a controlled delimiter such as OR or a JSON array.
IsDirectional	bit nullable	If true, do not swap source and destination during lookup. If false or null, lookup is bidirectional.

Table — DiagramConnections

Column	Type	Purpose
ConnectionID	uniqueidentifier or varchar(64)	Primary key of the actual connection.
DiagramID	uniqueidentifier or varchar(64)	Diagram that owns this connection.
SourceItemID	uniqueidentifier or varchar(64)	First connected item.
SourceItemKind	nvarchar(50)	Kind of the first item.
DestinationItemID	uniqueidentifier or varchar(64)	Second connected item.
DestinationItemKind	nvarchar(50)	Kind of the second item.
ConnectionType	nvarchar(100)	Selected connection type.
IsDeleted	bit	Soft-delete flag. Deleted connections shall not render.

Table — DiagramConnectionDetails

Column	Type	Purpose
ConnectionDetailID	uniqueidentifier or varchar(64)	Primary key of the detail row.
ConnectionID	uniqueidentifier or varchar(64)	Foreign key to DiagramConnections.
ConnectionRouteType	nvarchar(100)	Route family such as ShortestPathSingleLine or MultiLinkDelta.
SourceJunctionPointID	uniqueidentifier or varchar(64) nullable	Real source junction point endpoint.
DestinationJunctionPointID	uniqueidentifier or varchar(64) nullable	Real destination junction point endpoint.
LineType	nvarchar(50)	SolidLine, DottedLine, DashedLine, or NoLine.
LineColor	nvarchar(50)	Line color name or hex value.
LineWidth	float or decimal(18,6)	Numeric line width.
ThicknessName	nvarchar(50) nullable	Thin, Medium, or Thick.
SourceJunctionText	nvarchar(200) nullable	Text displayed near the source junction point.

DestinationJunctionText	nvarchar(200) nullable	Text displayed near the destination junction point.
MiddleLineText	nvarchar(200) nullable	Text displayed near the route midpoint.
IsDirectional	bit	Whether direction matters for rendering or later semantics.
UpdatedAt	datetime	Last detail update timestamp.

Table — ConnectionStyleDefaults

Column	Type	Purpose
ConnectionType	nvarchar(100)	Connection type whose default style is being defined.
DefaultLineType	nvarchar(50)	Default line style.
DefaultLineColor	nvarchar(50)	Default line color.
DefaultLineWidth	float or decimal(18,6)	Default numeric line width.
ThicknessName	nvarchar(50)	Thin, Medium, or Thick.
DefaultDashPattern	nvarchar(200) nullable	Optional dash pattern for dotted or dashed lines.
IsActive	bit	Whether this default connection type is available.

Recommended files for Milestone 7

File name	Type	Responsibility
connection-mode-state.js	JS	Stores Connect-To mode state, source geometry ID, target geometry ID, selected connection type, and cancellation state.
geometry-context-menu.js	JS	Handles right-click target detection and geometry context menu display.
connect-to-mode.js	JS	Enters, exits, and locks actions during Connect-To mode.
connection-type-lookup-csv.js	JS	Loads and searches the CSV lookup source first.
connection-type-lookup-db.js	JS	Calls the database lookup API if CSV is unavailable or has no matching row.
connection-options-menu.js	JS	Displays possible connection options after second target selection.

global-connection-defaults.js	JS	Reads and applies default connection styles from Global Variables -> Connections.
junction-point-geometry.js	JS	Calculates boundary attachment points and host location parameters for junction points.
junction-point-factory.js	JS	Creates real Square Junction Point endpoint objects.
junction-point-attachment.js	JS	Stores and validates host attachment data for junction points.
connection-shortest-path.js	JS	Calculates StartJunction and EndJunction for one-line shortest-path connections.
connection-delta-route.js	JS	Calculates two-line, three-line, and four-line derived route points.
connection-line.js	JS	Builds the renderable connection line/polyline object.
connection-label-position.js	JS	Calculates source junction, destination junction, and middle-line label positions.
connection-recalculation.js	JS	Recalculates connection geometry when connected hosts move or when persisted data is reloaded.
junction-point-recalculation-on-resize.js	JS	Runs resize-specific junction point recalculation rules for connected rectangles and circles.
render-connections.js	JS	Renders connection lines, route points, labels, and styles on DiagramCanvas.
build-connection-json.js	JS	Builds committed connection JSON from the in-memory diagram model.
connection-api.js	JS	Calls connection save/load/list and lookup APIs.
connection-menu.css	CSS	Styles the geometry context menu and connection options menu.
connection-line.css	CSS	Styles connection line classes where DOM/SVG rendering is used.
connection-labels.css	CSS	Styles source, destination, and middle-line labels.
global-variables-connections.css	CSS	Styles the Connections tab in Global Variables.

ConnectionController.cs	C#	Receives connection lookup, save, load, and list API requests.
ConnectionService.cs	C#	Performs trusted validation and orchestration for connection operations.
ConnectionRepository.cs	C#	Reads and writes DiagramConnections, DiagramConnectionDetails, lookup, and default style records.
ConnectionDto.cs	C#	Transfer model for actual connection identity.
ConnectionDetailDto.cs	C#	Transfer model for route, style, junction, and label details.
ConnectionTypeLookupDto.cs	C#	Transfer model for possible connection lookup results.

Function contract requirement

Every major Milestone 7 function shall be documented using the standard function contract already defined in this specification: Trigger, Input, Validation / Preconditions, Processing Responsibility, Output, State Change / Persistence Effect, Next Possible Call(s), and Error Output / Failure Path.

Mathematical Calculation Rules for Milestone 7

This section is the authoritative calculation scope for Milestone 7. All calculations shall use world-space values. Screen-space values are input/rendering values only and shall not be used as stored connection geometry truth.

JavaScript calculation ownership

Calculation area	Owning JS file	Primary output
Boundary attachment point and host parameter	junction-point-geometry.js	Junction point coordinate, HostEdgeName, HostLocationParameter
Real endpoint object creation	junction-point-factory.js	SourceJunctionPointID and DestinationJunctionPointID
Shortest source/destination endpoint selection	connection-shortest-path.js	StartJunction and EndJunction
Two-line, three-line, and four-line route points	connection-delta-route.js	IntermediateRoutePointList
Line/polyline object creation	connection-line.js	Renderable connection segment list
Label positions	connection-label-position.js	SourceLabelPosition, DestinationLabelPosition, MiddleLineLabelPosition

Move/reload recalculation	connection-recalculation.js	Updated connection endpoints and route points
Resize-specific junction recalculation	junction-point-recalculation-on-resize.js	Updated affected junction point positions only

Canonical geometry inputs

For an axis-aligned rectangle host R:

$$R_Left = R_CenterX - R_HalfWidth$$

$$R_Right = R_CenterX + R_HalfWidth$$

$$R_Top = R_CenterY + R_HalfHeight$$

$$R_Bottom = R_CenterY - R_HalfHeight$$

For a circle host C:

$$C_Center = (C_CenterX, C_CenterY)$$

$$C_Radius = Radius$$

The default connection attachment boundary is the visible host boundary unless a later rule explicitly says to attach to a padded boundary.

Rectangle boundary attachment calculation

Input: rectangle center (rx, ry), half sizes hw and hh, and direction vector d = (dx, dy) from the rectangle center toward the other host center.

If dx = 0 and dy = 0, use fallback direction d = (1, 0).

tx = hw / abs(dx), when dx != 0; otherwise tx = +Infinity

ty = hh / abs(dy), when dy != 0; otherwise ty = +Infinity

$$k = \min(tx, ty)$$

$$RectangleBoundaryPointX = rx + k * dx$$

$$RectangleBoundaryPointY = ry + k * dy$$

If RectangleBoundaryPointY = R_Top, HostEdgeName = TopEdge.

If RectangleBoundaryPointX = R_Right, HostEdgeName = RightEdge.

If RectangleBoundaryPointY = R_Bottom, HostEdgeName = BottomEdge.

If RectangleBoundaryPointX = R_Left, HostEdgeName = LeftEdge.

Rectangle host location parameter

$$\text{TopEdge: } t = (JunctionX - R_Left) / (R_Right - R_Left)$$

$$\text{BottomEdge: } t = (JunctionX - R_Left) / (R_Right - R_Left)$$

$$\text{LeftEdge: } t = (JunctionY - R_Bottom) / (R_Top - R_Bottom)$$

RightEdge: $t = (JunctionY - R_Bottom) / (R_Top - R_Bottom)$

The stored rectangle attachment identity shall include HostItemID, HostItemType, HostLocationType = RectangleBoundary, HostEdgeName, and HostLocationParameter = t.

Circle boundary attachment calculation

Input: circle center (cx, cy), radius r, and direction vector d = (dx, dy) from the circle center toward the other host center.

If dx = 0 and dy = 0, use fallback direction d = (1, 0).

Length = $\sqrt{dx^2 + dy^2}$

ux = dx / Length

uy = dy / Length

CircleBoundaryPointX = cx + ux * r

CircleBoundaryPointY = cy + uy * r

HostAngleRadians = $\text{atan2}(\text{CircleBoundaryPointY} - \text{cy}, \text{CircleBoundaryPointX} - \text{cx})$

The stored circle attachment identity shall include HostItemID, HostItemType = Circle, HostLocationType = CircleBoundary, and HostLocationParameter = HostAngleRadians.

Shortest-path endpoint calculation by geometry pair

The function calculateShortestConnectionJunctions(sourceHost, targetHost) shall return exactly two real endpoint coordinates:

StartJunction = (xs, ys)

EndJunction = (xe, ye)

dx_route = xe - xs

dy_route = ye - ys

Pair	StartJunction calculation	EndJunction calculation
Rectangle -> Rectangle	source rectangle boundary in direction target center - source center	target rectangle boundary in direction source center - target center
Rectangle -> Circle	source rectangle boundary in direction target circle center - source rectangle center	target circle boundary in direction source rectangle center - target circle center
Circle -> Rectangle	source circle boundary in direction target rectangle center - source circle center	target rectangle boundary in direction source circle center - target rectangle center
Circle -> Circle	source circle boundary in direction target circle center - source circle center	target circle boundary in direction source circle center - target circle center

The StartJunction and EndJunction shall become real Square Junction Point items. These two junction points shall have their own IDs and shall be persisted or restorable through DiagramConnectionDetails.

Connection route point calculation

Let StartJunction = S = (xs, ys) and EndJunction = E = (xe, ye).

$$dx = xe - xs$$

$$dy = ye - ys$$

alpha = default route split ratio, recommended 0.5

beta = default secondary route split ratio, recommended 0.5

One-line route

RoutePoints = [S, E]

ConnectionSegments = [(S, E)]

Two-line route

Horizontal-first two-line route:

$$B1 = (xe, ys)$$

RoutePoints = [S, B1, E]

Vertical-first two-line route:

$$B1 = (xs, ye)$$

RoutePoints = [S, B1, E]

Three-line route

Horizontal-first three-line route:

$$P1 = (xs + \alpha * dx, ys)$$

$$P2 = (xs + \alpha * dx, ye)$$

RoutePoints = [S, P1, P2, E]

Vertical-first three-line route:

$$P1 = (xs, ys + \alpha * dy)$$

$$P2 = (xe, ys + \alpha * dy)$$

RoutePoints = [S, P1, P2, E]

Four-line route

Horizontal-first four-line route:

$$P1 = (xs + \alpha * dx, ys)$$

$$P2 = (xs + \alpha * dx, ys + \beta * dy)$$

$$P3 = (xe, ys + \beta * dy)$$

RoutePoints = [S, P1, P2, P3, E]

Vertical-first four-line route:

$P1 = (xs, ys + \alpha * dy)$

$P2 = (xs + \beta * dx, ys + \alpha * dy)$

$P3 = (xs + \beta * dx, ye)$

RoutePoints = [S, P1, P2, P3, E]

The route orientation may be chosen by user selection, default connection type, or the shortest uncluttered route rule. If no explicit preference exists, the implementation shall choose a deterministic default and keep it stable for reload.

Intermediate route-point persistence rule

Intermediate route points are derived route points by default. They shall be recalculated from StartJunction, EndJunction, route type, and route parameters. They shall not be persisted as real Square Junction Point items unless a later milestone explicitly introduces editable bend handles.

Connection label position calculations

The label system shall display text near both real junction points and near the middle of the rendered route. Label placement shall use world-space route points, then convert to screen space for rendering.

LabelOffset = default positive world-space offset from the line, configurable later if needed.

For any segment A = (x1, y1) to B = (x2, y2):

$vx = x2 - x1$

$vy = y2 - y1$

$Length = \sqrt{vx^2 + vy^2}$

$ux = vx / Length$

$uy = vy / Length$

$nx = -uy$

$ny = ux$

Source junction text

Use the first rendered segment from RoutePoints[0] to RoutePoints[1].

$SourceLabelX = xs + nx * LabelOffset$

$SourceLabelY = ys + ny * LabelOffset$

Destination junction text

Use the last rendered segment from RoutePoints[n-2] to RoutePoints[n-1].

$DestinationLabelX = xe + nx_last * LabelOffset$

$DestinationLabelY = ye + ny_last * LabelOffset$

Middle-line text

For a polyline route, compute each segment length and total route length.

$$\text{TotalLength} = \text{sum}(\text{SegmentLength}_i)$$
$$\text{MiddleDistance} = \text{TotalLength} / 2$$

Find the segment where cumulative length first reaches MiddleDistance.

$$\text{LocalRatio} = (\text{MiddleDistance} - \text{LengthBeforeSegment}) / \text{SegmentLength}_{\text{current}}$$
$$\text{MiddleLineTextX} = \text{SegmentStartX} + \text{LocalRatio} * (\text{SegmentEndX} - \text{SegmentStartX})$$
$$\text{MiddleLineTextY} = \text{SegmentStartY} + \text{LocalRatio} * (\text{SegmentEndY} - \text{SegmentStartY})$$

Move recalculation rule

When a connected host geometry moves, connection-recalculation.js shall recalculate the source or destination endpoint using the current host geometry and stored host attachment parameters. If a junction point is stored as host-relative, it shall remain attached to the same host boundary location after the host moves.

Junction-point recalculation on resize

The file junction-point-recalculation-on-resize.js shall run only when a connected host resize can change the boundary location that owns the attached junction point.

Circle resize rule

Any resize of a connected circle shall recalculate all junction points attached to that circle, because Radius changes the full circle boundary.

Given stored HostAngleRadians = theta:

$$\text{JunctionX} = \text{CircleCenterX} + \text{Radius} * \cos(\text{theta})$$
$$\text{JunctionY} = \text{CircleCenterY} + \text{Radius} * \sin(\text{theta})$$

Rectangle resize rule

A rectangle junction point shall be recalculated only if the selected resize control point changes the edge that owns that junction point.

Attached edge	Resize control points that trigger recalculation
TopEdge	TopLeftResizeControlPoint, TopCenterResizeControlPoint, TopRightResizeControlPoint
RightEdge	TopRightResizeControlPoint, RightCenterResizeControlPoint, BottomRightResizeControlPoint
BottomEdge	BottomLeftResizeControlPoint, BottomCenterResizeControlPoint, BottomRightResizeControlPoint
LeftEdge	TopLeftResizeControlPoint, LeftCenterResizeControlPoint, BottomLeftResizeControlPoint

When the owning edge is affected, rebuild the junction point from HostEdgeName and HostLocationParameter t:

TopEdge: $\text{JunctionX} = R_Left + t * (R_Right - R_Left)$; $\text{JunctionY} = R_Top$

BottomEdge: $\text{JunctionX} = R_Left + t * (R_Right - R_Left)$; $\text{JunctionY} = R_Bottom$

LeftEdge: $\text{JunctionX} = R_Left$; $\text{JunctionY} = R_Bottom + t * (R_Top - R_Bottom)$

RightEdge: $\text{JunctionX} = R_Right$; $\text{JunctionY} = R_Bottom + t * (R_Top - R_Bottom)$

If the selected resize control point does not affect the attached edge, junction-point-recalculation-on-resize.js shall not run for that junction point.

Database-driven connection recalculation

When the diagram is loaded or refreshed, DiagramConnections shall identify the connected items and DiagramConnectionDetails shall provide route, style, junction, and label information using the shared ConnectionID.

If stored junction point coordinates are available and still valid on their host boundaries, the system may restore them. If they are missing or invalid, the system shall recalculate StartJunction and EndJunction from the current source and destination host geometry and update the active in-memory model.

Database-driven acceptance rule: when a row in DiagramConnections and its matching DiagramConnectionDetails row are inserted or updated, a reload or refresh shall update the rendered connection without requiring the user to manually recreate it.

Milestone 7 phases

Milestone 7 shall be implemented in eight phases.

Phase 7.1 — Right-click connection workflow and Connect-To mode lock

Goal

Detect a valid first geometry, show the geometry context menu, display Connect To, and enter Connect-To mode while locking unrelated diagram actions.

Scope

- right-click target detection
- geometry context menu rendering
- Connect To item display
- Connect-To mode state creation
- diagram action lock enforcement

Function contract deliverables

- detectRightClickTarget
- openGeometryContextMenu
- showConnectToMenuItem

- enterConnectToMode
- lockDiagramActionsDuringConnectToMode
- cancelConnectToMode

Validation / Preconditions

- the source target must be a connectable geometry
- the diagram must not already be in an incompatible modal action
- the context menu target must have a valid item ID

Error / failure expectations

- if target detection fails, do not open Connect-To mode
- if context menu rendering fails, keep diagram state unchanged
- if another action is active, reject Connect-To mode safely

Success criteria

- Connect-To mode can be entered only from a valid geometry
- drag/drop/move/resize/delete/paste/duplicate/new-item actions are blocked while Connect-To mode is active

Phase 7.2 — Select the second geometry and retrieve possible connection types

Goal

Allow left-click selection of the second geometry and retrieve possible connection types using CSV first and database second.

Scope

- left-click target detection while Connect-To mode is active
- source/target pair validation
- CSV lookup
- database fallback lookup
- bidirectional lookup

Function contract deliverables

- handleSecondGeometryClick
- validateConnectionTargetPair
- lookupConnectionTypesFromCsv
- lookupConnectionTypesFromDatabase
- resolveBidirectionalConnectionLookup

Validation / Preconditions

- Connect-To mode must be active
- source geometry must still exist
- target geometry must exist and be connectable
- CSV or database lookup source must be available

Error / failure expectations

- if the second target is invalid, keep Connect-To mode active and show a controlled message
- if CSV fails and DB is available, fall back to DB
- if both lookup paths fail, show no-available-connections state

Success criteria

- Rectangle to Circle works even if the lookup row is stored as Rectangle, Circle and the user selected Circle first
- AWS VPC to AWS VPC returns IPsec tunnel and VPC Peer link options

Phase 7.3 — Display connection options menu and apply default connection style

Goal

Display the retrieved possible connection options in a menu and prepare the selected connection type with its default visual properties.

Scope

- connection options menu rendering
- option selection
- default style lookup from Global Variables -> Connections
- style copy into pending connection detail state

Function contract deliverables

- renderConnectionOptionsMenu
- selectConnectionOption
- loadConnectionStyleDefault
- copyDefaultStyleToPendingConnectionDetail

Validation / Preconditions

- a valid source and target pair must be selected
- PossibleConnections must contain at least one option
- Global Variables -> Connections must provide defaults or fallback defaults must exist

Error / failure expectations

- if the menu cannot render, keep Connect-To mode active without calculating the line
- if style defaults are missing, apply safe fallback style
- if option selection is invalid, do not create a connection

Success criteria

- the menu appears after second target selection
- line calculation waits until a valid option is selected
- IPsec tunnel defaults to dotted orange thin line
- AWS VPC peer link defaults to solid black thick line

Phase 7.4 — Create a single-line shortest connection using junction points

Goal

Calculate the shortest valid boundary endpoints, create real source and destination Square Junction Point items, and render a single-line connection.

Scope

- source boundary attachment calculation
- destination boundary attachment calculation
- real junction point creation
- single-line segment creation
- initial label placement

Function contract deliverables

- calculateShortestConnectionJunctions
- calculateRectangleBoundaryAttachment
- calculateCircleBoundaryAttachment
- createSourceJunctionPoint
- createDestinationJunctionPoint
- createSingleLineConnection

Validation / Preconditions

- source and target geometries must have valid canonical geometry
- selected connection type must allow one-line route or shortest route
- junction point factory must be available

Error / failure expectations

- if endpoint calculation fails, do not create the connection
- if junction creation fails, roll back the pending connection

- if render fails, keep committed model unchanged

Success criteria

- the connection has a source junction point, a destination junction point, a line segment, correct default style, and optional source/middle/destination labels

Phase 7.5 — Create multi-link connection routes using derived intermediate route points

Goal

Calculate two-line, three-line, or four-line routes using derived route points while preserving only two real junction endpoints by default.

Scope

- route type selection
- intermediate route point calculation
- polyline segment creation
- line style application
- label midpoint recalculation

Function contract deliverables

- calculateDeltaRoutePoints
- chooseRouteOrientation
- createMultiLinkConnectionLine
- calculateMiddleLineTextPosition

Validation / Preconditions

- StartJunction and EndJunction must exist
- connection route type must be valid
- route split parameters must be valid positive numeric values

Error / failure expectations

- if route calculation fails, fall back to a safe shortest single-line route only if the selected connection type permits fallback
- if route points are invalid, reject the connection creation

Success criteria

- multi-link routes render through derived route points
- intermediate route points are not persisted as real Square Junction Point items by default

Phase 7.6 — Display source, destination, and middle-line connection text

Goal

Calculate and render connection text near the two junction points and near the route midpoint.

Scope

- source junction label placement
- destination junction label placement
- middle-line label placement
- label rendering
- label update after route recalculation

Function contract deliverables

- calculateSourceJunctionTextPosition
- calculateDestinationJunctionTextPosition
- calculateMiddleLineTextPosition
- renderConnectionLabels

Validation / Preconditions

- connection route points must be valid
- label text may be empty but must be handled safely
- label offset must be valid

Error / failure expectations

- if text is empty, render no label but keep the connection valid
- if label position cannot be calculated, skip the affected label and log a controlled warning

Success criteria

- SourceJunctionText appears near the source endpoint
- DestinationJunctionText appears near the destination endpoint
- MiddleLineText appears around the visual midpoint of the route

Phase 7.7 — Recalculate connections when connected geometries move or resize

Goal

Keep connection endpoints, route points, and labels valid when connected rectangles, circles, or containers move or resize.

Scope

- move-based connection recalculation
- circle resize recalculation
- rectangle edge-specific resize recalculation
- route recalculation
- label recalculation

Function contract deliverables

- getConnectionsForGeometry
- recalculateConnectionGeometry
- recalculateJunctionPointsOnCircleResize
- recalculateJunctionPointsOnRectangleResize
- renderConnections

Validation / Preconditions

- the moved or resized geometry must exist
- connections for the geometry must be retrievable
- host attachment parameters must be available or recalculable

Error / failure expectations

- if a connected item is missing, skip the broken connection and log the issue
- if a junction point is invalid after resize, recalculate from host parameters
- if recalculation fails, preserve the last valid committed connection where possible

Success criteria

- circle resize recalculates all attached circle junction points
- rectangle resize recalculates only junction points attached to affected edges
- labels and route points update after recalculation

Phase 7.8 — Save and reload connection and junction-point information from JSON and database

Goal

Persist actual connections and their details, then reload them without requiring the user to manually recreate the connections.

Scope

- JSON save/load extension
- database save/load extension
- DiagramConnections persistence
- DiagramConnectionDetails persistence
- database-driven refresh behavior

Function contract deliverables

- buildConnectionJsonObject
- extendSaveToJsonWithConnections
- extendSaveToDbWithConnections

- callLoadConnectionsApi
- applyLoadedConnectionData

Validation / Preconditions

- save-to-json.js and save-to-db.js must already work for previous milestones
- ConnectionID must link connection and detail records
- source and destination items must be valid or safely ignored

Error / failure expectations

- if detail row is missing, render using default style when possible
- if IsDeleted is true, do not render the connection
- if source or destination item is missing, ignore safely and log error

Success criteria

- after reload, the diagram renders the same connections, labels, styles, source/destination relationships, and route types without manual recreation

Connection database-driven test criteria

1. Insert or seed at least two connectable diagram items in the database.
2. Insert one row into DiagramConnections where SourceItemID references the first item and DestinationItemID references the second item.
3. Insert one matching row into DiagramConnectionDetails using the same ConnectionID.
4. Load or refresh the diagram by DiagramID.
5. The diagram shall retrieve the connection rows, calculate or restore source and destination junction points, calculate or restore route geometry, and render the connection.
6. The diagram shall display SourceJunctionText near the source junction point, DestinationJunctionText near the destination junction point, and MiddleLineText near the middle of the route.
7. If ConnectionType is IPsecTunnel and no row-specific style override exists, the diagram shall render a dotted orange thin line.
8. If ConnectionType is AWSVPCPeerLink and no row-specific style override exists, the diagram shall render a solid thick black line.
9. If LineType, LineColor, LineWidth, SourceJunctionText, DestinationJunctionText, or MiddleLineText changes in DiagramConnectionDetails, the diagram shall reflect the update after reload or refresh.
10. If IsDeleted is true in DiagramConnections, the connection shall not render.
11. If the source or destination item does not exist, the connection shall be ignored safely and an error shall be logged without breaking the diagram.

Milestone 7 save test criteria

The Milestone 7 save test shall check only the committed variables introduced by Milestone 7 while also running regression validation for all previously approved committed variables from Milestones 1 through 6.

- save-to-json.js writes DiagramConnections and DiagramConnectionDetails into the committed JSON payload.
- save-to-db.js writes DiagramConnections and DiagramConnectionDetails through the C# API and repository layer.
- reload from JSON restores source/destination item references, connection type, route type, line style, junction point references, and labels.
- reload from database restores the same visible connection state from ConnectionID-linked records.
- temporary Connect-To mode state is not persisted as committed diagram truth.

Final result after Milestone 7

- Connect-To mode works and locks unrelated actions.
- Second target selection triggers CSV-first and database-fallback connection lookup.
- Connection options are shown in a menu before line calculation.
- Global Variables -> Connections exists and shows default connection styles.
- IPsec tunnel defaults render as dotted orange thin line.
- AWS VPC peer link defaults render as solid black thick line.
- Single-line shortest routes are supported.
- Two-line, three-line, and four-line routes are supported through derived route points.
- Source and destination real Square Junction Point endpoints are created or restored.
- Intermediate multi-link route points are derived route points by default.
- Source junction text, destination junction text, and middle-line text render correctly.
- Connections recalculate correctly after connected host movement and relevant resize actions.
- DiagramConnections and DiagramConnectionDetails persist actual connections and details.
- Database-driven connection updates appear after reload or refresh without manual recreation.

Recommended implementation order

1. Implement Connect-To mode state and action lock.
2. Implement geometry context menu and Connect To action.
3. Implement second geometry left-click selection.
4. Implement CSV lookup and database fallback lookup.

5. Implement connection options menu.
6. Implement Global Variables -> Connections default style model.
7. Implement shortest-path junction point calculation.
8. Implement real source and destination junction point creation.
9. Implement one-line connection rendering.
10. Implement two-line, three-line, and four-line route-point calculation.
11. Implement source, destination, and middle-line labels.
12. Implement resize-based and move-based connection recalculation.
13. Extend save-to-json.js and save-to-db.js for connection persistence.
14. Implement database-driven reload and refresh tests.

Recommended coding files structure additions for Milestone 7

The following folder additions are recommended for Milestone 7. Existing project structure may be used if the same file responsibilities remain clear.

project/

```
|— css/
|   |— connection-menu.css
|   |— connection-line.css
|   |— connection-labels.css
|   └— global-variables-connections.css
|— js/
|   |— connection-mode-state.js
|   |— geometry-context-menu.js
|   |— connect-to-mode.js
|   |— connection-type-lookup-csv.js
|   |— connection-type-lookup-db.js
|   |— connection-options-menu.js
|   |— global-connection-defaults.js
|   |— junction-point-geometry.js
|   |— junction-point-factory.js
|   |— junction-point-attachment.js
|   └— connection-shortest-path.js
```

```
| └─ connection-delta-route.js
| └─ connection-line.js
| └─ connection-label-position.js
| └─ connection-recalculation.js
| └─ junction-point-recalculation-on-resize.js
| └─ render-connections.js
| └─ build-connection-json.js
| └─ connection-api.js
└─ csharp/
    | └─ ConnectionController.cs
    | └─ ConnectionService.cs
    | └─ ConnectionRepository.cs
    | └─ ConnectionDto.cs
    | └─ ConnectionDetailDto.cs
    | └─ ConnectionTypeLookupDto.cs
    └─ data/
        └─ connection_type_lookup.csv
```

More ideas you can define in Milestone 7

- editable bend handles for multi-link routes in a future milestone
- orthogonal routing around other geometries in a future milestone
- arrowheads for directional connections
- connection grouping and filtering by connection type
- apply current global connection style defaults to selected existing connections
- connection validation against business rules such as subnet-to-VPC or VPC-to-VPC compatibility
- connection audit log showing who created or changed a connection.

Milestone 8 — Circle On Container Edge Placement and Movement Behavior

Overview

Milestone 8 defines the implementation behavior for a new geometry: a Circle On Container whose center point is constrained to the boundary edge of an allowed rectangle container and can move on any edge of

that same container. This milestone introduces edge attachment, active-edge movement, dynamic nearest-edge projection, mandatory corner-transition behavior, recalculation when the parent container moves or resizes, and persistence of the committed Circle On Container state.

This milestone comes after Milestone 7 because Milestone 7 defines connection behavior using junction points and lines. The Circle On Container can later become a connection-related boundary object, port marker, or engineering interface point, but this milestone focuses only on defining and moving the circle on a container edge.

The new geometry is not a normal child circle inside a container. A normal child circle must remain fully inside the parent container. In this milestone, the circle center is on the parent container edge by design, so the implementation must use special edge-attachment rules rather than the normal child-circle containment rules.

Milestone 8 includes the following work:

- define the Circle On Container model and committed variables
- attach the circle center to one valid edge of an allowed rectangle container
- move the circle center across any valid edge using current active HostEdge and EdgeParameterT
- project mouse movement onto the nearest valid edge of the allowed container
- support deterministic corner transition between adjacent edges
- recalculate the circle position when the parent container moves or resizes
- validate sibling overlap and strict no-touch behavior where applicable
- save and reload the committed Circle On Container state through JSON and database paths

Mandatory movement clarification: after the Circle On Container is dropped on an allowed container edge, it must be movable across all four edges of that same allowed container. The initial drop edge is only the starting active edge; it is not a permanent movement restriction.

Milestone 8 excludes the following work unless a later milestone explicitly adds it:

- automatic creation of network connections from the Circle On Container
- new SVG rendering rules for the circle
- advanced port labeling
- multi-edge routing around complex polygon containers
- normal child containment behavior for a center-on-boundary circle

Recommended files for Milestone 8

File name	Type	Description	Explicit Trigger / Call	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Per	Next Possible Call(s)	Error Output / Failure Path
-----------	------	-------------	-------------------------	-------	----------------------------	---------------------------	--------	--------------------	-----------------------	-----------------------------

			Count					Consistence Effect		
circle-on-container.css	CS	Visual styling for Circle On Container states.	Page load and render refresh. Passed once per page load ; reapplied by browser.	Circle classes and render state.	Canvas and circle render target exist.	Style circle, hover state, selected state, and invalid preview state.	Readable Circle On Container visual state.	No persistence effect.	render-circle-on-container.js	Circle may render with default styling if selector mismatch occurs.
circle-on-container-state.js	JS	Holds live memory state for Circle On Containers.	App startup, JSON/D B load , creation, or accepted move. Call cou	Loaded edge circle data, parent container data, active diagram state.	Parent container exists and edge circle fields are valid.	Maintain committed and temporary Circle On Container state.	Updated memory state object.	Client-side state only until save.	render-circle-on-container.js ; build-diagram-save-payload.js	Reject invalid state and preserve previous valid state.

			nt: as nee ded per stat e upd ate.							
circle-on-container-model.js	JS	Builds normalized Circle OnContainer model.	Creation or load path. Call count: once per edge circle model build.	CircleOnContainerID, ParentContainerID, HostEdge, EdgeParameterT, radius and appearance values.	HostEdge valid, EdgeParameterT in range, Radius positive.	Normalize values and prepare model for calculation.	Structured Circle OnContainer object.	Incremental model creation only.	calculate-edge-circle-center.js	Return validation error and do not create model.
calculate-edge-circle-center.js	JS	Calculates Center X and Center Y from parent container edge and EdgeParameterT.	Render, load, parent move, parent resize, or edge-circle move	Parent boundaries, current active HostEdge, EdgeParameterT.	Parent container geometry valid.	Apply current active HostEdge equation and derive CenterX/CenterY.	Calculated world-space center.	Derived state only; committed after accepted update.	render-circle-on-container.js; validate-edge-circle-placement.js	Return failure if parent or edge is invalid.

			e. Call count: once per affected edge circle update.							
project- pointer-to- container- edge.js	J S	Proje cts pointer world positio n onto the nearest t valid contai ner edge.	Mou seM ove duri ng edg e- circl e dra g. Call cou nt: onc e per proc ess ed mov e- vali dati on upd ate.	Pointer WorldX , Pointer WorldY , parent bounda ries, all valid edges.	Drag mode active and parent contai ner valid.	Find neare st valid edge projec tion and derive candi date HostE dge plus Edge Para meter T.	Candi date Edge Param eterT and candid ate center .	Te mp orar y dra g can did ate only .	validate- edge- circle- placement. js	Cancel candidate and restore previous valid edge parameter.
move- circle-on- container.js	J S	Contro ls drag move ment along	Mou seD own on edg	Active edge circle, pointer world	Move mentE nable d true, circle	Comp ute candi date t, valida	Accep ted or reject ed edge-	Upd ate s in- me	render- canvas.js; update- previous-	Restore PreviousV alidEdgeP arameterT and

		the contai ner edge.	e circl e then proc ess ed Mou seM ove until Mou seU p. Call cou nt: onc e per proc ess ed mov e upd ate.	positio n, parent contain er.	select ed, drag state valid.	te it, apply or reject move ment.	circle center .	mor y stat e; per sist ed only whe n sav e is call ed.	valid-edge- position.js	previous valid center.
validate- edge- circle- placement. js	J S	Validat es edge attach ment and sibling overla p.	Can dida te edg e- circl e cent er calc ulat ed. Call cou nt: onc e per can dida te	Candid ate center, radius, protecti on paddin g, siblings in same parent/ root scope.	Candi date edge attach ment exists.	Chec k attach ment equati on, strict edge range , and siblin g non- overla p.	Valida tion result.	No per sist enc e effe ct.	apply- validated- edge- circle- center.js or restore- previous- valid-edge- position.js	Return invalid reason and keep previous valid state.

			update.							
recalculate-edge-circles-on-parent-change.js	JS	Recalculates Circle On Container centers when parent container moves or resizes.	Parent container new geometry and attached edge circles.	Parent geometry valid and attached circles exist.	Recalculate each attached circle from its current active HostEdgeParameter T.	Updated attached circle centers.	Updates client state; persisted only on save.	render-canvas.js; validate-edge-circle-placement.js	Skip invalid child and report controlled error.	
circle-on-container-api.js	JS	Calls backend save/load support through the existing save-to-db.js route.	Save or load action. Call count: once per API request if DB path is selected.	Committed diagram payload including Circle On Container collection.	Payload valid and API endpoint available.	Send and receive Circle On Container data through approved API path.	API response.	No direct DB write from JS.	C# controller/service/repository save flow.	Show API or validation error.

				cted						
CircleOnC ontainerDt o.cs	C #	DTO for Circle On Contai ner commi tted variabl es.	API sav e/lo ad req uest map ping . Call cou nt: onc e per map ped edg e circl e.	Reque st payloa d or reposit ory row.	Requir ed fields exist and data types are valid.	Carry Circle On Conta iner data betwe en API and servic e layers .	DTO object.	No dire ct per sist enc e effe ct.	CircleOnC ontainerSe rvice.cs	Model binding or validation failure.
CircleOnC ontainerSe rvice.cs	C #	Truste d validat ion for commi tted Circle On Contai ner state.	Con troll er sav e/lo ad call. Call cou nt: onc e per rout ed serv ice req uest .	DTOs, parent contain er data, user context .	User author ized, parent contai ner valid, attach ment equati on valid.	Recal culate center , valida te comm itted state, orche strate persis tence.	Save/l oad result.	Trig ger s rep osit ory per sist enc e thro ugh app rov ed flow .	CircleOnC ontainerRe pository.cs	Return controlled service failure.
CircleOnC ontainerRe pository.cs	C #	Datab ase acces	Ser vice sav	Persist ence model,	Datab ase conne	Insert, updat e,	Datab ase	Dire ct dat	Service return flow.	DB read/write failure.

		s for Circle On Contai ner record s.	e/lo ad ope ratio n. Call cou nt: onc e per data bas e ope ratio n step .	Diagra mID, CircleO nConta inerID.	cted and schem a availa ble.	load, and list Circle On Conta iner recor ds.	result rows.	aba se per sist enc e effe ct thro ugh C# only .
resize- circle-on- container.js	J S	Handl es Circle On Contai ner radius resize using global minim um and maxim um radius ratios.						
validate- circle-on- container- radius- range.js	J S	Validat es Candi dateR adius again st Minim umRa dius and						

		MaximumRadius derived from the current active edge length.
circle-on-container-subtree-follow.js	JS	Moves /recalculates Circle On Container centers when the host container or any higher nested parent container moves.

Recommended database table structure

Milestone 8 extends the database model with a table for Circle On Containers. It does not replace the existing Devices, Containers, DiagramCanvas, or connection tables.

Database table	Purpose	Important columns
CircleOnContainers	Stores Circle On Container items whose centers	CircleOnContainerID, DiagramID, ParentContainerID, HostEdge, EdgeParameterT, CenterX, CenterY, Radius, ZOrder, appearance fields, padding ratios, MovementEnabled, CornerTransitionEnabled, EdgePlacementPolicy

	are constrained to valid rectangle container edges and can move across any edge of the same allowed container.	
DiagramPayloads / DiagramShapePayloads	Optional full committed payload snapshot including Circle On Containers.	PayloadID, DiagramID, DiagramVersion, PayloadJson, SavedAt
RadiusResizeEnabled	Whether the user may resize the Circle On Container radius.	True
CircleOnContainerMinimumRadiusRatio	Global lower radius ratio applied to the current active edge length.	0.10
CircleOnContainerMaximumRadiusRatio	Global upper radius ratio applied to the current	0.20

	active edge length.	
CurrentActiveEdgeLength	Derived length of the current active edge. Top/Bottom use container width; Right/Left use container height.	Derived, not stored as independent truth
CalculatedMinimumRadius	Derived minimum allowed radius for the current active edge.	$\text{CurrentActiveEdgeLength} * 0.10$
CalculatedMaximumRadius	Derived maximum allowed radius for the current active edge.	$\text{CurrentActiveEdgeLength} * 0.20$
CircleOnContainerGlobalVariables	Stores global radius-limit and resize policy values for Circle On Container geometry.	CircleOnContainerMinimumRadiusRatio, CircleOnContainerMaximumRadiusRatio, CircleOnContainerMinimumAbsoluteRadius, CircleOnContainerMaximumAbsoluteRadius, CircleOnContainerResizeEnabled, CircleOnContainerContainerResizeInvalidRadiusPolicy

Table — CircleOnContainers

Column	Type	Purpose
CircleOnContainerID	uniqueidentifier or varchar(64)	Primary key for the Circle On Container.
DiagramID	uniqueidentifier or varchar(64)	Foreign key to the diagram.
ParentContainerID	uniqueidentifier or varchar(64)	Container whose edge owns the circle center.
HostEdge	Current active container edge on which the circle center lies. This value may change during movement because the circle can move on any edge of the same allowed container.	Top, Right, Bottom, or Left
EdgeParameterT	Normalized movement parameter along the current active edge.	0.0 to 1.0
CenterX	float or decimal(18,6)	Committed/calculated center X in world space.
CenterY	float or decimal(18,6)	Committed/calculated center Y in world space.
Radius	float or decimal(18,6)	Circle radius.
ZOrder	int	Stacking order.
FillType	nvarchar(50)	NoFill or SolidFill.
FillColor	nvarchar(50)	Fill color when used.
LineType	nvarchar(50)	NoLine or SolidLine.
LineColor	nvarchar(50)	Circle border color.
LineWidth	float or decimal(18,6)	Circle border width.
HoverPaddingRadiusRatio	float or decimal(18,6)	Hover padding ratio.
ProtectionPaddingRadiusRatio	float or decimal(18,6)	Protection padding ratio.
MovementEnabled	bit	Whether movement along the edge is allowed.
CornerTransitionEnabled	Whether movement may change HostEdge at a corner. Required True for this project. This project requires it to be enabled	True by default

	so the circle can move on any edge.	
EdgePlacementPolicy	nvarchar(100)	CenterOnBoundary by default.
CreatedAt	datetime	Creation timestamp.
UpdatedAt	datetime	Last update timestamp.
RadiusResizeEnabled	Whether the user may resize the Circle On Container radius.	True
CircleOnContainerMinimumRadiusRatio	Global lower radius ratio applied to the current active edge length.	0.10
CircleOnContainerMaximumRadiusRatio	Global upper radius ratio applied to the current active edge length.	0.20
CurrentActiveEdgeLength	Derived length of the current active edge. Top/Bottom use container width; Right/Left use container height.	Derived, not stored as independent truth
CalculatedMinimumRadius	Derived minimum allowed radius for the current active edge.	CurrentActiveEdgeLength * 0.10
CalculatedMaximumRadius	Derived maximum allowed radius for the current active edge.	CurrentActiveEdgeLength * 0.20
RadiusConstraintSource	nvarchar(100)	GlobalVariables by default; indicates radius constraints are not hard-coded per item.

Milestone 8 variables

Only variables treated as committed and persisted in both JSON and database are included here.

Variable name	Description	Recommended values
CircleOnContainerID	Unique identity of the Circle On Container.	GUID/UUID string
ParentContainerID	The rectangle container whose edge hosts the circle center.	Valid ContainerID

HostEdge	Current active container edge on which the circle center lies. This value may change during movement because the circle can move on any edge of the same allowed container.	Top, Right, Bottom, or Left
EdgeParameterT	Normalized movement parameter along the current active edge.	0.0 to 1.0
CenterX	Committed/calculated center X in world space.	Calculated from current active HostEdge and EdgeParameterT
CenterY	Committed/calculated center Y in world space.	Calculated from current active HostEdge and EdgeParameterT
Radius	Circle radius.	Positive numeric value
ZOrder	Stacking order.	Integer
FillType	Fill behavior.	NoFill / SolidFill
FillColor	Fill color.	Hex color or Transparent
LineType	Border behavior.	NoLine / SolidLine
LineColor	Circle border color.	Hex color
LineWidth	Circle border width.	Positive numeric value
HoverPaddingRadiusRatio	Hover padding ratio.	0.1
ProtectionPaddingRadiusRatio	Protection padding ratio.	0.2
MovementEnabled	Allows edge movement.	True
CornerTransitionEnabled	Whether movement may change HostEdge at a corner. Required True for this project. This project requires it to be enabled so the circle can move on any edge.	True by default
EdgePlacementPolicy	Boundary placement interpretation.	CenterOnBoundary
PreviousValidEdgeParameterT	Last valid accepted edge parameter during movement.	Updated after accepted move

PreviousValidCenterX	Last valid accepted center X.	Updated after accepted move
PreviousValidCenterY	Last valid accepted center Y.	Updated after accepted move
RadiusResizeEnabled	Whether the user may resize the Circle On Container radius.	True
CircleOnContainerMinimumRadiusRatio	Global lower radius ratio applied to the current active edge length.	0.10
CircleOnContainerMaximumRadiusRatio	Global upper radius ratio applied to the current active edge length.	0.20
CurrentActiveEdgeLength	Derived length of the current active edge. Top/Bottom use container width; Right/Left use container height.	Derived, not stored as independent truth
CalculatedMinimumRadius	Derived minimum allowed radius for the current active edge.	$\text{CurrentActiveEdgeLength} * 0.10$
CalculatedMaximumRadius	Derived maximum allowed radius for the current active edge.	$\text{CurrentActiveEdgeLength} * 0.20$

Prerequisites

Before Milestone 8 implementation starts, the following prerequisites must be confirmed.

- Milestone 7 connection workflow and junction-point rules are already defined and do not conflict with the new circle geometry.
- Rectangle Container geometry is already defined, including parent boundaries and containment rules.
- Circle geometry is already defined, including radius, appearance, hover padding, and protection padding.
- World-space, viewport, and screen-space conversion already works correctly.
- save-to-json.js and save-to-db.js already exist as the only approved top-level save entry files.
- Database connection is confirmed if the database save phase is implemented immediately.

Mandatory existing JavaScript before starting Milestone 8

JavaScript file that must already exist	Description	Reason this JavaScript must already be ready and working correctly
canvas-state.js	Maintains active diagram and item state.	Edge-attached circles must be stored in the active diagram model before save.
render-canvas.js	Coordinates canvas rendering.	The new circle must be drawn during the standard render cycle.
world-to-screen.js	Converts world coordinates to screen coordinates.	The Circle On Container center is world-space authoritative.
screen-to-world.js	Converts pointer screen coordinates to world coordinates.	Pointer movement must be projected onto a container edge.
derive-shape-geometry.js	Recalculates derived shape geometry.	Parent boundaries and circle padding values must be available.
save-to-json.js	Shared JSON save entry file.	Milestone 8 must extend the shared save path rather than create a separate top-level save file.
save-to-db.js	Shared database save entry file.	Milestone 8 must extend the shared database save path rather than create a separate top-level save file.

Shared save prerequisite. Before starting Milestone 8, the team shall verify that save-to-json.js and save-to-db.js are already saving all committed variables approved in Milestones 1 to 7 correctly. When Milestone 8 introduces Circle On Container variables, the developer shall extend only these same two files so the current milestone save test checks the Milestone 8 variables, while regression validation confirms that all previous milestone variables are still saved correctly.

Milestone 8 terminology

Term	Meaning
Circle On Container	A circle whose center is constrained to a rectangle container edge.
Parent Container	The rectangle container that owns the Circle On Container.

HostEdge	The current active edge on which the circle center lies: Top, Right, Bottom, or Left. It can change during movement so the circle is not locked to a single edge.
EdgeParameterT	Normalized position along the current active edge, from 0.0 to 1.0.
CenterOnBoundary	Placement policy where the circle center lies exactly on the parent visible boundary.
Corner Transition	Required behavior where moving past an edge end changes HostEdge to the adjacent edge.
Previous Valid Edge Position	The last accepted EdgeParameterT and center values that passed validation.

Important milestone rules

- The Circle On Container is a separate geometry behavior and shall not be treated as a normal child circle inside a container.
- The authoritative attachment variables are ParentContainerID, HostEdge, and EdgeParameterT.
- CenterX and CenterY shall be recalculated from the parent container edge before rendering, validation, and save acceptance.
- The circle center must lie exactly on the selected parent edge, subject to numeric epsilon tolerance.
- Movement along the edge shall be performed in world space.
- Mouse coordinates shall be converted from screen space to world space before edge projection.
- If CornerTransitionEnabled = False, movement is clamped to the current active edge.
- If CornerTransitionEnabled = True, corner crossing must update HostEdge deterministically.
- The circle shall move with the parent container because its center is derived from the parent container geometry.
- When the parent container is resized, the circle center shall be recalculated from HostEdge and EdgeParameterT.
- Normal child-circle containment rules shall not reject this geometry merely because the circle extends across the parent boundary.
- Sibling overlap and strict no-touch validation shall still be applied according to the selected validation scope.
- JavaScript handles live drag, projection, preview, validation, and rendering.
- C# handles trusted committed-state validation and database persistence.
- Browser JavaScript shall not write directly to the database.

Core Circle On Container equations used in this milestone

Parent boundaries

$$\text{ParentLeftX} = \text{ParentCenterX} - \text{ParentHalfWidth}$$
$$\text{ParentRightX} = \text{ParentCenterX} + \text{ParentHalfWidth}$$
$$\text{ParentTopY} = \text{ParentCenterY} + \text{ParentHalfHeight}$$
$$\text{ParentBottomY} = \text{ParentCenterY} - \text{ParentHalfHeight}$$

Edge equations using EdgeParameterT

Top edge: $\text{CenterX} = \text{ParentLeftX} + t * (\text{ParentRightX} - \text{ParentLeftX})$; $\text{CenterY} = \text{ParentTopY}$

Right edge: $\text{CenterX} = \text{ParentRightX}$; $\text{CenterY} = \text{ParentTopY} - t * (\text{ParentTopY} - \text{ParentBottomY})$

Bottom edge: $\text{CenterX} = \text{ParentRightX} - t * (\text{ParentRightX} - \text{ParentLeftX})$; $\text{CenterY} = \text{ParentBottomY}$

Left edge: $\text{CenterX} = \text{ParentLeftX}$; $\text{CenterY} = \text{ParentBottomY} + t * (\text{ParentTopY} - \text{ParentBottomY})$

Clamping rule

$$\text{EdgeParameterT_clamped} = \min(1, \max(0, \text{EdgeParameterT_candidate}))$$

Circle Protection Padding

$$\text{ProtectionPaddingRadius} = \text{Radius} * \text{ProtectionPaddingRadiusRatio}$$
$$\text{ProtectionPaddedRadius} = \text{Radius} + \text{ProtectionPaddingRadius}$$

CSV relationship and drop eligibility rules for Milestone 8

Milestone 8 shall use device-on-container-edge-rules.csv to define which Circle On Container device types are allowed on which container types. The CSV file shall have exactly two columns: device on container edge and container.

device on container edge	container
Internet Gateway	VPC

A Circle On Container drop is valid only when the dragged DeviceOnContainerEdgeType and the target ParentContainerType match a CSV row. For example, Internet Gateway can be placed on VPC edge, but not on Region edge unless Internet Gateway,Region is explicitly added.

Drop eligibility begins when any part of the dragged Circle On Container overlaps the Hover Padding area of a candidate container. After eligibility is detected, the system shall validate the CSV rule, choose the nearest valid container edge, project the center to that edge, and then calculate HostEdge and EdgeParameterT.

ZOrder and border-occlusion rules for Milestone 8

CircleOnContainerZOrder must always be greater than ParentContainerZOrder. Recommended default is ParentContainerZOrder + 1. The parent container border line shall not be visible through Circle On

Container. The renderer must either use an opaque Circle On Container fill or mask/suppress the container border segment covered by the circle.

Additional committed Milestone 8 variables

Add these committed variables to the Milestone 8 save set: DeviceOnContainerEdgeType, ParentContainerType, BorderOcclusionPolicy, and the CSV-derived allowed relationship reference where applicable. Candidate hover state, candidate container ID, temporary drag preview values, and invalid drop candidates shall remain runtime-only values and shall not be saved as committed diagram truth.

Mathematical Calculation Rules for Milestone 8

Milestone 8 calculations derive a circle center from a parent rectangle container edge, project pointer movement onto that edge, clamp or transition the edge parameter, validate the accepted candidate, and preserve the previous valid position if validation fails.

Primary JavaScript files for this calculation scope: calculate-edge-circle-center.js, project-pointer-to-container-edge.js, move-circle-on-container.js, validate-edge-circle-placement.js, recalculate-edge-circles-on-parent-change.js.

Container edge segment parameter calculation

For a horizontal edge, derive t from the candidate X coordinate. For a vertical edge, derive t from the candidate Y coordinate. The t formula must match the current active edge orientation defined in this milestone.

- Top edge: $t = (\text{PointerWorldX} - \text{ParentLeftX}) / (\text{ParentRightX} - \text{ParentLeftX})$.
- Right edge: $t = (\text{ParentTopY} - \text{PointerWorldY}) / (\text{ParentTopY} - \text{ParentBottomY})$.
- Bottom edge: $t = (\text{ParentRightX} - \text{PointerWorldX}) / (\text{ParentRightX} - \text{ParentLeftX})$.
- Left edge: $t = (\text{PointerWorldY} - \text{ParentBottomY}) / (\text{ParentTopY} - \text{ParentBottomY})$.

Candidate center projection/clamping

After t is calculated, clamp it to the range 0.0 to 1.0 unless corner transition is enabled. The center is then recalculated using the current active edge equation. The raw pointer position shall not become the circle center unless it lies on the edge after projection.

Edge movement calculation

During a drag action, the system shall store PreviousValidEdgeParameterT, PreviousValidCenterX, and PreviousValidCenterY. Each processed mouse update creates a candidate t and candidate center. If the candidate is valid, apply it and update the previous valid variables. If invalid, restore the previous valid values.

Protection Padding and sibling validation calculation

The Circle On Container ProtectionPaddedRadius shall be used for overlap checks against other Circle On Containers and sibling geometries in the same relevant scope. If strict no-touch is required, touching counts as invalid. Normal child-circle containment is not applied because the center intentionally lies on the parent edge.

Milestone 8 phases

Milestone 8 shall be implemented in 8 phases.

Phase 8.1 — Define Circle On Container geometry model and committed variables

Goal

Define the model, variables, default values, database fields, and in-memory state for a circle whose center is attached to a rectangle container edge.

Scope

- define CircleOnContainerID, ParentContainerID, HostEdge, EdgeParameterT, CenterX, CenterY, Radius, appearance values, padding values, movement flags, and previous valid values
- extend the active diagram model to include an CircleOnContainers collection
- separate committed variables from derived helper values

Function contract deliverables

- createCircleOnContainerModel
- initializeCircleOnContainerDefaults
- validateCircleOnContainerModel
- addCircleOnContainerToDiagramState

Output

A normalized Circle On Container model exists in the active diagram state.

Success criteria

This phase succeeds when the developer can create an Circle On Container object with valid committed variables without rendering or movement errors.

Phase 8.2 — Attach the circle center to a valid rectangle container edge

Goal

Create the first valid center-on-edge placement using ParentContainerID, current active HostEdge, and EdgeParameterT. The initial HostEdge is only the starting active edge; it must not prevent later movement to other edges of the same allowed container.

Scope

- select a rectangle container as parent
- select the initial active edge of the parent container
- calculate CenterX and CenterY from edge equations
- render the circle centered on the current active container edge

Function contract deliverables

- selectParentContainerForEdgeCircle

- selectHostEdge
- calculateEdgeCircleCenter
- renderCircleOnContainer

Output

The circle appears with its center exactly on one valid parent container edge.

Success criteria

This phase succeeds when all four HostEdge values can place the circle center on the correct rectangle container edge.

Phase 8.3 — Move the circle center across any valid edge of the container

Goal

Allow the user to drag the circle across any edge of the same allowed container while preserving the center-on-boundary constraint.

Scope

- convert pointer screen position to world space
- project pointer world position onto the nearest valid container edge
- calculate candidate EdgeParameterT
- clamp candidate t when corner transition is disabled
- recalculate candidate CenterX and CenterY

Function contract deliverables

- beginEdgeCircleMove
- projectPointerToContainerEdge
- calculateCandidateEdgeParameterT
- applyValidatedEdgeCircleCenter
- endEdgeCircleMove

Output

The circle moves smoothly along the full container boundary and cannot leave the allowed container edge set.

Success criteria

This phase succeeds when dragging does not allow the circle center to drift inside or outside the edge line.

Phase 8.4 — Handle edge endpoints and mandatory corner transition behavior

Goal

Define deterministic behavior when the user drags the circle to a corner so movement can continue onto the adjacent edge.

Scope

- corner transition is required; the circle shall not be permanently clamped to the first edge
- transfer the circle to the adjacent edge at the corner using deterministic clockwise/counterclockwise rules
- update HostEdge and EdgeParameterT consistently whenever the active edge changes

Function contract deliverables

- clampEdgeParameterT
- detectEdgeEndpoint
- resolveCornerTransition
- updateHostEdgeAfterCornerTransition

Output

Endpoint behavior is deterministic and does not create ambiguous circle positions.

Success criteria

This phase succeeds when edge-end behavior is predictable for all four container edges.

Phase 8.5 — Validate Protection Padding and sibling overlap while moving along edge

Goal

Validate the moving Circle On Container against other relevant geometries without applying the normal child-circle containment rule incorrectly.

Scope

- recalculate ProtectionPaddedRadius
- validate edge-attachment equation
- validate against sibling Circle On Containers and sibling geometries
- restore previous valid position if strict no-touch overlap is violated

Function contract deliverables

- recalculateEdgeCircleProtectionPadding
- validateEdgeCircleAttachment

- validateEdgeCircleSiblingNonOverlap
- restorePreviousValidEdgePosition

Output

Invalid overlap or touch candidates are rejected and the circle returns to the previous valid edge position.

Success criteria

This phase succeeds when the circle remains attached to the container edge and does not violate the selected sibling overlap rule.

Phase 8.6 — Resize Circle On Container within global radius constraints

Goal

Allow the user to resize a Circle On Container while keeping its Radius inside the globally defined radius range for the current active edge length.

Scope

This phase is about Circle On Container radius resize only. The resize action shall not resize the parent container, and parent container resize shall not automatically resize the Circle On Container.

Required calculations

CurrentActiveEdgeLength = ParentRightX - ParentLeftX for Top/Bottom edges, and
CurrentActiveEdgeLength = ParentTopY - ParentBottomY for Right/Left edges.

MinimumRadius = CurrentActiveEdgeLength * CircleOnContainerMinimumRadiusRatio. MaximumRadius = CurrentActiveEdgeLength * CircleOnContainerMaximumRadiusRatio.

CandidateRadius is valid only when MinimumRadius <= CandidateRadius <= MaximumRadius. The recommended live interaction behavior is to clamp the resize preview to the nearest valid radius boundary.

Success criteria

This phase succeeds when a Circle On Container can be resized within the allowed radius boundaries, cannot be resized below the minimum or above the maximum, and does not resize the parent container.

Phase 8.7 — Recalculate Circle On Container centers when the parent container moves or resizes

Goal

Keep Circle On Containers correctly attached after parent container movement or resize.

Parent container movement or ancestor-subtree movement shall move the Circle On Container with the host container. Parent container resize shall recalculate the Circle On Container center from HostEdge and EdgeParameterT but shall not automatically change Radius.

Scope

- when the parent container moves, recalculate all attached circle centers from the moved parent boundaries
- when the parent container resizes, preserve EdgeParameterT and recalculate CenterX/CenterY

- validate resulting positions after recalculation

Function contract deliverables

- getEdgeCirclesByParentContainer
- recalculateEdgeCirclesOnParentMove
- recalculateEdgeCirclesOnParentResize
- validateEdgeCirclesAfterParentChange

Output

All attached circles remain on valid edges of their assigned parent containers after parent movement or resize.

Success criteria

This phase succeeds when parent movement and resize do not detach or misplace any Circle On Container.

Phase 8.8 — Recalculate committed Circle On Container geometry and persist to JSON and database

Exact variables to save in this phase

Exact committed Milestone 8 save variables: CircleOnContainerID, ParentContainerID, HostEdge, EdgeParameterT, CenterX, CenterY, Radius, ZOrder, FillType, FillColor, LineType, LineColor, LineWidth, HoverPaddingRadiusRatio, ProtectionPaddingRadiusRatio, MovementEnabled, CornerTransitionEnabled, EdgePlacementPolicy. HostEdge stores the current active edge at the time of save, not a permanent single-edge restriction.

PreviousValidEdgeParameterT, PreviousValidCenterX, and PreviousValidCenterY are movement helper values. They may be retained in client runtime state during movement, but they should not be treated as permanent committed diagram truth unless the project explicitly decides to persist editor recovery state.

Save regression rule for this phase

Milestone 8 adds a new committed circle-on-container variable set. The phase fails if save-to-json.js or save-to-db.js omits any committed Milestone 8 variable, or if either save path breaks any committed variables from Milestones 1 to 7.

Save test criteria for this phase

Verify that save-to-json.js writes every listed Milestone 8 variable into the committed JSON payload, that save-to-db.js writes the same committed variable set to the trusted database model, that reload restores the circle on the saved current active parent edge at the same EdgeParameterT, and that derived helper values are recalculated rather than trusted blindly.

Goal

Persist the committed Circle On Container state and rebuild it correctly from JSON and database.

Scope

- recalculate CenterX and CenterY from ParentContainerID, HostEdge, and EdgeParameterT before save acceptance

- extend save-to-json.js with the CircleOnContainers collection
- extend save-to-db.js and C# persistence with CircleOnContainers
- reload the diagram and reconstruct Circle On Container centers from parent geometry

Function contract deliverables

- recalculateCommittedCircleOnContainerGeometry
- buildCircleOnContainerJsonPayload
- appendCircleOnContainersToSavePayload
- validateCircleOnContainerSavePayload
- saveCircleOnContainersToDatabase
- loadCircleOnContainersFromDatabase
- rebuildCircleOnContainersAfterLoad

Output

The Circle On Container is saved and reopened correctly through both JSON and database paths.

Success criteria

This phase succeeds when a saved diagram reloads the Circle On Container on the same parent container edge with the same normalized edge position and valid recalculated world-space center.

Final result after Milestone 8

At the end of Milestone 8, the developer should have:

- a formally defined Circle On Container geometry model
- a circle whose center can be constrained to any valid edge of an allowed rectangle container
- any-edge movement using current active HostEdge and EdgeParameterT
- safe projection from pointer movement to edge-constrained movement
- deterministic endpoint and mandatory corner-transition behavior
- recalculation after parent container move or resize
- Protection Padding and sibling overlap validation adapted for this geometry
- JSON and database persistence through the shared save files

Recommended implementation order

1. Define the CircleOnContainer model and variables.
2. Attach a circle center to one valid parent container edge as the initial active edge.
3. Render the Circle On Container.
4. Implement drag movement across any edge of the same allowed container.

5. Add endpoint and mandatory corner-transition logic.
6. Add sibling overlap validation and previous valid position restore.
7. Recalculate attached circles after parent move/resize.
8. Extend JSON and database persistence through save-to-json.js and save-to-db.js.

Recommended coding files structure additions for Milestone 8

For this milestone, add the following files or modules while keeping the shared save-entry rule unchanged.

CSS

- circle-on-container.css

JavaScript

- circle-on-container-state.js
- circle-on-container-model.js
- calculate-edge-circle-center.js
- project-pointer-to-container-edge.js
- move-circle-on-container.js
- validate-edge-circle-placement.js
- recalculate-edge-circles-on-parent-change.js
- render-circle-on-container.js

C#

- CircleOnContainerDto.cs
- CircleOnContainerModel.cs
- CircleOnContainerService.cs
- CircleOnContainerRepository.cs

Shared save files to extend, not replace

- save-to-json.js
- save-to-db.js

Best folder layout additions for Milestone 8

project/

├─ css/

| └─ circle-on-container.css

├─ js/

- | └─ circle-on-container-state.js
- | └─ circle-on-container-model.js
- | └─ calculate-edge-circle-center.js
- | └─ project-pointer-to-container-edge.js
- | └─ move-circle-on-container.js
- | └─ validate-edge-circle-placement.js
- | └─ recalculate-edge-circles-on-parent-change.js
- | └─ render-circle-on-container.js
- └─ csharp/
 - └─ CircleOnContainerDto.cs
 - └─ CircleOnContainerModel.cs
 - └─ CircleOnContainerService.cs
 - └─ CircleOnContainerRepository.cs

More ideas you can define in Milestone 8

- support inside-biased and outside-biased placement policies where the circle center is offset from the boundary by a controlled amount
- allow Circle On Containers to act as connection ports in a later connection milestone
- support edge labels that move with EdgeParameterT
- support snapping to edge divisions such as quarters or thirds
- support multiple Circle On Containers on the same edge with automatic spacing
- support lock-to-corner behavior for special port symbols

Milestone 9 — Protection Padding Calculation and Containment Behavior Inside Rectangle Containers

Overview

Milestone 9 defines the first focused implementation milestone for Protection Padding calculation inside parent containers.

For this milestone, all containers shall be treated as:

Rectangle Containers only

not circle containers

This milestone uses the existing document rules that state:

Protection Padding is the mathematical safety region used for overlap avoidance, spacing rules, child-inside-container calculations, and containment validation

all Protection Padding calculations must happen in world space

contained geometries inside a parent rectangle container must remain fully inside the immediate parent container and must not get out of it, overlap its border line, or even touch its border line

clamped movement is the recommended safe behavior for contained items

This milestone shall be implemented in 7 phases.

The first 3 phases apply the containment and Protection Padding logic to:

a single child rectangle inside a parent rectangle container

The next 3 phases apply the same containment and Protection Padding logic to:

a single child circle inside a parent rectangle container

The last phase ensures that all committed Protection Padding geometry and containment state can be recalculated and persisted correctly.

A core design rule for this milestone is:

JavaScript is responsible for immediate Protection Padding calculation, live containment validation, drag behavior, resize behavior, clamped movement, parent-child reference updates, and live rendering

C# is responsible only for trusted validation, persistence, API handling, and database communication when save/load is required

Another required persistence rule for this milestone is: ParentContainerID = NULL means the item belongs directly to DiagramCanvas root rather than to a visible parent container.

Trigger

Input

Validation / Preconditions

Processing Responsibility

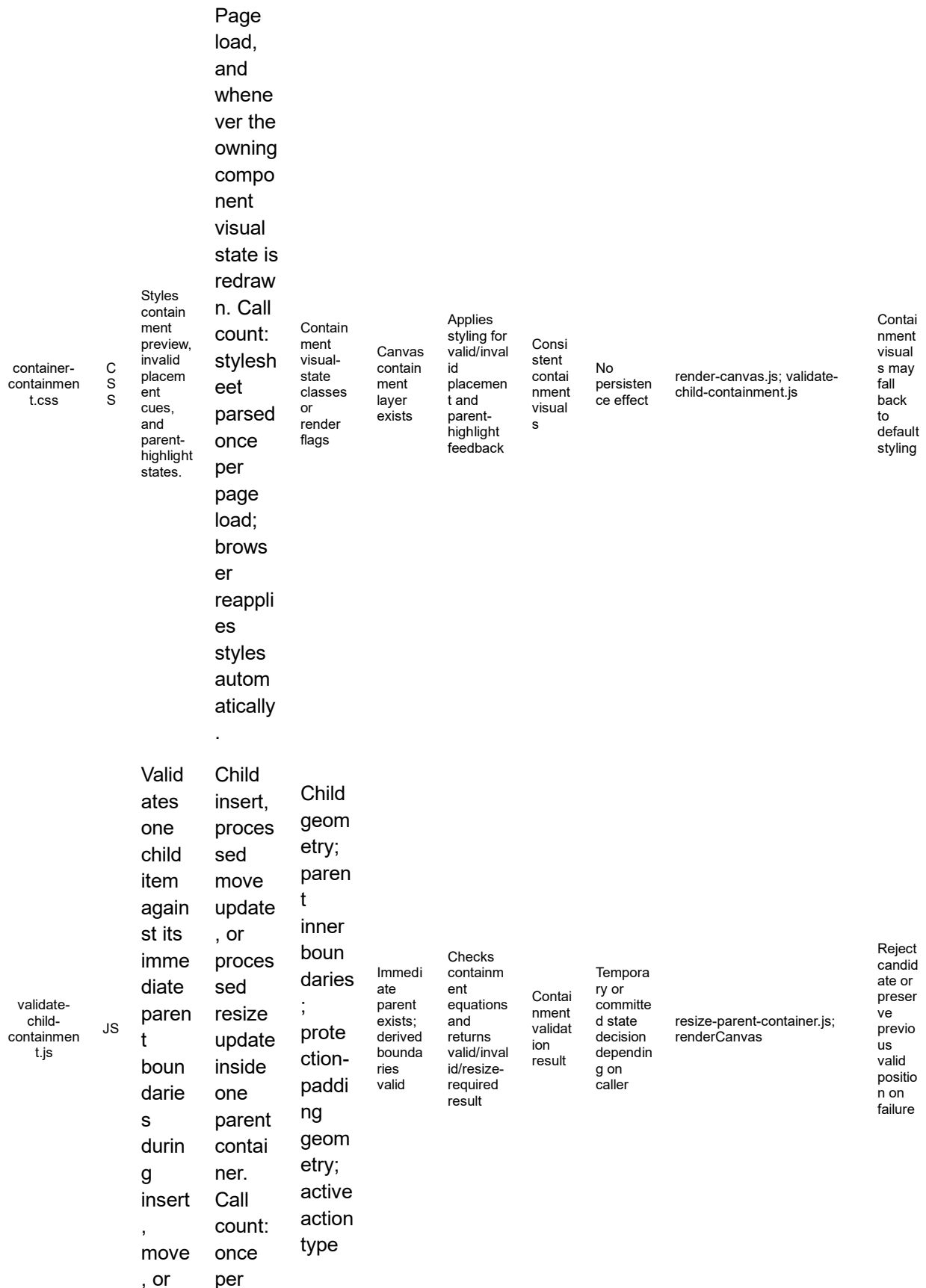
Output

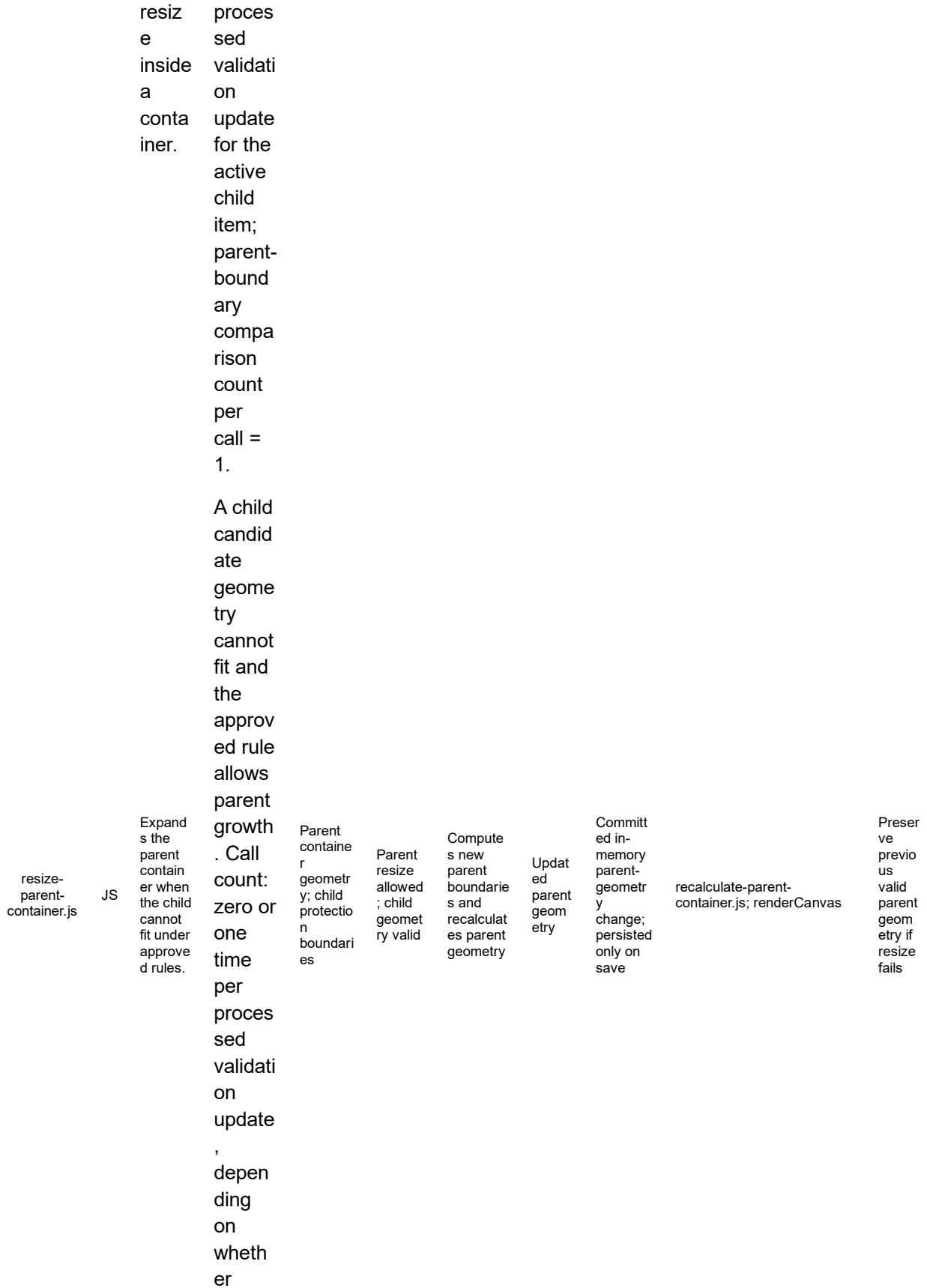
State Change / Persistence Effect

Next Possible Call(s)

Error Output / Failure Path

File name	Type	Description	Explicit Trigger / Call Count	Input	Validati on / Precon ditions	Processi ng Respons ibility	Output	State Change / Persiste nce Effect	Next Possible Call(s)	Error Output / Failur e Path
-----------	------	-------------	--	-------	---------------------------------------	--------------------------------------	--------	---	-----------------------	--







ContainerService.cs	C#	Trusted validation and orchestration for committed containment state.	API request. Only when the DB/API persistence path is enabled and the controller routes a save/load request. Call count: once per routed service request.	Validated DTOs; Diagram ID; user context	Request valid; parent/child relationships valid	Validates committed containment and coordinates repository operations	Service result object	Triggers trusted persistence through repository	ContainerRepository.SaveCommittedContainmentState	Return structured failure result if validation fails
---------------------	----	---	---	--	---	---	-----------------------	---	---	--

The table below follows the Milestone 2 file-definition style and includes CSS, JavaScript, and C# files relevant to protection padding calculation and containment behavior inside rectangle containers. Validation rows now state the practical call count per processed update.

Recommended files for Milestone 9

Database table	Type	Purpose
Containers	Core source	Stores container geometry and ParentContainerID. Containers are rendered as Rectangle Containers in this milestone.
Devices	Core source	Stores non-container items, host geometry type, and ParentContainerID.

DiagramPayloads	Optional snapshot	Stores committed full-diagram snapshots including containment state.
ContainmentChangeLog	Optional audit	Stores committed containment insert/move/resize history if auditability is needed later.

The table below lists the main database tables used or extended by Milestone 9.

Recommended database table structure

Variable name	Description	Recommended values
ParentContainerID	Identity of the immediate parent Container. NULL means direct ownership by DiagramCanvas root.	Valid ContainerID or NULL
ContainedItemID	Identity of the current contained geometry under validation.	Valid DeviceID or ContainerID
ContainedItemType	Logical contained type inside the parent container.	Device / Container
ParentInnerLeftX	Derived usable left boundary of the parent interior.	Numeric world-space value
ParentInnerRightX	Derived usable right boundary of the parent interior.	Numeric world-space value
ParentInnerTopY	Derived usable top boundary of the parent interior.	Numeric world-space value
ParentInnerBottomY	Derived usable bottom boundary of the parent interior.	Numeric world-space value
CandidateContainedCenterX	Candidate contained-item center X during validation.	Numeric world-space value
CandidateContainedCenterY	Candidate contained-item center Y during validation.	Numeric world-space value
ContainedProtectionBounds	Derived protection boundaries of the contained item.	Structured boundary object
PreviousValidCenterX	Previous accepted center X of the contained item.	Numeric world-space value
PreviousValidCenterY	Previous accepted center Y of the contained item.	Numeric world-space value
ContainmentValidationResult	Result of parent-boundary and sibling-overlap validation.	Valid / Invalid / Clamped

The table below includes the main Milestone 9 working variables.

Milestone 9 variables

Minimum readiness rule: Milestone 9 should not be considered ready to begin until the team confirms at least this: one contained geometry can already be identified together with its immediate parent container, and ParentContainerID = NULL is reserved for direct ownership by DiagramCanvas root.

Required confirmations: Milestones 3 through 7 are already working; Rectangle Container rules are defined; single-child containment equations are agreed; Protection Padding calculation helpers exist for Rectangle and Circle items; immediate-parent relationships are available; and parent resize behavior is conceptually approved.

Before Milestone 9 implementation starts, the following prerequisites should be confirmed.

Prerequisites

Mandatory existing JavaScript before starting Milestone 9

Only JavaScript files that must already exist and work correctly before this milestone starts are listed here.

Shared save prerequisite. Before starting Milestone 9, the team shall verify that save-to-json.js and save-to-db.js are already saving all committed variables approved in Milestones 1 to 8 correctly. When Milestone 9 introduces new committed variables, the developer shall extend only these same two files so the current milestone save test checks the Milestone 9 variables, while regression validation confirms that all previous milestone variables are still saved correctly.

JavaScript file that must already exist	Description	Reason this JavaScript must already be ready and working correctly
rectangle-hover.js	Detects Rectangle hover-padding activity and shows interaction cues.	Single-child containment starts from already interactive Rectangle geometries.
rectangle-selection.js	Handles Rectangle selection from Hover Padding.	The child geometry must already support stable selection before containment behavior is added.
rectangle-resize.js	Handles Rectangle resize workflow and candidate size changes.	Milestone 9 validates containment during child resize, so resize behavior must already exist.
circle-hover.js	Detects Circle hover-padding activity and shows interaction cues.	Containment in this milestone also includes Circle children.
circle-selection.js	Handles Circle selection from Hover Padding.	Circle child items must already support stable selection before container validation is added.
circle-resize.js	Handles Circle resize workflow and candidate radius changes.	Milestone 9 validates containment during Circle resize as well as move.
derive-shape-geometry.js	Provides stable derived geometry for child items.	Containment equations compare padded child boundaries against parent boundaries.

recalculate-rectangle-protection-padding.js	Recalculates Rectangle Protection Padding boundaries from the current Rectangle geometry.	Milestone 9 containment and spacing rules depend on Rectangle Protection Padding being correct before child validation begins.
recalculate-circle-protection-padding.js	Recalculates Circle Protection Padding radius and derived boundaries from the current Circle geometry.	Milestone 9 also validates Circle children inside Rectangle Containers, so Circle Protection Padding must already be stable.
validate-child-containment.js	Validates one child item against its immediate parent boundaries.	Milestone 9 is built on immediate-parent containment validation and cannot continue unless this validation path is already ready.
resize-parent-container.js	Expands the parent container when a child cannot fit under approved rules.	Milestone 9 includes the parent-growth rule when a valid child cannot fit inside the current parent boundary.

Milestone 9 terminology

Main developer names

The following names are recommended for this milestone:

Parent container

ParentRectangleContainer

Parent container geometry

ParentInnerLeftX

ParentInnerRightX

ParentInnerTopY

ParentInnerBottomY

ParentVisibleLeftX

ParentVisibleRightX

ParentVisibleTopY

ParentVisibleBottomY

Child rectangle

ChildRectangleItem

Child rectangle geometry

ChildProtectionLeftX

ChildProtectionRightX

ChildProtectionTopY

ChildProtectionBottomY

Child circle

ChildCircleItem

Child circle geometry

ChildProtectionPaddedRadius

ChildProtectionLeftX

ChildProtectionRightX

ChildProtectionTopY

ChildProtectionBottomY

Reference-center variables

ChildParentOffsetX

ChildParentOffsetY

Containment state

IsChildContainmentValid

IsStrictNoTouchEnabled

NeedsProtectionPaddingRecalculation

Recommended implementation rule

The parent container shall expose a usable interior boundary, and all child containment checks shall use the parent inner boundaries together with the child Protection Padding boundaries. This follows the document's recommended internal usable boundary rule and the general containment rule.

Important milestone rules

Rule 1 — Protection Padding is the containment authority

For this milestone, containment inside a parent rectangle container shall be determined by:

the child item's Protection Padding boundaries

the parent rectangle container's inner usable boundaries

Hover Padding shall not be used for containment.

Rule 2 — All containment checks shall use boundary equations, not center-only equations

Containment shall not be decided by checking only whether the child center is inside the parent.

Instead, containment shall be decided by comparing the full Protection Padding boundaries of the child item against the parent inner boundaries. The document explicitly recommends boundary equations rather than center-only equations.

Rule 3 — Strict no-touch shall remain enabled by default

The child must not:

get out of the parent

overlap the parent border line

even touch the parent border line

Therefore, the containment inequalities should remain strict, and a small epsilon margin may be used for implementation safety. This follows the document's strict no-touch rule.

Rule 4 — Parent move shall preserve child-parent reference center

When the parent rectangle container moves, the child item shall move with it and preserve the same reference-center offset.

Recommended reference-center variables:

$$\text{ChildParentOffsetX} = \text{ChildCenterX} - \text{ParentCenterX}$$
$$\text{ChildParentOffsetY} = \text{ChildCenterY} - \text{ParentCenterY}$$

If the parent later moves by:

$$\Delta\text{ParentX}, \Delta\text{ParentY}$$

then the child shall move by the same translation:

$$\text{NewChildCenterX} = \text{OldChildCenterX} + \Delta\text{ParentX}$$
$$\text{NewChildCenterY} = \text{OldChildCenterY} + \Delta\text{ParentY}$$

Equivalent offset-preserving form:

$$\text{NewChildCenterX} = \text{NewParentCenterX} + \text{ChildParentOffsetX}$$
$$\text{NewChildCenterY} = \text{NewParentCenterY} + \text{ChildParentOffsetY}$$

This is the recommended behavior for your requested "same reference center" rule.

Rule 5 — Parent resize does not automatically scale the child unless explicitly added later

For this milestone, resizing the parent rectangle container shall change the available containment area, but it shall not automatically scale the child item.

Instead:

the child remains geometrically unchanged

the child is revalidated against the new parent inner boundaries

if the child becomes invalid under the new parent size, the resize shall be rejected or clamped according to the chosen policy

That is the cleanest behavior for version 1.

Core Protection Padding equations used in this milestone

Parent rectangle inner usable boundary

For this milestone, the parent rectangle container uses:

ParentInnerLeftX

ParentInnerRightX

ParentInnerTopY

ParentInnerBottomY

In version 1, these may equal the parent visible boundaries, which matches the current document recommendation.

Child rectangle Protection Padding boundaries

For a child rectangle:

$\text{ChildProtectionLeftX} = \text{ChildCenterX} - \text{ChildHalfWidth} - \text{ChildProtectionPaddingX}$

$\text{ChildProtectionRightX} = \text{ChildCenterX} + \text{ChildHalfWidth} + \text{ChildProtectionPaddingX}$

$\text{ChildProtectionTopY} = \text{ChildCenterY} + \text{ChildHalfHeight} + \text{ChildProtectionPaddingY}$

$\text{ChildProtectionBottomY} = \text{ChildCenterY} - \text{ChildHalfHeight} - \text{ChildProtectionPaddingY}$

Containment rule:

$\text{ChildProtectionLeftX} > \text{ParentInnerLeftX}$

$\text{ChildProtectionRightX} < \text{ParentInnerRightX}$

$\text{ChildProtectionTopY} < \text{ParentInnerTopY}$

$\text{ChildProtectionBottomY} > \text{ParentInnerBottomY}$

This follows the child-rectangle containment section already defined in the file.

Child rectangle clamped center equations

If the child rectangle is dragged to a target center position, the safe clamped center is:

$\text{ClampedChildCenterX} = \text{clamp}(\text{TargetChildCenterX}, \text{ParentInnerLeftX} + \text{ChildHalfWidth} + \text{ChildProtectionPaddingX} + \epsilon, \text{ParentInnerRightX} - \text{ChildHalfWidth} - \text{ChildProtectionPaddingX} - \epsilon)$

$\text{ClampedChildCenterY} = \text{clamp}(\text{TargetChildCenterY}, \text{ParentInnerBottomY} + \text{ChildHalfHeight} + \text{ChildProtectionPaddingY} + \epsilon, \text{ParentInnerTopY} - \text{ChildHalfHeight} - \text{ChildProtectionPaddingY} - \epsilon)$

ParentInnerTopY - ChildHalfHeight - ChildProtectionPaddingY - ϵ
)

This is the direct practical form of the document's general clamped movement rule plus strict no-touch safety margin.

Child circle Protection Padding boundaries

For a child circle:

ChildProtectionPaddedRadius = ChildCircleRadius + ChildCircleProtectionPadding

ChildProtectionLeftX = ChildCircleCenterX - ChildProtectionPaddedRadius

ChildProtectionRightX = ChildCircleCenterX + ChildProtectionPaddedRadius

ChildProtectionTopY = ChildCircleCenterY + ChildProtectionPaddedRadius

ChildProtectionBottomY = ChildCircleCenterY - ChildProtectionPaddedRadius

Containment rule:

ChildProtectionLeftX > ParentInnerLeftX

ChildProtectionRightX < ParentInnerRightX

ChildProtectionTopY < ParentInnerTopY

ChildProtectionBottomY > ParentInnerBottomY

This follows the document's rule that circles inside containers should be converted to protection-padded left/right/top/bottom boundaries and then checked using the same rectangle-container logic.

Child circle clamped center equations

ClampedChildCircleCenterX = clamp(
TargetChildCircleCenterX,
ParentInnerLeftX + ChildProtectionPaddedRadius + ϵ ,
ParentInnerRightX - ChildProtectionPaddedRadius - ϵ
)

ClampedChildCircleCenterY = clamp(
TargetChildCircleCenterY,
ParentInnerBottomY + ChildProtectionPaddedRadius + ϵ ,
ParentInnerTopY - ChildProtectionPaddedRadius - ϵ
)

Mathematical Calculation Rules for Milestone 9

Milestone 9 calculations validate one child item inside one rectangle container using Protection Padding. The child may be a rectangle or circle. The parent container uses an inner usable boundary as the containment authority.

Primary JavaScript files for this calculation scope: validate-child-containment.js, resize-parent-container.js, recalculate-container-derived-geometry-on-resize.js, apply-validated-center.js, restore-previous-valid-position.js.

General containment calculation

ChildProtectionLeftX > ParentInnerLeftX. ChildProtectionRightX < ParentInnerRightX.

ChildProtectionTopY < ParentInnerTopY. ChildProtectionBottomY > ParentInnerBottomY.

Child rectangle center clamp

ClampedCenterX = clamp(TargetCenterX, ParentInnerLeftX + ChildHalfWidth + ChildProtectionPaddingX + Epsilon, ParentInnerRightX - ChildHalfWidth - ChildProtectionPaddingX - Epsilon).

ClampedCenterY = clamp(TargetCenterY, ParentInnerBottomY + ChildHalfHeight + ChildProtectionPaddingY + Epsilon, ParentInnerTopY - ChildHalfHeight - ChildProtectionPaddingY - Epsilon).

Child circle center clamp

ClampedCenterX = clamp(TargetCenterX, ParentInnerLeftX + ChildProtectionPaddedRadius + Epsilon, ParentInnerRightX - ChildProtectionPaddedRadius - Epsilon).

ClampedCenterY = clamp(TargetCenterY, ParentInnerBottomY + ChildProtectionPaddedRadius + Epsilon, ParentInnerTopY - ChildProtectionPaddedRadius - Epsilon).

Parent expansion calculation

If insertion or resize requires parent growth: NewParentInnerLeftX = min(CurrentParentInnerLeftX, ChildProtectionLeftX - Epsilon), and apply the corresponding max/min expansion to right, top, and bottom boundaries.

Milestone 9 phases

Milestone 9 shall be implemented in 8 phases.

Phase 9.1 — Single child rectangle inside parent rectangle container: free movement under Protection Padding

For more information, please refer to section Protection Padding, Overlap, Containment, and Persistence Calculations ([place in this document](#)).

Goal

Allow a single child rectangle to move freely inside a parent rectangle container without ever getting out of it.

Scope

This phase is about:

a single parent rectangle container

a single child rectangle

free child movement inside the parent

Protection Padding calculation during movement

clamped containment validation

Required test behaviors

The child rectangle shall be movable freely inside the parent rectangle container.

Expected result:

the child rectangle moves smoothly

the child rectangle never gets out of the parent

the child rectangle never touches the parent border line

the child rectangle's Protection Padding boundaries remain valid during movement

Variables to define

At minimum:

ParentInnerLeftX

ParentInnerRightX

ParentInnerTopY

ParentInnerBottomY

ChildCenterX

ChildCenterY

ChildHalfWidth

ChildHalfHeight

ChildProtectionPaddingX

ChildProtectionPaddingY

IsChildContainmentValid

Required event-to-function routing

Child-rectangle move flow

left click and drag child rectangle

→ screenToWorld

→ computeCandidateChildRectangleCenter

→ recalculateChildRectangleProtectionPadding

→ clampChildRectangleCenterWithinParent

→ applyValidatedChildRectangleCenter

→ renderCanvas

State rule

During this phase:

only the child rectangle moves

the parent rectangle container remains fixed

all containment checks happen in world space

no persistence is required yet

Success criteria

This phase is successful if:

the child rectangle can move freely inside the parent rectangle container

the child rectangle never leaves the parent

the child rectangle never touches the border line

Function interaction contracts

Function Name: recalculateChildRectangleProtectionPadding

Layer: JavaScript

File: child-rectangle-protection-padding.js

Purpose: Recalculate the child rectangle's Protection Padding boundaries from its canonical geometry.

Trigger:

child rectangle movement begins or candidate child rectangle center changes

Input:

ChildCenterX

ChildCenterY

ChildHalfWidth

ChildHalfHeight

ChildProtectionPaddingX

ChildProtectionPaddingY

Validation / Preconditions:

child rectangle exists

child rectangle geometry is valid

padding values are numeric and non-negative

Processing Responsibility:

calculate ChildProtectionLeftX

ChildProtectionRightX

ChildProtectionTopY

ChildProtectionBottomY

prepare child rectangle padded boundaries for containment validation

Output:

updated child rectangle Protection Padding boundaries

State Change / Persistence Effect:

updates temporary in-memory derived geometry only

no persistence effect

Next Possible Call(s):

clampChildRectangleCenterWithinParent()

validateChildRectangleContainment()

Error Output / Failure Path:

return invalid recalculation result and preserve previous valid child rectangle state if geometry is invalid

Function Name: clampChildRectangleCenterWithinParent

Layer: JavaScript

File: child-rectangle-container-clamp.js

Purpose: Clamp a candidate child rectangle center so the child remains fully inside the parent rectangle container under strict no-touch rules.

Trigger:

candidate child rectangle center generated during drag

Input:

TargetChildCenterX

TargetChildCenterY

ParentInnerLeftX

ParentInnerRightX

ParentInnerTopY

ParentInnerBottomY

ChildHalfWidth

ChildHalfHeight

ChildProtectionPaddingX

ChildProtectionPaddingY

epsilon

Validation / Preconditions:

parent container exists

child rectangle exists

parent inner boundaries are valid

child padded geometry is derivable

Processing Responsibility:

compute clamped child rectangle center using parent inner boundaries and child Protection Padding dimensions

enforce strict no-touch safety margin

Output:

ClampedChildCenterX

ClampedChildCenterY

State Change / Persistence Effect:

no direct persistence effect

returns a validated or clamped center for later application

Next Possible Call(s):
applyValidatedChildRectangleCenter()
renderCanvas()

Error Output / Failure Path:
preserve previous valid child rectangle center if clamp calculation fails

Phase 9.2 — Single child rectangle inside parent rectangle container: child resize under Protection Padding

For more information, please refer to section [Protection Padding, Overlap, Containment, and Persistence Calculations](#) ([place in this document](#)).

Goal

Allow the single child rectangle to be resized while confirming that it never gets out of the parent rectangle container.

Scope

This phase is about:

child rectangle resize

Protection Padding recalculation after resize

containment validation during resize

resize rejection or clamping if invalid

Required test behaviors

The child rectangle shall be resized from its rectangle resize control points.

Expected result:

the child rectangle resizes correctly

the child rectangle never gets out of the parent

the child rectangle never touches the parent border line

the child rectangle's resized Protection Padding boundaries remain valid

Variables to define

At minimum:

CandidateChildHalfWidth

CandidateChildHalfHeight

CandidateChildCenterX

CandidateChildCenterY

MinChildHalfWidth

MinChildHalfHeight

Required event-to-function routing

Child-rectangle resize flow

left click and drag child rectangle resize control point

→ updateChildRectangleResizeCandidate

→ recalculateChildRectangleProtectionPadding

→ validateChildRectangleResizeWithinParent

→ applyValidatedChildRectangleResize

→ renderCanvas

State rule

During this phase:

the child rectangle changes size

the parent remains fixed

containment is enforced using the child's Protection Padding boundaries and the parent's inner usable boundaries

Success criteria

This phase is successful if:

the child rectangle can resize

the resized child rectangle never gets out of the parent

the resized child rectangle never touches the parent border line

Function interaction contracts

Function Name: validateChildRectangleResizeWithinParent

Layer: JavaScript

File: child-rectangle-resize-validation.js

Purpose: Validate a candidate child rectangle resize against parent rectangle containment rules.

Trigger:

candidate child rectangle resize geometry generated during resize

Input:

CandidateChildCenterX

CandidateChildCenterY

CandidateChildHalfWidth

CandidateChildHalfHeight

ChildProtectionPaddingX

ChildProtectionPaddingY

ParentInnerLeftX

ParentInnerRightX

ParentInnerTopY
ParentInnerBottomY

Validation / Preconditions:

parent container exists
child rectangle exists
candidate child geometry is valid
padding values are valid

Processing Responsibility:

derive candidate child rectangle Protection Padding boundaries
compare them against the parent inner boundaries
return valid or invalid resize result

Output:

valid / invalid child resize validation result

State Change / Persistence Effect:

no persistence effect
no direct permanent state change

Next Possible Call(s):

applyValidatedChildRectangleResize()
restorePreviousValidChildRectangleGeometry()

Error Output / Failure Path:

reject the resize and preserve previous valid child rectangle geometry if validation fails

Function Name: applyValidatedChildRectangleResize

Layer: JavaScript

File: child-rectangle-resize.js

Purpose: Commit a validated child rectangle resize and update derived Protection Padding geometry.

Trigger:

successful child rectangle resize validation result

Input:

ValidatedChildCenterX
ValidatedChildCenterY
ValidatedChildHalfWidth
ValidatedChildHalfHeight
child rectangle ID

Validation / Preconditions:

validated resize result exists
child rectangle exists
validated geometry values are numeric and above minimum size

Processing Responsibility:

update child rectangle canonical geometry
recalculate Protection Padding boundaries
request render update

Output:

updated child rectangle geometry

State Change / Persistence Effect:

updates temporary in-memory child rectangle model only

no persistence effect

Next Possible Call(s):

recalculateChildRectangleProtectionPadding()

renderCanvas()

Error Output / Failure Path:

preserve previous valid child rectangle geometry if the validated resize result is invalid

Phase 9.3 — Parent rectangle resize and parent-child reference-center preservation for child rectangle

For more information, please refer to section Parent-container expansion calculations ([place in this document](#)) and section Protection Padding, Overlap, Containment, and Persistence Calculations ([place in this document](#)).

Goal

Resize the parent rectangle container and then move the child rectangle and the parent container while preserving the child-parent reference-center relationship.

Scope

This phase is about:

Whenever parent container resize changes the container geometry, recalculate-container-derived-geometry-on-resize.js shall run so that visible boundaries, Hover Padding, Protection Padding, inner usable boundaries, and containment helper boundaries are recalculated before child-preservation or further validation.

parent rectangle container resize

revalidation of child rectangle against the resized parent

parent move with child-follow behavior

preserving child-parent offset

Required test behaviors

Expected result:

resizing the parent changes the available containment area

the child remains valid inside the resized parent or the parent resize is rejected/clamped

when the parent moves, the child moves with it

the child preserves the same reference-center offset relative to the parent

Variables to define

At minimum:

ParentResizeCandidate

ChildParentOffsetX

ChildParentOffsetY

ParentMoveDeltaX

ParentMoveDeltaY

Required event-to-function routing

Parent resize flow

left click and drag parent rectangle resize control point

→ updateParentRectangleResizeCandidate

→ validateParentResizeAgainstCurrentChild

→ applyValidatedParentResize

→ renderCanvas

Parent move flow

left click and drag parent rectangle container

→ computeParentMoveDelta

→ moveChildRectangleWithParentReference

→ applyValidatedParentMove

→ renderCanvas

State rule

During this phase:

the parent may resize or move

the child remains associated with the same parent

when the parent moves, the child moves with it by translation and preserves its reference-center offset

Success criteria

This phase is successful if:

resizing the parent is validated against the current child rectangle

moving the parent also moves the child rectangle

the child keeps the same reference-center offset relative to the parent

Function interaction contracts

Function Name: validateParentResizeAgainstCurrentChild

Layer: JavaScript

File: parent-rectangle-resize-validation.js

Purpose: Validate a candidate parent rectangle resize against the current child rectangle Protection Padding boundaries.

Trigger:
candidate parent rectangle resize generated during parent resize

Input:
CandidateParentInnerLeftX
CandidateParentInnerRightX
CandidateParentInnerTopY
CandidateParentInnerBottomY
current child rectangle Protection Padding boundaries

Validation / Preconditions:
parent container exists
child rectangle exists
candidate parent geometry is valid
child padded boundaries are valid

Processing Responsibility:
compare the current child rectangle Protection Padding boundaries against the candidate parent inner boundaries
return valid or invalid parent resize result

Output:
valid / invalid parent resize validation result

State Change / Persistence Effect:
no persistence effect

Next Possible Call(s):
applyValidatedParentResize()
restorePreviousValidParentGeometry()

Error Output / Failure Path:
reject the parent resize if the current child would become invalid

Function Name: moveChildRectangleWithParentReference

Layer: JavaScript
File: parent-child-reference-center.js
Purpose: Move the child rectangle with the parent rectangle while preserving the stored child-parent reference-center offset.

Trigger:
validated parent move delta generated during parent drag

Input:
NewParentCenterX
NewParentCenterY
ChildParentOffsetX
ChildParentOffsetY
child rectangle ID

Validation / Preconditions:

parent exists

child exists

stored child-parent reference offset exists

Processing Responsibility:

compute new child rectangle center from the new parent center and the stored reference offset

recalculate child rectangle derived geometry

preserve the same offset relationship

Output:

updated child rectangle center after parent move

State Change / Persistence Effect:

updates temporary in-memory parent and child positions only

no persistence effect

Next Possible Call(s):

recalculateChildRectangleProtectionPadding()

renderCanvas()

Error Output / Failure Path:

preserve previous valid parent and child positions if reference-offset computation fails

Phase 9.4 — Single child circle inside parent rectangle container: free movement under Protection Padding

For more information, please refer to section [Protection Padding, Overlap, Containment, and Persistence Calculations](#) ([place in this document](#)).

Goal

Allow a single child circle to move freely inside a parent rectangle container without ever getting out of it.

Scope

This phase is about:

a single parent rectangle container

a single child circle

free child circle movement inside the parent

Protection Padding calculation during movement

clamped containment validation

Required test behaviors

Expected result:

the child circle moves smoothly

the child circle never gets out of the parent

the child circle never touches the parent border line

the child circle's Protection Padding boundaries remain valid during movement

Variables to define

At minimum:

ChildCircleCenterX

ChildCircleCenterY

ChildCircleRadius

ChildCircleProtectionPadding

ChildProtectionPaddedRadius

Required event-to-function routing

Child-circle move flow

left click and drag child circle

→ screenToWorld

→ computeCandidateChildCircleCenter

→ recalculateChildCircleProtectionPadding

→ clampChildCircleCenterWithinParent

→ applyValidatedChildCircleCenter

→ renderCanvas

State rule

During this phase:

only the child circle moves

the parent rectangle container remains fixed

containment uses the circle's protection-padded left/right/top/bottom boundaries against the parent inner boundaries

Success criteria

This phase is successful if:

the child circle can move freely inside the parent rectangle container

the child circle never leaves the parent

the child circle never touches the border line

Function interaction contracts

Function Name: recalculateChildCircleProtectionPadding

Layer: JavaScript

File: child-circle-protection-padding.js

Purpose: Recalculate the child circle's Protection Padding values and protection-padded boundaries.

Trigger:

child circle movement begins or candidate child circle center changes

Input:

ChildCircleCenterX

ChildCircleCenterY

ChildCircleRadius

ChildCircleProtectionPadding

Validation / Preconditions:

child circle exists

circle radius is valid

protection padding value is numeric and non-negative

Processing Responsibility:

calculate ChildProtectionPaddedRadius

ChildProtectionLeftX

ChildProtectionRightX

ChildProtectionTopY

ChildProtectionBottomY

prepare child circle padded boundaries for containment validation

Output:

updated child circle Protection Padding values and boundaries

State Change / Persistence Effect:

updates temporary in-memory derived geometry only

no persistence effect

Next Possible Call(s):

clampChildCircleCenterWithinParent()

validateChildCircleContainment()

Error Output / Failure Path:

return invalid recalculation result and preserve previous valid child circle state if geometry is invalid

Function Name: clampChildCircleCenterWithinParent

Layer: JavaScript

File: child-circle-container-clamp.js

Purpose: Clamp a candidate child circle center so the child remains fully inside the parent rectangle container under strict no-touch rules.

Trigger:

candidate child circle center generated during drag

Input:

TargetChildCircleCenterX

TargetChildCircleCenterY

ParentInnerLeftX
ParentInnerRightX
ParentInnerTopY
ParentInnerBottomY
ChildProtectionPaddedRadius
epsilon

Validation / Preconditions:

parent container exists
child circle exists
parent inner boundaries are valid
child protection-padded radius is valid

Processing Responsibility:

compute clamped child circle center using parent inner boundaries and child protection-padded radius
enforce strict no-touch safety margin

Output:

ClampedChildCircleCenterX
ClampedChildCircleCenterY

State Change / Persistence Effect:

no direct persistence effect
returns a validated or clamped center for later application

Next Possible Call(s):

applyValidatedChildCircleCenter()
renderCanvas()

Error Output / Failure Path:

preserve previous valid child circle center if clamp calculation fails

Phase 9.5 — Single child circle inside parent rectangle container: child resize under Protection Padding

For more information, please refer to section [Protection Padding, Overlap, Containment, and Persistence Calculations](#) ([place in this document](#)).

Goal

Allow the single child circle to be resized while confirming that it never gets out of the parent rectangle container.

Scope

This phase is about:

child circle resize

Protection Padding recalculation after resize

containment validation during resize

resize rejection or clamping if invalid

Required test behaviors

Expected result:

the child circle resizes correctly

the child circle never gets out of the parent

the child circle never touches the parent border line

the child circle's protection-padded boundaries remain valid

Variables to define

At minimum:

CandidateChildCircleRadius

ValidatedChildCircleRadius

MinChildCircleRadius

Required event-to-function routing

Child-circle resize flow

left click and drag child circle resize control point

→ updateChildCircleResizeCandidate

→ recalculateChildCircleProtectionPadding

→ validateChildCircleResizeWithinParent

→ applyValidatedChildCircleResize

→ renderCanvas

State rule

During this phase:

the child circle changes size

the parent remains fixed

containment uses the circle's protection-padded boundaries against the parent inner boundaries

Success criteria

This phase is successful if:

the child circle can resize

the resized child circle never gets out of the parent

the resized child circle never touches the parent border line

Function interaction contracts

Function Name: validateChildCircleResizeWithinParent

Layer: JavaScript

File: child-circle-resize-validation.js

Purpose: Validate a candidate child circle resize against parent rectangle containment rules.

Trigger:

candidate child circle radius generated during resize

Input:

CandidateChildCircleCenterX

CandidateChildCircleCenterY

CandidateChildCircleRadius

ChildCircleProtectionPadding

ParentInnerLeftX

ParentInnerRightX

ParentInnerTopY

ParentInnerBottomY

Validation / Preconditions:

parent container exists

child circle exists

candidate circle geometry is valid

protection padding value is valid

Processing Responsibility:

derive candidate protection-padded radius and padded left/right/top/bottom boundaries

compare them against the parent inner boundaries

return valid or invalid resize result

Output:

valid / invalid child circle resize validation result

State Change / Persistence Effect:

no persistence effect

Next Possible Call(s):

applyValidatedChildCircleResize()

restorePreviousValidChildCircleGeometry()

Error Output / Failure Path:

reject the resize and preserve previous valid child circle geometry if validation fails

Function Name: applyValidatedChildCircleResize

Layer: JavaScript

File: child-circle-resize.js

Purpose: Commit a validated child circle resize and update derived Protection Padding geometry.

Trigger:

successful child circle resize validation result

Input:

ValidatedChildCircleCenterX

ValidatedChildCircleCenterY

ValidatedChildCircleRadius
child circle ID

Validation / Preconditions:

validated resize result exists

child circle exists

validated radius is numeric and above minimum size

Processing Responsibility:

update child circle canonical geometry

recalculate child circle Protection Padding values and boundaries

request render update

Output:

updated child circle geometry

State Change / Persistence Effect:

updates temporary in-memory child circle model only

no persistence effect

Next Possible Call(s):

recalculateChildCircleProtectionPadding()

renderCanvas()

Error Output / Failure Path:

preserve previous valid child circle geometry if the validated resize result is invalid

Phase 9.6 — Parent rectangle resize and parent-child reference-center preservation for child circle

For more information, please refer to section Parent-container expansion calculations ([place in this document](#)) and section Protection Padding, Overlap, Containment, and Persistence Calculations ([place in this document](#)).

Goal

Resize the parent rectangle container and then move the child circle and the parent container while preserving the child-parent reference-center relationship.

Scope

This phase is about:

parent rectangle container resize

revalidation of child circle against the resized parent

parent move with child-follow behavior

preserving child-parent offset

Required test behaviors

Expected result:

resizing the parent changes the available containment area

the child circle remains valid inside the resized parent or the parent resize is rejected/clamped

when the parent moves, the child circle moves with it

the child circle preserves the same reference-center offset relative to the parent

Variables to define

At minimum:

ChildCircleParentOffsetX

ChildCircleParentOffsetY

ParentMoveDeltaX

ParentMoveDeltaY

Required event-to-function routing

Parent resize flow with child circle

left click and drag parent rectangle resize control point

→ updateParentRectangleResizeCandidate

→ validateParentResizeAgainstCurrentChildCircle

→ applyValidatedParentResize

→ renderCanvas

Parent move flow with child circle

left click and drag parent rectangle container

→ computeParentMoveDelta

→ moveChildCircleWithParentReference

→ applyValidatedParentMove

→ renderCanvas

State rule

During this phase:

the parent may resize or move

the child circle remains associated with the same parent rectangle container

when the parent moves, the child circle moves by translation and preserves its reference-center offset

Success criteria

This phase is successful if:

resizing the parent is validated against the current child circle

moving the parent also moves the child circle

the child keeps the same reference-center offset relative to the parent

Function interaction contracts

Function Name: validateParentResizeAgainstCurrentChildCircle

Layer: JavaScript

File: parent-rectangle-resize-validation.js

Purpose: Validate a candidate parent rectangle resize against the current child circle protection-padded boundaries.

Trigger:

candidate parent rectangle resize generated during parent resize

Input:

CandidateParentInnerLeftX

CandidateParentInnerRightX

CandidateParentInnerTopY

CandidateParentInnerBottomY

current child circle protection-padded boundaries

Validation / Preconditions:

parent container exists

child circle exists

candidate parent geometry is valid

child padded boundaries are valid

Processing Responsibility:

compare the current child circle protection-padded boundaries against the candidate parent inner boundaries

return valid or invalid parent resize result

Output:

valid / invalid parent resize validation result

State Change / Persistence Effect:

no persistence effect

Next Possible Call(s):

applyValidatedParentResize()

restorePreviousValidParentGeometry()

Error Output / Failure Path:

reject the parent resize if the current child circle would become invalid

Function Name: moveChildCircleWithParentReference

Layer: JavaScript

File: parent-child-reference-center.js

Purpose: Move the child circle with the parent rectangle while preserving the stored child-parent reference-center offset.

Trigger:

validated parent move delta generated during parent drag

Input:

NewParentCenterX

NewParentCenterY

ChildCircleParentOffsetX

ChildCircleParentOffsetY

child circle ID

Validation / Preconditions:

parent exists

child circle exists

stored child-parent reference offset exists

Processing Responsibility:

compute new child circle center from the new parent center and the stored reference offset

recalculate child circle derived geometry

preserve the same offset relationship

Output:

updated child circle center after parent move

State Change / Persistence Effect:

updates temporary in-memory parent and child positions only

no persistence effect

Next Possible Call(s):

recalculateChildCircleProtectionPadding()

renderCanvas()

Error Output / Failure Path:

preserve previous valid parent and child positions if reference-offset computation fails

Phase 9.7 — Recalculate committed Protection Padding geometry and persist the containment state to JSON and database

Exact variables to save in this phase

Milestone 9 introduces no new independent permanent persistence field names. The exact save requirement in this phase is: keep saving all previously approved Milestone 2, Milestone 3, Milestone 7, and Milestone 8 committed variables, together with the already approved ParentContainerID relationships on Device and Container records, after containment recalculation has been accepted.

Do not save Milestone 9 recalculation or validation helpers as milestone-required persisted data:

ParentInnerLeftX, ParentInnerRightX, ParentInnerTopY, ParentInnerBottomY,

CandidateContainedCenterX, CandidateContainedCenterY, ContainedProtectionBounds,

PreviousValidCenterX, PreviousValidCenterY, ContainmentValidationResult. These values shall be recalculated or used only for validation.

Save regression rule for this phase

All approved Milestones 2 through 8 committed variables shall already remain saved correctly in both persistence paths. This phase passes only if save-to-json.js and save-to-db.js keep the prior approved

save set intact while persisting the accepted containment result through canonical geometry and parent relations only.

Save test criteria for this phase

Verify that both shared save files persist the accepted canonical parent and child records with the correct ParentContainerID relations, that the recalculation helpers listed above are excluded as milestone-required saved data, and that reopen from either source restores the same containment result.

For more information, please refer to section Protection Padding, Overlap, Containment, and Persistence Calculations ([place in this document](#)).

Goal

Recalculate all committed Protection Padding geometry for parent-child containment and persist the diagram to both JSON and database.

Scope

This phase is about:

recalculating committed child Protection Padding geometry

revalidating parent-child containment

rebuilding the save model

saving the diagram to JSON

saving the diagram to the database

Important recalculation rule

Before save, the system shall recalculate from the committed geometry:

parent inner usable boundaries

child rectangle Protection Padding boundaries if the child is a rectangle

child circle protection-padded radius and protection-padded boundaries if the child is a circle

child-parent reference-center offsets if they are part of the committed containment relationship

This follows the document's general rule that derived geometry shall be recalculated from canonical stored values and that Protection Padding is the authoritative safety region for containment validation.

Required event-to-function routing

Save-commit flow

click Save button

→ recalculateAllContainmentProtectionPaddingGeometry

→ validateCommittedContainmentState

→ buildDiagramJsonObject

→ serializeDiagramToJson

→ saveDiagramJsonFile

→ buildDiagramSavePayload

→ callSaveDiagramApi

→ DiagramController.SaveDiagram
→ DiagramService.SaveDiagram
→ DiagramRepository.SaveDiagram
→ handleSaveCommitResult

State rule

During this phase:

recalculation must occur before persistence

JSON save and database save must use the same committed recalculated containment state

Success criteria

This phase is successful if:

all committed parent-child Protection Padding geometry is recalculated before save

the saved JSON and database state both represent the same committed containment logic

reopening from file or database restores the same parent-child containment state

Function interaction contracts

Function Name: recalculateAllContainmentProtectionPaddingGeometry

Layer: JavaScript

File: containment-protection-padding-recalculation.js

Purpose: Recalculate all parent-child Protection Padding geometry required for committed containment validation and persistence.

Trigger:

explicit save flow

diagram load reconstruction

committed geometry update requiring full containment refresh

Input:

parent container collection

child rectangle collection

child circle collection

committed geometry values

parent-child relation data

Validation / Preconditions:

all referenced parent containers exist

all referenced child items exist

canonical geometry values are present

Processing Responsibility:

recalculate parent inner usable boundaries

recalculate child rectangle Protection Padding boundaries

recalculate child circle protection-padded radius and padded boundaries

prepare committed containment geometry for validation

Output:

recalculated containment geometry collection

State Change / Persistence Effect:

updates in-memory derived containment geometry only

no direct persistence by this function

Next Possible Call(s):

validateCommittedContainmentState()

buildDiagramJsonObject()

renderCanvas()

Error Output / Failure Path:

stop save flow if required containment geometry cannot be recalculated

Function Name: validateCommittedContainmentState

Layer: JavaScript

File: containment-save-validation.js

Purpose: Verify that the committed parent-child containment state is valid before persistence.

Trigger:

save flow after containment recalculation

Input:

committed parent geometry

committed child rectangle or child circle geometry

recalculated Protection Padding boundaries

parent-child reference-center data if used

Validation / Preconditions:

recalculation already completed

required parent and child fields exist

parent-child relations are valid

Processing Responsibility:

validate child rectangle containment if the child is a rectangle

validate child circle containment if the child is a circle

validate strict no-touch conditions

validate that saved reference-center data is consistent if present

Output:

valid / invalid committed containment result

State Change / Persistence Effect:

no persistence effect

no direct permanent state change

Next Possible Call(s):

buildDiagramJsonObject()

buildDiagramSavePayload()

Error Output / Failure Path:

stop save flow and report containment-validation failure if committed state is invalid

Phase 9.8 — Validate Circle On Container Protection Padding and radius constraints on a single rectangle container

Goal

Extend the single-container Protection Padding milestone so a Circle On Container on the parent boundary is validated without being treated as a normal child circle fully contained inside the container.

Scope

This phase is about one rectangle container with one or more Circle On Container items attached to its boundary. The Circle On Container shall keep normal Circle Protection Padding and shall be validated against sibling geometries and other boundary circles using strict no-touch rules when required.

Required validation rules

Do not apply normal child-circle containment to Circle On Container. Validate attachment to the allowed container edge, CSV relationship eligibility, radius range, ProtectionPaddedRadius, and sibling/root non-overlap.

If the parent container is resized, the Circle On Container Radius shall not automatically change. The candidate container resize is valid only if the existing Radius remains within the recomputed min/max range for the active edge length.

Success criteria

This phase succeeds when Circle On Container items participate in Protection Padding validation for the container system, preserve their independent Radius, and reject or prevent invalid overlap/touch or invalid radius range states.

Final result after Milestone 9

At the end of Milestone 9, the developer should have:

single child rectangle containment inside a parent rectangle container

free child rectangle movement inside the parent under Protection Padding rules

child rectangle resize under Protection Padding rules

parent rectangle resize validation against child rectangle containment

parent move with child rectangle follow behavior and preserved reference-center offset

single child circle containment inside a parent rectangle container

free child circle movement inside the parent under Protection Padding rules

child circle resize under Protection Padding rules

parent rectangle resize validation against child circle containment

parent move with child circle follow behavior and preserved reference-center offset

recalculation and persistence of committed Protection Padding containment state

Circle On Container boundary Protection Padding validation with independent radius-range validation.

This milestone makes Protection Padding and parent-child containment behavior a real interactive and persistent part of the diagram system.

Only after containment behavior is stable should JSON/database persistence be added.

Do not add multi-child logic here; keep the milestone focused on one child and one immediate parent at a time.

Implement single-child containment validation first for Rectangle children, then extend to Circle children, then add approved parent-resize behavior.

Recommended implementation order

Recommended coding files structure additions for Milestone 9

JavaScript

Recommended new modules:

child-rectangle-protection-padding.js

child-rectangle-container-clamp.js

child-rectangle-resize-validation.js

child-rectangle-resize.js

child-circle-protection-padding.js

child-circle-container-clamp.js

child-circle-resize-validation.js

child-circle-resize.js

parent-rectangle-resize-validation.js

parent-child-reference-center.js

containment-protection-padding-recalculation.js

recalculate-container-derived-geometry-on-resize.js

containment-save-validation.js

circle-on-container-protection-padding-validation.js

circle-on-container-radius-range-validation.js

circle-on-container-container-resize-radius-validation.js

Optional C# structure

ContainmentDto.cs

ParentChildRelationDto.cs

ProtectionPaddingStateDto.cs

Do not move live containment validation, clamped movement, or parent-child reference updates into C#.

Best folder layout additions for Milestone 9

```
project/
├─ js/
│   ├─ child-rectangle-protection-padding.js
│   ├─ child-rectangle-container-clamp.js
│   ├─ child-rectangle-resize-validation.js
│   ├─ child-rectangle-resize.js
│   ├─ child-circle-protection-padding.js
│   ├─ child-circle-container-clamp.js
│   ├─ child-circle-resize-validation.js
│   ├─ child-circle-resize.js
│   ├─ parent-rectangle-resize-validation.js
│   ├─ parent-child-reference-center.js
│   ├─ containment-protection-padding-recalculation.js
│   └─ containment-save-validation.js
├─ csharp/
│   ├─ ContainmentDto.cs
│   ├─ ParentChildRelationDto.cs
│   └─ ProtectionPaddingStateDto.cs
```

More ideas you can define in Milestone 9

You may either include these now or mark them as future-ready additions:

1. Multiple child items inside one parent container
2. Sibling-overlap validation inside the same parent container
3. Nested nested rectangle containers with recursive parent-child offset preservation
4. Child snapping inside the parent container
5. Parent-center scaling policy if you later want parent resize to reposition children proportionally
6. Visual debug overlay for Protection Padding boundaries
7. Temporary warning color when a candidate move or resize becomes invalid
8. Save/load of the parent-child reference-center offset as explicit committed relation metadata

The strongest design recommendation here is:

Use **parent inner usable boundaries** as the parent containment authority, use **child Protection Padding boundaries** as the child containment authority, and keep the parent-move behavior based on a **stored child-parent reference-center offset**. That gives you the cleanest and most predictable implementation for both child rectangles and child circles inside rectangle containers.

Milestone 10 — Protection Padding Calculation and Containment Behavior for Multiple Children Inside One Parent Rectangle Container

Overview

Milestone 10 defines the multi-child extension of the parent-container Protection Padding logic.

For this milestone, all containers shall be treated as:

Rectangle Containers only

not circle containers

This milestone keeps the same parent-container rules already defined in the document:

Protection Padding is the mathematical safety region used for overlap avoidance, spacing rules, child-inside-container calculations, and containment validation

all Protection Padding calculations must happen in world space

child geometries inside a parent rectangle container must remain fully inside the immediate parent container and must not get out of it, overlap its border line, or even touch its border line when strict no-touch is enabled

clamped movement is the recommended safe behavior for contained items

The difference in Milestone 10 is:

the parent rectangle container now contains more than one child item at the same time

therefore, each candidate child move or resize must be validated against:

the parent rectangle container inner usable boundary

all sibling child items already inside the same immediate parent container

So for each child item, the direct validation scope becomes:

parent containment validation

sibling non-overlap validation

This is consistent with the document's earlier Protection Padding rules, which state that child geometry inside a parent container must be validated against both the immediate parent and sibling geometries inside the same immediate parent container.

This milestone shall be implemented in 7 phases.

The first 3 phases apply to:
multiple child rectangles inside one parent rectangle container

The next 3 phases apply to:
multiple child circles inside one parent rectangle container

The last phase ensures that all committed multi-child Protection Padding geometry and containment state can be recalculated and persisted correctly.

A core design rule for this milestone is:

JavaScript is responsible for immediate Protection Padding calculation, sibling-overlap validation, live containment validation, drag behavior, resize behavior, clamped movement, parent-child reference updates, and live rendering

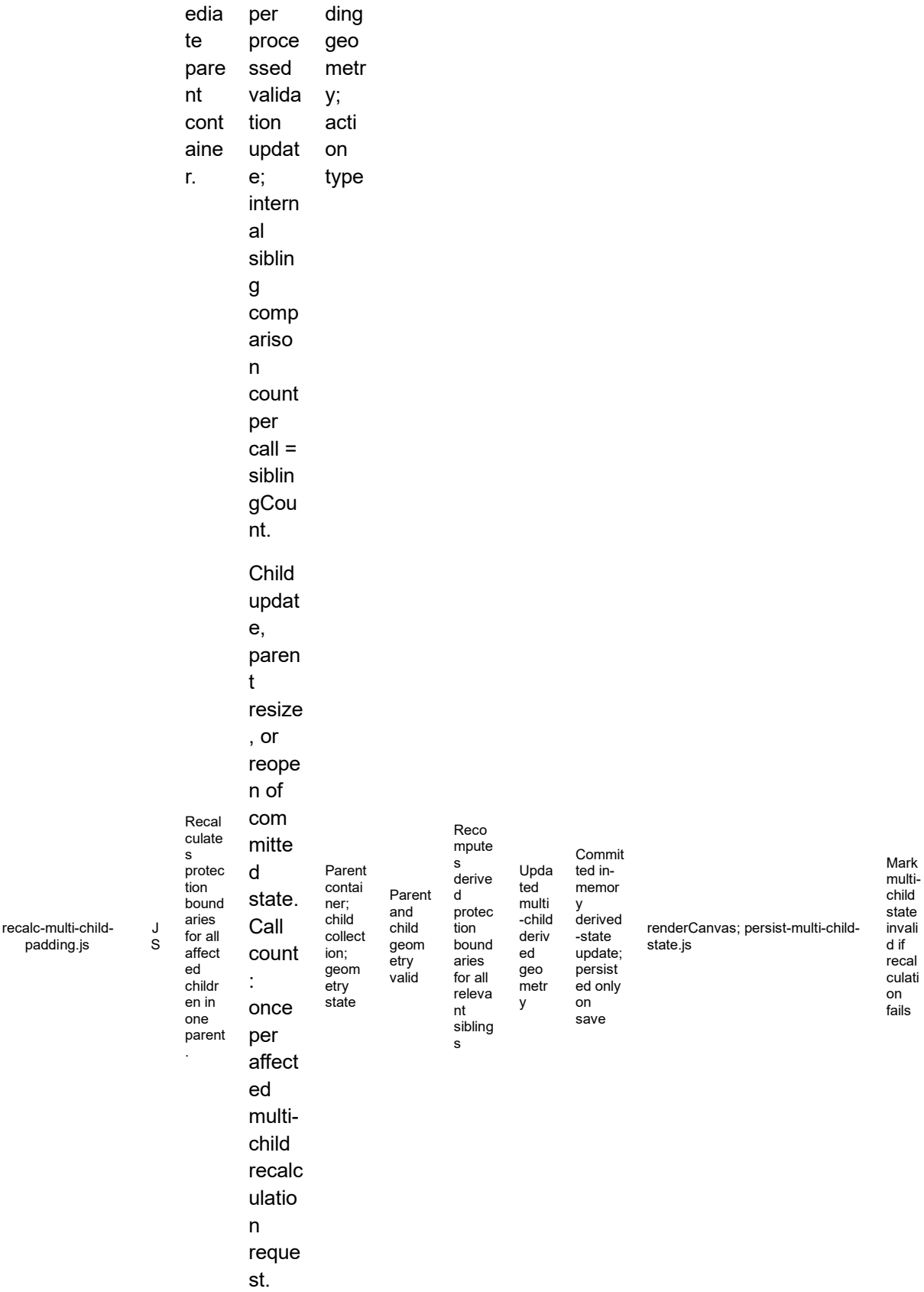
C# is responsible only for trusted validation, persistence, API handling, and database communication when save/load is required

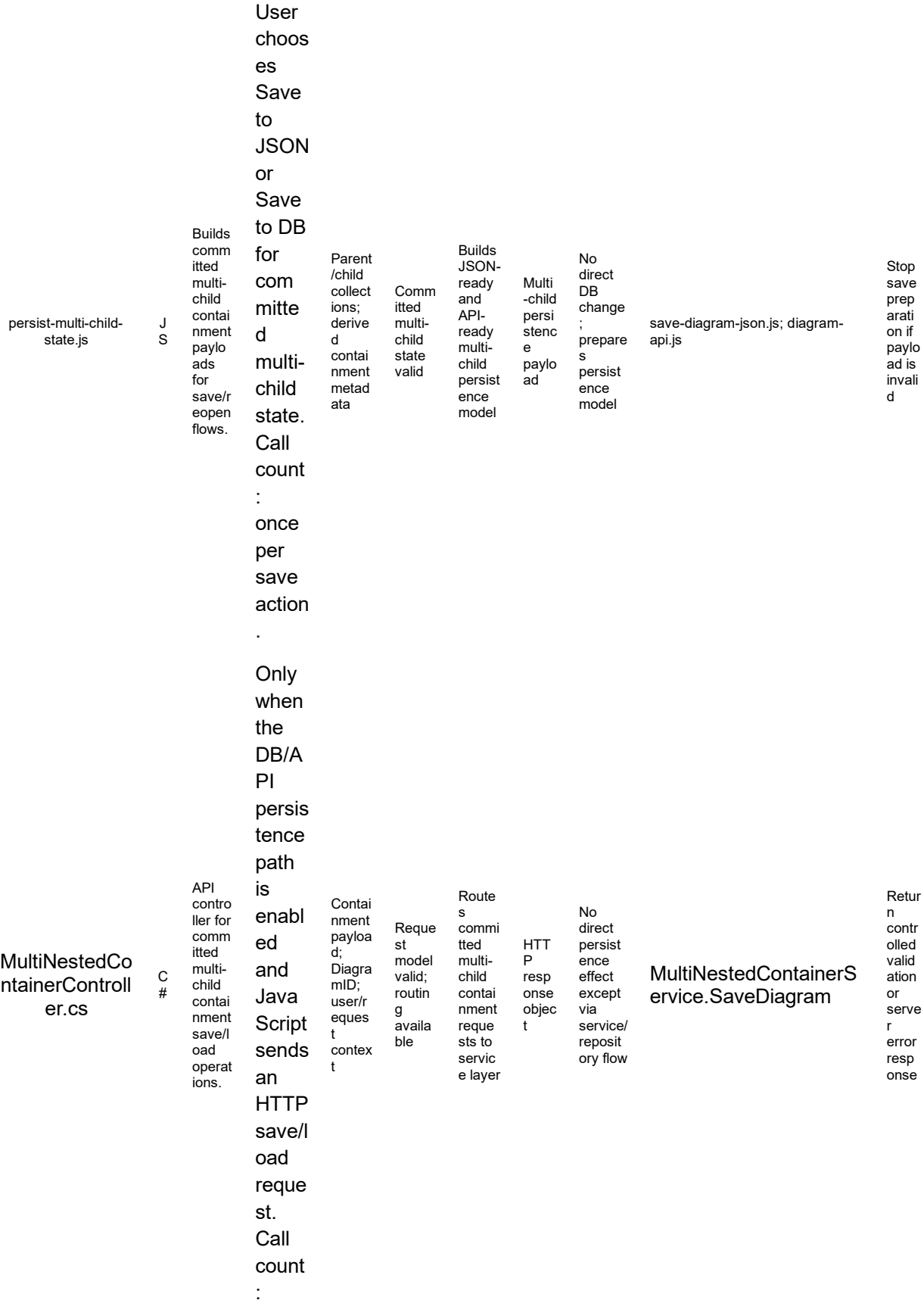
This milestone shall use the same function-contract structure already defined in the document:

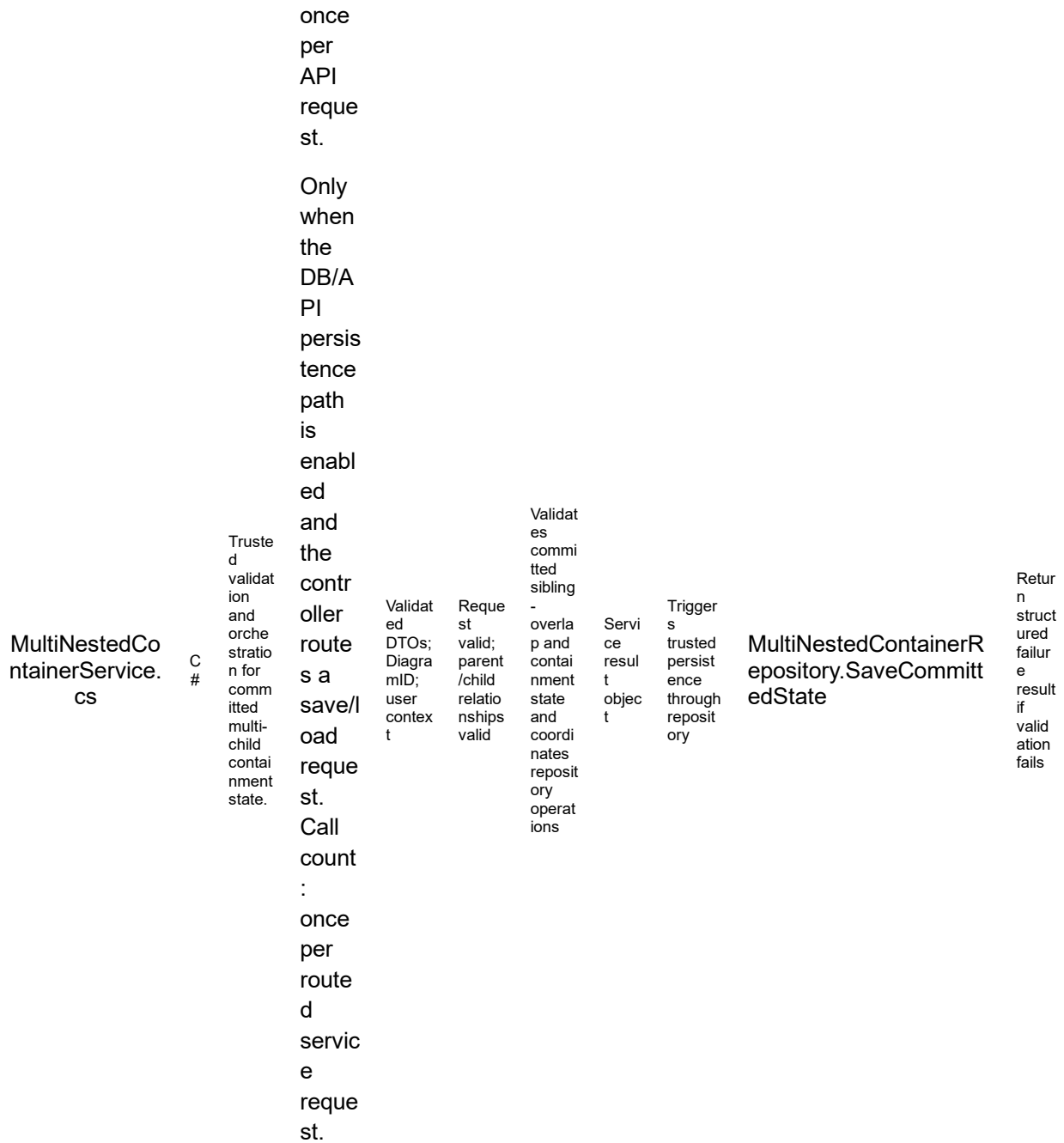
- Trigger
- Input
- Validation / Preconditions
- Processing Responsibility
- Output
- State Change / Persistence Effect
- Next Possible Call(s)
- Error Output / Failure Path

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
multi-child-containment.css	CSS	Styles multi-child valid/invalid placement cues and parent feedback.	Page load, and whenever the owning component	Containment visual-state flags	Canvas containment layer exists	Applies visual feedback for multi-child containment and overlap results	Consistent multi-child containment visuals	No persistence effect	render-canvas.js; validate-sibling-overlap.js	Visuals may fall back to default styling

onent visual state is redra wn. Call count : styles heet parse d once per page load; brows er reapp lies styles auto matic ally.										
validate-sibling-overlap.js	J S	Vali date s the acti ve chil d agai nst eac h sibli ng geo metr y insi de the sam e imm	Child insert , proce ssed move updat e, or proce ssed resize updat e in a paren t with multip le childr en. Call count : once	Acti ve child geo metr y; sibli ng colle ctio n in the sam e imm edia te pare nt; prot ecti on- pad	Siblin g collect ion availla ble; bound aries valid	Check s sibling overla p and return s conflic t/valid result	Sibli ng valid ation resul t	Tempor ary or commit ted state decisio n depend ing on caller	recalc-multi-child-padding.js; renderCanvas	Rest ore previ ous valid child state on failur e







The table below follows the Milestone 2 file-definition style and includes CSS, JavaScript, and C# files relevant to protection padding calculation and containment behavior for multiple children inside one parent rectangle container. Validation rows now state the practical call count per processed update.

The table below lists the main database tables used or extended by Milestone 10.

Database table	Type	Purpose
Containers	Core source	Stores parent container geometry, nested container

		geometry, and ParentContainerID values.
Devices	Core source	Stores contained device geometry, host type, and ParentContainerID values.
DiagramPayloads	Optional snapshot	Stores committed full-diagram snapshots including multi-child containment state.
ContainmentChangeLog	Optional audit	Stores committed multi-child containment history if auditability is needed later.

The table below lists the main database tables used or extended by Milestone 10.

The table below includes the main Milestone 10 working variables.

Variable name	Description	Recommended values
ParentContainerID	Identity of the immediate parent container. NULL means direct ownership by DiagramCanvas root.	Valid ContainerID or NULL
ContainedItemsCollection	Collection of all contained items inside the same immediate parent.	Ordered item list
ActiveContainedItemId	Current contained item being inserted, moved, or resized.	Valid DeviceID or ContainerID
SiblingValidationScope	Current sibling set used for overlap checks.	All siblings except active item
CandidateContainedCenterX	Candidate contained-item center X during multi-child validation.	Numeric world-space value
CandidateContainedCenterY	Candidate contained-item center Y during multi-child validation.	Numeric world-space value
CandidateContainedProtectionBounds	Derived protection boundaries of the active contained item.	Structured boundary object
ParentResizeRequirement	Indicates whether parent expansion is required under approved rules.	True / False

MultiChildValidationResult	Combined containment and sibling-overlap result for the active item.	Valid / Invalid / Clamped
RecalculatedSiblingSet	Collection of children whose derived geometry must be refreshed after commit.	Ordered item list

The table below includes the main Milestone 10 working variables.

Milestone 10 variables

Minimum readiness rule: Milestone 10 should not be considered ready to begin until the team confirms at least this: all children inside one immediate parent can be enumerated in a deterministic order and validated against both the parent and their siblings.

Required confirmations: Milestone 9 single-child containment is already stable; immediate-parent relationships are already persisted; sibling item collections can be enumerated; Rectangle and Circle protection boundaries can be recalculated on demand; and parent resize rules are already approved.

Before Milestone 10 implementation starts, the following prerequisites should be confirmed.

Prerequisites

Mandatory existing JavaScript before starting Milestone 10

Only JavaScript files that must already exist and work correctly before this milestone starts are listed here.

Shared save prerequisite. Before starting Milestone 10, the team shall verify that save-to-json.js and save-to-db.js are already saving all committed variables approved in Milestones 1 to 9 correctly. When Milestone 10 introduces new committed variables, the developer shall extend only these same two files so the current milestone save test checks the Milestone 10 variables, while regression validation confirms that all previous milestone variables are still saved correctly.

JavaScript file that must already exist	Description	Reason this JavaScript must already be ready and working correctly
validate-child-containment.js	Validates one child item against its immediate parent boundaries.	Milestone 10 extends stable single-child containment into multi-child containment.
resize-parent-container.js	Expands the parent container when a child cannot fit under approved rules.	Multi-child containment still depends on the parent-expansion path from Milestone 9.
recalc-multi-child-padding.js	Recalculates protection boundaries for all affected children in one parent.	The multi-child milestone requires consistent recalculation across sibling collections.
render-canvas.js	Renders the updated container and children after each accepted change.	The developer must visually confirm multi-child validation results.

Milestone 10 terminology

Main developer names

The following names are recommended for this milestone:

Parent container

ParentRectangleContainer

Parent container geometry

ParentInnerLeftX

ParentInnerRightX

ParentInnerTopY

ParentInnerBottomY

ParentVisibleLeftX

ParentVisibleRightX

ParentVisibleTopY

ParentVisibleBottomY

Child collections

ParentChildRectangleItems[]

ParentChildCircleItems[]

Child rectangle geometry

ChildRectangleID

ChildCenterX

ChildCenterY

ChildHalfWidth

ChildHalfHeight

ChildProtectionLeftX

ChildProtectionRightX

ChildProtectionTopY

ChildProtectionBottomY

Child circle geometry

ChildCircleID

ChildCircleCenterX

ChildCircleCenterY

ChildCircleRadius

ChildProtectionPaddedRadius

ChildProtectionLeftX

ChildProtectionRightX

ChildProtectionTopY

ChildProtectionBottomY

Sibling-validation collections

SiblingRectangleSet[]

SiblingCircleSet[]

ParentChildItems[]

Reference-center variables

ChildParentOffsetX

ChildParentOffsetY

Containment state

IsChildContainmentValid

IsSiblingOverlapValid

IsStrictNoTouchEnabled

NeedsProtectionPaddingRecalculation

Recommended implementation rule

Each child item shall have exactly one immediate parent rectangle container. Every validation in this milestone shall be done relative to:

that immediate parent container

the sibling child items that share the same immediate parent container

This follows the document's immediate-parent rule and sibling-validation rule for contained Protection Padding behavior.

Important milestone rules

Rule 1 — Protection Padding is the containment and sibling-overlap authority

For this milestone, validation inside one parent rectangle container shall be determined by:

the child item's Protection Padding boundaries

the parent rectangle container's inner usable boundaries

the sibling child items' Protection Padding boundaries

Hover Padding shall not be used for containment or sibling overlap.

Rule 2 — Every candidate child move or resize must satisfy two conditions

A candidate child item state is valid only if both are true:

1. the child remains fully inside the parent rectangle container under strict no-touch rules
2. the child does not overlap any sibling child item inside the same immediate parent container

Rule 3 — All containment and sibling-overlap checks shall use boundary equations, not center-only equations

Containment and sibling overlap shall not be decided by center-only tests.

They shall be decided by comparing:

child Protection Padding boundaries

sibling Protection Padding boundaries

parent inner usable boundaries

Rule 4 — Strict no-touch shall remain enabled by default

Each child must not:

get out of the parent

overlap the parent border line

touch the parent border line

Likewise, if strict sibling no-touch is enabled, one child item shall not touch another child item's Protection Padding boundary either.

A small epsilon margin may be used for implementation safety.

Rule 5 — Parent move shall preserve each child-parent reference-center offset

When the parent rectangle container moves, every child item inside it shall move with it and preserve its own stored reference-center offset.

For each child item:

$\text{ChildParentOffsetX} = \text{ChildCenterX} - \text{ParentCenterX}$

$\text{ChildParentOffsetY} = \text{ChildCenterY} - \text{ParentCenterY}$

Then after parent movement:

$\text{NewChildCenterX} = \text{NewParentCenterX} + \text{ChildParentOffsetX}$

$\text{NewChildCenterY} = \text{NewParentCenterY} + \text{ChildParentOffsetY}$

This shall apply to all children of the same parent.

Rule 6 — Parent resize does not automatically scale children unless explicitly added later

For this milestone, resizing the parent rectangle container shall change the available containment area, but it shall not automatically scale any child item.

Instead:

all current children are revalidated against the new parent

if one or more children become invalid under the new parent size, the parent resize shall be rejected or clamped according to the chosen policy

This is the cleanest version-1 behavior for multiple children.

Core Protection Padding equations used in this milestone

Parent rectangle inner usable boundary

For this milestone, the parent rectangle container uses:

ParentInnerLeftX

ParentInnerRightX

ParentInnerTopY

ParentInnerBottomY

In version 1, these may equal the parent visible boundaries.

Child rectangle Protection Padding boundaries

For a child rectangle:

$\text{ChildProtectionLeftX} = \text{ChildCenterX} - \text{ChildHalfWidth} - \text{ChildProtectionPaddingX}$

$\text{ChildProtectionRightX} = \text{ChildCenterX} + \text{ChildHalfWidth} + \text{ChildProtectionPaddingX}$

$\text{ChildProtectionTopY} = \text{ChildCenterY} + \text{ChildHalfHeight} + \text{ChildProtectionPaddingY}$

$\text{ChildProtectionBottomY} = \text{ChildCenterY} - \text{ChildHalfHeight} - \text{ChildProtectionPaddingY}$

Parent containment rule:

$\text{ChildProtectionLeftX} > \text{ParentInnerLeftX}$

$\text{ChildProtectionRightX} < \text{ParentInnerRightX}$

$\text{ChildProtectionTopY} < \text{ParentInnerTopY}$

$\text{ChildProtectionBottomY} > \text{ParentInnerBottomY}$

Sibling non-overlap rule between two child rectangles A and B:

they do not overlap if at least one of the following is true:

$\text{A.RightProtectionX} \leq \text{B.LeftProtectionX}$

$\text{B.RightProtectionX} \leq \text{A.LeftProtectionX}$

$A.TopProtectionY \leq B.BottomProtectionY$

$B.TopProtectionY \leq A.BottomProtectionY$

If strict no-touch between siblings must be enforced, replace $\leq$ with $<$.

This is the same rectangle Protection Padding non-overlap logic already defined in the file, now applied to sibling children inside the same parent.

Child circle Protection Padding boundaries

For a child circle:

$ChildProtectionPaddedRadius = ChildCircleRadius + ChildCircleProtectionPadding$

$ChildProtectionLeftX = ChildCircleCenterX - ChildProtectionPaddedRadius$

$ChildProtectionRightX = ChildCircleCenterX + ChildProtectionPaddedRadius$

$ChildProtectionTopY = ChildCircleCenterY + ChildProtectionPaddedRadius$

$ChildProtectionBottomY = ChildCircleCenterY - ChildProtectionPaddedRadius$

Parent containment rule:

$ChildProtectionLeftX > ParentInnerLeftX$

$ChildProtectionRightX < ParentInnerRightX$

$ChildProtectionTopY < ParentInnerTopY$

$ChildProtectionBottomY > ParentInnerBottomY$

Sibling non-overlap rule between two child circles A and B:

let:

$dx = B.CenterX - A.CenterX$

$dy = B.CenterY - A.CenterY$

$d^2 = dx^2 + dy^2$

Then the circles do not overlap if:

$d^2 \geq (A.ProtectionPaddedRadius + B.ProtectionPaddedRadius)^2$

If strict sibling no-touch must be enforced, use:

$d^2 > (A.ProtectionPaddedRadius + B.ProtectionPaddedRadius)^2$

This is the same circle Protection Padding non-overlap logic already defined in the file, now applied to sibling children inside the same parent.

Child rectangle clamped center equations

For a candidate child rectangle center:

```

ClampedChildCenterX = clamp(
TargetChildCenterX,
ParentInnerLeftX + ChildHalfWidth + ChildProtectionPaddingX + ε,
ParentInnerRightX - ChildHalfWidth - ChildProtectionPaddingX - ε
)

ClampedChildCenterY = clamp(
TargetChildCenterY,
ParentInnerBottomY + ChildHalfHeight + ChildProtectionPaddingY + ε,
ParentInnerTopY - ChildHalfHeight - ChildProtectionPaddingY - ε
)

```

After clamping against the parent, the candidate state must still be validated against siblings.

Child circle clamped center equations

```

ClampedChildCircleCenterX = clamp(
TargetChildCircleCenterX,
ParentInnerLeftX + ChildProtectionPaddedRadius + ε,
ParentInnerRightX - ChildProtectionPaddedRadius - ε
)

ClampedChildCircleCenterY = clamp(
TargetChildCircleCenterY,
ParentInnerBottomY + ChildProtectionPaddedRadius + ε,
ParentInnerTopY - ChildProtectionPaddedRadius - ε
)

```

After clamping against the parent, the candidate state must still be validated against siblings.

Mathematical Calculation Rules for Milestone 10

Milestone 10 extends Milestone 9 from one child to multiple children inside the same immediate parent container. Each moved or resized child must be validated against its immediate parent and all sibling geometries in that same parent.

Primary JavaScript files for this calculation scope: validate-sibling-non-overlap.js, validate-child-containment.js, rectangle-overlap-on-move.js, circle-overlap-on-move.js, rectangle-circle-overlap-on-move.js, move-update-scheduler.js.

Validation set calculation

If a parent contains N child items including the moved item, the moved item is validated against N - 1 siblings plus one immediate-parent containment check.

ProcessedMoveUpdatesApprox = DragDurationSeconds * MaxMoveValidationRate.

SiblingOverlapChecksApprox = ProcessedMoveUpdatesApprox * (N - 1).

ParentContainmentChecksApprox = ProcessedMoveUpdatesApprox.

Sibling non-overlap calculation

Rectangle-rectangle, circle-circle, and rectangle-circle checks shall use Protection Padding geometry only. Hover Padding shall not be used for collision or containment validation.

If the candidate item overlaps or touches a sibling under the strict no-touch rule, the candidate movement shall be rejected and the item shall return to its previous valid position.

Parent containment calculation

After sibling validation passes, validate the candidate against the immediate parent inner boundary. If containment alone fails, clamping may be applied according to the containment rules. If sibling conflict remains, restore previous valid position.

Milestone 10 phases

Milestone 10 shall be implemented in 8 phases.

Phase 10.1 — Multiple child rectangles inside one parent rectangle container: free movement under Protection Padding

For more information, please refer to section Protection Padding, Overlap, Containment, and Persistence Calculations ([place in this document](#)).

Goal

Allow multiple child rectangles to move freely inside one parent rectangle container while ensuring that each child stays inside the parent and does not overlap any sibling child rectangle.

Scope

This phase is about:

one parent rectangle container

more than one child rectangle inside that same parent

free child movement inside the parent

Protection Padding calculation during movement

clamped parent containment validation

sibling non-overlap validation

Required test behaviors

Each child rectangle shall be movable freely inside the parent rectangle container.

Expected result:

the selected child rectangle moves smoothly

the selected child rectangle never gets out of the parent

the selected child rectangle never touches the parent border line

the selected child rectangle never overlaps any sibling child rectangle

the selected child rectangle's Protection Padding boundaries remain valid during movement

Variables to define

At minimum:

ParentInnerLeftX

ParentInnerRightX

ParentInnerTopY

ParentInnerBottomY

ActiveChildRectangleID

SiblingRectangleSet[]

ChildHalfWidth

ChildHalfHeight

ChildProtectionPaddingX

ChildProtectionPaddingY

IsChildContainmentValid

IsSiblingOverlapValid

Required event-to-function routing

Child-rectangle move flow

left click and drag one child rectangle

→ screenToWorld

→ computeCandidateChildRectangleCenter

→ recalculateChildRectangleProtectionPadding

→ clampChildRectangleCenterWithinParent

→ validateChildRectangleAgainstSiblingRectangles

→ applyValidatedChildRectangleCenter

→ renderCanvas

State rule

During this phase:

only the selected child rectangle moves

the parent rectangle container remains fixed

all containment and sibling-overlap checks happen in world space

no persistence is required yet

Success criteria

This phase is successful if:

one child rectangle can move freely inside the parent rectangle container

it never leaves the parent

it never touches the parent border line

it never overlaps any sibling child rectangle

Function interaction contracts

Function Name: recalculateChildRectangleProtectionPadding

Layer: JavaScript

File: child-rectangle-protection-padding.js

Purpose: Recalculate the active child rectangle's Protection Padding boundaries from its canonical geometry.

Trigger:

active child rectangle movement begins or candidate child rectangle center changes

Input:

ChildRectangleID

ChildCenterX

ChildCenterY

ChildHalfWidth

ChildHalfHeight

ChildProtectionPaddingX

ChildProtectionPaddingY

Validation / Preconditions:

child rectangle exists

child rectangle geometry is valid

padding values are numeric and non-negative

Processing Responsibility:

calculate ChildProtectionLeftX

ChildProtectionRightX

ChildProtectionTopY

ChildProtectionBottomY

prepare the padded boundaries for containment and sibling validation

Output:

updated active child rectangle Protection Padding boundaries

State Change / Persistence Effect:

updates temporary in-memory derived geometry only

no persistence effect

Next Possible Call(s):

clampChildRectangleCenterWithinParent()

validateChildRectangleAgainstSiblingRectangles()

Error Output / Failure Path:

return invalid recalculation result and preserve previous valid child rectangle state if geometry is invalid

Function Name: validateChildRectangleAgainstSiblingRectangles

Layer: JavaScript

File: sibling-rectangle-overlap-validation.js

Purpose: Validate the active child rectangle against all sibling child rectangles inside the same parent rectangle container.

Trigger:

candidate child rectangle center or geometry generated during child movement

Input:

ActiveChildRectangleID

active child rectangle Protection Padding boundaries

SiblingRectangleSet[]

strict sibling no-touch rule

Validation / Preconditions:

active child rectangle exists

sibling rectangle set exists

all sibling padded boundaries are valid

Processing Responsibility:

compare the active child rectangle's Protection Padding boundaries against each sibling child rectangle's Protection Padding boundaries

determine whether the active child rectangle overlaps or touches any sibling according to the current sibling-spacing rule

Output:

valid / invalid sibling-overlap validation result

State Change / Persistence Effect:

no persistence effect

no direct permanent state change

Next Possible Call(s):

applyValidatedChildRectangleCenter()

restorePreviousValidChildRectangleGeometry()

Error Output / Failure Path:

reject the candidate move and preserve previous valid child rectangle state if sibling validation fails

Phase 10.2 — Multiple child rectangles inside one parent rectangle container: child resize under Protection Padding

For more information, please refer to section [Protection Padding, Overlap, Containment, and Persistence Calculations](#) ([place in this document](#)).

Goal

Allow one child rectangle to be resized while confirming that it remains inside the parent and does not overlap any sibling child rectangle.

Scope

This phase is about:

active child rectangle resize

Protection Padding recalculation after resize

parent containment validation during resize

sibling non-overlap validation during resize

resize rejection or clamping if invalid

Required test behaviors

Expected result:

the active child rectangle resizes correctly

the resized child rectangle never gets out of the parent

the resized child rectangle never touches the parent border line

the resized child rectangle never overlaps any sibling child rectangle

Variables to define

At minimum:

ActiveChildRectangleID

CandidateChildHalfWidth

CandidateChildHalfHeight

CandidateChildCenterX

CandidateChildCenterY

MinChildHalfWidth

MinChildHalfHeight

SiblingRectangleSet[]

Required event-to-function routing

Child-rectangle resize flow

left click and drag active child rectangle resize control point

- updateChildRectangleResizeCandidate
- recalculateChildRectangleProtectionPadding
- validateChildRectangleResizeWithinParent
- validateChildRectangleResizeAgainstSiblingRectangles
- applyValidatedChildRectangleResize
- renderCanvas

State rule

During this phase:

the active child rectangle changes size

the parent remains fixed

containment and sibling-overlap are both enforced

Success criteria

This phase is successful if:

the active child rectangle can resize

the resized child rectangle never gets out of the parent

the resized child rectangle never overlaps any sibling child rectangle

Function interaction contracts

Function Name: validateChildRectangleResizeWithinParent

Layer: JavaScript

File: child-rectangle-resize-validation.js

Purpose: Validate a candidate child rectangle resize against parent rectangle containment rules.

Trigger:

candidate child rectangle resize geometry generated during resize

Input:

ActiveChildRectangleID
CandidateChildCenterX
CandidateChildCenterY
CandidateChildHalfWidth
CandidateChildHalfHeight
ChildProtectionPaddingX
ChildProtectionPaddingY
ParentInnerLeftX
ParentInnerRightX
ParentInnerTopY
ParentInnerBottomY

Validation / Preconditions:

- parent container exists
- active child rectangle exists
- candidate child geometry is valid
- padding values are valid

Processing Responsibility:

- derive candidate child rectangle Protection Padding boundaries
- compare them against the parent inner boundaries
- return valid or invalid parent-containment resize result

Output:

- valid / invalid parent-containment resize validation result

State Change / Persistence Effect:

- no persistence effect

Next Possible Call(s):

- `validateChildRectangleResizeAgainstSiblingRectangles()`
- `applyValidatedChildRectangleResize()`

Error Output / Failure Path:

- reject the resize and preserve previous valid child rectangle geometry if parent containment fails

Function Name: `validateChildRectangleResizeAgainstSiblingRectangles`

Layer: JavaScript

File: `sibling-rectangle-overlap-validation.js`

Purpose: Validate a candidate child rectangle resize against all sibling child rectangles inside the same parent rectangle container.

Trigger:

- candidate child rectangle resize geometry generated during resize

Input:

- `ActiveChildRectangleID`
- candidate child rectangle Protection Padding boundaries
- `SiblingRectangleSet[]`
- strict sibling no-touch rule

Validation / Preconditions:

- active child rectangle exists
- sibling rectangle set exists
- candidate padded geometry is valid

Processing Responsibility:

- compare the candidate child rectangle Protection Padding boundaries against each sibling child rectangle's Protection Padding boundaries
- return valid or invalid sibling-overlap resize result

Output:

- valid / invalid sibling-overlap resize validation result

State Change / Persistence Effect:
no persistence effect

Next Possible Call(s):
applyValidatedChildRectangleResize()
restorePreviousValidChildRectangleGeometry()

Error Output / Failure Path:
reject the resize and preserve previous valid child rectangle geometry if sibling validation fails

Phase 10.3 — Parent rectangle resize and parent-child reference-center preservation for multiple child rectangles

For more information, please refer to section Parent-container expansion calculations ([place in this document](#)) and section Protection Padding, Overlap, Containment, and Persistence Calculations ([place in this document](#)).

Goal

Resize the parent rectangle container and then move the parent rectangle while preserving each child rectangle's reference-center offset.

Scope

This phase is about:

- parent rectangle container resize
- revalidation of all child rectangles against the resized parent
- revalidation of all child rectangles against each other
- parent move with all child rectangles following
- preserving every child-parent offset

Required test behaviors

Expected result:

- resizing the parent changes the available containment area
- all child rectangles remain valid inside the resized parent or the parent resize is rejected/clamped
- when the parent moves, all child rectangles move with it
- each child rectangle preserves the same reference-center offset relative to the parent

Variables to define

At minimum:

ParentResizeCandidate

ParentChildRectangleItems[]

ChildParentOffsetX

ChildParentOffsetY

ParentMoveDeltaX

ParentMoveDeltaY

Required event-to-function routing

Parent resize flow

left click and drag parent rectangle resize control point

→ updateParentRectangleResizeCandidate

→ validateParentResizeAgainstAllChildRectangles

→ applyValidatedParentResize

→ renderCanvas

Parent move flow

left click and drag parent rectangle container

→ computeParentMoveDelta

→ moveAllChildRectanglesWithParentReference

→ applyValidatedParentMove

→ renderCanvas

State rule

During this phase:

the parent may resize or move

all child rectangles remain associated with the same parent

when the parent moves, all child rectangles move with it by translation and preserve their own reference offsets

Success criteria

This phase is successful if:

resizing the parent is validated against all current child rectangles

moving the parent also moves all child rectangles

each child keeps the same reference-center offset relative to the parent

Function interaction contracts

Function Name: validateParentResizeAgainstAllChildRectangles

Layer: JavaScript

File: parent-rectangle-resize-validation.js

Purpose: Validate a candidate parent rectangle resize against all child rectangles currently inside the parent container.

Trigger:

candidate parent rectangle resize generated during parent resize

Input:

CandidateParentInnerLeftX

CandidateParentInnerRightX

CandidateParentInnerTopY

CandidateParentInnerBottomY

ParentChildRectangleItems[]

current child rectangle Protection Padding boundaries

Validation / Preconditions:

parent container exists

child rectangle collection exists

candidate parent geometry is valid

all child padded boundaries are valid

Processing Responsibility:

for each child rectangle

compare its current Protection Padding boundaries against the candidate parent inner boundaries

return valid only if all child rectangles remain valid under the candidate parent size

Output:

valid / invalid parent resize validation result for all child rectangles

State Change / Persistence Effect:

no persistence effect

Next Possible Call(s):

applyValidatedParentResize()

restorePreviousValidParentGeometry()

Error Output / Failure Path:

reject the parent resize if one or more child rectangles would become invalid

Function Name: moveAllChildRectanglesWithParentReference

Layer: JavaScript

File: parent-child-reference-center.js

Purpose: Move all child rectangles with the parent rectangle while preserving each child's stored reference-center offset.

Trigger:

validated parent move delta generated during parent drag

Input:

NewParentCenterX

NewParentCenterY

ParentChildRectangleItems[]

stored child-parent offsets for each child rectangle

Validation / Preconditions:

parent exists

child rectangle collection exists
stored offsets exist for each child rectangle

Processing Responsibility:
for each child rectangle
compute the new child center from the new parent center and that child's stored offset
recalculate the child rectangle derived geometry
preserve the same offset relationship for all children

Output:
updated child rectangle centers after parent move

State Change / Persistence Effect:
updates temporary in-memory parent and child positions only
no persistence effect

Next Possible Call(s):
recalculateChildRectangleProtectionPadding()
renderCanvas()

Error Output / Failure Path:
preserve previous valid parent and child positions if one or more reference-offset computations fail

Phase 10.4 — Multiple child circles inside one parent rectangle container: free movement under Protection Padding

For more information, please refer to section [Protection Padding, Overlap, Containment, and Persistence Calculations](#) ([place in this document](#)).

Goal

Allow multiple child circles to move freely inside one parent rectangle container while ensuring that each child stays inside the parent and does not overlap any sibling child circle.

Scope

This phase is about:

one parent rectangle container
more than one child circle inside that same parent
free child-circle movement inside the parent
Protection Padding calculation during movement
clamped parent containment validation
sibling non-overlap validation

Required test behaviors

Expected result:

the active child circle moves smoothly

the active child circle never gets out of the parent

the active child circle never touches the parent border line

the active child circle never overlaps any sibling child circle

Variables to define

At minimum:

ActiveChildCircleID

SiblingCircleSet[]

ChildCircleRadius

ChildCircleProtectionPadding

ChildProtectionPaddedRadius

IsChildContainmentValid

IsSiblingOverlapValid

Required event-to-function routing

Child-circle move flow

left click and drag active child circle

→ screenToWorld

→ computeCandidateChildCircleCenter

→ recalculateChildCircleProtectionPadding

→ clampChildCircleCenterWithinParent

→ validateChildCircleAgainstSiblingCircles

→ applyValidatedChildCircleCenter

→ renderCanvas

State rule

During this phase:

only the active child circle moves

the parent rectangle container remains fixed

containment and sibling-overlap both use world-space Protection Padding calculations

Success criteria

This phase is successful if:

the active child circle can move freely inside the parent rectangle container

it never leaves the parent

it never overlaps any sibling child circle

Function interaction contracts

Function Name: recalculateChildCircleProtectionPadding

Layer: JavaScript

File: child-circle-protection-padding.js

Purpose: Recalculate the active child circle's Protection Padding values and boundaries.

Trigger:

active child circle movement begins or candidate child circle center changes

Input:

ChildCircleID

ChildCircleCenterX

ChildCircleCenterY

ChildCircleRadius

ChildCircleProtectionPadding

Validation / Preconditions:

child circle exists

circle radius is valid

protection padding value is numeric and non-negative

Processing Responsibility:

calculate ChildProtectionPaddedRadius

ChildProtectionLeftX

ChildProtectionRightX

ChildProtectionTopY

ChildProtectionBottomY

prepare the padded boundaries for containment and sibling validation

Output:

updated child circle Protection Padding values and boundaries

State Change / Persistence Effect:

updates temporary in-memory derived geometry only

no persistence effect

Next Possible Call(s):

clampChildCircleCenterWithinParent()

validateChildCircleAgainstSiblingCircles()

Error Output / Failure Path:

return invalid recalculation result and preserve previous valid child circle state if geometry is invalid

Function Name: validateChildCircleAgainstSiblingCircles

Layer: JavaScript

File: sibling-circle-overlap-validation.js

Purpose: Validate the active child circle against all sibling child circles inside the same parent rectangle container.

Trigger:

candidate child circle center or geometry generated during child movement

Input:

ActiveChildCircleID

candidate child circle center

candidate child protection-padded radius

SiblingCircleSet[]

strict sibling no-touch rule

Validation / Preconditions:

active child circle exists

sibling circle set exists

all sibling padded radii are valid

Processing Responsibility:

for each sibling child circle

compute center distance relation using protection-padded radii

determine whether the active child circle overlaps or touches any sibling according to the current sibling-spacing rule

Output:

valid / invalid sibling-overlap validation result

State Change / Persistence Effect:

no persistence effect

Next Possible Call(s):

applyValidatedChildCircleCenter()

restorePreviousValidChildCircleGeometry()

Error Output / Failure Path:

reject the candidate move and preserve previous valid child circle state if sibling validation fails

Phase 10.5 — Multiple child circles inside one parent rectangle container: child resize under Protection Padding

For more information, please refer to section Protection Padding, Overlap, Containment, and Persistence Calculations ([place in this document](#)).

Goal

Allow one child circle to be resized while confirming that it remains inside the parent and does not overlap any sibling child circle.

Scope

This phase is about:

active child circle resize

Protection Padding recalculation after resize

parent containment validation during resize

sibling non-overlap validation during resize

resize rejection or clamping if invalid

Required test behaviors

Expected result:

the active child circle resizes correctly

the resized child circle never gets out of the parent

the resized child circle never touches the parent border line

the resized child circle never overlaps any sibling child circle

Variables to define

At minimum:

ActiveChildCircleID

CandidateChildCircleRadius

ValidatedChildCircleRadius

MinChildCircleRadius

SiblingCircleSet[]

Required event-to-function routing

Child-circle resize flow

left click and drag active child circle resize control point

→ updateChildCircleResizeCandidate

→ recalculateChildCircleProtectionPadding

→ validateChildCircleResizeWithinParent

→ validateChildCircleResizeAgainstSiblingCircles

→ applyValidatedChildCircleResize

→ renderCanvas

State rule

During this phase:

the active child circle changes size

the parent remains fixed

containment and sibling-overlap are both enforced

Success criteria

This phase is successful if:

the active child circle can resize

the resized child circle never gets out of the parent

the resized child circle never overlaps any sibling child circle

Function interaction contracts

Function Name: validateChildCircleResizeWithinParent

Layer: JavaScript

File: child-circle-resize-validation.js

Purpose: Validate a candidate child circle resize against parent rectangle containment rules.

Trigger:

candidate child circle radius generated during resize

Input:

ActiveChildCircleID

CandidateChildCircleCenterX

CandidateChildCircleCenterY

CandidateChildCircleRadius

ChildCircleProtectionPadding

ParentInnerLeftX

ParentInnerRightX

ParentInnerTopY

ParentInnerBottomY

Validation / Preconditions:

parent container exists

active child circle exists

candidate circle geometry is valid

protection padding value is valid

Processing Responsibility:

derive candidate protection-padded radius and padded boundaries

compare them against the parent inner boundaries

return valid or invalid parent-containment resize result

Output:

valid / invalid parent-containment resize validation result

State Change / Persistence Effect:

no persistence effect

Next Possible Call(s):

validateChildCircleResizeAgainstSiblingCircles()

applyValidatedChildCircleResize()

Error Output / Failure Path:

reject the resize and preserve previous valid child circle geometry if parent containment fails

Function Name: validateChildCircleResizeAgainstSiblingCircles

Layer: JavaScript

File: sibling-circle-overlap-validation.js

Purpose: Validate a candidate child circle resize against all sibling child circles inside the same parent rectangle container.

Trigger:

candidate child circle radius generated during resize

Input:

ActiveChildCircleID

candidate child circle center

candidate child protection-padded radius

SiblingCircleSet[]

strict sibling no-touch rule

Validation / Preconditions:

active child circle exists

sibling circle set exists

candidate padded radius is valid

Processing Responsibility:

compare the candidate child circle against each sibling child circle using protection-padded radii

return valid or invalid sibling-overlap resize result

Output:

valid / invalid sibling-overlap resize validation result

State Change / Persistence Effect:

no persistence effect

Next Possible Call(s):

applyValidatedChildCircleResize()

restorePreviousValidChildCircleGeometry()

Error Output / Failure Path:

reject the resize and preserve previous valid child circle geometry if sibling validation fails

Phase 10.6 — Parent rectangle resize and parent-child reference-center preservation for multiple child circles

For more information, please refer to section Parent-container expansion calculations ([place in this document](#)) and section Protection Padding, Overlap, Containment, and Persistence Calculations ([place in this document](#)).

Goal

Resize the parent rectangle container and then move the parent rectangle while preserving each child circle's reference-center offset.

Scope

This phase is about:

parent rectangle container resize

revalidation of all child circles against the resized parent

revalidation of all child circles against each other

parent move with all child circles following

preserving every child-parent offset

Required test behaviors

Expected result:

resizing the parent changes the available containment area

all child circles remain valid inside the resized parent or the parent resize is rejected/clamped

when the parent moves, all child circles move with it

each child circle preserves the same reference-center offset relative to the parent

Variables to define

At minimum:

ParentChildCircleItems[]

ChildCircleParentOffsetX

ChildCircleParentOffsetY

ParentMoveDeltaX

ParentMoveDeltaY

Required event-to-function routing

Parent resize flow

left click and drag parent rectangle resize control point

→ updateParentRectangleResizeCandidate

→ validateParentResizeAgainstAllChildCircles

→ applyValidatedParentResize

→ renderCanvas

Parent move flow

left click and drag parent rectangle container

→ computeParentMoveDelta

→ moveAllChildCirclesWithParentReference

→ applyValidatedParentMove

→ renderCanvas

State rule

During this phase:

the parent may resize or move

all child circles remain associated with the same parent

when the parent moves, all child circles move with it by translation and preserve their own reference offsets

Success criteria

This phase is successful if:

resizing the parent is validated against all current child circles

moving the parent also moves all child circles

each child keeps the same reference-center offset relative to the parent

Function interaction contracts

Function Name: validateParentResizeAgainstAllChildCircles

Layer: JavaScript

File: parent-rectangle-resize-validation.js

Purpose: Validate a candidate parent rectangle resize against all child circles currently inside the parent container.

Trigger:

candidate parent rectangle resize generated during parent resize

Input:

CandidateParentInnerLeftX

CandidateParentInnerRightX

CandidateParentInnerTopY

CandidateParentInnerBottomY

ParentChildCircleItems[]

current child circle protection-padded boundaries

Validation / Preconditions:

parent container exists

child circle collection exists

candidate parent geometry is valid

all child padded boundaries are valid

Processing Responsibility:

for each child circle

compare its current protection-padded boundaries against the candidate parent inner boundaries

return valid only if all child circles remain valid under the candidate parent size

Output:

valid / invalid parent resize validation result for all child circles

State Change / Persistence Effect:

no persistence effect

Next Possible Call(s):

applyValidatedParentResize()

restorePreviousValidParentGeometry()

Error Output / Failure Path:

reject the parent resize if one or more child circles would become invalid

Function Name: moveAllChildCirclesWithParentReference

Layer: JavaScript

File: parent-child-reference-center.js

Purpose: Move all child circles with the parent rectangle while preserving each child's stored reference-center offset.

Trigger:

validated parent move delta generated during parent drag

Input:

NewParentCenterX

NewParentCenterY

ParentChildCircleItems[]

stored child-parent offsets for each child circle

Validation / Preconditions:

parent exists

child circle collection exists

stored offsets exist for each child circle

Processing Responsibility:

for each child circle

compute the new child center from the new parent center and that child's stored offset

recalculate the child circle derived geometry

preserve the same offset relationship for all children

Output:

updated child circle centers after parent move

State Change / Persistence Effect:

updates temporary in-memory parent and child positions only

no persistence effect

Next Possible Call(s):

recalculateChildCircleProtectionPadding()

renderCanvas()

Error Output / Failure Path:

preserve previous valid parent and child positions if one or more reference-offset computations fail

Phase 10.7 — Recalculate committed multi-child Protection Padding geometry and persist the containment state to JSON and database

Exact variables to save in this phase

Milestone 10 introduces no new independent permanent persistence field names. The exact save requirement in this phase is: keep saving all previously approved committed variables, now with the accepted multi-child geometry and parent-child relations already reflected in the canonical saved item

records. Multi-child containment is persisted through the saved canonical item geometry and existing ParentContainerID relations, not through temporary validation collections.

Do not save Milestone 10 runtime or validation helpers as milestone-required persisted data: ContainedItemsCollection, ActiveContainedItemId, SiblingValidationScope, CandidateContainedCenterX, CandidateContainedCenterY, CandidateContainedProtectionBounds, ParentResizeRequirement, MultiChildValidationResult, RecalculatedSiblingSet.

Save regression rule for this phase

All approved Milestones 2 through 8 committed variables shall already remain saved correctly in both persistence paths. This phase passes only if save-to-json.js and save-to-db.js keep the prior approved save set intact while persisting the accepted multi-child result through canonical geometry and existing parent relations only.

Save test criteria for this phase

Verify that both shared save files persist the accepted canonical item records and ParentContainerID relations for the multi-child layout, that the Milestone 10 validation helpers listed above are excluded as milestone-required saved data, and that reopen from either source restores the same multi-child containment result.

For more information, please refer to section Protection Padding, Overlap, Containment, and Persistence Calculations ([place in this document](#)).

Goal

Recalculate all committed multi-child Protection Padding geometry for parent-child containment and sibling-overlap validation, then persist the diagram to both JSON and database.

Scope

This phase is about:

recalculating committed child Protection Padding geometry

revalidating parent-child containment

revalidating sibling non-overlap inside the same parent rectangle container

rebuilding the save model

saving the diagram to JSON

saving the diagram to the database

Important recalculation rule

Before save, the system shall recalculate from the committed geometry:

parent inner usable boundaries

all child rectangle Protection Padding boundaries

all child circle protection-padded radii and protection-padded boundaries

all child-parent reference-center offsets if they are part of the committed containment relationship

all sibling-overlap validation state inside each immediate parent container

This follows the document's general rule that derived geometry shall be recalculated from canonical stored values and that Protection Padding is the authoritative safety region for containment and overlap validation.

Required event-to-function routing

Save-commit flow

click Save button

- recalculateAllMultiChildContainmentProtectionPaddingGeometry
- validateCommittedMultiChildContainmentState
- buildDiagramJsonObject
- serializeDiagramToJson
- saveDiagramJsonFile
- buildDiagramSavePayload
- callSaveDiagramApi
- DiagramController.SaveDiagram
- DiagramService.SaveDiagram
- DiagramRepository.SaveDiagram
- handleSaveCommitResult

State rule

During this phase:

recalculation must occur before persistence

JSON save and database save must use the same committed recalculated multi-child containment state

Success criteria

This phase is successful if:

all committed parent-child and sibling Protection Padding geometry is recalculated before save

the saved JSON and database state both represent the same committed multi-child containment logic

reopening from file or database restores the same multi-child containment state

Function interaction contracts

Function Name: recalculateAllMultiChildContainmentProtectionPaddingGeometry

Layer: JavaScript

File: multi-child-containment-protection-padding-recalculation.js

Purpose: Recalculate all parent-child and sibling Protection Padding geometry required for committed multi-child containment validation and persistence.

Trigger:

explicit save flow

diagram load reconstruction

committed geometry update requiring full containment refresh

Input:

parent container collection
child rectangle collection
child circle collection
committed geometry values
parent-child relation data
sibling grouping by immediate parent

Validation / Preconditions:

all referenced parent containers exist
all referenced child items exist
canonical geometry values are present
immediate-parent grouping data exists

Processing Responsibility:

recalculate parent inner usable boundaries
recalculate all child rectangle Protection Padding boundaries
recalculate all child circle protection-padded radii and padded boundaries
group children by immediate parent
prepare committed containment and sibling-overlap geometry for validation

Output:

recalculated multi-child containment geometry collection

State Change / Persistence Effect:

updates in-memory derived containment geometry only
no direct persistence by this function

Next Possible Call(s):

validateCommittedMultiChildContainmentState()
buildDiagramJsonObject()
renderCanvas()

Error Output / Failure Path:

stop save flow if required multi-child containment geometry cannot be recalculated

Function Name: validateCommittedMultiChildContainmentState

Layer: JavaScript

File: multi-child-containment-save-validation.js

Purpose: Verify that the committed multi-child containment state is valid before persistence.

Trigger:

save flow after containment recalculation

Input:

committed parent geometry
committed child rectangle geometry
committed child circle geometry
recalculated Protection Padding boundaries
parent-child reference-center data if used
sibling grouping by immediate parent

Validation / Preconditions:

- recalculation already completed
- required parent and child fields exist
- parent-child relations are valid
- sibling grouping is valid

Processing Responsibility:

- validate parent containment for all child rectangles
- validate parent containment for all child circles
- validate sibling non-overlap for all child rectangles inside the same parent
- validate sibling non-overlap for all child circles inside the same parent
- validate strict no-touch conditions
- validate that saved reference-center data is consistent if present

Output:

- valid / invalid committed multi-child containment result

State Change / Persistence Effect:

- no persistence effect
- no direct permanent state change

Next Possible Call(s):

- buildDiagramJsonObject()
- buildDiagramSavePayload()

Error Output / Failure Path:

- stop save flow and report multi-child containment-validation failure if committed state is invalid

Phase 10.8 — Validate multiple Circle On Container items on one container boundary system

Goal

Extend the multi-child container milestone so one rectangle container can have multiple Circle On Container items on its boundary while preserving strict non-overlap and independent radius rules.

Scope

This phase is about multiple Circle On Container items attached to the same parent container, possibly on the same edge or different edges, together with normal child rectangles, child circles, and nested containers.

Required validation rules

Each Circle On Container must pass CSV relationship validation, active-edge attachment validation, radius-range validation, and ProtectionPaddedRadius overlap validation against every relevant sibling geometry and every other Circle On Container on the same parent container.

If two Circle On Container items approach each other on the same edge or at a corner, the strict no-touch rule shall use their ProtectionPaddedRadius values. The candidate move or resize shall be rejected or clamped according to the selected UI rule.

Container resize shall not automatically resize any Circle On Container. If the resized container would make any attached Circle On Container radius invalid, reject the container resize by default and restore the previous valid container geometry.

Success criteria

This phase succeeds when multiple Circle On Container items can coexist on a container boundary without overlap/touch violations, without automatic radius scaling, and without breaking normal child containment behavior.

Final result after Milestone 10

At the end of this milestone, the developer should also have multiple Circle On Container items on one container boundary with radius-range and Protection Padding validation.

At the end of Milestone 10, the developer should have:

multiple child rectangle containment inside one parent rectangle container

free child rectangle movement inside the parent under Protection Padding rules

child rectangle resize under Protection Padding rules

sibling non-overlap validation between child rectangles inside the same parent

parent rectangle resize validation against all child rectangles

parent move with all child rectangles following and preserving their reference-center offsets

multiple child circle containment inside one parent rectangle container

free child circle movement inside the parent under Protection Padding rules

child circle resize under Protection Padding rules

sibling non-overlap validation between child circles inside the same parent

parent rectangle resize validation against all child circles

parent move with all child circles following and preserving their reference-center offsets

recalculation and persistence of committed multi-child Protection Padding containment state

This milestone makes multi-child parent-container containment and sibling Protection Padding interaction a real part of the diagram system.

Do not introduce nested-container movement here; keep the focus on one immediate parent with multiple direct children.

Implement sibling validation first, then add recursive parent-resize handling, then add persistence.

Keep the single-child rules from Milestone 9 intact and add only sibling overlap plus collection-level recalculation.

Recommended implementation order

Recommended coding files structure additions for Milestone 10

JavaScript

Recommended new modules:

sibling-rectangle-overlap-validation.js

sibling-circle-overlap-validation.js

multi-child-containment-protection-padding-recalculation.js

multi-child-containment-save-validation.js

parent-child-grouping.js

Optional C# structure

MultiChildContainmentDto.cs

SiblingGroupDto.cs

MultiChildProtectionPaddingStateDto.cs

Do not move live sibling-overlap validation, clamped movement, or parent-child reference updates into C#.

Best folder layout additions for Milestone 10

project/

├─ js/

| └─ sibling-rectangle-overlap-validation.js

| └─ sibling-circle-overlap-validation.js

| └─ multi-child-containment-protection-padding-recalculation.js

| └─ multi-child-containment-save-validation.js

| └─ parent-child-grouping.js

├─ csharp/

| └─ MultiChildContainmentDto.cs

| └─ SiblingGroupDto.cs

| └─ MultiChildProtectionPaddingStateDto.cs

More ideas you can define in Milestone 10

You may either include these now or mark them as future-ready additions:

1. Mixed children inside the same parent rectangle container, such as rectangles and circles together
2. Child insertion into the parent with automatic nearest valid placement

3. Stable sibling ordering rules inside the same parent
4. Parent resize strategies that try to preserve all current children automatically
5. Visual debug overlay for sibling Protection Padding boundaries
6. Automatic spacing or packing inside the parent container
7. Nested nested rectangle containers with multiple siblings at each nesting level

The strongest design recommendation here is:

Use the same **parent inner usable boundaries** and **child Protection Padding boundaries** from Milestone 9, but add a second validation layer for **sibling non-overlap inside the same immediate parent container**. That gives you the cleanest multi-child extension without changing the document's original containment model.

Milestone 11 — Moving a Rectangle Container with Multiple Levels of Nested Containers and Multiple Geometric Children

Overview

Milestone 11 defines the movement behavior of a rectangle container that contains:

multiple levels of nested rectangle containers

multiple child rectangles

multiple child circles

This milestone focuses on **hierarchical container movement**.

The central rule of this milestone is:

when a rectangle container moves, its full descendant subtree shall move with it

That means:

the moved container changes its world position

all direct contained items move with it

all nested containers move with it

all geometric descendants inside those nested containers also move with it

the relative reference-center offset between each child and its immediate parent shall remain preserved

This milestone uses the document's existing model that:

each child item has exactly one immediate parent container

multiple nesting levels are handled by applying the same immediate-parent rules recursively

if containment is enforced correctly between every child and its immediate parent, then the full multi-level containment hierarchy remains valid

all geometry and containment calculations shall happen in world space

Protection Padding is the mathematical safety region used for overlap avoidance, spacing rules, and containment validation.

For this milestone, all containers shall be treated as:

Rectangle Containers only

not circle containers

A core design rule for this milestone is:

JavaScript is responsible for live hierarchical movement, subtree translation, parent-child reference preservation, containment validation, sibling-overlap validation, Protection Padding recalculation, and immediate rendering

C# is responsible only for trusted validation, persistence, API handling, and database communication when save/load is required

This milestone shall use the same function-contract structure already defined in the document:

Trigger

Input

Validation / Preconditions

Processing Responsibility

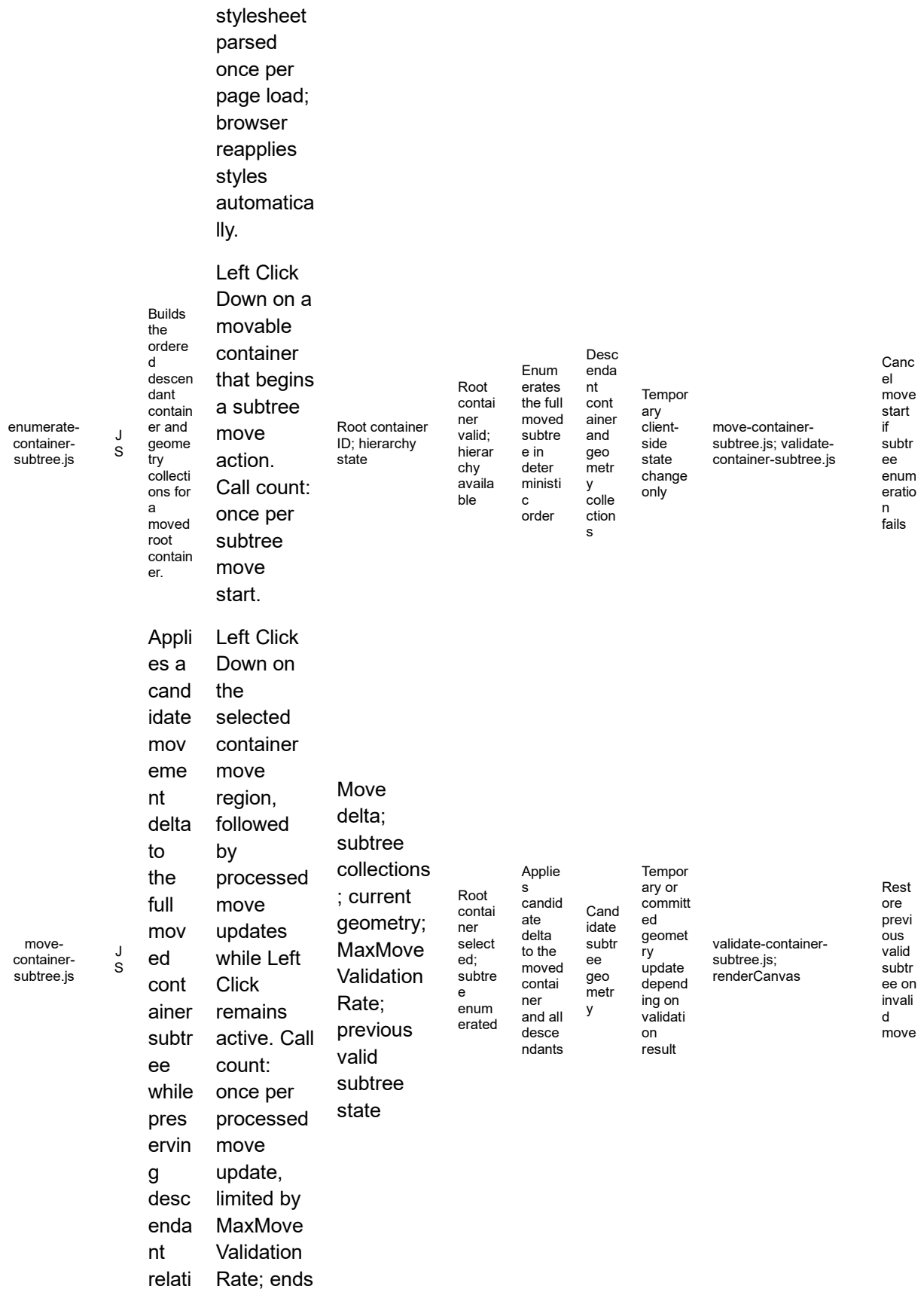
Output

State Change / Persistence Effect

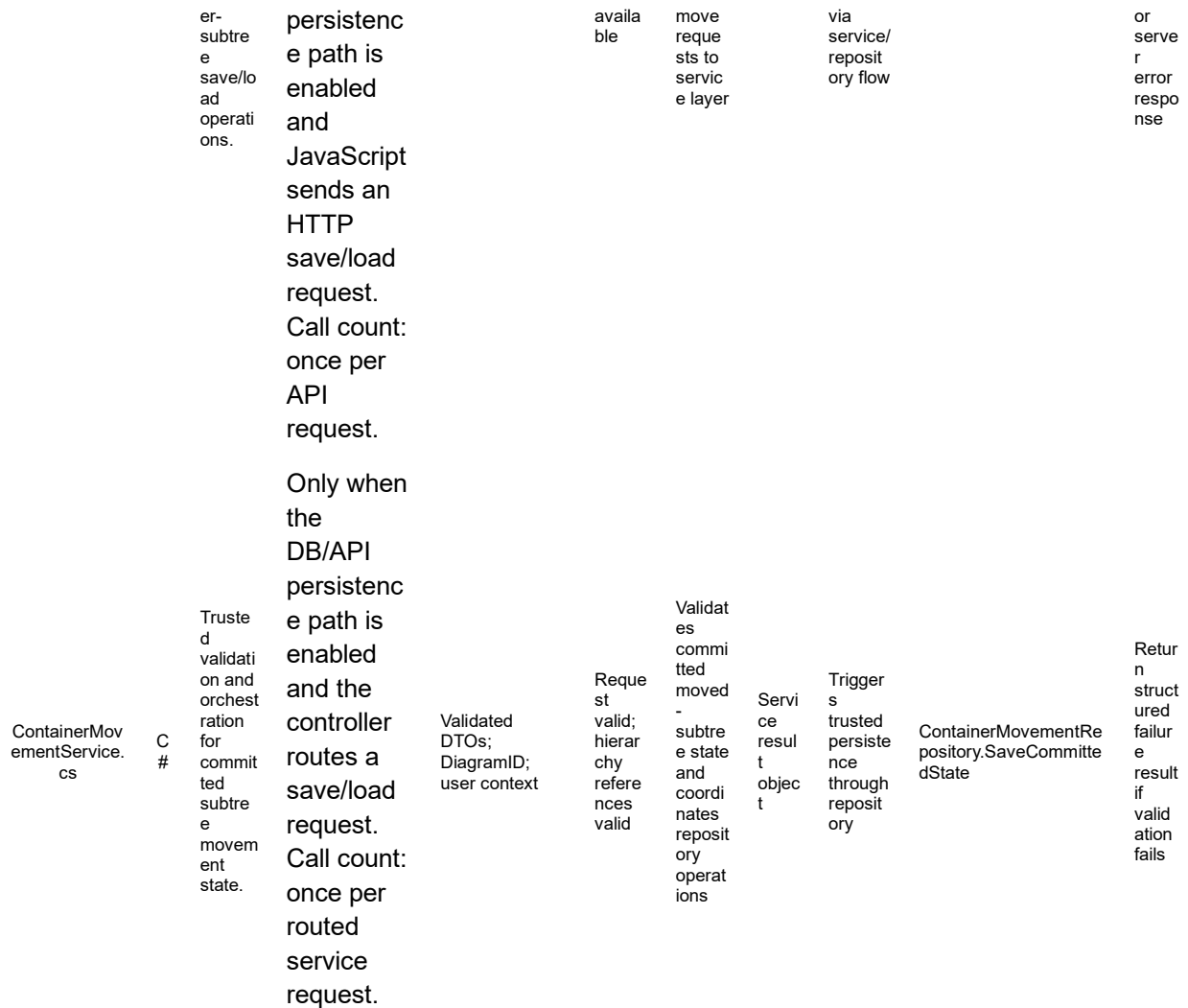
Next Possible Call(s)

Error Output / Failure Path

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
nested-container-move.css	CSS	Styles moved - container preview, selected subtree, and invalid-placement cues.	Page load, and whenever the owning component visual state is redrawn. Call count:	Subtree visual-state flags	Canvas container-move layer exists	Applies visual feedback for subtree selection, preview, and invalid placement	Consistent moved-subtree visuals	No persistence effect	render-canvas.js; move-container-subtree.js	Visuals may fall back to default styling



validate-container-subtree.js	J S	Validates the moved subtree against immediate parent and sibling rules for the moved root container.	on Left Click Up.	Candidate container-subtree move during an active subtree move action. Call count: once per processed move update; internal comparison cost scales with sibling count of the moved root plus the size of the moved subtree.	Candidate subtree geometry; sibling collections; parent container rules; subtree metadata	Candidate subtree exists; validation scope available	Check sibling overlap, ancestor containment, and no-touch rules for the moved subtree	Subtree validation result	Temporary decision state only until accepted	recalc-moved-subtree.js; renderCanvas	Restore previous valid subtree state on failure
recalc-moved-subtree.js	J S	Recalculates derived geometry and protection boundaries for the accepted moved subtree.		Accepted subtree move or reopen of committed subtree state. Call count: once per accepted subtree move step or rebuild request.	Committed subtree geometry	Committed subtree geometry valid	Recompute derived geometry for moved containers and descendants	Updated committed subtree-derived geometry	Committed in-memory state change; persisted only on save	persist-container-move-state.js; renderCanvas	Mark subtree invalid if recalculation fails
ContainerMovementController.cs	C #	API controller for committed containers		Only when the DB/API	Moved-subtree payload; DiagramID; user/request context	Request model valid; routing	Routes committed container-	HTTP response object	No direct persistence effect except	ContainerMovementService.SaveDiagram	Return controlled validation



The table below follows the Milestone 2 file-definition style and includes CSS, JavaScript, and C# files relevant to moving a rectangle container with multiple levels of nested containers and multiple geometric children. For move-related rows, call count is based on processed updates limited by `MaxMoveValidationRate`, not raw hardware mouse reports.

Recommended files for Milestone 11

Database table	Type	Purpose
Containers	Core source	Stores moved-container rows, nested container rows, and <code>ParentContainerID</code> values.
Devices	Core source	Stores descendant device geometry and <code>ParentContainerID</code> values.

DiagramPayloads	Optional snapshot	Stores committed full-diagram snapshots including subtree move state.
SubtreeMoveChangeLog	Optional audit	Stores committed subtree movement history if auditability is needed later.

The table below lists the main database tables used or extended by Milestone 11.

Recommended database table structure

Variable name	Description	Recommended values
RootMovedContainerId	Identity of the topmost container currently being moved.	Valid ContainerID
DescendantContainerIds	Ordered collection of descendant nested containers in the moved subtree.	Ordered container list
DescendantGeometryIds	Ordered collection of descendant Devices in the moved subtree.	Ordered device list
CandidateMoveDeltaX	Candidate world-space movement delta X for the subtree.	Numeric world-space value
CandidateMoveDeltaY	Candidate world-space movement delta Y for the subtree.	Numeric world-space value
PreviousValidSubtreeState	Last accepted committed geometry state of the full moved subtree.	Structured subtree snapshot
SubtreeValidationResult	Result of validating the moved subtree against siblings and parent boundaries.	Valid / Invalid
RootParentContainerID	Immediate parent of the moved root container. NULL means the root container belongs directly to DiagramCanvas root.	Valid ContainerID or NULL
AffectedLinksCollection	Connections that must be recalculated after the subtree move.	Ordered link list
SubtreeRecalculationScope	Ordered list of containers and devices whose derived geometry must be refreshed.	Ordered subtree collection

The table below includes the main Milestone 11 working variables.

Milestone 11 variables

Minimum readiness rule: Milestone 11 should not be considered ready to begin until the team confirms at least this: a moved container can be expanded into a deterministic subtree of nested containers and devices, and the root container may have ParentContainerID = NULL when it belongs directly to DiagramCanvas root.

Required confirmations: Milestones 8 and 9 containment behavior are already stable; nested parent-child relationships are persisted; subtree enumeration of descendant containers and geometries is available; and derived geometry recalculation is already working for containers, rectangles, and circles.

Before Milestone 11 implementation starts, the following prerequisites should be confirmed.

Prerequisites

Mandatory existing JavaScript before starting Milestone 11

Only JavaScript files that must already exist and work correctly before this milestone starts are listed here.

Shared save prerequisite. Before starting Milestone 11, the team shall verify that save-to-json.js and save-to-db.js are already saving all committed variables approved in Milestones 1 to 10 correctly. When Milestone 11 introduces new committed variables, the developer shall extend only these same two files so the current milestone save test checks the Milestone 11 variables, while regression validation confirms that all previous milestone variables are still saved correctly.

JavaScript file that must already exist	Description	Reason this JavaScript must already be ready and working correctly
validate-sibling-overlap.js	Validates the active child against sibling geometries in the same parent.	Nested-container movement builds on stable sibling-overlap behavior from earlier containment milestones.
validate-child-containment.js	Validates one child item against its immediate parent boundaries.	A moved container subtree still has to remain valid relative to its immediate parent.
recalc-multi-child-padding.js	Recalculates affected protection boundaries in one parent collection.	Container-subtree movement needs stable recalculation after each accepted move.
resize-parent-container.js	Expands the parent container where approved by the rules.	The container hierarchy still depends on valid parent and child boundary updates.
render-canvas.js	Renders moved containers, descendants, and validation feedback.	Nested movement must be visible and verifiable on the canvas.

Milestone 11 terminology

Main developer names

The following names are recommended for this milestone:

Moved container

ActiveMovedContainer

MovedContainerID

MovedContainerSubtree

Hierarchy terms

ImmediateParentContainerID

DirectChildItems[]

DirectNestedContainers[]

DescendantItems[]

DescendantContainers[]

SubtreeRootContainerID

SubtreeItemCollection[]

SubtreeContainerCollection[]

SubtreeGeometryCollection[]

Reference-center variables

ChildParentOffsetX

ChildParentOffsetY

SubtreeMoveDeltaX

SubtreeMoveDeltaY

Containment state

IsSubtreeContainmentValid

IsSubtreeSiblingOverlapValid

NeedsSubtreeRecalculation

Recommended implementation rule

The movement of a container subtree should be implemented as a translation operation applied recursively through the full descendant hierarchy.

For any moved container:

all descendants receive the same movement delta in world space

their parent-child reference offsets remain unchanged

their derived Protection Padding boundaries are recalculated after the move

This is the cleanest implementation and fits the document's world-space and immediate-parent logic.

Important milestone rules

Rule 1 — Container movement is subtree movement

When a rectangle container moves, it shall not move alone.

Instead, the full descendant subtree shall move with it.

This includes:

direct child rectangles

direct child circles

direct nested rectangle containers

all deeper descendants inside any nested container

Rule 1A — Circle On Container moves with host-container subtree

A Circle On Container attached to a container edge is part of the movement subtree of its host container.

If the attached container moves, the Circle On Container shall move with it.

If a parent container or any higher nested parent container moves and the attached container moves as part of that subtree, the Circle On Container shall also move.

After the movement delta is applied, the Circle On Container center shall be recalculated from the moved host container boundaries, current active HostEdge, and EdgeParameterT.

Rule 2 — Immediate parent offsets remain preserved

For each child item relative to its immediate parent container:

$$\text{ChildParentOffsetX} = \text{ChildCenterX} - \text{ParentCenterX}$$
$$\text{ChildParentOffsetY} = \text{ChildCenterY} - \text{ParentCenterY}$$

When the parent container moves, those offsets remain unchanged.

So for any child after movement:

$$\text{NewChildCenterX} = \text{NewParentCenterX} + \text{ChildParentOffsetX}$$
$$\text{NewChildCenterY} = \text{NewParentCenterY} + \text{ChildParentOffsetY}$$

This same rule applies recursively at every nesting level.

Rule 3 — Subtree movement is a world-space translation

If the moved container changes by:

$$\Delta X, \Delta Y$$

then every descendant in its subtree shall also change by:

$$\Delta X, \Delta Y$$

This means subtree movement can be implemented as:

$\text{NewDescendantCenterX} = \text{OldDescendantCenterX} + \Delta X$

$\text{NewDescendantCenterY} = \text{OldDescendantCenterY} + \Delta Y$

This is equivalent to preserving the immediate-parent offsets as long as the whole subtree is translated consistently.

Rule 4 — Validation remains immediate-parent based

Even during subtree movement, correctness shall still be interpreted using the immediate-parent model.

That means:

the moved subtree root must remain valid relative to its own immediate parent if it has one

siblings of the moved subtree root inside that same immediate parent must still be respected

descendants inside the moved subtree do not need to be revalidated against ancestors separately if their immediate-parent relationships remain preserved by the subtree translation

This follows the document's recursive immediate-parent rule.

Rule 5 — Protection Padding remains the containment and overlap authority

All containment and sibling-overlap validation in this milestone shall use:

the moved subtree root's Protection Padding boundaries relative to its immediate parent

the relevant sibling items or sibling containers' Protection Padding boundaries inside that same immediate parent

Hover Padding shall not be used for subtree movement validation.

Rule 6 — A valid subtree move must preserve internal hierarchy consistency

If a subtree move is accepted:

all internal parent-child offsets remain valid

all descendants remain inside their own immediate parents automatically because the full subtree moved together

only the subtree root needs direct revalidation against its immediate parent and siblings in most cases

That is the main optimization rule for hierarchical movement.

Core movement equations used in this milestone

Subtree root move delta

If the subtree root container moves from:

$\text{OldRootCenter} = (x_0, y_0)$

to:

$\text{NewRootCenter} = (x_1, y_1)$

then:

$$\Delta X = x_1 - x_0$$

$$\Delta Y = y_1 - y_0$$

Descendant translation rule

For any descendant item or container in the subtree:

$$\text{NewDescendantCenterX} = \text{OldDescendantCenterX} + \Delta X$$

$$\text{NewDescendantCenterY} = \text{OldDescendantCenterY} + \Delta Y$$

Immediate-parent offset preservation rule

For any child relative to its immediate parent:

$$\text{ChildParentOffsetX} = \text{OldChildCenterX} - \text{OldParentCenterX}$$

$$\text{ChildParentOffsetY} = \text{OldChildCenterY} - \text{OldParentCenterY}$$

After subtree translation:

$$\text{NewChildCenterX} - \text{NewParentCenterX} = \text{ChildParentOffsetX}$$

$$\text{NewChildCenterY} - \text{NewParentCenterY} = \text{ChildParentOffsetY}$$

So subtree translation preserves all immediate-parent offsets automatically.

Root containment rule

If the moved subtree root is itself a nested rectangle container inside another parent rectangle container, then the moved subtree root must satisfy:

$$\text{RootProtectionLeftX} > \text{ParentInnerLeftX}$$

$$\text{RootProtectionRightX} < \text{ParentInnerRightX}$$

$$\text{RootProtectionTopY} < \text{ParentInnerTopY}$$

$$\text{RootProtectionBottomY} > \text{ParentInnerBottomY}$$

Root sibling non-overlap rule

If the moved subtree root has sibling items or sibling containers in the same immediate parent, then the moved subtree root shall not overlap those siblings using the same Protection Padding overlap rules already defined earlier for rectangles and circles.

Large moved-container overlap equation for Milestone 11

When the moved subtree root is a rectangle container, the same rectangle Protection Padding overlap rule shall be applied to the subtree root after its candidate movement delta is applied. This explicitly covers a very large moving container suddenly overlapping a smaller sibling container.

For moved root container A and sibling container B:

```
A_Left   = NewRootCenterX - A_HalfWidth - A_ProtectionPaddingX
A_Right  = NewRootCenterX + A_HalfWidth + A_ProtectionPaddingX
A_Top    = NewRootCenterY + A_HalfHeight + A_ProtectionPaddingY
A_Bottom = NewRootCenterY - A_HalfHeight - A_ProtectionPaddingY

B_Left   = B_CenterX - B_HalfWidth - B_ProtectionPaddingX
B_Right  = B_CenterX + B_HalfWidth + B_ProtectionPaddingX
B_Top    = B_CenterY + B_HalfHeight + B_ProtectionPaddingY
B_Bottom = B_CenterY - B_HalfHeight - B_ProtectionPaddingY
```

The moved root container is valid against B only if one of the strict no-touch separation conditions is true:

```
A_Right < B_Left
OR B_Right < A_Left
OR A_Top < B_Bottom
OR B_Top < A_Bottom
```

If none of these conditions is true, the candidate movement shall be rejected and the root container shall be restored to PreviousValidCenterX and PreviousValidCenterY. After restoration, all descendants shall also return using the restored root movement delta so the subtree remains internally consistent.

Mathematical Calculation Rules for Milestone 11

Milestone 11 calculations move a rectangle container together with its complete descendant subtree. The moved container is validated against its immediate parent and sibling/root geometries. Descendants move by the same delta as the container and remain internally consistent.

Primary JavaScript files for this calculation scope: collect-container-subtree.js, compute-container-subtree-move-delta.js, apply-container-subtree-candidate-move.js, validate-container-subtree-move.js, restore-container-subtree-previous-valid-position.js, connection-recalculation.js.

Subtree movement delta

$\Delta X = \text{CandidateContainerCenterX} - \text{PreviousContainerCenterX}$. $\Delta Y = \text{CandidateContainerCenterY} - \text{PreviousContainerCenterY}$.

For every descendant item in the moved subtree: $\text{CandidateDescendantCenterX} = \text{PreviousDescendantCenterX} + \Delta X$ and $\text{CandidateDescendantCenterY} = \text{PreviousDescendantCenterY} + \Delta Y$.

Validation scope calculation

If the moved container is inside another parent container, validate the moved container against all sibling containers/geometries inside the same immediate parent and against the immediate parent inner boundary.

If the moved container is at DiagramCanvas root, validate it against all other root-level geometries. Do not validate it against its own descendants because descendants move with the subtree.

Rejection calculation

If the candidate moved container overlaps or touches a sibling/root geometry using strict Protection Padding validation, reject the whole candidate subtree move and restore every affected item from its previous valid world-space position.

After an accepted subtree move, recalculate derived geometry and affected connection geometry for the moved container and all connected descendants.

Circle On Container subtree movement equation

For any Circle On Container attached to a descendant container inside the moved subtree, apply the subtree movement delta to the host container first, then recalculate the circle center from the moved host boundaries.

$\text{CandidateCircleCenterX} = \text{OldCircleCenterX} + \text{DeltaX}$ and $\text{CandidateCircleCenterY} = \text{OldCircleCenterY} + \text{DeltaY}$ for pure translation. The equivalent reconstruction is `calculateCircleOnContainerCenter(MovedParentContainer, HostEdge, EdgeParameterT)`.

Previous valid restoration must restore both the moved host container and every Circle On Container attached to its subtree. Do not leave a boundary circle at a candidate position if the subtree move is rejected.

Milestone 11 phases

Milestone 11 shall be implemented in 8 phases.

Phase 11.1 — Define the movable container subtree and its descendant collections

For more information, please refer to section [Move-update calculations and MaxMoveValidationRate \(place in this document\)](#) and section [Protection Padding, Overlap, Containment, and Persistence Calculations \(place in this document\)](#).

Primary JavaScript files for this phase: `move-update-scheduler.js`; `detect-render-rate.js`; `validate-child-containment.js`; `restore-previous-valid-position.js`; `apply-validated-center.js`; `update-previous-valid-position.js`.

Goal

Define the full subtree of a moved rectangle container, including all nested containers and all geometric descendants.

Scope

This phase is about:

identifying the moved container as the subtree root

collecting all descendant containers

collecting all descendant geometric items

building the subtree model used for recursive movement

Required visible / measurable behaviors

Examples:

the system can identify the selected container as a movable subtree root

the system can list all direct children of the root container

the system can list all deeper descendants

the system can distinguish between:

direct children

descendants

siblings outside the subtree

Variables to define

At minimum:

SubtreeRootContainerID

DirectChildItems[]

DirectNestedContainers[]

DescendantItems[]

DescendantContainers[]

SubtreeItemCollection[]

SubtreeContainerCollection[]

Required event-to-function routing

Subtree-definition flow

select rectangle container for movement

→ buildContainerSubtree

→ collectDirectChildItems

→ collectDescendantContainers

→ collectDescendantGeometries

→ prepareSubtreeMoveState

State rule

During this phase:

only the subtree definition state is built

no actual movement happens yet

no persistence is required

Success criteria

This phase is successful if:

the system can build the full subtree of the moved rectangle container

the system can distinguish subtree members from outside siblings

Function interaction contracts

Function Name: buildContainerSubtree

Layer: JavaScript

File: container-subtree-builder.js

Purpose: Build the full descendant subtree for a selected rectangle container.

Trigger:

rectangle container selected for movement

Input:

SubtreeRootContainerID

canvas item collection

parent-child relation collection

Validation / Preconditions:

selected container exists

parent-child relations are available

canvas hierarchy data is valid

Processing Responsibility:

identify the selected container as subtree root

collect all direct child items

collect all direct nested containers

recursively collect all deeper descendant containers and geometric items

Output:

SubtreeItemCollection[]

SubtreeContainerCollection[]

subtree metadata object

State Change / Persistence Effect:

updates temporary in-memory subtree move state only

no persistence effect

Next Possible Call(s):

prepareSubtreeMoveState()

computeSubtreeMoveDelta()

Error Output / Failure Path:

return structured failure and do not enter subtree movement mode if hierarchy data is invalid

Function Name: collectDescendantGeometries

Layer: JavaScript

File: container-subtree-builder.js

Purpose: Collect all geometric descendants inside a selected container subtree.

Trigger:

called during subtree construction

Input:

current container ID

parent-child relation collection

canvas item collection

Validation / Preconditions:

current container exists

hierarchy relations are valid

Processing Responsibility:

traverse child relations recursively

collect child rectangles

collect child circles

exclude items outside the subtree

Output:

DescendantItems[]

State Change / Persistence Effect:

no persistence effect

returns descendant collection for subtree construction

Next Possible Call(s):

buildContainerSubtree()

prepareSubtreeMoveState()

Error Output / Failure Path:

return empty or failed descendant collection if traversal data is invalid

Phase 11.2 — Move a nested rectangle container inside its immediate parent container

For more information, please refer to section [Move-update calculations and MaxMoveValidationRate \(place in this document\)](#) and section [Protection Padding, Overlap, Containment, and Persistence Calculations \(place in this document\)](#).

Primary JavaScript files for this phase: move-update-scheduler.js; detect-render-rate.js; validate-child-containment.js; restore-previous-valid-position.js; apply-validated-center.js; update-previous-valid-position.js.

Goal

Move a rectangle container that is itself a child of another rectangle container, while preserving its full subtree.

Scope

This phase is about:

moving one nested rectangle container

translating its full descendant subtree

validating the moved subtree root against its immediate parent

validating the moved subtree root against its siblings in the same immediate parent

Required test behaviors

Expected result:

the selected nested container moves

all descendants inside it move with it

the moved nested container remains inside its immediate parent

the moved nested container does not overlap its siblings in the same immediate parent

all descendants preserve their relative parent-child offsets

Variables to define

At minimum:

MovedContainerID

ImmediateParentContainerID

SiblingItems[]

SiblingContainers[]

SubtreeMoveDeltaX

SubtreeMoveDeltaY

Required event-to-function routing

Nested container move flow

left click and drag nested container

→ screenToWorld

→ computeSubtreeMoveDelta

→ translateSubtreeByDelta

→ validateMovedSubtreeRootWithinImmediateParent

→ validateMovedSubtreeRootAgainstImmediateSiblings

→ applyValidatedSubtreeMove

→ renderCanvas

State rule

During this phase:

the moved nested container is treated as the subtree root

all descendants move with it

only the subtree root is directly validated against its immediate parent and siblings

Success criteria

This phase is successful if:

the nested container can move

its descendants move with it

the moved subtree root remains valid inside its immediate parent

the moved subtree root remains valid relative to its siblings

Function interaction contracts

Function Name: computeSubtreeMoveDelta

Layer: JavaScript

File: container-subtree-move.js

Purpose: Compute the world-space movement delta for a selected subtree root container.

Trigger:

candidate container move generated during drag

Input:

OldRootCenterX

OldRootCenterY

CandidateRootCenterX

CandidateRootCenterY

Validation / Preconditions:

subtree root exists

old and candidate centers are valid numeric world coordinates

Processing Responsibility:

compute SubtreeMoveDeltaX

compute SubtreeMoveDeltaY

prepare the move delta for subtree translation

Output:

SubtreeMoveDeltaX

SubtreeMoveDeltaY

State Change / Persistence Effect:

no persistence effect

returns delta for later subtree application

Next Possible Call(s):

translateSubtreeByDelta()

validateMovedSubtreeRootWithinImmediateParent()

Error Output / Failure Path:

return failed delta computation and preserve previous valid subtree state if coordinates are invalid

Function Name: translateSubtreeByDelta

Layer: JavaScript

File: container-subtree-move.js

Purpose: Translate the subtree root container and all its descendants by the same world-space movement delta.

Trigger:

successful subtree move delta computation

Input:

SubtreeRootContainerID

SubtreeItemCollection[]

SubtreeContainerCollection[]

SubtreeMoveDeltaX

SubtreeMoveDeltaY

Validation / Preconditions:

subtree root exists

subtree collections exist

movement delta is valid

Processing Responsibility:

add the same delta to the root container center

add the same delta to every descendant container center

add the same delta to every descendant geometric item center

preserve all internal offsets

Output:

translated subtree preview state

State Change / Persistence Effect:

updates temporary in-memory subtree preview geometry only

no persistence effect

Next Possible Call(s):

validateMovedSubtreeRootWithinImmediateParent()

validateMovedSubtreeRootAgainstImmediateSiblings()

Error Output / Failure Path:

restore previous valid subtree geometry if subtree translation fails

Function Name: validateMovedSubtreeRootWithinImmediateParent

Layer: JavaScript

File: container-subtree-validation.js

Purpose: Validate the moved subtree root container against the inner usable boundaries of its immediate parent container.

Trigger:

translated subtree preview state generated during container move

Input:

MovedSubtreeRootProtectionLeftX

MovedSubtreeRootProtectionRightX

MovedSubtreeRootProtectionTopY

MovedSubtreeRootProtectionBottomY

ImmediateParentInnerLeftX

ImmediateParentInnerRightX

ImmediateParentInnerTopY

ImmediateParentInnerBottomY

Validation / Preconditions:

subtree root exists

immediate parent exists

root Protection Padding boundaries are derivable

parent inner boundaries are valid

Processing Responsibility:

compare the moved subtree root Protection Padding boundaries against the immediate parent inner boundaries

enforce strict no-touch rule

Output:

valid / invalid root-parent containment result

State Change / Persistence Effect:

no persistence effect

Next Possible Call(s):

validateMovedSubtreeRootAgainstImmediateSiblings()

applyValidatedSubtreeMove()

Error Output / Failure Path:

reject the candidate subtree move if root-parent containment fails

Phase 11.3 — Move an intermediate rectangle container with mixed descendants

For more information, please refer to section [Move-update calculations and MaxMoveValidationRate \(place in this document\)](#) and section [Protection Padding, Overlap, Containment, and Persistence Calculations \(place in this document\)](#).

Primary JavaScript files for this phase: move-update-scheduler.js; detect-render-rate.js; validate-child-containment.js; restore-previous-valid-position.js; apply-validated-center.js; update-previous-valid-position.js.

Goal

Move an intermediate rectangle container that contains both nested containers and geometric items.

Scope

This phase is about:

moving an intermediate container

preserving mixed descendants such as:

child rectangles

child circles

nested rectangle containers

revalidating the moved subtree root only against its immediate parent and siblings

Required test behaviors

Expected result:

the intermediate container moves

all mixed descendants move with it

all internal hierarchy relations remain preserved

the moved subtree root remains valid relative to its immediate parent and siblings

Variables to define

At minimum:

SubtreeMixedItems[]

SubtreeMixedContainers[]

ImmediateSiblingSet[]

Required event-to-function routing

Intermediate container move flow

left click and drag intermediate container

→ buildContainerSubtree

→ computeSubtreeMoveDelta

→ translateSubtreeByDelta

→ validateMovedSubtreeRootWithinImmediateParent

→ validateMovedSubtreeRootAgainstImmediateSiblings

→ applyValidatedSubtreeMove

→ renderCanvas

State rule

During this phase:

the internal subtree hierarchy remains unchanged except for translation

mixed descendants all move together by the same world-space delta

Success criteria

This phase is successful if:

an intermediate container with mixed descendants can move as one subtree

all descendants preserve their immediate-parent offsets

Function interaction contracts

Function Name: `validateMovedSubtreeRootAgainstImmediateSiblings`

Layer: JavaScript

File: `container-subtree-validation.js`

Purpose: Validate the moved subtree root container against sibling items and sibling containers inside the same immediate parent container.

Trigger:

translated subtree preview state generated during subtree move

Input:

`MovedSubtreeRootProtectionPadding` boundaries

`ImmediateSiblingSet[]`

strict sibling no-touch rule

Validation / Preconditions:

subtree root exists

sibling set exists

all sibling Protection Padding boundaries are valid

Processing Responsibility:

compare the moved subtree root Protection Padding boundaries against each sibling item or sibling container Protection Padding boundary

determine whether overlap or forbidden touching occurs

Output:

valid / invalid root-sibling validation result

State Change / Persistence Effect:

no persistence effect

Next Possible Call(s):

`applyValidatedSubtreeMove()`

`restorePreviousValidSubtreeGeometry()`

Error Output / Failure Path:

reject the candidate subtree move if root-sibling validation fails

Function Name: `applyValidatedSubtreeMove`

Layer: JavaScript

File: `container-subtree-move.js`

Purpose: Commit a validated subtree translation to the live hierarchy model.

Trigger:

successful subtree root validation result

Input:

validated subtree translation state

SubtreeRootContainerID

SubtreeItemCollection[]

SubtreeContainerCollection[]

Validation / Preconditions:

validated subtree state exists

all subtree members exist

Processing Responsibility:

commit the translated centers of the subtree root and all descendants

update the live world-space hierarchy model

mark the diagram dirty

Output:

updated live subtree hierarchy state

State Change / Persistence Effect:

updates temporary in-memory hierarchy model only

no persistence effect yet

Next Possible Call(s):

recalculateSubtreeDerivedGeometry()

renderCanvas()

Error Output / Failure Path:

preserve previous valid subtree geometry if the validated move state is invalid

Phase 11.4 — Move a top-level rectangle container with multiple nested containers and multiple geometric descendants

For more information, please refer to section [Move-update calculations and MaxMoveValidationRate \(place in this document\)](#) and section [Protection Padding, Overlap, Containment, and Persistence Calculations \(place in this document\)](#).

Primary JavaScript files for this phase: move-update-scheduler.js; detect-render-rate.js; validate-child-containment.js; restore-previous-valid-position.js; apply-validated-center.js; update-previous-valid-position.js.

Goal

Move a top-level rectangle container that contains several nested containers and several geometric descendants.

Scope

This phase is about:

moving a top-level container

translating all descendants in the hierarchy

preserving all child-parent offsets

since the container is top-level, no immediate parent containment validation is needed

Required test behaviors

Expected result:

the top-level container moves

all nested containers move with it

all geometric descendants move with it

all internal reference-center offsets remain unchanged

Variables to define

At minimum:

TopLevelContainerID

TopLevelSubtreeCollection[]

Required event-to-function routing

Top-level container move flow

left click and drag top-level container

→ buildContainerSubtree

→ computeSubtreeMoveDelta

→ translateSubtreeByDelta

→ applyValidatedSubtreeMove

→ renderCanvas

State rule

During this phase:

the whole subtree moves by translation only

there is no immediate parent validation for the top-level root

internal hierarchy consistency must remain preserved

Success criteria

This phase is successful if:

the top-level container and all descendants move together

all internal parent-child offsets remain unchanged

Function interaction contracts

Function Name: moveTopLevelContainerSubtree

Layer: JavaScript

File: container-subtree-move.js

Purpose: Move a top-level container subtree by world-space translation.

Trigger:

candidate move generated during top-level container drag

Input:

TopLevelContainerID

SubtreeItemCollection[]

SubtreeContainerCollection[]

SubtreeMoveDeltaX

SubtreeMoveDeltaY

Validation / Preconditions:

top-level container exists

subtree collections exist

movement delta is valid

Processing Responsibility:

translate the top-level container and all descendants by the same world-space delta

preserve all internal offsets

Output:

translated top-level subtree state

State Change / Persistence Effect:

updates temporary in-memory hierarchy model only

no persistence effect

Next Possible Call(s):

applyValidatedSubtreeMove()

renderCanvas()

Error Output / Failure Path:

preserve previous valid top-level subtree geometry if translation fails

Phase 11.5 — Recalculate derived geometry and Protection Padding for the full moved subtree

For more information, please refer to section [Protection Padding, Overlap, Containment, and Persistence Calculations](#) ([place in this document](#)).

Goal

After a subtree move is accepted, recalculate all required derived geometry and Protection Padding values for the full subtree.

Scope

This phase is about:

recalculating moved container boundaries

recalculating moved nested container boundaries

recalculating moved child rectangle Protection Padding boundaries

recalculating moved child circle protection-padded values

Required test behaviors

Expected result:

after the subtree moves, all derived geometry is updated correctly

containment and overlap checks can continue using the recalculated values

Variables to define

At minimum:

NeedsSubtreeRecalculation

RecalculatedSubtreeGeometryState

Required event-to-function routing

Subtree recalculation flow

applyValidatedSubtreeMove

→ recalculateSubtreeDerivedGeometry

→ recalculateSubtreeProtectionPaddingGeometry

→ renderCanvas

State rule

During this phase:

only derived geometry is recalculated

canonical moved world positions remain the primary truth

Success criteria

This phase is successful if:

all descendant derived geometry is updated after subtree movement

all Protection Padding values remain usable for future validation

Function interaction contracts

Function Name: recalculateSubtreeDerivedGeometry

Layer: JavaScript

File: container-subtree-recalculation.js

Purpose: Recalculate derived geometry for the moved container subtree after a committed subtree move.

Trigger:
successful subtree move committed

Input:
SubtreeRootContainerID
SubtreeItemCollection[]
SubtreeContainerCollection[]
current canonical world positions

Validation / Preconditions:
subtree exists
canonical moved positions are valid

Processing Responsibility:
recalculate rectangle visible boundaries for all moved containers and rectangles
recalculate circle visible boundaries for all moved circles
prepare derived geometry for further Protection Padding recalculation

Output:
updated subtree derived geometry collection

State Change / Persistence Effect:
updates in-memory derived geometry only
no persistence effect

Next Possible Call(s):
recalculateSubtreeProtectionPaddingGeometry()
renderCanvas()

Error Output / Failure Path:
mark subtree recalculation failure and preserve previous valid derived geometry if recalculation fails

Function Name: recalculateSubtreeProtectionPaddingGeometry

Layer: JavaScript
File: container-subtree-recalculation.js
Purpose: Recalculate all Protection Padding geometry for the moved subtree after derived geometry is updated.

Trigger:
successful subtree derived-geometry recalculation

Input:
SubtreeItemCollection[]
SubtreeContainerCollection[]
padding values for rectangles and circles

Validation / Preconditions:
subtree collections exist
derived geometry is available
padding values are valid

Processing Responsibility:

recalculate Protection Padding boundaries for rectangles and rectangle containers

recalculate protection-padded radii and boundaries for circles

prepare subtree validation-ready geometry

Output:

updated subtree Protection Padding geometry collection

State Change / Persistence Effect:

updates in-memory derived Protection Padding geometry only

no persistence effect

Next Possible Call(s):

renderCanvas()

validateCommittedSubtreeMoveState()

Error Output / Failure Path:

mark subtree Protection Padding recalculation failure and preserve previous valid derived state if

recalculation fails

Phase 11.6 — Move one sibling container among multiple sibling containers inside the same immediate parent

For more information, please refer to section [Move-update calculations and MaxMoveValidationRate \(place in this document\)](#) and section [Protection Padding, Overlap, Containment, and Persistence Calculations \(place in this document\)](#).

Primary JavaScript files for this phase: move-update-scheduler.js; detect-render-rate.js; validate-child-containment.js; restore-previous-valid-position.js; apply-validated-center.js; update-previous-valid-position.js.

Goal

Move one rectangle container among multiple sibling nested containers inside the same immediate parent while preserving its internal subtree and respecting sibling validation.

Scope

This phase is about:

multiple sibling nested containers inside one immediate parent

moving one of them

preserving its subtree

validating against sibling nested containers and sibling geometric items

Required test behaviors

Expected result:

one sibling nested container can move

its full subtree moves with it

it does not overlap sibling containers

it does not overlap sibling geometric items

it remains inside its immediate parent

Variables to define

At minimum:

ActiveSiblingContainerID

ImmediateSiblingContainers[]

ImmediateSiblingItems[]

Required event-to-function routing

Sibling container move flow

left click and drag one sibling nested container

→ buildContainerSubtree

→ computeSubtreeMoveDelta

→ translateSubtreeByDelta

→ validateMovedSubtreeRootWithinImmediateParent

→ validateMovedSubtreeRootAgainstImmediateSiblings

→ applyValidatedSubtreeMove

→ renderCanvas

State rule

During this phase:

the moved sibling container is the subtree root

its internal descendants move with it

its external siblings do not move

Success criteria

This phase is successful if:

one sibling nested container can move independently

its subtree remains intact

it remains valid relative to the immediate parent and sibling items

Function interaction contracts

Function Name: validateSiblingContainerSubtreeMove

Layer: JavaScript

File: container-subtree-validation.js

Purpose: Validate the move of one sibling nested container subtree against its immediate parent and all external siblings.

Trigger:

translated subtree preview state generated during sibling container move

Input:

MovedSubtreeRootProtectionPadding boundaries

ImmediateParent inner boundaries

ImmediateSiblingContainers[]

ImmediateSiblingItems[]

strict sibling no-touch rule

Validation / Preconditions:

moved subtree root exists

immediate parent exists

sibling sets exist

Protection Padding boundaries are valid

Processing Responsibility:

validate moved subtree root against immediate parent inner boundaries

validate moved subtree root against sibling containers

validate moved subtree root against sibling items

Output:

valid / invalid sibling-container subtree move result

State Change / Persistence Effect:

no persistence effect

Next Possible Call(s):

applyValidatedSubtreeMove()

restorePreviousValidSubtreeGeometry()

Error Output / Failure Path:

reject the candidate subtree move and preserve previous valid subtree geometry if validation fails

Phase 11.7 — Recalculate committed hierarchical move state and persist the subtree move to JSON and database

Exact variables to save in this phase

Milestone 11 introduces no new independent permanent persistence field names. The exact save requirement in this phase is: keep saving all previously approved committed variables for every container, descendant container, geometric child, and connection affected by the accepted subtree move. The canonical saved truth remains the updated committed geometry, identities, and ParentContainerID relations already approved in earlier milestones.

Do not save Milestone 11 hierarchy-processing helpers as milestone-required persisted data:

CandidateMoveDeltaX, CandidateMoveDeltaY, PreviousValidSubtreeState, SubtreeValidationResult, AffectedLinksCollection, SubtreeRecalculationScope, RootMovedContainerId, DescendantContainerIds,

and DescendantGeometryIds may be used during orchestration, but the committed saved truth shall remain the already approved persistent item and relation records.

Save regression rule for this phase

All approved Milestones 2 through 9 committed variables shall already remain saved correctly in both persistence paths. This phase passes only if save-to-json.js and save-to-db.js keep the prior approved save set intact while persisting the accepted subtree move through canonical geometry and existing parent relations only.

Save test criteria for this phase

Verify that both shared save files persist the updated committed item records for the full moved subtree, that the Milestone 11 hierarchy helpers listed above are excluded as milestone-required saved data, and that reopen from either source restores the same accepted hierarchical move result.

For more information, please refer to section Protection Padding, Overlap, Containment, and Persistence Calculations ([place in this document](#)).

Goal

Recalculate all committed hierarchy movement state for a moved subtree and persist the diagram to both JSON and database.

Scope

This phase is about:

recalculating subtree geometry

revalidating committed hierarchy movement state

rebuilding the save model

saving the diagram to JSON

saving the diagram to the database

Important recalculation rule

Before save, the system shall recalculate from the committed hierarchy state:

all moved container centers

all descendant geometric centers

all immediate-parent reference offsets if they are part of the committed hierarchy relation

all moved subtree derived geometry

all moved subtree Protection Padding geometry

This follows the document's general rule that derived geometry must be recalculated from canonical stored values before persistence.

Required event-to-function routing

Save-commit flow

click Save button
→ recalculateAllHierarchicalMoveGeometry
→ validateCommittedHierarchicalMoveState
→ buildDiagramJsonObject
→ serializeDiagramToJson
→ saveDiagramJsonFile
→ buildDiagramSavePayload
→ callSaveDiagramApi
→ DiagramController.SaveDiagram
→ DiagramService.SaveDiagram
→ DiagramRepository.SaveDiagram
→ handleSaveCommitResult

State rule

During this phase:

recalculation must occur before persistence

JSON save and database save must use the same committed recalculated hierarchy state

Success criteria

This phase is successful if:

all committed subtree movement geometry is recalculated before save

the saved JSON and database state both represent the same committed hierarchy movement logic

reopening from file or database restores the same moved subtree state

Function interaction contracts

Function Name: recalculateAllHierarchicalMoveGeometry

Layer: JavaScript

File: hierarchical-move-recalculation.js

Purpose: Recalculate all committed geometry and Protection Padding state for moved container hierarchies before persistence.

Trigger:

explicit save flow

diagram load reconstruction

committed subtree move requiring full hierarchy refresh

Input:

container hierarchy collection

geometry item collection

parent-child relation data

committed moved world positions

Validation / Preconditions:

all referenced containers exist

all referenced items exist

canonical moved positions are present
parent-child relations are valid

Processing Responsibility:

recalculate moved container derived geometry
recalculate moved descendant item derived geometry
recalculate subtree Protection Padding geometry
prepare committed hierarchical move state for validation

Output:

recalculated hierarchical move geometry collection

State Change / Persistence Effect:

updates in-memory derived hierarchy geometry only
no direct persistence by this function

Next Possible Call(s):

validateCommittedHierarchicalMoveState()
buildDiagramJsonObject()
renderCanvas()

Error Output / Failure Path:

stop save flow if required hierarchy movement geometry cannot be recalculated

Function Name: validateCommittedHierarchicalMoveState

Layer: JavaScript

File: hierarchical-move-save-validation.js

Purpose: Verify that the committed moved subtree hierarchy state is valid before persistence.

Trigger:

save flow after hierarchical move recalculation

Input:

committed container hierarchy data
committed geometric item data
recalculated derived geometry
recalculated Protection Padding geometry
immediate-parent relation data
reference-center offset data if used

Validation / Preconditions:

recalculation already completed
required hierarchy fields exist
parent-child relations are valid

Processing Responsibility:

validate subtree root containment relative to immediate parent if applicable
validate subtree root sibling non-overlap if applicable
validate internal hierarchy consistency
validate reference-center offset consistency if saved

Output:

valid / invalid committed hierarchical move result

State Change / Persistence Effect:

no persistence effect

no direct permanent state change

Next Possible Call(s):

buildDiagramJsonObject()

buildDiagramSavePayload()

Error Output / Failure Path:

stop save flow and report hierarchical-move validation failure if committed state is invalid

Phase 11.8 — Move container subtrees with Circle On Container items attached to descendant container edges

Goal

Ensure Circle On Container items move correctly when their attached container, a parent container, or any higher nested parent container moves.

Scope

This phase is about subtree movement. A Circle On Container attached to a descendant container edge must be included in the moved subtree collection for movement, restoration, rendering, and save/load recalculation.

Required movement rules

If the host container moves by DeltaX and DeltaY, every attached Circle On Container moves by the same delta and then recalculates CenterX and CenterY from HostEdge and EdgeParameterT.

If a higher nested parent container moves, the host container moves as part of that parent subtree, and every Circle On Container attached to that host container also moves.

If the candidate subtree move is rejected because of containment or sibling/root overlap, restore the previous valid position of the moved container subtree and every affected Circle On Container.

Success criteria

This phase succeeds when Circle On Container items remain visually and mathematically attached to their container edges after moving any valid ancestor container subtree, and when rejected subtree moves restore all affected Circle On Container positions correctly.

Final result after Milestone 11

At the end of this milestone, Circle On Container items attached to moved descendant container edges are translated, recalculated, validated, and restored with the same subtree.

At the end of Milestone 11, the developer should have:

movement of a nested rectangle container subtree

movement of an intermediate rectangle container with mixed descendants

movement of a top-level rectangle container with multiple nested containers and multiple geometric descendants

preservation of immediate-parent reference-center offsets across multiple hierarchy levels

subtree translation in world space

validation of moved subtree roots against immediate parent containers

validation of moved subtree roots against immediate siblings

recalculation of derived geometry and Protection Padding for the full moved subtree

recalculation and persistence of committed hierarchical movement state

This milestone makes hierarchical container movement with multiple nesting levels and multiple geometric descendants a real interactive and persistent part of the diagram system.

Only after subtree movement is stable should JSON/database persistence be enabled.

Keep all movement in world space and treat the moved container plus descendants as one logical validation unit.

Implement subtree enumeration first, then candidate move application, then subtree validation, then recalculation and persistence.

Recommended implementation order

Recommended coding files structure additions for Milestone 11

JavaScript

Recommended new modules:

container-subtree-builder.js

container-subtree-move.js

container-subtree-validation.js

container-subtree-recalculation.js

recalculate-container-derived-geometry-on-resize.js

hierarchical-move-recalculation.js

hierarchical-move-save-validation.js

Optional C# structure

HierarchicalMoveDto.cs

ContainerSubtreeDto.cs

HierarchyReferenceOffsetDto.cs

Do not move live subtree translation, sibling validation, or parent-child offset preservation into C#.

Best folder layout additions for Milestone 11

```
project/  
├─ js/  
|   ├─ container-subtree-builder.js  
|   ├─ container-subtree-move.js  
|   ├─ container-subtree-validation.js  
|   ├─ container-subtree-recalculation.js  
|   ├─ hierarchical-move-recalculation.js  
|   └─ hierarchical-move-save-validation.js  
├─ csharp/  
|   ├─ HierarchicalMoveDto.cs  
|   ├─ ContainerSubtreeDto.cs  
|   └─ HierarchyReferenceOffsetDto.cs
```

More ideas you can define in Milestone 11

You may either include these now or mark them as future-ready additions:

1. Moving multiple selected sibling containers together
2. Locking one subtree while another subtree moves
3. Optional proportional repositioning of descendants during parent resize
4. Visual debug overlay for subtree roots and descendant offsets
5. Undo/redo action records for subtree movement
6. Partial subtree collapse/expand in the UI for easier hierarchy editing

The strongest design recommendation here is:

Treat a moved container as a **subtree root**, apply one **world-space translation delta** to the full subtree, preserve all **immediate-parent reference offsets**, and validate only the **moved subtree root** directly against its immediate parent and external siblings. That is the cleanest extension of the document's existing recursive containment model

Milestone 12 — Attaching an SVG Image to a Rectangle or a Circle Using Host Geometry Rules

Overview

Milestone 12 defines the behavior for attaching an SVG image to a Device that already uses Rectangle or Circle as its host geometry on the DiagramCanvas.

Circle On Container SVG support is intentionally left for a later update. This milestone continues to define SVG attachment for normal Rectangle and Circle host geometry only unless a future requirement explicitly extends it.

For this milestone, the supported Device host geometries are:

Rectangle

Circle

The SVG image shall behave as a **host-attached visual item**.

That means:

the SVG has its own identity

the SVG is saved as an SVG asset, including its SVG source text

the SVG knows which Device it belongs to

the SVG follows host movement

the SVG is recalculated when the host resizes

the SVG may follow host rotation when the host is a rectangle

the SVG does not replace the Device host geometry

the host geometry remains the main authority for:

padding rules

container restrictions

overlap rules

selection rules

movement rules

resize rules

The SVG file itself may contain any valid visual SVG content such as:

rect

circle

ellipse

path

symbol

defs

use

or mixed SVG content

For this milestone, the internal SVG tag types do **not** determine the diagram geometry type.

The real diagram geometry type is determined only by the native Device host geometry:

Rectangle

or

Circle

This milestone also introduces one common user-controlled sizing variable for both host types:

SvgAttachmentScaleRatio

If the user increases the SVG size, the containing rectangle or circle shall reflect that size change by updating its host geometry.

So the common sizing concept is shared, while the resulting host geometry remains shape-specific:

Rectangle updates:

HalfWidth

HalfHeight

Circle updates:

Radius

A core design rule for this milestone is:

JavaScript is responsible for SVG upload parsing, SVG asset validation, SVG attachment interaction, SVG fitting calculation, host-size update from attachment scale, live rendering, host-follow behavior, and immediate recalculation of SVG placement and host geometry

C# is responsible only for trusted validation, persistence, API handling, and database communication when save/load is required

Another important rule for this milestone is:

all SVG attachment calculations shall be based on host geometry in world space

all padding, container, and overlap rules shall be applied through the authoritative host geometry

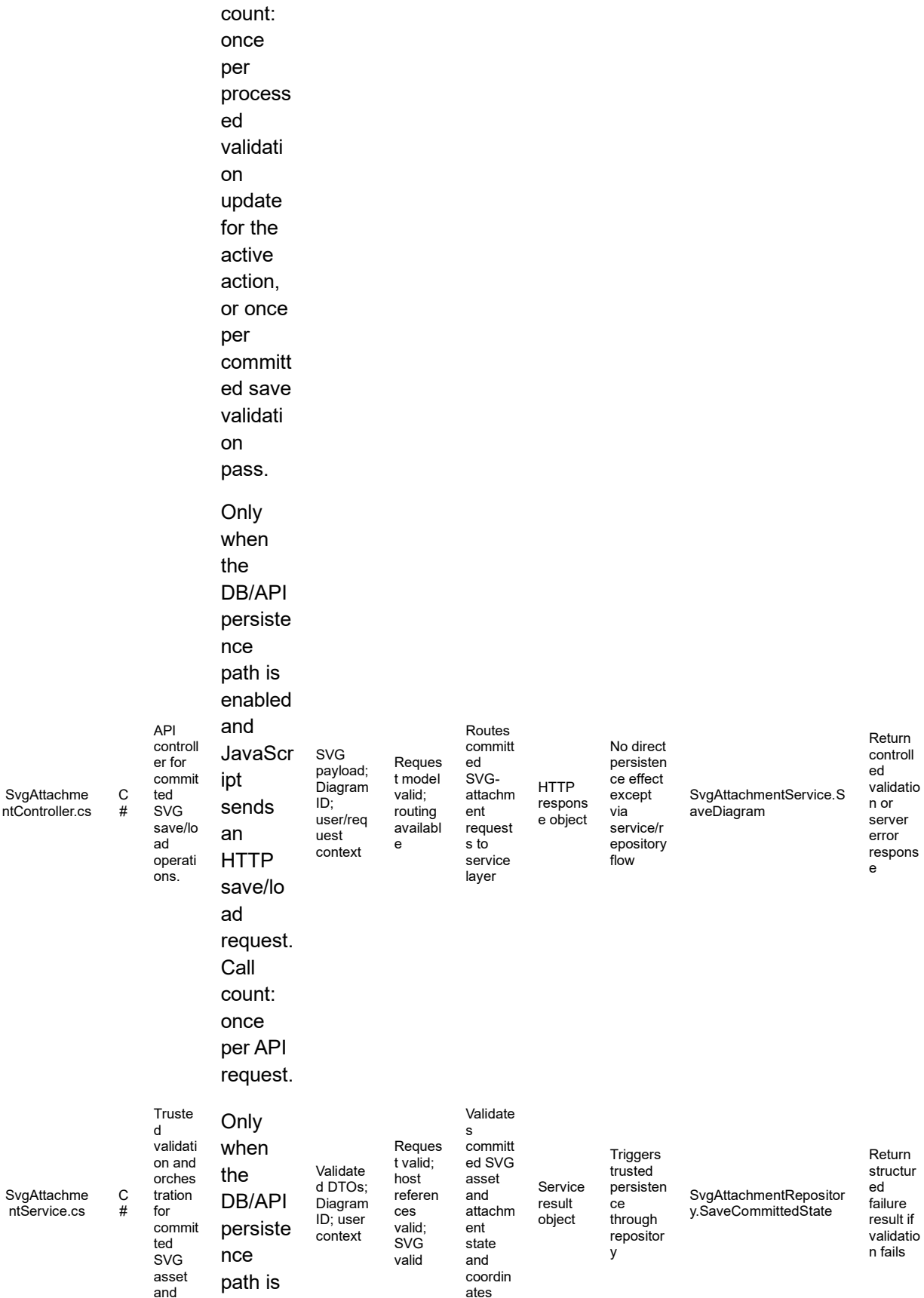
the SVG shall be treated as visual content attached to the host, not as the primary rule geometry

This milestone shall use the same function-contract structure already defined in the document, and it keeps the same world-space and Protection Padding rules already used by rectangles, circles, and containers.

File name	Type	Description	Explicit Trigger / Call Count	Input	Validation / Precon ditions	Process ing Respo nsibilit y	Output	State Change / Persiste nce Effect	Next Possible Call(s)	Error Output / Failure Path
-----------	------	-------------	--	-------	-----------------------------------	--	--------	---	-----------------------	---







attach ment state.	enabled and the controll er routes a save/lo ad request. Call count: once per routed service request.	reposito ry operati ons
--------------------------	---	----------------------------------

The table below follows the Milestone 2 file-definition style and includes CSS, JavaScript, and C# files relevant to attaching an SVG image to a Rectangle or a Circle using host geometry rules.

Recommended files for Milestone 12

Database table	Type	Purpose
Devices	Core source	Stores the Device rows whose host geometry is Rectangle or Circle before SVG attachment.
SvgAssets	Core	Stores validated SVG asset identity, source text, and metadata.
SvgAttachments	Core	Stores Device-to-SVG relationship, attachment mode, and committed positioning metadata.
DiagramPayloads	Optional snapshot	Stores committed full-diagram snapshots including SVG attachment state.
ObjectStorageReferences	Optional storage map	Stores object-storage references if SVG assets are externalized.

The table below lists the main database tables used or extended by Milestone 12.

Recommended database table structure

Variable name	Description	Recommended values
DeviceID	Identity of the Device that owns the SVG attachment.	Valid DeviceID

DeviceHostGeometryType	Host geometry used by the Device before SVG attachment.	Rectangle / Circle
SvgAssetId	Permanent identity of the SVG asset.	GUID / UUID string
SvgSourceText	Committed SVG markup text after validation.	Valid SVG XML text
SvgRenderWidth	Current render width used for the SVG asset.	Positive numeric value
SvgRenderHeight	Current render height used for the SVG asset.	Positive numeric value
AttachmentMode	Rule by which the SVG is attached to the Device host.	HostCentered / HostBounded / HostScaled
SvgAttachmentScaleRatio	User-controlled sizing ratio applied to the SVG relative to the Device host.	Positive numeric value such as 1.0
HostOffsetX	Optional X offset from the host reference point.	Numeric world-space value such as 0
HostOffsetY	Optional Y offset from the host reference point.	Numeric world-space value such as 0
FollowHostRotation	Indicates whether the SVG follows Rectangle host rotation.	True / False
SvgAttachmentValidationResult	Result of validating SVG-host size, containment, and overlap rules.	Valid / Invalid

The table below includes the main Milestone 12 working variables.

Milestone 12 variables

Minimum readiness rule: Milestone 12 should not be considered ready to begin until the team confirms at least this: a valid SVG asset can be loaded as text and one existing Rectangle or Circle can already be resolved as the host geometry.

Required confirmations: Milestones 3 through 10 are already working; Rectangle and Circle host geometry rules are stable; upload/validation of SVG text is available; host recalculation after move or resize is already working; and object-storage or file-persistence rules for SVG assets are agreed.

Before Milestone 12 implementation starts, the following prerequisites should be confirmed.

Prerequisites

Mandatory existing JavaScript before starting Milestone 12

Only JavaScript files that must already exist and work correctly before this milestone starts are listed here.

Shared save prerequisite. Before starting Milestone 12, the team shall verify that save-to-json.js and save-to-db.js are already saving all committed variables approved in Milestones 1 to 11 correctly. When Milestone 12 introduces new committed variables, the developer shall extend only these same two files so the current milestone save test checks the Milestone 12 variables, while regression validation confirms that all previous milestone variables are still saved correctly.

JavaScript file that must already exist	Description	Reason this JavaScript must already be ready and working correctly
render-rectangle.js	Renders Rectangle host geometry.	SVG attachment to Rectangle depends on a stable visible host geometry.
render-circle.js	Renders Circle host geometry.	SVG attachment to Circle depends on a stable visible host geometry.
derive-shape-geometry.js	Provides authoritative derived host geometry and padding boundaries.	SVG sizing, padding, containment, and overlap all depend on existing host-geometry calculations.
world-to-screen.js	Converts host/world geometry into screen space for preview and rendering.	The attached SVG must render in the correct visible place relative to its host.
screen-to-world.js	Converts pointer positions into world coordinates.	SVG placement, resize, and interaction must stay in world-space logic.
render-canvas.js	Renders the host item plus attached SVG result.	The milestone requires visible SVG attachment feedback on the existing canvas render path.

Milestone 12 terminology

Main developer names

The following names are recommended for this milestone:

SVG asset

SvgAsset

SvgAssetID

SvgSourceText

SvgSourceType

SvgViewBoxMinX

SvgViewBoxMinY

SvgViewBoxWidth

SvgViewBoxHeight

SvgIntrinsicAspectRatio

SvgUsesRect

SvgUsesCircle

SvgUsesEllipse

SvgUsesPath

SvgUsesSymbol

SvgUsesDefs

SvgUsesUse

SVG attachment item

SvgAttachmentItem

SvgAttachmentID

SvgAssetID

Host relation

HostItemID

HostItemType

Supported host types

Rectangle

Circle

Host-based SVG rule mode

SvgRuleGeometryMode

Recommended value for this milestone:

UseHostGeometry

SVG attachment placement modes

CenterFitInsideRectangle

CenterFitInsideCircle

PreserveAspectRatio

KeepSvgOwnSizeCentered

SVG attachment visual state

SvgVisible

SvgOpacity

SvgZOrder

SvgOffsetX

SvgOffsetY

SvgLocalRotationAngle

SvgFollowHostRotation

Common shared sizing variable

SvgAttachmentScaleRatio

Host-size coupling mode

SvgHostSizeCouplingMode

Recommended value for this milestone:

SvgDrivesHostGeometry

Rectangle host geometry affected by SVG scale

HalfWidth

HalfHeight

Circle host geometry affected by SVG scale

Radius

Derived rendered SVG placement

SvgRenderedCenterX

SvgRenderedCenterY

SvgRenderedWidth

SvgRenderedHeight

SvgRenderedRotationAngle

Rectangle host fitting box

RectangleHostBoxWidth

RectangleHostBoxHeight

Circle host fitting box

CircleHostBoxSide

Stored base fit values

SvgBaseFitWidth

SvgBaseFitHeight

Recommended version-1 rule

For version 1, Recommendation:

SvgRuleGeometryMode = UseHostGeometry

SvgHostSizeCouplingMode = SvgDrivesHostGeometry

PreserveAspectRatio = true

Rectangle mode = CenterFitInsideRectangle

Circle mode = CenterFitInsideCircle

Important milestone rules

Rule 1 — The SVG is a host-attached visual item, not the authoritative rule geometry

The SVG shall be treated as a visual child of the host geometry.

It is not the main containment authority.

It is not the main overlap authority.

It is not the primary source of padding rules.

The host geometry remains the authoritative item for:

hover padding

protection padding

container restrictions

overlap rules

movement rules

resize rules

Rule 2 — The uploaded SVG may be arbitrary valid SVG content

The uploaded SVG does not need to contain only:

<rect>

or

<circle>

It may contain:

<path>

<symbol>

<defs>

<use>

or mixed shape content.

That is acceptable, because the real diagram geometry is still the native host item.

Rule 3 — The SVG shall be saved as an asset, not merged into host geometry fields

The SVG asset shall be saved separately from the host geometry.

The host rectangle or host circle shall still save its normal native variables.

The SVG attachment shall save its own attachment settings separately.

So persistence shall remain split into:

1. host geometry record
2. SVG asset record
3. SVG attachment record

Rule 4 — The same padding, container, and overlap rules apply through the host geometry

If the SVG is attached to a rectangle:

all rectangle padding, containment, and overlap rules shall apply through the host rectangle geometry

If the SVG is attached to a circle:

all circle padding, containment, and overlap rules shall apply through the host circle geometry

This is the correct way to reuse the document's existing rule system without turning arbitrary SVG XML into a separate geometry engine.

Rule 5 — The common size variable shall be `SvgAttachmentScaleRatio`

The user-controlled common size variable for both rectangle-hosted SVG and circle-hosted SVG shall be:

`SvgAttachmentScaleRatio`

Recommended default:

`SvgAttachmentScaleRatio = 1.0`

Meaning:

1.0 = default fitted size

greater than 1.0 = larger

less than 1.0 = smaller

Rule 6 — Increasing SVG size shall update the host geometry

For this milestone, the default size-coupling behavior shall be:

`SvgHostSizeCouplingMode = SvgDrivesHostGeometry`

So if the user increases the SVG size:

rectangle host geometry shall update its:

`HalfWidth`

`HalfHeight`

circle host geometry shall update its:

Radius

After that host update, all normal rules continue to apply through the updated host geometry.

Rule 7 — Rectangle host may rotate and the SVG may follow it

Because the rectangle already supports RotationAngle, a rectangle-hosted SVG should normally support:

`SvgFollowHostRotation = true`

So the SVG rendered rotation may be:

$\text{SvgRenderedRotationAngle} = \text{RectangleRotationAngle} + \text{SvgLocalRotationAngle}$

Rule 8 — Circle host uses a circle-safe fitting region

For a circle host, the safest fitting box is the **inscribed square** of the circle.

If the circle radius is:

Radius

then:

$\text{CircleHostBoxSide} = \sqrt{2} \times \text{Radius}$

This is the recommended fitting rule for version 1 when preserving aspect ratio inside a circle.

Core SVG attachment equations used in this milestone

SVG asset metadata

From the uploaded SVG asset, the system shall extract or derive at least:

`SvgViewBoxMinX`

`SvgViewBoxMinY`

`SvgViewBoxWidth`

`SvgViewBoxHeight`

$\text{SvgIntrinsicAspectRatio} = \text{SvgViewBoxWidth} / \text{SvgViewBoxHeight}$

The full SVG source shall be stored as:

`SvgSourceText`

This preserves the exact visual asset.

Rectangle-hosted SVG with host-size coupling

Let the SVG base fitted size at initial attach time be:

`SvgBaseFitWidth`

`SvgBaseFitHeight`

Let:

$$\text{SvgAttachmentScaleRatio} = s$$

Then:

$$\text{SvgRenderedWidth} = \text{SvgBaseFitWidth} \times s$$
$$\text{SvgRenderedHeight} = \text{SvgBaseFitHeight} \times s$$

For the rectangle host:

$$\text{HalfWidth} = \text{SvgRenderedWidth} / 2$$
$$\text{HalfHeight} = \text{SvgRenderedHeight} / 2$$

Then:

$$\text{RectangleHostBoxWidth} = 2 \times \text{HalfWidth}$$
$$\text{RectangleHostBoxHeight} = 2 \times \text{HalfHeight}$$

And the rendered center is:

$$\text{SvgRenderedCenterX} = \text{CenterX} + \text{SvgOffsetX}$$
$$\text{SvgRenderedCenterY} = \text{CenterY} + \text{SvgOffsetY}$$

If host-follow-rotation is enabled:

$$\text{SvgRenderedRotationAngle} = \text{RotationAngle} + \text{SvgLocalRotationAngle}$$

otherwise:

$$\text{SvgRenderedRotationAngle} = \text{SvgLocalRotationAngle}$$

Circle-hosted SVG with host-size coupling

Let the SVG base fitted size at initial attach time be:

$$\text{SvgBaseFitWidth}$$
$$\text{SvgBaseFitHeight}$$

Let:

$$\text{SvgAttachmentScaleRatio} = s$$

Then:

$$\text{SvgRenderedWidth} = \text{SvgBaseFitWidth} \times s$$
$$\text{SvgRenderedHeight} = \text{SvgBaseFitHeight} \times s$$

Because the SVG must fit inside the inscribed square of the circle, define:

$$\text{RequiredCircleHostBoxSide} = \max(\text{SvgRenderedWidth}, \text{SvgRenderedHeight})$$

Then:

$\text{Radius} = \text{RequiredCircleHostBoxSide} / \sqrt{2}$

Then:

$\text{CircleHostBoxSide} = \sqrt{2} \times \text{Radius}$

And:

$\text{SvgRenderedCenterX} = \text{CenterX} + \text{SvgOffsetX}$

$\text{SvgRenderedCenterY} = \text{CenterY} + \text{SvgOffsetY}$

For version 1, Recommendation:

$\text{SvgRenderedRotationAngle} = \text{SvgLocalRotationAngle}$

because the circle itself does not normally carry visible geometric rotation unless later extended.

Mathematical Calculation Rules for Milestone 12

Milestone 12 calculations attach an SVG visual asset to an existing rectangle or circle host geometry. The host geometry remains the authoritative object for movement, containment, overlap validation, and persistence.

Primary JavaScript files for this calculation scope: `svg-asset-validation.js`, `svg-host-attachment.js`, `svg-scale-to-host.js`, `svg-resize-on-host.js`, `svg-host-padding-validation.js`, `recalculate-svg-host-linked-geometry.js`.

SVG host placement calculation

For a rectangle host, $\text{SvgCenterX} = \text{RectangleCenterX}$ and $\text{SvgCenterY} = \text{RectangleCenterY}$ unless an explicit host-local offset is introduced.

For a circle host, $\text{SvgCenterX} = \text{CircleCenterX}$ and $\text{SvgCenterY} = \text{CircleCenterY}$ unless an explicit host-local offset is introduced.

Rectangle host fit calculation

$\text{SvgMaxWidth} = 2 * \text{RectangleHalfWidth}$. $\text{SvgMaxHeight} = 2 * \text{RectangleHalfHeight}$.

If aspect ratio preservation is enabled, $\text{SvgScale} = \min(\text{SvgMaxWidth} / \text{SvgOriginalWidth}, \text{SvgMaxHeight} / \text{SvgOriginalHeight})$.

Circle host fit calculation

For a circle host, the initial SVG bounding square should not exceed the circle diameter unless the product intentionally allows clipping. $\text{SvgMaxWidth} = 2 * \text{CircleRadius}$ and $\text{SvgMaxHeight} = 2 * \text{CircleRadius}$.

Persistence and validation calculation

The SVG asset text, attachment identity, host item ID, host item type, SVG scale, SVG offset, and host-linked size values shall be recalculated before save. Host Protection Padding remains the authority for overlap and containment validation.

Milestone 12 phases

Milestone 12 shall be implemented in 7 phases.

Phase 12.1 — Upload, validate, and save the SVG asset as text

Goal

Allow the user to upload an SVG, validate it as an SVG asset, extract basic SVG metadata, and save the SVG source text as an asset record.

Scope

This phase is about:

uploading SVG text

validating that the file is SVG

extracting viewBox and basic SVG metadata

saving the full SVG source text

This phase is not yet about attaching the SVG to a host.

Required visible behaviors

Examples:

the system accepts a valid SVG upload

the system extracts the root viewBox if present

the system stores the SVG source text

the system reports that the SVG may contain arbitrary SVG content but will still be used only as a visual asset

Variables to define

At minimum:

SvgAssetID

SvgSourceText

SvgSourceType

SvgViewBoxMinX

SvgViewBoxMinY

SvgViewBoxWidth

SvgViewBoxHeight

SvgIntrinsicAspectRatio

SvgUsesRect

SvgUsesCircle

SvgUsesEllipse

SvgUsesPath

SvgUsesSymbol

SvgUsesDefs

SvgUsesUse

Required event-to-function routing

SVG asset upload flow

upload SVG file

→ validateSvgAssetSource

→ extractSvgAssetMetadata

→ createSvgAssetRecord

→ storeSvgSourceText

State rule

During this phase:

only the SVG asset record is created

no host geometry is affected yet

no attachment exists yet

Success criteria

This phase is successful if:

a valid SVG file can be uploaded

its source text can be stored

its viewBox and basic metadata can be extracted

Function interaction contracts

Function Name: validateSvgAssetSource

Layer: JavaScript

File: svg-asset-validation.js

Purpose: Validate that the uploaded file is a valid SVG asset source for the diagram system.

Trigger:

user uploads an SVG file

Input:

uploaded file data

uploaded text content
file metadata

Validation / Preconditions:

file exists
uploaded text is available
file size is within allowed range

Processing Responsibility:

check that the uploaded text is valid SVG/XML content
reject unsupported unsafe content if your application restricts it
allow arbitrary valid SVG drawing content because the SVG is treated as a visual asset, not authoritative geometry

Output:

valid / invalid SVG asset validation result

State Change / Persistence Effect:

no persistence effect by itself
no host state change

Next Possible Call(s):

extractSvgAssetMetadata()
createSvgAssetRecord()

Error Output / Failure Path:

reject the upload and do not create an SVG asset record if validation fails

Function Name: extractSvgAssetMetadata

Layer: JavaScript

File: svg-asset-metadata.js

Purpose: Extract basic SVG metadata needed for fitting, scaling, and persistence.

Trigger:

successful SVG asset validation result

Input:

SvgSourceText

Validation / Preconditions:

SVG asset source is valid
SVG text content exists

Processing Responsibility:

extract the root viewBox values
derive the intrinsic aspect ratio
detect whether the SVG uses rect, circle, ellipse, path, symbol, defs, and use elements
prepare the metadata required for later attachment calculations

Output:

SVG metadata object

State Change / Persistence Effect:
no persistence effect by itself
returns metadata for asset creation

Next Possible Call(s):
createSvgAssetRecord()
storeSvgSourceText()

Error Output / Failure Path:
fail asset metadata extraction and stop SVG asset creation if required metadata cannot be derived

Phase 12.2 — Attach the SVG asset to a rectangle host

For more information, please refer to section [Coordinate conversion calculations](#) ([place in this document](#)).

Goal

Allow an uploaded SVG asset to be attached to an existing rectangle host using host-based rectangle fitting.

Scope

This phase is about:

- selecting a rectangle host
- selecting an SVG asset
- creating an SVG attachment item
- calculating the initial rectangle-based SVG placement

Required visible behaviors

Examples:

- the SVG appears attached to the rectangle
- the SVG is centered inside the rectangle
- the SVG preserves its aspect ratio by default
- the rectangle remains the authoritative host item

Variables to define

At minimum:

SvgAttachmentID

SvgAssetID

HostItemID

HostItemType = Rectangle

SvgRuleGeometryMode = UseHostGeometry

HostAttachmentMode = CenterFitInsideRectangle

PreserveAspectRatio = true

SvgAttachmentScaleRatio = 1.0

SvgHostSizeCouplingMode = SvgDrivesHostGeometry

SvgOffsetX = 0

SvgOffsetY = 0

SvgLocalRotationAngle = 0

SvgFollowHostRotation = true

SvgBaseFitWidth

SvgBaseFitHeight

Required event-to-function routing

Rectangle SVG attachment flow

select rectangle host

→ selectSvgAsset

→ createRectangleSvgAttachment

→ calculateRectangleSvgBaseFit

→ applySvgScaleToRectangleHost

→ calculateRectangleSvgPlacement

→ renderAttachedSvg

State rule

During this phase:

a real SVG attachment item is created

the rectangle remains the authoritative host item

the attachment stores only host relation and SVG settings

Success criteria

This phase is successful if:

the SVG can be attached to a rectangle

the base fit can be calculated

the rectangle host and SVG placement are synchronized correctly

Function interaction contracts

Function Name: createRectangleSvgAttachment

Layer: JavaScript

File: svg-attachment-rectangle.js

Purpose: Create an SVG attachment item whose host is a rectangle.

Trigger:

user chooses an SVG asset while a rectangle host is selected

Input:

SvgAssetID

SvgSourceText

HostItemID

HostItemType

default SVG attachment settings

Validation / Preconditions:

selected rectangle host exists

SVG asset exists

host type is Rectangle

attachment mode is supported

Processing Responsibility:

create the SVG attachment item

bind it to the rectangle host

store the default rectangle SVG attachment settings

set the rule geometry mode to UseHostGeometry

Output:

new SvgAttachmentItem for rectangle host

State Change / Persistence Effect:

updates temporary in-memory canvas item model only

no persistence effect yet

Next Possible Call(s):

calculateRectangleSvgBaseFit()

applySvgScaleToRectangleHost()

calculateRectangleSvgPlacement()

Error Output / Failure Path:

do not create the SVG attachment if the host or SVG asset is invalid

Function Name: calculateRectangleSvgBaseFit

Layer: JavaScript

File: svg-placement-rectangle.js

Purpose: Calculate the base fitted SVG size inside a rectangle host before scale is applied.

Trigger:

new rectangle SVG attachment created

rectangle host selected for first-time SVG fitting

Input:

rectangle host geometry

SvgViewBoxWidth
SvgViewBoxHeight
PreserveAspectRatio
HostAttachmentMode

Validation / Preconditions:

rectangle host exists
rectangle geometry is valid
SVG asset metadata exists
SVG viewBox dimensions are valid

Processing Responsibility:

calculate the rectangle host box width and height
calculate the base fit scale using the SVG intrinsic aspect ratio
calculate SvgBaseFitWidth and SvgBaseFitHeight

Output:

SvgBaseFitWidth
SvgBaseFitHeight

State Change / Persistence Effect:

updates temporary in-memory attachment-derived values only
no persistence effect

Next Possible Call(s):

applySvgScaleToRectangleHost()
calculateRectangleSvgPlacement()

Error Output / Failure Path:

preserve previous valid base fit state if the host geometry or SVG metadata is invalid

Phase 12.3 — Resize the SVG on a rectangle and make the rectangle host reflect that size

For more information, please refer to section [Protection Padding, Overlap, Containment, and Persistence Calculations](#) ([place in this document](#)).

Goal

Allow the user to change the SVG size using the common variable SvgAttachmentScaleRatio, and make the rectangle host geometry reflect that size change.

Scope

This phase is about:

changing SvgAttachmentScaleRatio

recalculating SvgRenderedWidth and SvgRenderedHeight

updating rectangle HalfWidth and HalfHeight

keeping the SVG aligned to the rectangle after move, resize, and rotation

Required visible behaviors

Examples:

increasing the SVG size makes the rectangle larger

decreasing the SVG size makes the rectangle smaller

moving the rectangle moves the SVG with it

rotating the rectangle rotates the SVG if enabled

Variables to define

At minimum:

SvgAttachmentScaleRatio

SvgHostSizeCouplingMode = SvgDrivesHostGeometry

HalfWidth

HalfHeight

SvgRenderedWidth

SvgRenderedHeight

SvgRenderedRotationAngle

Required event-to-function routing

Rectangle SVG resize flow

user changes SvgAttachmentScaleRatio

→ applySvgScaleToRectangleHost

→ calculateRectangleSvgPlacement

→ renderAttachedSvg

→ renderCanvas

Rectangle host move / resize / rotation flow

move or resize or rotate rectangle

→ updateAttachedSvgAfterRectangleGeometryChange

→ calculateRectangleSvgPlacement

→ renderAttachedSvg

→ renderCanvas

State rule

During this phase:

the user changes SVG size through the common attachment scale variable

the rectangle host geometry is updated from that scale

the SVG rules still apply through the rectangle host geometry

Success criteria

This phase is successful if:

changing the SVG scale changes the rectangle host geometry correctly

moving or rotating the rectangle keeps the SVG aligned correctly

Function interaction contracts

Function Name: `applySvgScaleToRectangleHost`

Layer: JavaScript

File: `svg-scale-host-coupling.js`

Purpose: Apply the common SVG attachment scale to a rectangle host and update the rectangle geometry.

Trigger:

user changes `SvgAttachmentScaleRatio`

rectangle SVG attachment initialized

Input:

`SvgAttachmentScaleRatio`

`SvgBaseFitWidth`

`SvgBaseFitHeight`

`HostItemID`

`SvgHostSizeCouplingMode`

Validation / Preconditions:

rectangle host exists

SVG attachment exists

`SvgBaseFitWidth` and `SvgBaseFitHeight` are valid

coupling mode is `SvgDrivesHostGeometry`

Processing Responsibility:

calculate `SvgRenderedWidth` and `SvgRenderedHeight` from the base fit and scale ratio

update rectangle `HalfWidth` and `HalfHeight` from the scaled SVG size

keep the rectangle center unchanged

Output:

updated rectangle `HalfWidth`

updated rectangle `HalfHeight`

updated scaled SVG size

State Change / Persistence Effect:

updates temporary in-memory rectangle geometry and attachment-derived values only

no persistence effect yet

Next Possible Call(s):

`calculateRectangleSvgPlacement()`

`updateAttachedSvgAfterRectangleGeometryChange()`

`renderCanvas()`

Error Output / Failure Path:

preserve previous valid rectangle geometry and SVG scale state if scale application fails

Function Name: updateAttachedSvgAfterRectangleGeometryChange

Layer: JavaScript

File: svg-follow-host.js

Purpose: Recalculate the attached SVG placement after the rectangle host moves, resizes, or rotates.

Trigger:

rectangle move committed

rectangle resize committed

rectangle rotation committed

rectangle preview geometry updated

Input:

updated rectangle host geometry

updated rectangle RotationAngle

SVG attachment settings

SvgBaseFitWidth

SvgBaseFitHeight

SvgAttachmentScaleRatio

Validation / Preconditions:

rectangle host exists

attached SVG exists

rectangle geometry is valid

Processing Responsibility:

recalculate the rectangle-hosted SVG rendered center

recalculate rendered size from the current scale ratio

apply host rotation if enabled

preserve host attachment mode and offsets

Output:

updated rectangle-hosted SVG rendered placement

State Change / Persistence Effect:

updates temporary in-memory derived SVG placement only

no persistence effect

Next Possible Call(s):

calculateRectangleSvgPlacement()

renderAttachedSvg()

renderCanvas()

Error Output / Failure Path:

preserve previous valid SVG placement if rectangle geometry change data is invalid

Phase 12.4 — Attach the SVG asset to a circle host

For more information, please refer to section [Coordinate conversion calculations](#) ([place in this document](#)).

Goal

Allow an uploaded SVG asset to be attached to an existing circle host using host-based circle-safe fitting.

Scope

This phase is about:

selecting a circle host

selecting an SVG asset

creating an SVG attachment item

calculating the initial circle-based SVG placement

Required visible behaviors

Examples:

the SVG appears attached to the circle

the SVG is centered inside the circle

the SVG preserves its aspect ratio by default

the circle remains the authoritative host item

Variables to define

At minimum:

SvgAttachmentID

SvgAssetID

HostItemID

HostItemType = Circle

SvgRuleGeometryMode = UseHostGeometry

HostAttachmentMode = CenterFitInsideCircle

PreserveAspectRatio = true

SvgAttachmentScaleRatio = 1.0

SvgHostSizeCouplingMode = SvgDrivesHostGeometry

SvgOffsetX = 0

SvgOffsetY = 0

SvgLocalRotationAngle = 0

SvgBaseFitWidth

SvgBaseFitHeight

Required event-to-function routing

Circle SVG attachment flow

select circle host

→ selectSvgAsset

→ createCircleSvgAttachment

→ calculateCircleSvgBaseFit

→ applySvgScaleToCircleHost

→ calculateCircleSvgPlacement

→ renderAttachedSvg

State rule

During this phase:

a real SVG attachment item is created

the circle remains the authoritative host item

the attachment stores only host relation and SVG settings

Success criteria

This phase is successful if:

the SVG can be attached to a circle

the base fit can be calculated

the circle host and SVG placement are synchronized correctly

Function interaction contracts

Function Name: createCircleSvgAttachment

Layer: JavaScript

File: svg-attachment-circle.js

Purpose: Create an SVG attachment item whose host is a circle.

Trigger:

user chooses an SVG asset while a circle host is selected

Input:

SvgAssetID

SvgSourceText

HostItemID

HostItemType

default SVG attachment settings

Validation / Preconditions:

selected circle host exists

SVG asset exists
host type is Circle
attachment mode is supported

Processing Responsibility:
create the SVG attachment item
bind it to the circle host
store the default circle SVG attachment settings
set the rule geometry mode to UseHostGeometry

Output:
new SvgAttachmentItem for circle host

State Change / Persistence Effect:
updates temporary in-memory canvas item model only
no persistence effect yet

Next Possible Call(s):
calculateCircleSvgBaseFit()
applySvgScaleToCircleHost()
calculateCircleSvgPlacement()

Error Output / Failure Path:
do not create the SVG attachment if the host or SVG asset is invalid

Function Name: calculateCircleSvgBaseFit

Layer: JavaScript
File: svg-placement-circle.js
Purpose: Calculate the base fitted SVG size inside a circle host before scale is applied.

Trigger:
new circle SVG attachment created
circle host selected for first-time SVG fitting

Input:
circle host geometry
SvgViewBoxWidth
SvgViewBoxHeight
PreserveAspectRatio
HostAttachmentMode

Validation / Preconditions:
circle host exists
circle geometry is valid
SVG asset metadata exists
SVG viewBox dimensions are valid

Processing Responsibility:
calculate the circle-safe fitting box using the inscribed square rule
calculate the base fit scale using the SVG intrinsic aspect ratio
calculate SvgBaseFitWidth and SvgBaseFitHeight

Output:

SvgBaseFitWidth

SvgBaseFitHeight

State Change / Persistence Effect:

updates temporary in-memory attachment-derived values only

no persistence effect

Next Possible Call(s):

applySvgScaleToCircleHost()

calculateCircleSvgPlacement()

Error Output / Failure Path:

preserve previous valid base fit state if the host geometry or SVG metadata is invalid

Phase 12.5 — Resize the SVG on a circle and make the circle host reflect that size

For more information, please refer to section [Protection Padding, Overlap, Containment, and Persistence Calculations](#) ([place in this document](#)).

Goal

Allow the user to change the SVG size using the common variable SvgAttachmentScaleRatio, and make the circle host geometry reflect that size change.

Scope

This phase is about:

changing SvgAttachmentScaleRatio

recalculating SvgRenderedWidth and SvgRenderedHeight

updating circle Radius

keeping the SVG aligned to the circle after move and resize

Required visible behaviors

Examples:

increasing the SVG size makes the circle larger

decreasing the SVG size makes the circle smaller

moving the circle moves the SVG with it

resizing the circle through host rules refits the SVG correctly

Variables to define

At minimum:

SvgAttachmentScaleRatio

SvgHostSizeCouplingMode = SvgDrivesHostGeometry

Radius

SvgRenderedWidth

SvgRenderedHeight

CircleHostBoxSide

Required event-to-function routing

Circle SVG resize flow

user changes SvgAttachmentScaleRatio

→ applySvgScaleToCircleHost

→ calculateCircleSvgPlacement

→ renderAttachedSvg

→ renderCanvas

Circle host move / resize flow

move or resize circle

→ updateAttachedSvgAfterCircleGeometryChange

→ calculateCircleSvgPlacement

→ renderAttachedSvg

→ renderCanvas

State rule

During this phase:

the user changes SVG size through the common attachment scale variable

the circle host geometry is updated from that scale

the SVG rules still apply through the circle host geometry

Success criteria

This phase is successful if:

changing the SVG scale changes the circle host geometry correctly

moving or resizing the circle keeps the SVG aligned correctly

Function interaction contracts

Function Name: applySvgScaleToCircleHost

Layer: JavaScript

File: svg-scale-host-coupling.js

Purpose: Apply the common SVG attachment scale to a circle host and update the circle geometry.

Trigger:

user changes SvgAttachmentScaleRatio

circle SVG attachment initialized

Input:

SvgAttachmentScaleRatio
SvgBaseFitWidth
SvgBaseFitHeight
HostItemID
SvgHostSizeCouplingMode

Validation / Preconditions:

circle host exists
SVG attachment exists
SvgBaseFitWidth and SvgBaseFitHeight are valid
coupling mode is SvgDrivesHostGeometry

Processing Responsibility:

calculate scaled SVG rendered width and height
calculate the required circle-safe fitting box side
update circle Radius from the required fitting box side using the inscribed-square rule
keep the circle center unchanged

Output:

updated circle Radius
updated scaled SVG size

State Change / Persistence Effect:

updates temporary in-memory circle geometry and attachment-derived values only
no persistence effect yet

Next Possible Call(s):

calculateCircleSvgPlacement()
updateAttachedSvgAfterCircleGeometryChange()
renderCanvas()

Error Output / Failure Path:

preserve previous valid circle geometry and SVG scale state if scale application fails

Function Name: updateAttachedSvgAfterCircleGeometryChange

Layer: JavaScript

File: svg-follow-host.js

Purpose: Recalculate the attached SVG placement after the circle host moves or resizes.

Trigger:

circle move committed
circle resize committed
circle preview geometry updated

Input:

updated circle host geometry
SVG attachment settings
SvgBaseFitWidth
SvgBaseFitHeight
SvgAttachmentScaleRatio

Validation / Preconditions:

- circle host exists
- attached SVG exists
- circle geometry is valid

Processing Responsibility:

- recalculate the circle-hosted SVG rendered center
- recalculate rendered size from the current scale ratio
- preserve host attachment mode and offsets

Output:

- updated circle-hosted SVG rendered placement

State Change / Persistence Effect:

- updates temporary in-memory derived SVG placement only
- no persistence effect

Next Possible Call(s):

- calculateCircleSvgPlacement()
- renderAttachedSvg()
- renderCanvas()

Error Output / Failure Path:

- preserve previous valid SVG placement if circle geometry change data is invalid

Phase 12.6 — Apply padding, container restrictions, and overlap rules to SVG through the host geometry

For more information, please refer to section [Protection Padding, Overlap, Containment, and Persistence Calculations](#) ([place in this document](#)) and section [Parent-container expansion calculations](#) ([place in this document](#)).

Goal

Apply the same document rules for hover padding, protection padding, container restrictions, and overlap to an attached SVG by using the authoritative host geometry.

Scope

This phase is about:

- rectangle-hosted SVG using rectangle rules

- circle-hosted SVG using circle rules

- container restrictions through the host

- overlap rules through the host

Required visible / measurable behaviors

Examples:

if the rectangle host is inside a parent rectangle container, the attached SVG follows the same container restrictions because the host rectangle is restricted

if the circle host is inside a parent rectangle container, the attached SVG follows the same container restrictions because the host circle is restricted

if overlap is forbidden for the host geometry, the attached SVG is also effectively constrained because the host is constrained

Variables to define

At minimum:

SvgRuleGeometryMode = UseHostGeometry

HostItemID

HostItemType

HostHoverPaddingState

HostProtectionPaddingState

HostContainmentState

HostOverlapState

Required event-to-function routing

SVG host-rule flow

host move or resize or containment validation or overlap validation

→ resolveSvgRuleGeometryFromHost

→ validateSvgThroughHostGeometry

→ renderAttachedSvg

→ renderCanvas

State rule

During this phase:

the SVG does not introduce a second independent rule geometry

the host geometry remains the only authoritative rule geometry

the SVG inherits the host's rule results

Success criteria

This phase is successful if:

the same document rules apply to attached SVGs through their hosts

rectangle-hosted SVG obeys rectangle rules

circle-hosted SVG obeys circle rules

Function interaction contracts

Function Name: resolveSvgRuleGeometryFromHost

Layer: JavaScript

File: svg-rule-geometry.js

Purpose: Resolve the authoritative rule geometry of an attached SVG from its native host item.

Trigger:

SVG attachment validation requested

host geometry changed

host containment or overlap validation requested

Input:

SvgAttachmentID

HostItemID

HostItemType

SvgRuleGeometryMode

Validation / Preconditions:

SVG attachment exists

host item exists

SvgRuleGeometryMode is valid

Processing Responsibility:

resolve the rectangle host geometry if HostItemType = Rectangle

resolve the circle host geometry if HostItemType = Circle

return the authoritative rule geometry used for validation

Output:

authoritative host rule geometry for the SVG attachment

State Change / Persistence Effect:

no persistence effect

no direct permanent state change

Next Possible Call(s):

validateSvgThroughHostGeometry()

renderAttachedSvg()

Error Output / Failure Path:

do not validate the SVG attachment if the host rule geometry cannot be resolved

Function Name: validateSvgThroughHostGeometry

Layer: JavaScript

File: svg-rule-geometry.js

Purpose: Apply the existing diagram rules to an attached SVG by validating its authoritative host geometry.

Trigger:

resolved host rule geometry available for an SVG attachment

Input:

resolved host rule geometry

host containment state
host overlap state
host padding state
host parent relation if applicable

Validation / Preconditions:

host rule geometry exists
host item is valid
host validation context is valid

Processing Responsibility:

use the host geometry for hover padding rules
use the host geometry for Protection Padding rules
use the host geometry for parent-container validation if applicable
use the host geometry for overlap validation if applicable
return the effective validation state for the attached SVG

Output:

effective SVG validation state through host geometry

State Change / Persistence Effect:

no persistence effect
no direct permanent state change

Next Possible Call(s):

renderAttachedSvg()
renderCanvas()

Error Output / Failure Path:

preserve previous valid host-derived SVG rule state if validation fails

Phase 12.7 — Recalculate committed SVG asset, attachment, and host-linked geometry and persist to JSON and database

Exact variables to save in this phase

Exact committed Milestone 12 save variables: SVG asset fields = SvgAssetID, SvgSourceText, SvgSourceType, SvgViewBoxMinX, SvgViewBoxMinY, SvgViewBoxWidth, SvgViewBoxHeight, SvgIntrinsicAspectRatio. SVG attachment fields = SvgAttachmentID, SvgAssetID, HostItemID, HostItemType, SvgRuleGeometryMode, HostAttachmentMode, PreserveAspectRatio, SvgAttachmentScaleRatio, SvgHostSizeCouplingMode, SvgOffsetX, SvgOffsetY, SvgLocalRotationAngle, SvgFollowHostRotation, SvgOpacity, SvgZOrder, SvgVisible, SvgBaseFitWidth, SvgBaseFitHeight. Host geometry remains saved in its native Rectangle or Circle record using the previously approved fields.

Save regression rule for this phase

All approved Milestones 2 through 10 committed variables shall already remain saved correctly in both persistence paths. This phase passes only if save-to-json.js and save-to-db.js add the Milestone 12 SVG asset and attachment variables without breaking the earlier approved save sets.

Save test criteria for this phase

Verify that both shared save files persist every listed Milestone 12 SVG asset and attachment variable, that host geometry remains saved in its native Rectangle or Circle record, and that reopen from either source restores the same SVG attachment behavior and placement.

Goal

Recalculate all committed SVG asset, attachment, and host-linked geometry and persist the diagram to both JSON and database.

Scope

This phase is about:

recalculating rectangle-hosted SVG placement

recalculating circle-hosted SVG placement

rebuilding the save model

saving SVG asset data

saving SVG attachment data

saving host geometry data

Important recalculation rule

Before save, the system shall recalculate from the committed state:

the authoritative host geometry

the SVG asset metadata

the base fit size

the scaled rendered placement

the host-linked rule geometry mode

The authoritative saved truth shall remain separated into:

1. host geometry
2. SVG asset
3. SVG attachment settings

The fully rendered SVG placement shall always be derivable again from those committed values.

Recommended saved fields

For an SVG asset

SvgAssetID

SvgSourceText

SvgSourceType

SvgViewBoxMinX

SvgViewBoxMinY

SvgViewBoxWidth

SvgViewBoxHeight

SvgIntrinsicAspectRatio

For an SVG attachment

SvgAttachmentID

SvgAssetID

HostItemID

HostItemType

SvgRuleGeometryMode

HostAttachmentMode

PreserveAspectRatio

SvgAttachmentScaleRatio

SvgHostSizeCouplingMode

SvgOffsetX

SvgOffsetY

SvgLocalRotationAngle

SvgFollowHostRotation

SvgOpacity

SvgZOrder

SvgVisible

SvgBaseFitWidth

SvgBaseFitHeight

Required event-to-function routing

Save-commit flow

click Save button

→ recalculateAllSvgAttachmentGeometry

→ validateCommittedSvgAttachmentState

→ buildSvgAsset.JsonData

→ buildSvgAttachment.JsonData

→ buildDiagram.JsonObject

→ serializeDiagramToJson

- saveDiagramJsonFile
- buildSvgAssetSavePayload
- buildSvgAttachmentSavePayload
- buildDiagramSavePayload
- callSaveDiagramApi
- DiagramController.SaveDiagram
- DiagramService.SaveDiagram
- DiagramRepository.SaveDiagram
- handleSaveCommitResult

State rule

During this phase:

recalculation must occur before persistence

JSON save and database save must use the same committed recalculated SVG asset and attachment state

host geometry remains saved in its own native rectangle or circle record

Success criteria

This phase is successful if:

all committed SVG asset and attachment geometry is recalculated before save

the saved JSON and database state both represent the same committed SVG-host logic

reopening from file or database restores the same host relation, SVG source, scale rule, and rendered behavior

Function interaction contracts

Function Name: recalculateAllSvgAttachmentGeometry

Layer: JavaScript

File: svg-attachment-recalculation.js

Purpose: Recalculate all committed SVG attachment placement derived from committed host geometry and committed SVG attachment settings.

Trigger:

explicit save flow

diagram load reconstruction

committed host geometry update requiring full SVG refresh

Input:

SVG attachment collection

SVG asset collection

host geometry collection

committed SVG attachment settings

Validation / Preconditions:

all referenced host items exist

all referenced SVG attachments exist

all referenced SVG assets exist
required attachment and asset fields are present

Processing Responsibility:
for each rectangle-hosted SVG
recalculate rectangle-based fitted placement and host-linked rendered placement
for each circle-hosted SVG
recalculate circle-based fitted placement and host-linked rendered placement
prepare committed SVG placement for validation and persistence

Output:
recalculated SVG attachment geometry collection

State Change / Persistence Effect:
updates in-memory derived SVG placement only
no direct persistence by this function

Next Possible Call(s):
validateCommittedSvgAttachmentState()
buildSvgAssetJsonData()
buildSvgAttachmentJsonData()
renderCanvas()

Error Output / Failure Path:
stop save flow if required SVG attachment geometry cannot be recalculated

Function Name: validateCommittedSvgAttachmentState

Layer: JavaScript
File: svg-attachment-save-validation.js
Purpose: Verify that the committed SVG asset and attachment state is valid before persistence.

Trigger:
save flow after SVG attachment recalculation

Input:
committed SVG asset data
committed SVG attachment data
host geometry data
recalculated SVG rendered placement

Validation / Preconditions:
recalculation already completed
required SVG asset fields exist
required SVG attachment fields exist
host references are valid
asset references are valid

Processing Responsibility:
validate host references
validate attachment mode
validate size and placement inputs
validate scale-to-host coupling rule

validate that recalculated SVG placement is consistent with the committed asset and attachment definitions

Output:

valid / invalid committed SVG attachment result

State Change / Persistence Effect:

no persistence effect

no direct permanent state change

Next Possible Call(s):

buildSvgAssetJsonData()

buildSvgAttachmentJsonData()

buildSvgAssetSavePayload()

buildSvgAttachmentSavePayload()

Error Output / Failure Path:

stop save flow and report SVG asset/attachment validation failure if committed state is invalid

Function Name: buildSvgAttachmentSavePayload

Layer: JavaScript

File: svg-attachment-api.js

Purpose: Build the committed SVG attachment payload that will be sent to the server as part of the full diagram save.

Trigger:

explicit diagram save

Input:

committed SVG attachment model

diagram metadata

host relation data

linked SVG asset identity

Validation / Preconditions:

SVG attachment exists

diagram identity exists

required SVG attachment fields are valid

Processing Responsibility:

extract committed SVG attachment save data

exclude temporary preview-only state

prepare payload shape expected by backend

Output:

SvgAttachmentSavePayload

State Change / Persistence Effect:

no direct persistence by this function

no direct state change beyond building the payload

Next Possible Call(s):
buildDiagramSavePayload()
callSaveDiagramApi()

Error Output / Failure Path:
return structured failure and stop save flow if required SVG attachment data is invalid

Final result after Milestone 12

At the end of Milestone 12, the developer should have:

SVG upload and validation as a text/XML asset

saved SVG asset source text and SVG metadata

SVG attachment to rectangles

rectangle-hosted SVG fitting and centering

common SVG size variable using SvgAttachmentScaleRatio

rectangle host geometry updated from SVG size changes

rectangle move / resize / rotation follow behavior for attached SVG

SVG attachment to circles

circle-safe SVG fitting using the inscribed square rule

circle host geometry updated from SVG size changes

circle move / resize follow behavior for attached SVG

application of padding, container restrictions, and overlap rules through the authoritative host geometry

recalculation and persistence of committed SVG asset and attachment state

This milestone makes host-attached SVG imagery a real visual and persistent part of the diagram system while keeping the host rectangle or circle as the authoritative rule geometry.

Keep the SVG rules subordinate to host geometry, overlap, padding, and containment rules already defined earlier in the document.

After base attachment works, add host-linked resize behavior and only then add persistence.

Implement SVG upload/validation first, then host attachment for Rectangle, then host attachment for Circle.

Recommended implementation order

Recommended coding files structure additions for Milestone 12

JavaScript

Recommended new modules:

svg-asset-validation.js

svg-asset-metadata.js

svg-attachment-rectangle.js

svg-placement-rectangle.js

svg-attachment-circle.js

svg-placement-circle.js

svg-scale-host-coupling.js

svg-follow-host.js

svg-rule-geometry.js

svg-attachment-recalculation.js

svg-attachment-save-validation.js

svg-attachment-json.js

svg-attachment-api.js

Optional C# structure

SvgAssetDto.cs

SvgAttachmentDto.cs

SvgAttachmentPlacementDto.cs

Do not move live SVG fitting, host-size coupling, host-follow recalculation, or host-rule validation into C#.

Best folder layout additions for Milestone 12

project/

└─ js/

| └─ svg-asset-validation.js

| └─ svg-asset-metadata.js

| └─ svg-attachment-rectangle.js

| └─ svg-placement-rectangle.js

| └─ svg-attachment-circle.js

| └─ svg-placement-circle.js

| └─ svg-scale-host-coupling.js

| └─ svg-follow-host.js

| └─ svg-rule-geometry.js

| └─ svg-attachment-recalculation.js

| └─ svg-attachment-save-validation.js

```
| ├── svg-attachment-json.js
| └─ svg-attachment-api.js
|   ├── csharp/
|   |   ├── SvgAssetDto.cs
|   |   ├── SvgAttachmentDto.cs
|   |   └─ SvgAttachmentPlacementDto.cs
```

More ideas you can define in Milestone 12

You may either include these now or mark them as future-ready additions:

1. Multiple SVG attachments on one host geometry
2. Explicit SVG hit testing and selection behavior
3. Clipping the SVG exactly to the host rectangle or circle
4. SVG anchor points other than center
5. SVG color overrides or theme mapping
6. Replacing one SVG asset while keeping the same host attachment settings
7. Standalone SVG items later using normalized rule geometry rather than host geometry

The strongest design recommendation here is:

Treat the SVG as a **host-attached visual child**, save the SVG as a **text/XML asset**, keep the **rectangle or circle as the authoritative rule geometry**, use **SvgAttachmentScaleRatio** as the shared user-controlled size variable, and let that shared scale update the host geometry through **SvgHostSizeCouplingMode = SvgDrivesHostGeometry**. That gives you one clean rule system for both shapes without breaking the document's existing geometry model.